

Syllabus

DATS 6101—Introduction to Data Science • Fall 2026

NOTE

This page is the authoritative syllabus for the course. If anything posted elsewhere (Blackboard, a printed handout, an older PDF) disagrees with this page, this page is correct.

Course and Contact Information

Term	Fall 2026
Section	11
Time	Mon 6:10–8:40 p.m.
Location	<u>1776 G Street, Room C-115</u>

Instructor: [Dr. Sushovan Majhi](#)

Office: Samson Hall 313 (easiest entry from the Corcoran 2nd floor, through the end-of-hallway stairs)

Email: s.majhi@gwu.edu

Office hours: Monday 4:00–6:00 p.m., Samson Hall 313, immediately before this class. No appointment needed—walk in. They can also be joined online at gwu-edu.zoom.us/my/smajhi, so that anyone who cannot be on campus at that hour can still drop in. These are joint office hours shared with DATS 6103, so students from either course are welcome at any point during the two hours.

Teaching Assistant: none for this section.

Course Description

This course introduces the concepts and practice of data science: framing a research question that data can answer, preparing and exploring real data, determining what the evidence supports, and communicating the result. Topics progress from exploratory analysis through probability and statistical inference to linear and logistic modeling, where the emphasis is on what a fitted model licenses you to claim.

R is used throughout, with **RStudio**, **RMarkdown**, and **Git**. Every code example in the course notes runs in the browser, with no installation needed to try it.

Students are expected to use AI assistants and to identify when those assistants are wrong. Assignments are graded on that judgment as well as on correctness; see [Use of AI Assistants](#).

Learning Outcomes

As a result of completing this course, students will be able to:

1. Frame a research question on real data using S.M.A.R.T. principles, and say what evidence would answer it;
2. Import, clean, and reshape messy real-world data in R;
3. Explore data and describe it honestly, choosing summaries and visualizations that do not mislead;
4. Reason with random variables: compute and interpret expectation and variance, apply conditional probability and Bayes' rule, and explain why variances of independent quantities add;
5. Derive and use the fact that the standard error of a mean is $\sigma/\sqrt{n}$, and reason about sampling distributions and the central limit theorem by simulation rather than taking them on faith;
6. Construct and interpret confidence intervals, and state precisely what they do and do not claim;

7. Choose and carry out an appropriate hypothesis test, interpret a p-value without overstating it, compute the power of a study and say what a non-significant result is worth, and recognize when multiple comparisons have inflated a finding;
8. Design and analyze a randomized experiment end to end—assignment, sample size, guardrail metrics and a stopping rule fixed in advance—and explain what randomization licenses that an observational comparison does not;
9. Judge whether a sample can answer the question asked of it, identifying selection and survivorship bias and reasoning about why data is missing rather than only removing it;
10. Fit a linear model and treat each coefficient as an estimate: report its standard error and confidence interval, test it, and state precisely what “holding the other variables constant” does and does not buy;
11. Check a fitted model rather than trusting it—read residual plots, identify influential observations, and explain how confounding and omitted variables change a conclusion, and state what would be needed to license a causal claim without an experiment;
12. Distinguish a confidence interval from a prediction interval, and use resampling—the bootstrap and permutation tests—to quantify uncertainty when the standard formulas do not apply;
13. Fit and interpret a logistic model, report odds ratios with intervals, and explain why linear regression will not do for a binary outcome;
14. Collaborate with teammates using Git, and defend a shared analysis out loud to both technical and general audiences.

Course Prerequisites and Software

You are expected to have a basic understanding of introductory statistics and some exposure to R, whether through prior coursework or online resources. This course does not teach programming from zero.

No class session is spent on R itself. The language is instead covered by the [R Primer](#), a self-paced part of the notes that runs alongside the course from day one: the environment and RStudio, R's core objects, coercion and missing values, factors and data frames, plotting with ggplot2, reproducible documents, and Git, with browser-based exercises throughout that check themselves and say which misconception you hit. Work through it in the opening weeks and use office hours for anything that will not come right.

The GW bootcamp's [Programming](#) track covers the habits behind it—Git, debugging, and environments—but it is taught in Python, so the R itself is what the Primer is for. The bootcamp's [Math](#) track is the foundation for DATS 6103 rather than for this course.

Install the following before Aug 24:

1. **R**—<https://www.r-project.org/>
2. **RStudio**—<https://posit.co/products/open-source/rstudio/>
3. **Git**, and a free **GitHub** account—<https://github.com/>
4. Access to an **AI assistant** (ChatGPT, Claude, Copilot, Gemini, or similar). Free tiers are sufficient; no paid subscription is required.

The [Data Science Help Desk](#) exists for exactly this: it is staffed for students in the program or taking DATS classes, in Samson Hall 304—the same stairs as my office, one floor up—and dsheelpdesk@gwu.edu will arrange a remote session. Take a setup problem there in the first two weeks rather than to the week an assignment is due.

Textbooks

No single book covers this course. All of the following are free online.

1. **OpenIntro Statistics**, Diez et al.—<https://openintro.org/book/os/>
2. **Introductory Statistics**, Illowsky et al.—<https://openstax.org/details/books/introductory-statistics>
3. **An Introduction to Statistical Learning**, James et al.—<https://www.statlearning.com/>
4. **R for Data Science**, Wickham, Çetinkaya-Rundel and Golemund—<https://r4ds.hadley.nz/>

Course Format

This course is taught as a flipped workshop. Lecture material is delivered as assigned reading; class time is spent on discussion and supervised lab work. Slides are not presented in class, and content is not re-covered from the front of the room. Reading before each session is therefore required.

Each session runs in three modes:

- **Question triage.** Instructor-led, drawn entirely from the questions students posted on the reading.
- **Reasoning and discussion.** Students predict outputs, select tests, and defend conclusions to one another.
- **Lab.** Supervised hands-on work at the keyboard.

Before class: readings and questions

Each class day has an assigned reading, listed in the course schedule. Every chapter of the course notes carries a comment box; students post questions there as they read, and may also answer one another. **Questions are due Monday at 12:00 noon**, the morning of class. One substantive question earns the week’s credit—a question that identifies what the reading left unclear, rather than a request to restate it. **Every week counts—no reading question is dropped.** You have the whole week to post one, and a reading you skipped is the hardest thing in this course to catch up on later.

The first class is an exception: no reading is due before Aug 24. Students should arrive with R and RStudio installed. Questions and Data is read *after* that session; assigned reading proper begins with Exploratory Data Analysis, due before Aug 31, with the first questions due Monday, Aug 31 at noon. One chapter of Questions and Data—Where the Data Came From—is due with it, because the Aug 31 lab is built on it.

The R Primer sits outside this cycle. It is not tied to any session and has no question deadline; post there whenever something does not come right, and those questions are picked up in office hours or at the start of a session.

THE FIRST SESSION IS UNGRADED

No work on Aug 24 counts toward the final grade. It is for installing software, registering with the comment system, and becoming familiar with the format.

Session structure

TIME	ACTIVITY
0:00–0:20	Question triage
0:20–0:45	Reasoning and discussion
0:45–1:10	Lab, part 1
1:10–1:20	Break
1:20–1:35	Checkpoint
1:35–2:20	Lab, part 2
2:20–2:30	Wrap-up

Recordings

Sessions are recorded with GW’s lecture capture and posted through Echo360. The midterm is not recorded.

What that captures here is the question triage and the discussion, because there is no lecture to capture—the reading is the lecture. Use it to review, and for a session you have to miss.

It is not a substitute for being in the room. In-class work is assessed on observed participation, so a session watched afterwards earns none of its three points. That is what the two dropped sessions are for.

In-class work and how it is graded

In-class work is assessed on observed participation rather than on submitted files. Each session is worth three points:

COMPONENT	CRITERION
Interview	Took both roles in the discussion, confirmed by a signed rubric card from whoever interviewed you.
Discussion	Contributed substantively at the checkpoint.
Build	Engaged with the lab and able to describe the current difficulty when asked.

None of the three criteria requires a correct answer. **The two lowest sessions are dropped**, without documentation or prior permission.

Participation is recorded at **eleven sessions**—every meeting except Aug 24, which is ungraded, Oct 26, which is the midterm, and Dec 10, which is the presentation session. No points are recorded on those three days, and there is no class on Nov 16. With the two lowest dropped, nine sessions count.

Students work in a pair for the discussion and a group of two or three for the lab. When an odd number is present, one discussion group is a trio and the interview goes round it—A interviews B, B interviews C, C interviews A—so everyone still takes both roles.

In the lab every student is at a keyboard. One job rotates within each group, so that every student takes it over the semester: the **skeptic** is responsible for identifying what would falsify each result the group accepts.

Homework

Homework is individual and due most weeks, **Sunday at 11:59 p.m.** Each assignment is graded on the correctness of the analysis and on the quality of the AI disclosure described under [Use of AI Assistants](#). Details are posted on Blackboard. **Every homework counts—no homework score is dropped.**

Project

A team project takes a real dataset through the full life cycle: S.M.A.R.T. questions, acquisition, cleaning, exploration, modeling, and a written and spoken account of the findings.

Deliverables are a **GitHub repository**, a **written report** stating what the analysis does and does not support, an **AI disclosure statement**, and a **presentation**. Presentation questions are answered individually, so members of one team may receive different grades.

The project is proposed Oct 19 and presented on **Thursday, December 10**. The start time is announced during the term.

Midterm Exam

The midterm is held in class on **Monday, October 26** and covers the material through Oct 19: the data science life cycle, exploratory analysis, probability, random variables, expectation and variance, the central limit theorem, interval estimation, hypothesis testing, and the choice of an appropriate test.

It runs the **full session, 2 hours 30 minutes**, and is worth 100 points.

The exam is individual and taken without AI assistance. As with in-class work, this does not mean from memory.

It is written on paper, and no device may be open—no laptop, tablet, phone or watch. That is not a remark about trust. Everything this course runs, it runs in the browser, and anything the browser can reach is reachable from the page itself; a laptop in an exam is a door no rule written here can close. Paper has nothing to open.

You may bring **printed material only**, in any quantity:

- the chapters covered,
- the **labs whose solutions have been released**,
- the homework covered,
- and your own notes.

A PDF of exactly that material is posted two weeks before, so none of it has to be assembled by hand. Nothing on the paper has to be computed that cannot be done on paper: every critical value a question needs is printed in the question that needs it.

A **full-length sample paper** is posted on **Oct 12**, with its answers released after the Oct 19 session.

Late and Missed Work

Homework may be submitted up to **48 hours late, twice in the term**, with no penalty and no reason required. Say in the submission that you are using one. Beyond those two, and beyond 48 hours, homework is not accepted and scores zero. All eight assignments count, which is why the two extensions exist: there is no dropped homework score behind them.

No documentation or prior permission is needed to use an extension. In-class work is the component that carries drops—the two lowest sessions—and those cover illness, travel and everything else. Reading questions carry none: the window is a full week rather than an evening, so there is nothing a drop would be absorbing.

The midterm and the project presentation are fixed dates and cannot be dropped. A make-up is arranged only for a documented emergency, and only if you contact me as early as you are able.

Use of AI Assistants

Generative AI tools are permitted in this course under the conditions below, and students are responsible for the correctness of anything they submit.

ASSESSMENT	AI POLICY
Homework	Encouraged, with disclosure
In-class work	Not permitted
Midterm exam	Not permitted
Project	Permitted, must be disclosed in the report

Homework submitted with AI assistance must include the **prompt log** and a short **critique** identifying what the assistant got wrong or would have got wrong if accepted unchanged. The critique is graded alongside the analysis.

In-class work is unaided with respect to AI. It is not closed-book: the R documentation and these course notes may be consulted freely.

Submitting a fabricated or altered prompt log constitutes falsification of data under the Academic Integrity Code and will be reported. Declaring that no AI was used is always an acceptable answer.

THIS POLICY DIFFERS FROM OTHER SECTIONS

Other sections of DATS 6101 may set a different AI policy. **Students are bound by the policy of the section in which they are enrolled.** The policies for this section and for DATS 6103 with this instructor are identical.

Course Schedule

Monday sections lose three meetings to Labor Day, Fall Break, and Thanksgiving, and a fourth to a conference on November 16. No session is spent on R itself; that material is in the self-paced [R Primer](#). The university designates **Wednesday, December 9** a Designated Monday; Monday classes meet that day, and it is the last teaching session. Project presentations are held on the Make-Up Day, **Thursday, December 10**, at a start time announced during the term.

DATE	MODULE	TOPICS	READING	DUE
Aug 24	Foundations	Course introduction and how this course runs; S.M.A.R.T. questions and the data science life cycle; importing and cleaning real data · <i>Lab</i> : Getting messy data into shape (<i>ungraded</i>)	Nothing due in advance. Install R and RStudio; read Questions and Data in full and start the R Primer afterwards—neither carries a deadline	
Aug 31		Where the data came from —selection and survivorship bias, convenience samples, and missing data as a mechanism rather than a cleaning chore; exploratory data analysis, and summaries and visualizations that do not mislead · <i>Lab</i> : EDA on a real dataset, and asking who is missing from it	Exploratory Data Analysis · Where the Data Came From	
Sep 7		Labor Day—no class		
Sep 14	Inference	Probability and random variables : outcomes and events, conditional probability and Bayes, independence; the distributions we actually use · <i>Lab</i> : Simulating probabilities before computing them	Probability	HW 1
Sep 21		Expectation and variance , and the one result the rest of the term rests on: variances of independent quantities add, so $\text{Var}(\bar{X}) = \sigma^2/n$ and the standard error shrinks like $1/\sqrt{n}$ · <i>Lab</i> : Watching $\sigma/\sqrt{n}$ emerge by simulation	Expectation and Variance	HW 2
Sep 28		The central limit theorem and confidence intervals; what “95% confident” does and does not claim; the <i>t</i> distribution, and why the standard interval for a proportion fails at the rates commercial data actually has · <i>Lab</i> : Building 100 intervals and counting how many cover	Confidence Intervals	HW 3
Oct 5		Hypothesis testing and p-values; what a small p-value licenses you to say; statistical power —what a non-significant result is worth, and why underpowered studies overstate every effect they publish; multiple comparisons —why testing many things and reporting the winner is how most irreproducible findings happen · <i>Lab</i> : Testing, and misreading tests	Hypothesis Testing	HW 4
Oct 12		Fall Break—no class		
Oct 19		Experiments and causal claims : randomization, and what it licenses that no observational comparison can; designing an A/B test end to end—assignment, sample ratio mismatch , guardrail metrics, and a stopping rule fixed in advance; practical versus statistical significance; choosing a test—chi-squared and contingency tables, with a decision table for the rest · <i>Lab</i> : Designing and analyzing an A/B test	Experiments and Causal Claims	Project proposal
Oct 26		Midterm exam (in class)		
Nov 2	Modeling	Linear models and coefficient inference : fitting; a coefficient as an estimate with a sampling distribution; standard errors, t-tests and confidence intervals for coefficients; R^2 and what it does not tell you · <i>Lab</i> : Fitting and reading a model	Linear Models	HW 5
Nov 9		Model checking, and what a coefficient may claim : residual diagnostics, leverage and influence; confounding and omitted-variable bias —why “holding the others constant” depends on what you put in the model; what would license a causal claim without the randomization of Oct 19, and why	Model Checking	HW 6

DATE	MODULE	TOPICS	READING	DUE
		“controlling for” is not the same as intervening · <i>Lab</i> : Breaking a model on purpose		
Nov 16		No class; instructor at a conference		
Nov 23		Thanksgiving—no class		
Nov 30		Inference when the assumptions fail: prediction intervals versus confidence intervals for a fitted value; the bootstrap and permutation tests—inference for a coefficient, a median, a correlation, or anything else the formulas do not cover · <i>Lab</i> : Bootstrapping a coefficient end to end	<u>Resampling</u>	HW 7
Dec 7		Logistic regression: odds ratios and their intervals, deviance, and why linear regression will not do for a binary outcome · <i>Lab</i> : Logistic regression end to end	<u>Logistic Regression</u>	HW 8
Dec 9 (W)		From question to report: one analysis carried end to end, and what the write-up may claim · <i>Lab</i> : Project clinic (<i>Designated Monday</i>)	<u>From Question to Report</u>	
Dec 10 (R)		Project presentations · (<i>Make-Up Day; start time announced during the term</i>)		Project due

Grading

COMPONENT	WEIGHT
Project	20%
Midterm exam	20%
In-class work (observed)	30%
Homework	20%
Reading questions	10%

Letter Grades

A	A–	B+	B	B–	C+	C	C–	F
≥ 93	90–92	87–89	83–86	80–82	77–79	73–76	70–72	< 70

Independent or Out-of-Class Learning

The average minimum amount of out-of-class or independent learning expected is **5 hours per week**, for our 2.5 hours of classroom meetings. Do not wait until it is too late to get help.

University Policies

Observance of Religious Holidays

Students must notify faculty during the first week of the semester in which they are enrolled in the course, or as early as possible, but no later than three weeks prior to the absence, of their intention to be absent from class on their day(s) of religious

observance. If the holiday falls within the first three weeks of class, the student must inform the faculty in the first week of the semester. For details, see “Religious Holidays” at provost.gwu.edu/policies-procedures-and-guidelines.

Academic Integrity Code

Academic integrity is an essential part of the educational process, and all members of the GW community take these matters very seriously. As the instructor of record for this course, my role is to provide clear expectations and uphold them in all assessments. Violations of academic integrity occur when students fail to cite research sources properly, engage in unauthorized collaboration, falsify data, use resources not permitted for a given assessment, and otherwise violate the Code of Academic Integrity. **The permitted use of AI assistants in this course is defined in the [Use of AI Assistants](#) section above; use outside those bounds is a violation.** If you have any questions about whether particular academic practices or resources are permitted, you should ask me for clarification. If you are reported for an academic integrity violation, you should contact the Office of Student Rights and Responsibilities (SRR) to learn more about your rights and options in the process. Consequences can range from failure of the assignment to expulsion from the university and may include a transcript notation. For more information, see the [SRR website](#), email rights@gwu.edu, or call 202-994-6757.

Use of Electronic Course Materials and Class Recordings

Students are encouraged to use electronic course materials, including recorded class sessions, for private personal use in connection with their academic program of study. Electronic course materials and recorded class sessions should not be shared or used for non-course-related purposes unless express permission has been granted by the instructor. Students who impermissibly share any electronic course materials are subject to discipline under the Student Code of Conduct. Please contact the instructor if you have questions regarding what constitutes permissible or impermissible use of electronic course materials and/or recorded class sessions. Please contact Disability Support Services at disabilitysupport.gwu.edu if you have questions or need assistance in accessing electronic course materials.

Academic Support

Writing Center. GW’s Writing Center cultivates confident writers in the University community by facilitating collaborative, critical, and inclusive conversations at all stages of the writing process. Working alongside peer mentors, writers develop strategies to write independently in academic and public settings. Appointments can be booked at writingcenter.gwu.edu/appointments.

Academic Commons. Academic Commons provides tutoring and other academic support resources to students in many courses. Students can schedule virtual one-on-one appointments or attend virtual drop-in sessions at academiccommons.gwu.edu.

Support for Students Outside the Classroom

Disability Support Services (DSS)—202-994-8250. Any student who may need an accommodation based on the potential impact of a disability should contact Disability Support Services at disabilitysupport.gwu.edu to establish eligibility and to coordinate reasonable accommodations.

Counseling and Psychological Services—202-994-5300. GW’s Colonial Health Center offers counseling and psychological services, supporting mental health and personal development by collaborating directly with students to overcome challenges and difficulties that may interfere with academic, emotional, and personal success. healthcenter.gwu.edu/counseling-and-psychological-services

Safety and Security

- In an emergency: call GUPD at 202-994-6111 or 911
- For situation-specific actions: review the Emergency Response Handbook at safety.gwu.edu/emergency-response-handbook
- In an active violence situation: Get Out, Hide Out, or Take Out. See go.gwu.edu/shooterprep
- Stay informed: safety.gwu.edu/stay-informed

Monday, August 31

Last week you got data into shape. This week you look at it.

That sounds like the easy part, and it is where most of the damage gets done. Exploratory data analysis is where you form the beliefs that everything afterwards is built on: which variables matter, which relationships look real, which observations are strange. If you form those beliefs badly, no amount of careful modeling downstream will save you—you will simply get a very precise answer to the wrong question.

The awkward fact about EDA is that it has no right answer to check yourself against. A regression coefficient is either computed correctly or it is not. A histogram is not wrong; it is a *choice*, and a different choice would have shown you something else. So the skill this week is not “which function makes the plot.” It is judgment: knowing what each summary throws away, noticing when two honest-looking pictures of the same data tell opposite stories, and being able to say out loud what you are claiming and what you are not.

What this week covers

DESCRIBING A SINGLE VARIABLE

Center, spread, quantiles, skew, and outliers. Why the mean and the median disagree and what that disagreement is telling you. Why the variance formula divides by $n - 1$ —stated honestly, not waved at. Histograms and boxplots, and what each of them hides.

RELATIONSHIPS BETWEEN VARIABLES

Two-way tables, scatterplots, covariance and correlation. What a correlation coefficient cannot see. And the difference between association and cause, stated carefully rather than as a slogan you repeat and then ignore.

VISUALIZATION THAT DOES NOT MISLEAD

Truncated axes, aspect ratio, bin widths, dual axes, and aggregation that hides a reversal in the subgroups. The through-line: the same data supports several pictures, and choosing one is an argument you are making.

WHERE THE DATA CAME FROM

One chapter borrowed from [Questions and Data](#), because this week’s lab is built on it. Survivorship and selection bias, the three ways a sample goes wrong, and missing data as a mechanism rather than a gap to be patched. Every summary in the chapters above describes the rows you were given; this one asks which rows you were not.

ALONGSIDE THIS WEEK: PLOTTING WITH GGPLOT2

The tool you will actually be asked to use: data, an aesthetic mapping, a geom, and layers that add. It lives in the self-paced [R Primer](#) rather than in this week’s reading, because it is the mechanics of a package rather than an idea about data. Work through it whenever you need it—the labs assume it from Aug 31 onward.

PRACTICE PROBLEMS

Predict-before-you-run, and code that runs perfectly and is still wrong. Not graded—bring your answers and your disagreements to class.

Where this is heading

This is the last week before the inference block, and it is deliberately placed there. Everything you compute this week is computed *on a sample*.

When you write `mean(salary)` you get one number, and the machine prints it to four decimal places with total confidence. But that number is a property of the 240 people who happened to end up in your file, not of the population you actually care about.

Draw a different 240 people and you get a different number. Nothing in this week's toolkit tells you how different.

Hold on to that. From [Probability](#) onward, the entire question is: **given that this sample is one draw out of many possible draws, what am I entitled to say about the thing it was drawn from?** This week's job is to make you fluent at describing the draw in front of you—and slightly uncomfortable about how easily that description passes itself off as a fact about the world.

DUE THIS WEEK

No homework is due this week. Post your questions on these pages by Monday before—Monday's session is built out of them.

A single column of numbers can contain thousands of values, and you cannot hold thousands of values in your head. So you replace them with a handful of summaries. Every summary is a deliberate act of forgetting: you throw away almost everything in exchange for something you can actually think about.

The question is never “what is the summary of this variable.” It is **what did I just throw away, and did it matter?**

We will work with a made-up but realistic variable throughout: the annual salaries of 240 alumni, five years after graduation. Run this cell, and every cell after it in order—each one builds on the last.

R Code	
1	<code>set.seed(6101)</code>
2	<code>salary <- round(rlnorm(240, meanlog = 11, sdlog = 0.55))</code>
3	<code>head(salary, 12)</code>
4	<code>length(salary)</code>
<pre>[1] 47036 41998 111617 45725 33962 36473 51090 39327 111823 34392 [11] 40142 76164 [1] 240</pre>	

`set.seed()` fixes R’s random number generator, so this “random” data is exactly the same for you as it is for me and for the person next to you. Every figure in these notes is reproducible on your machine. If you get a different number from the one in the text, something real has gone wrong and you should say so.

Center

Two summaries claim to be the center of a distribution, and it is worth being precise about what each one actually is.

The **mean** is the balance point:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

If you put the data on a see-saw with equal weights at each value, the mean is where you put the fulcrum. That is a physical fact about it, and it explains everything that follows: a value far out on one side has to be balanced by a lot of mass on the other, so a single extreme value moves the fulcrum.

The **median** is the middle by rank: sort the values, take the one in the middle (or average the two middle ones). It does not care how far away the extremes are, only that they are on one side or the other.

R Code	
1	<code>mean(salary)</code>
2	<code>median(salary)</code>
<pre>[1] 70583.4 [1] 59225.5</pre>	

The mean is about \$70,600. The median is about \$59,200. They differ by roughly \$11,000, which is not a rounding error—it is nearly a fifth of the median. Both numbers are correctly computed. They are answers to different questions.

What the disagreement tells you

THE SINGLE MOST USEFUL DIAGNOSTIC IN THIS CHAPTER

If $\text{mean} > \text{median}$, the distribution has a longer tail to the **right**. If $\text{mean} < \text{median}$, the longer tail is to the **left**. The bigger the gap relative to the spread, the more skewed the variable.

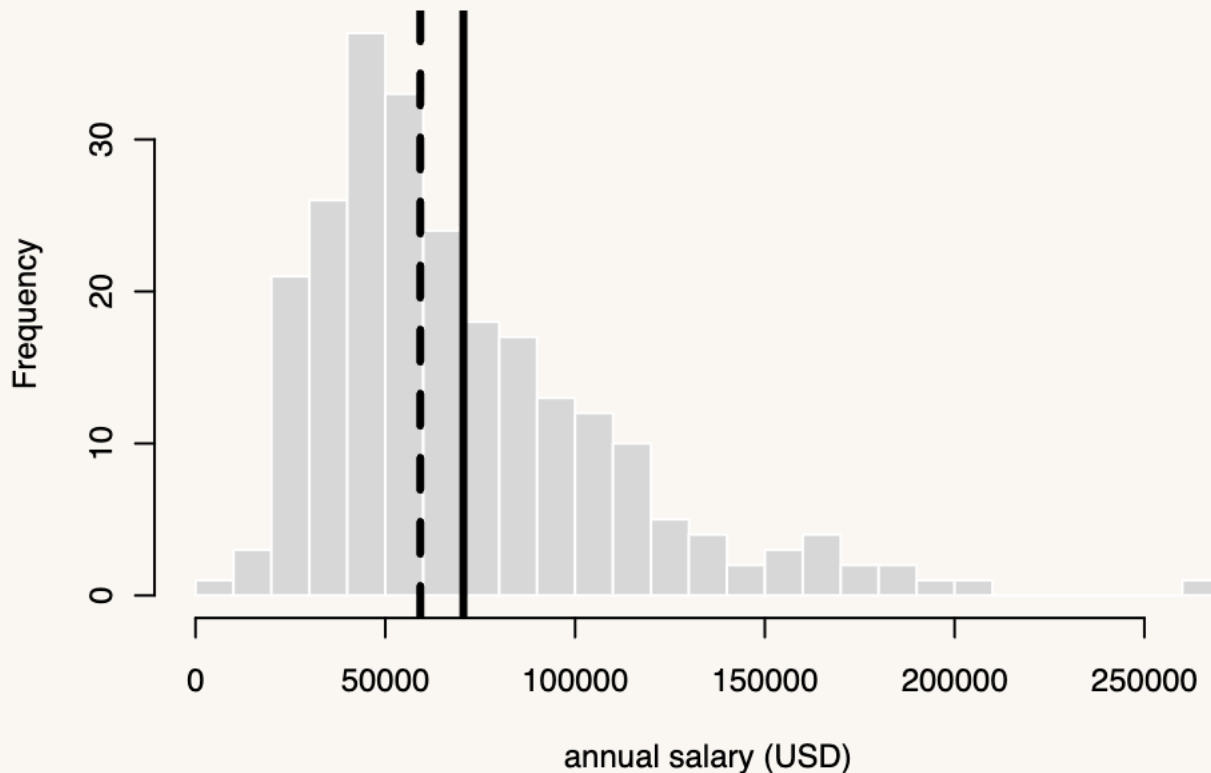
You can diagnose the shape of a distribution before you ever plot it, just by computing two numbers and comparing them.

With one condition, and it matters. The rule reads the gap as a *tail*, and that reading is only available if the distribution has a single peak. What the gap actually tells you is that the mean and the median disagree. It does not tell you *why* they disagree. Skew is one reason. Two populations mixed together is another, and it produces the same gap while having no tail at all. You will meet exactly that case [later in this chapter](#), where $\text{mean} < \text{median}$ and there is nothing stretched to the left anywhere in the data.

Here it is in the picture. The solid line is the mean, the dashed line the median:

R Code

```
1 hist(salary, breaks = 30, col = "gray85", border = "white",
2      main = "", xlab = "annual salary (USD)")
3 abline(v = mean(salary), lwd = 3)
4 abline(v = median(salary), lwd = 3, lty = 2)
```



The bulk of the alumni sit between \$30,000 and \$90,000. A thin tail of high earners stretches out to nearly \$270,000. Those few people cannot move the median—it only matters that they are above it, not how far above. But they pull the fulcrum to the right, and so the mean lands in a region where hardly anyone actually is.

Notice what that implies. Ask how many alumni earn less than the average:

R Code

```

1 sum(salary < mean(salary))
2 mean(salary < mean(salary))

[1] 145
[1] 0.6041667

```

Sixty percent of them are “below average”. This is not a paradox and it is not a trick; it is what a right-skewed variable does. Any time you see a headline about average income, average house price, average time-to-hire, or average customer spend, the number being quoted is one that a clear majority of the population falls below.

MEAN, MEDIAN, AND SHAPE



A city reports that the **mean** household income is \$88,000 and the **median** household income is \$61,000. Nothing was computed incorrectly.

Which statement is best supported by these two numbers alone?

- ☐ The income distribution has a long right tail, and more than half of households earn below \$88,000.
- ☐ The income distribution has a long left tail, and more than half of households earn above \$88,000.
- ☐ The data contain errors, because the mean and the median of the same variable should agree.
- ☐ Roughly half of households earn below \$88,000, since the mean and the median both estimate the center.



Which one should you report?

Neither answer is automatic, and “use the median because it is robust” is a bad habit dressed up as a principle. Ask what the number is *for*.

- **The university wants to budget financial aid based on alumni giving capacity.** The mean. Giving comes out of total dollars, and total dollars is $n \times \bar{x}$. The high earners genuinely are most of the money.
- **A prospective student asks “what will I earn?”** The median. They are asking about a typical outcome for a person like them, and the mean describes an outcome almost nobody has.

The honest move, when you have room, is to report both and let the gap between them do some work. `summary()` gives you both plus the quartiles:

```

R Code
1 summary(salary)

  Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
 8747  42570   59226   70583   90007  269815

```

Spread

Two variables can share a center and be nothing alike. Spread is the second thing you need.

The obvious idea—average how far each point is from the mean—fails immediately, because the deviations sum to exactly zero by construction. That is what “balance point” means. So we square them first:

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

This is the **variance**. Its units are the square of the original units, which for salary means “squared dollars”, a quantity nobody has an intuition for. So we take the square root and get the **standard deviation** s , back in dollars.

R Code	
1	<code>var(salary)</code>
2	<code>sd(salary)</code>

```
[1] 1631780458
[1] 40395.3
```

An SD of about \$40,400 against a median of \$59,200 is large—the typical deviation is more than two-thirds of the typical value. That ratio is itself informative, and computing it is a habit worth forming.

Why keep the variance at all

If the SD is the readable one, why carry the variance around? Why not square-root immediately and never speak of squared dollars again?

Because **variances add and standard deviations do not**. Combine two independent sources of variation and the variances sum; the SDs do not, and there is no simple rule that makes them. That single fact is why the variance survives:

$$\text{SD} = \sqrt{s_1^2 + s_2^2} \quad \text{not} \quad s_1 + s_2$$

Every formula in the rest of this course is additive in the variance and awkward in the SD. The standard error of a mean divides a variance by n . Regression splits the variance of y into a part the model explains and a part it does not, and R^2 is the ratio of those two pieces. None of those decompositions work on standard deviations.

So the division of labour is: **the variance is what the mathematics adds up, and the SD is what you report to a human**. Compute in one, communicate in the other. You will do this for the rest of the course.

Why $n - 1$, honestly

This is the most-asked question in the course, and it deserves a real answer rather than “because that’s the formula”.

Start with the question that comes *before* it, because the correction makes no sense until that one is settled: **whose** variance are you computing? If all you want is to describe the 240 numbers in front of you—these alumni, this file, no claim about anybody else—then there is no bias to fix and $n - 1$ is unjustified. Divide by n . That quantity has a name, the **population variance**, and it is the correct and complete answer to “how spread out is this set of numbers?”

You are almost never asking that. You are asking about a population you did not measure, on the evidence of a sample you did: what do GW alumni earn, not what did these particular 240 earn. The sample is a means, not the subject. `var()` divides by $n - 1$ because it assumes you are asking the second question—and the second question is the only one of the two that has a bias to correct.

Here is the situation. There is a true population standard deviation σ that you do not know. You have a sample, and you want to estimate σ . The natural thing is to measure how spread out the sample is around **its own mean** $\bar{x}$ —because you do not know the true mean μ either.

And that is the problem. $\bar{x}$ is computed *from this very sample*. It is, by construction, the point that makes $\sum (x_i - \bar{x})^2$ as small as it can possibly be—smaller than $\sum (x_i - \mu)^2$ would be for the true μ , for any sample where $\bar{x} \neq \mu$, which is essentially every sample. Squared distances measured from the sample’s own mean are systematically too small.

Dividing by $n - 1$ instead of n inflates the result by exactly the right amount to cancel that bias out. Do not take my word for it. This simulation draws 20,000 samples of size 5 from a population with a true variance of exactly 1, and computes both versions on each:

R Code	
1	<code>set.seed(6101)</code>
2	<code>n <- 5</code>
3	<code>est <- replicate(20000, {</code>

```

4 | s <- rnorm(n, mean = 0, sd = 1) # true variance is 1
5 | c(divide_by_n = mean((s - mean(s))^2),
6 |   divide_by_n1 = var(s),
7 |   sd_estimate = sd(s))
8 | })
9 | round(rowMeans(est), 4)

```

```

divide_by_n divide_by_n1 sd_estimate
0.8074      1.0093      0.9440

```

Dividing by n gives about **0.807** on average. It is not noisy—it is *wrong*, and it is wrong in the same direction every time. Dividing by $n - 1$ gives about **1.009**, which is 1 up to simulation noise. With $n = 5$ the correction is large, because $\bar{x}$ is doing a lot of the fitting; with $n = 5000$ the two would be indistinguishable, which is why nobody argues about it on big data.

THE PART THAT IS USUALLY LEFT OUT

Look at the third number: the average of `sd(s)` is about **0.944**, not 1.

So `var()` is an unbiased estimate of σ^2 , but `sd()` is *not* an unbiased estimate of σ . It is biased low. The $n - 1$ fixes the variance, and then the square root breaks it again—squaring and averaging do not commute with taking roots.

We use `sd()` anyway, because it is in the right units, the bias shrinks fast with n , and nothing better is simple. But if someone tells you $n - 1$ makes “the standard deviation unbiased”, they are repeating something they were told. It makes the *variance* unbiased. Being able to state exactly what a correction does, and what it does not, is the kind of precision this course is trying to build.

The “ $n - 1$ ” is called the **degrees of freedom**. The intuition: once you know $\bar{x}$ and any $n - 1$ of the observations, the last one is determined. Only $n - 1$ of the deviations are free to vary. You will meet the same count again in t -tests and in regression, and it will mean the same thing every time: observations minus the number of quantities you estimated from the data.

WHAT $n - 1$ BUYS YOU

You draw a sample of size n and compute $\frac{1}{n} \sum_i (x_i - \bar{x})^2$ instead of dividing by $n - 1$.

What exactly goes wrong?

- ☐ Nothing systematic—it is a different but equally good estimate, just noisier.
- ☐ It underestimates the population variance on average, because the deviations are measured from the sample’s own mean rather than the unknown true mean.
- ☐ It overestimates the population variance on average, because squaring exaggerates large deviations.
- ☐ It gives the wrong answer only when the data are skewed; for symmetric data the two divisors agree.



Exercise: the correction, by hand

Exercise

```

1 | # The vector `s` holds five values. R's var(s) is 16.
2 | # Compute the OTHER version: the average squared deviation from the mean,
3 | # dividing by n rather than n - 1. Write it out; do not use var().
4 | -----

```

Please replace _____ with valid code.

Exercise: below average

Exercise

```
1 # `salary` holds the 240 alumni salaries.
2 # Return the PROPORTION of alumni who earn less than the mean salary
3 # (a single number between 0 and 1).
4 _____
```

Please replace _____ with valid code.

Quantiles

The median generalizes. The *p*-th **quantile** is the value below which proportion *p* of the data falls. The median is the 0.5 quantile; the **quartiles** are at 0.25 and 0.75.

R Code

```
1 quantile(salary, probs = c(0.05, 0.25, 0.5, 0.75, 0.95))
```

```
      5%      25%      50%      75%      95%
26181.90 42570.00 59225.50 90007.25 156691.10
```

Read that out loud: the bottom 5% of these alumni earn under about \$26,200, the middle half earn between roughly \$42,600 and \$90,000, and the top 5% earn more than about \$156,700. That is a far richer description than “the mean is \$70,600”, and it is barely more work.

The **interquartile range** is the width of that middle half:

R Code

```
1 IQR(salary)
```

```
[1] 47437.25
```

The IQR is a spread measure like the SD, but built from ranks, so extreme values cannot inflate it. Comparing the two is a quick tail check. For a Normal distribution the IQR is about $1.35 \times$ the SD; here:

R Code

```
1 IQR(salary) / sd(salary)
```

```
[1] 1.174326
```

It comes out around 1.17, comfortably below 1.35. The SD is larger than the middle half of the data can account for, which is the arithmetic signature of a heavy tail.

THE IQR DOES THREE DIFFERENT JOBS. KEEP THEM APART.

The same quantity turns up in three roles in this chapter, and they answer three different questions:

1. **The IQR alone** says *where the middle half sits*. It is a description of the bulk, and it is the width of the box in a boxplot.
2. **The IQR compared to the SD** says *how heavy the tails are*. The ratio you just computed is a shape diagnostic—nothing to do with any individual point.
3. **1.5 IQR beyond a quartile** flags *individual points* as far away. That is the next section, and it is the only one of the three about outliers.

Reaching for the IQR does not mean you are hunting outliers. Two of its three jobs never look at a single observation.

Skew

Skew is the asymmetry of the distribution, and you now have three ways to detect it without any new machinery:

1. **mean** versus **median**. Mean above median means a right tail.
2. The distance from the median up to Q3 versus down to Q1. If the upper gap is bigger, the right side is stretched.
3. The histogram, which shows you directly.

R Code

```
1 q <- quantile(salary, c(0.25, 0.5, 0.75))
2 c(lower_gap = q[[2]] - q[[1]], upper_gap = q[[3]] - q[[2]])
```

```
lower_gap upper_gap
16655.50  30781.75
```

The upper gap is over 80% larger than the lower one. Consistent with the mean sitting above the median, and consistent with what the histogram showed. Three independent signs pointing the same way is what confidence in a description should look like.

When two of the three agree and are both wrong

Read that last sentence carefully, because the word doing the work is **independent**, and the first two signs are not.

`quakes$depth` records the depth of a thousand earthquakes near Fiji. Run the same two checks on it, and put the answers beside the salary data:

R Code

```
1 d <- quakes$depth
2 qd <- quantile(d, c(0.25, 0.5, 0.75))
3
4 c(gap_in_sd = (mean(d) - median(d)) / sd(d),
5   gap_ratio = (qd[[3]] - qd[[2]]) / (qd[[2]] - qd[[1]]))
```

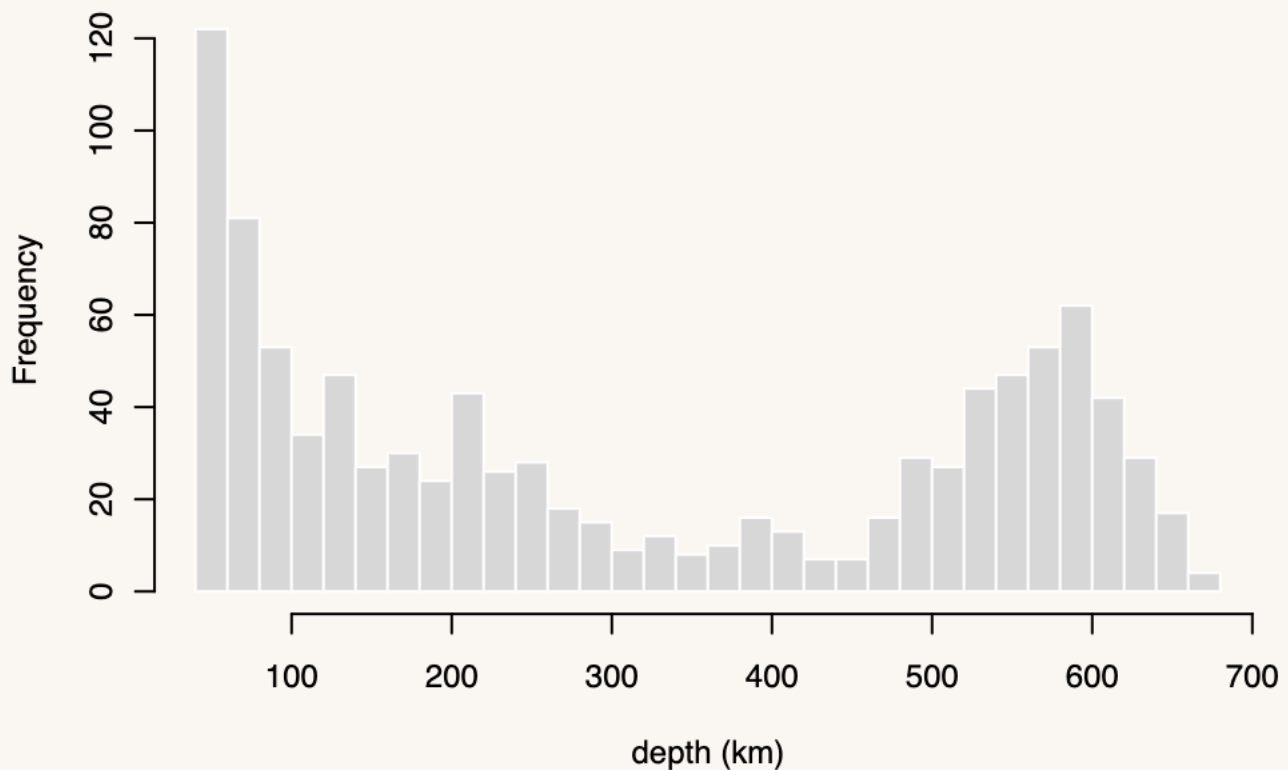
```
gap_in_sd gap_ratio
0.2986561 2.0000000
```

	MEAN – MEDIAN	UPPER GAP ÷ LOWER
<code>salary</code> —a clear right tail	0.28 sd	1.85
<code>depth</code>	0.30 sd	2.00

Both checks fire, and both fire *harder* on `depth` than on the variable you just used to learn what a right tail looks like. Every number is correct. Now draw it:

R Code

```
1 hist(quakes$depth, breaks = 30, col = "gray85", border = "white",
2       main = "", xlab = "depth (km)")
```



There is no right tail. There are two populations—shallow events near the trench, deep ones down the subduction zone—and about six percent of the data anywhere near the mean.

WHY THE TWO NUMERIC SIGNS FAILED TOGETHER

They are not two independent tests. Both are measuring **one** thing: that mass sits further from the middle on one side than the other. That is true here, and it is not a tail. A tail is a thin stretch of rare values; this is a second hump of four hundred events.

Neither number can tell those apart, and no third number of that kind would have, because the information they need is not in any of them. **The histogram is not a fourth opinion. It is the only one of the three that can see shape at all,** which is why the other two are checks and it is the evidence.

Right-skewed variables are extremely common in practice, and they tend to be the same kinds of variables: money, waiting times, city populations, file sizes, counts of anything where a few cases dominate. If your variable is one of those, expect skew and check for it rather than being surprised by it.

Outliers

Now the part that gets handled badly more often than anything else in this chapter.

A common rule of thumb flags a point as an outlier if it lies more than $1.5 \times \text{IQR}$ beyond the nearer quartile. It is what `boxplot()` draws as separate dots.

R Code

```
1 upper_fence <- quantile(salary, 0.75) + 1.5 * IQR(salary)
```

```

2 | upper_fence
3 | sum(salary > upper_fence)

      75%
161163.1
[1] 11

```

Eleven alumni are flagged. And now a question that only *looks* technical:

AN OUTLIER IS A QUESTION, NOT A VERDICT

The 1.5 IQR rule does not detect errors. It detects **distance**. It has no access to whether a value is possible, and no opinion about whether it is interesting. All it reports is that a point is far from the middle.

Deleting flagged points because they are flagged is not cleaning your data. It is deciding, before you have looked, that your variable has no tail—and for salary, waiting time, insurance claims, or network traffic, the tail is very often the entire reason anyone cares.

So what should you do? Go and look at the point, and work out which of these it is:

- **An error.** A blood pressure of 999, a height of 2.4 meters in a sample of eight-year-olds, an age of -1 . These are impossible, they came from a broken sensor or a sentinel value nobody documented, and you should fix or remove them—and say in the write-up that you did.
- **A different population.** The “customer” who is actually your own test account. The “household” that is a nursing home. Real data, wrong table. Removing them can be right, but only if you state the rule you used, and the rule must be about *what the row is*, not about how large the number is.
- **A real, rare value.** The alumnus who founded a company. The genuinely hundred-year-old respondent. These are your data. They are frequently the most informative rows you have.

The one thing you may not do is delete a point *because it was inconvenient for your result* and then not mention it. That is not a judgment call; that is falsifying the record.

Watch how much the choice matters here:

```

R Code

1 | kept <- salary[salary <= upper_fence]
2 | rbind(mean = c(all = mean(salary), trimmed = mean(kept)),
3 |       median = c(all = median(salary), trimmed = median(kept)))

      all trimmed
mean  70583.4 65023.9
median 59225.5 57587.0

```

Dropping eleven of 240 rows—under 5% of the file—takes about \$5,600 off the mean, close to 8% of it. The median moves by about \$1,600, under 3%. That contrast is the practical argument for reporting the median alongside the mean whenever there is a tail: the gap between how much each one moves tells your reader how much of your headline number was resting on a handful of rows.

Exercise: counting the flagged points

```

Exercise

1 | # Count how many salaries lie BELOW the lower fence,
2 | # that is, below Q1 - 1.5 * IQR.
3 | -----

```

Please replace _____ with valid code.

Exercise: what the flag licenses you to do

A hospital records length of stay, in days, for 900 patients. The 1.5 IQR rule flags 34 of them as high outliers. Following up on all 34, the analyst finds:

- 3 have a recorded stay of 999 days (the system's "unknown" code)
- 6 are patients transferred in from long-term care
- 25 are ordinary patients who were genuinely very sick

Exercise

```
1 # Which single course of action is defensible? Assign the letter,
2 # for example: answer <- "A"
3 #
4 # A. Drop all 34 flagged rows; they are outliers by the standard rule.
5 # B. Drop the 3 coded as 999. Handle the 6 transfers by a stated rule about
6 # admission source, reported in the write-up. Keep the 25.
7 # C. Keep all 34, since removing any data point is manipulation.
8 # D. Keep all 34, but report the median instead of the mean so the tail
9 # stops mattering.
10 answer <- _____
```

Please replace _____ with valid code.

Histograms and boxplots, and what each hides

You now have two pictures of one variable. They fail in opposite directions, and knowing how is most of the value in owning both.

Here is a second variable: the one-way commute times, in minutes, of 400 employees at a company with an office downtown and a second site out by the airport.

R Code

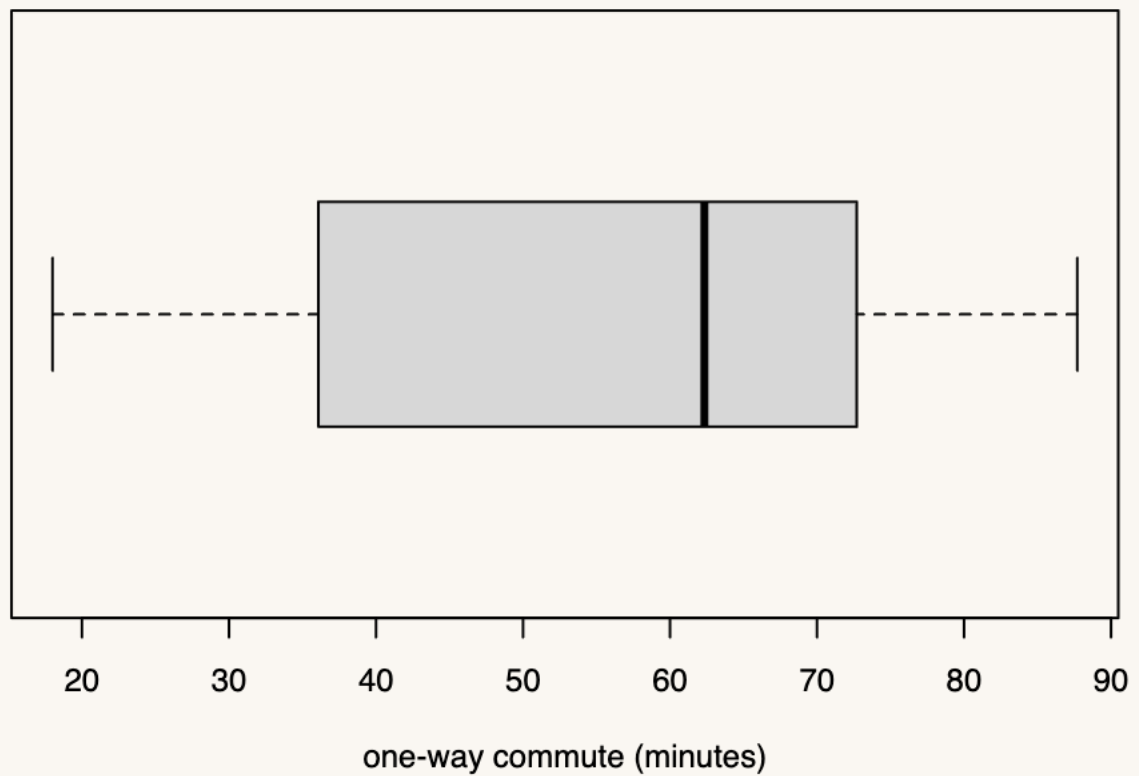
```
1 set.seed(6101)
2 grp <- sample(c(0, 1), 400, replace = TRUE)
3 commute <- rnorm(400, mean = ifelse(grp == 1, 72, 34), sd = 6)
4 summary(commute)
5 sd(commute)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
[1] 18.00  36.10   62.33   54.32  72.70   87.70
[1] 19.52988
```

Nothing alarming. The median is about 62 minutes, the mean about 54, the SD about 19.5. Now draw the boxplot:

R Code

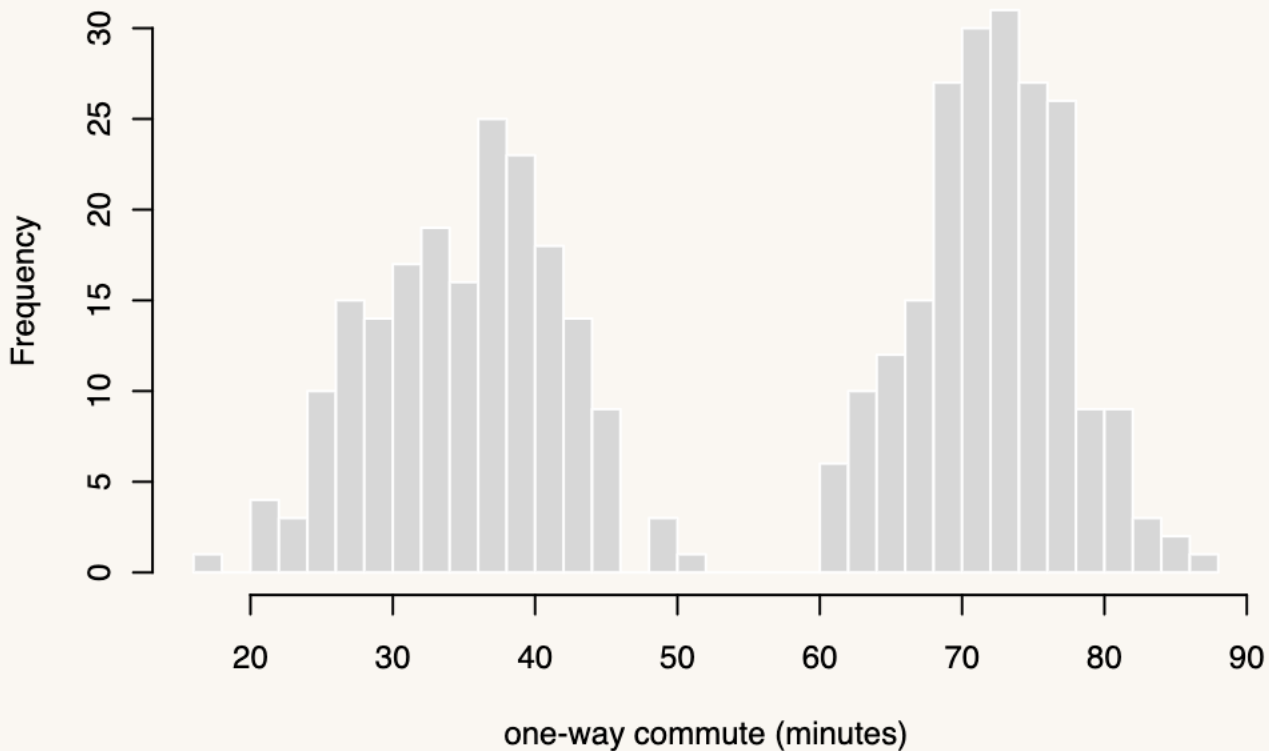
```
1 boxplot(commute, horizontal = TRUE, col = "gray85",
2         xlab = "one-way commute (minutes)")
```



A wide, slightly left-leaning box. No flagged outliers. A perfectly ordinary picture, and you would move on. Now the histogram of the *same 400 numbers*:

R Code

```
1 hist(commute, breaks = 30, col = "gray85", border = "white",  
2       main = "", xlab = "one-way commute (minutes)")
```



There are two groups of employees and almost nobody in between. The median, at 62 minutes, sits in the sparse valley—it describes a commute that essentially no employee has. Every summary statistic on the page was correct, and every one of them was describing a distribution that does not exist.

Now go back and apply [the mean-median rule](#) to these numbers. The mean is 54 and the median is 62, so $\text{mean} < \text{median}$, and the rule says *longer tail to the left*. Look at the histogram: there is no left tail. There is no tail at all. The rule has not malfunctioned—it was answering the only question it can answer, which is whether the mean and median disagree. They do, emphatically, and the reason is that one of the two groups is pulling the balance point down while the rank-based median stays stranded in the gap between them. **The gap is a signal that something is asymmetric about the distribution. Naming that something as skew is a second step, and it needs the picture.**

WHAT EACH ONE HIDES

A boxplot hides shape. It is built from five numbers, all of them ranks. It cannot see multiple modes, gaps, clumping at round numbers, or a spike at zero. Two wildly different distributions can give identical boxplots.

A histogram hides the tails, and depends on its bins. Rare values may not be visible at all when the bar is one pixel tall—`hist()` will not tell you that one person commutes for four hours. And the bin width is a choice you make that changes the shape you see, which is [Visualization That Does Not Mislead](#)'s problem.

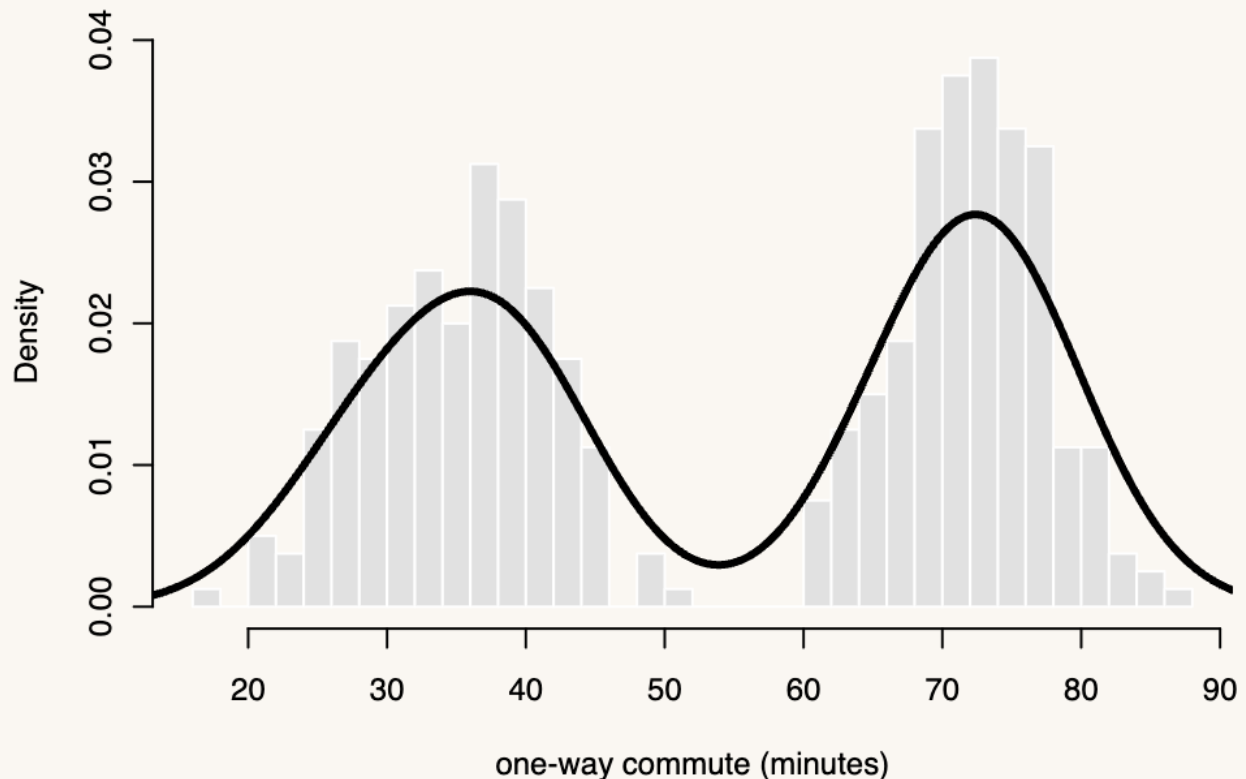
The practical rule: **use the histogram to learn the shape, the boxplot to compare groups.** A boxplot's real strength is that you can line up eight of them side by side and read them at a glance—but only once you have checked, one variable at a time, that there is no bimodality for it to conceal.

A THIRD OPTION WORTH KNOWING

`density()` draws a smoothed version of the histogram, with no bins to argue about—though it substitutes a *bandwidth* to argue about instead. Overlaying it on the histogram is a cheap way to check that the shape you think you see is not an artifact of where the bin edges happened to land.

R Code

```
1 hist(commute, breaks = 30, freq = FALSE, col = "gray90", border = "white",
2     main = "", xlab = "one-way commute (minutes)")
3 lines(density(commute), lwd = 3)
```



A note on tools

Everything in these notes uses base R graphics: `hist`, `boxplot`, `plot`, `barplot`. That is a deliberate constraint, so that every cell runs in your browser with nothing installed.

In your own work, and in your projects, you should be using **ggplot2**. It is the standard in the R world, it scales to complicated figures far better than base graphics, and its grammar—map variables to aesthetics, add layers—will make you think about figures more clearly. A grouped histogram that takes twenty fiddly lines in base R is one `facet_wrap()` in ggplot2. Learn it; the concepts in this week transfer directly, only the syntax changes.

[Plotting with ggplot2](#) is the working minimum, and it is a chapter of this part rather than an appendix.

The first thing to run on a new dataset

Before any of the above, you want a single view of every column at once: what type it is, how much of it is missing, and roughly what shape it has. Doing that by hand is a dozen calls; doing it column by column is how you spend an hour learning something you could have known in ten seconds.

R Code

```
1 set.seed(11)
2 n <- 200
3 staff <- data.frame(
4   id      = 1:n,
5   dept    = sample(c("eng", "ops", "sales"), n, replace = TRUE),
6   salary  = round(rlnorm(n, 11.2, 0.35)),
7   years   = sample(0:30, n, replace = TRUE)
8 )
9 staff$salary[sample(n, 24)] <- NA
10 staff$rating <- NA_real_      # a column somebody added and never filled
11
12 profile <- function(df) {
13   data.frame(
14     type      = sapply(df, function(x) class(x)[1]),
15     missing   = sapply(df, function(x) sum(is.na(x))),
16     unique    = sapply(df, function(x) length(unique(x[!is.na(x)]))),
17     min       = sapply(df, function(x) if (is.numeric(x)) min(x, na.rm = TRUE) else NA),
18     median    = sapply(df, function(x) if (is.numeric(x)) median(x, na.rm = TRUE) else NA),
19     max       = sapply(df, function(x) if (is.numeric(x)) max(x, na.rm = TRUE) else NA)
20   )
21 }
22 profile(staff)
```

R Warning in `min(x, na.rm = TRUE)`

Warning: no non-missing arguments to min; returning Inf

R Warning in `max(x, na.rm = TRUE)`

Warning: no non-missing arguments to max; returning -Inf

	type	missing	unique	min	median	max
id	integer	0	200	1	100.5	200
dept	character	0	3	NA	NA	NA
salary	numeric	24	176	39133	72844.5	198351
years	integer	0	31	0	14.0	30
rating	numeric	200	0	Inf	NA	-Inf

Three things are visible in one glance and none of them are visible in `head(staff)`: `salary` is 12% missing, `rating` is entirely empty and should be dropped rather than analyzed, and `id` has as many unique values as there are rows, which is how you confirm it really is an identifier and not a variable.

You do not have to write that function. `skimr::skim(df)` does the same job properly, with miniature histograms, and is one line. It is not available in the browser, so install it locally—it is worth being the first thing you type against any new file.

Summary

- Every summary is a decision about what to forget. Know what you forgot.
- The mean is the balance point and moves with extreme values; the median is the middle by rank and does not. Their **gap tells you the two disagree** before you plot anything—which is skew if the variable has a single peak, and may be two populations if it does not. The gap raises the question; the picture answers it.

- Which to report depends on the question: totals and budgets want the mean, typical outcomes want the median. Reporting both is usually right.
- The variance divides by $n - 1$ because the sample mean was fitted to this very sample, making deviations systematically too small. That makes `var()` unbiased—and `sd()` still slightly biased low.
- Quantiles and the IQR describe spread by rank; an SD much larger than the IQR signals heavy tails.
- The 1.5 IQR rule flags **distance**, not error. Every flagged point is a question to investigate, and whatever you do with it goes in the write-up.
- Boxplots hide shape. Histograms hide the tails and depend on the bins. Use both.

Next: what happens when you have two variables and want to know whether they have anything to do with each other.

None of these are graded and none of them are auto-checked. They are here so you arrive on Monday with something to argue about.

Work them in order. Several come in two parts: **predict**, then **run**. The prediction is the part that teaches you something—if you skip straight to the Run button you will nod along at the output and learn nothing, because output you did not expect and output you did expect look identical after the fact.

Write your prediction down. On paper, in a comment, in a note on your phone. Then run it.

WHAT TO BRING TO CLASS

For each problem, bring **an answer you are prepared to be wrong about**. The in-class discussion is graded on whether you said something substantive and could defend it, not on whether it was correct. A confidently wrong answer you can explain is worth more here than a silence that was technically not incorrect.

Predict before you run

A1. Two vectors, one summary

Predict all four numbers before running. Then run.

R Code				
1	a <- c(10, 20, 30, 40, 50)			
2	b <- c(10, 20, 30, 40, 500)			
3				
4	c(mean_a = mean(a), mean_b = mean(b),			
5	med_a = median(a), med_b = median(b))			
	mean_a	mean_b	med_a	med_b
	30	120	30	30

Then answer: one summary changed by a factor of roughly two and the other did not change at all. Explain *why* in terms of what each summary is—balance point versus rank—rather than by describing what happened.

DISCUSSION



A2. Which one moves

Before running: a and b from A1 again. Which of sd and IQR will change more when 50 becomes 500, and by roughly how much?

R Code				
1	c(sd_a = sd(a), sd_b = sd(b),			
2	iqr_a = IQR(a), iqr_b = IQR(b))			
	sd_a	sd_b	iqr_a	iqr_b
	15.81139	212.72047	20.00000	20.00000

Then answer: the IQR did not move at all. Is that a point in its favor, or against it? Give a situation where each answer is the right one.

DISCUSSION



A3. Adding a constant

Predict every one of these six numbers before running.

R Code

```
1 set.seed(6101)
2 z <- rnorm(50, mean = 100, sd = 15)
3
4 c(mean_z = mean(z), sd_z = sd(z),
5   mean_z10 = mean(z + 10), sd_z10 = sd(z + 10),
6   mean_2z = mean(2 * z), sd_2z = sd(2 * z))
```

```
mean_z sd_z mean_z10 sd_z10 mean_2z sd_2z
96.05849 15.47873 106.05849 15.47873 192.11698 30.95747
```

Then answer: state the general rules for what happens to the mean and to the SD under $x \mapsto x + c$ and $x \mapsto cx$. Then explain why the SD behaves differently from the mean under the first transformation but not the second.

DISCUSSION

>

A4. A correlation you can predict

R Code

```
1 set.seed(6101)
2 p <- rnorm(60)
3 q <- -3 * p + 7
4
5 c(cor_pq = cor(p, q), cor_qp = cor(q, p))
```

```
cor_pq cor_qp
-1 -1
```

Predict both numbers exactly—not approximately—before running, and be ready to say why the answer does not depend on the -3 or the 7 .

DISCUSSION

>

A5. Same r , different data

Predict which of these three pairs has the largest correlation. Then run, then plot each one.

R Code

```
1 set.seed(6101)
2 u1 <- runif(60, 0, 10); v1 <- u1 + rnorm(60, 0, 0.5)
3 u2 <- runif(60, 0, 10); v2 <- 40 * u2 + rnorm(60, 0, 20)
4 u3 <- runif(60, 0, 10); v3 <- exp(u3 / 2) + rnorm(60, 0, 1)
5
6 round(c(pair1 = cor(u1, v1), pair2 = cor(u2, v2), pair3 = cor(u3, v3)), 3)
```

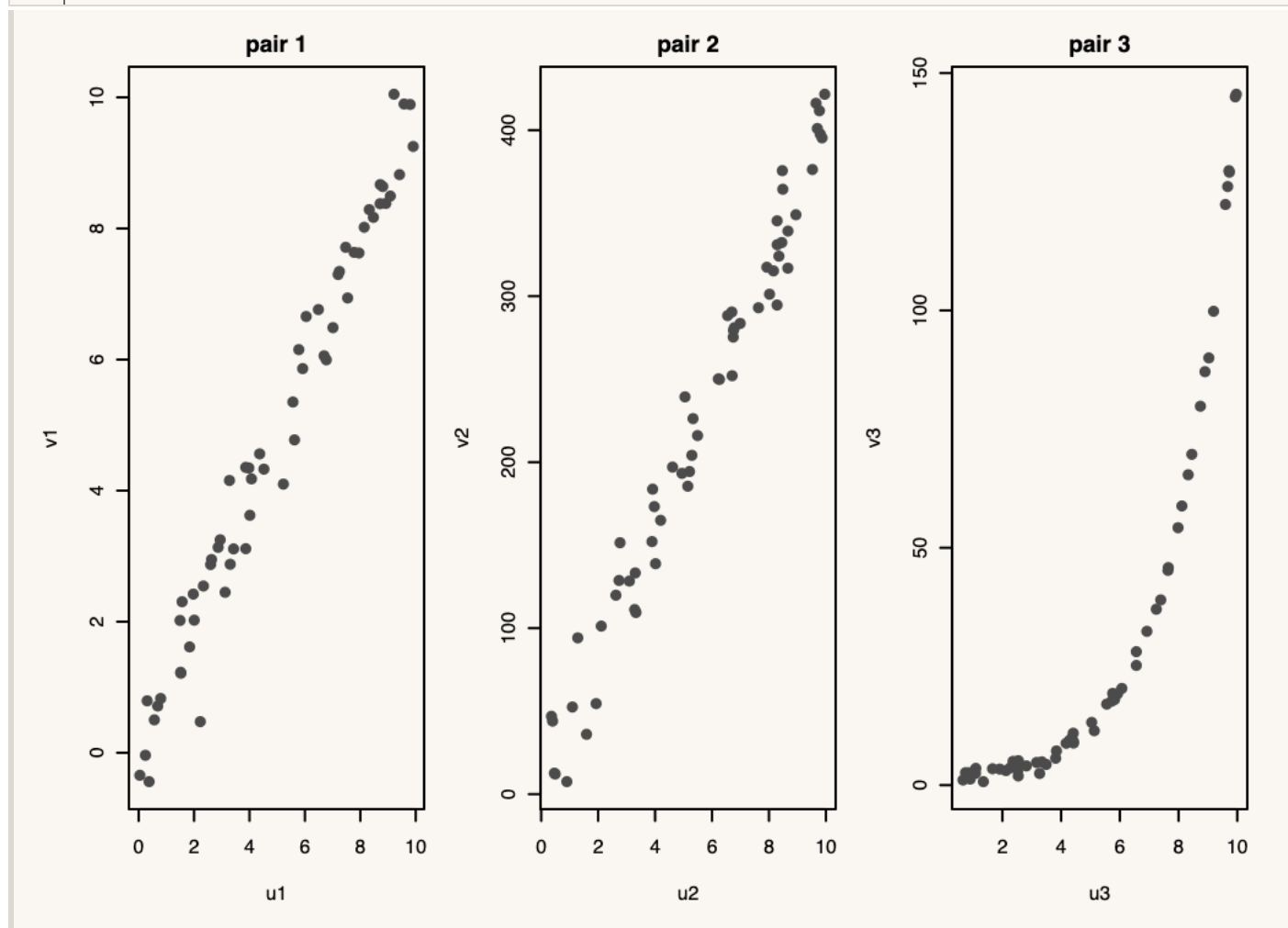
```
pair1 pair2 pair3
0.986 0.987 0.872
```

R Code

```

1 par(mfrow = c(1, 3), mar = c(4, 4, 2, 1))
2 plot(u1, v1, pch = 19, col = "gray30", main = "pair 1")
3 plot(u2, v2, pch = 19, col = "gray30", main = "pair 2")
4 plot(u3, v3, pch = 19, col = "gray30", main = "pair 3")

```



Then answer: pair 3 has a very high correlation. Would you be comfortable reporting “ v increases by about k for every unit of u ” for it? What does the plot tell you that the coefficient does not?

DISCUSSION



This runs, and it is still wrong

Every cell below executes without an error or a warning. Each one produces a result that is wrong, misleading, or answers a different question from the one asked. Find the problem, then fix it.

This is the harder skill and the more important one. Code that crashes tells you it is broken. Code like this does not.

B1. The average rating

Task: a product has ratings from two stores. Store 1 sold 900 units with an average rating of 4.6. Store 2 sold 20 units with an average rating of 2.0. Report the overall average rating.

R Code

```
1 store_means <- c(4.6, 2.0)
2 mean(store_means)
```

```
[1] 3.3
```

NOTE



B2. The typical house price

Task: report the typical house price in a neighborhood, for a family deciding whether they can afford to live there.

R Code

```
1 set.seed(6101)
2 price <- round(rlnorm(150, meanlog = 12.6, sdlog = 0.7))
3 mean(price)
```

```
[1] 367451.3
```

The code is correct R, the number is correctly computed, and the answer will mislead the family. Say precisely how. Then produce a better answer—more than one number is allowed—and justify the choice by reference to what the family is going to do with it.

DISCUSSION

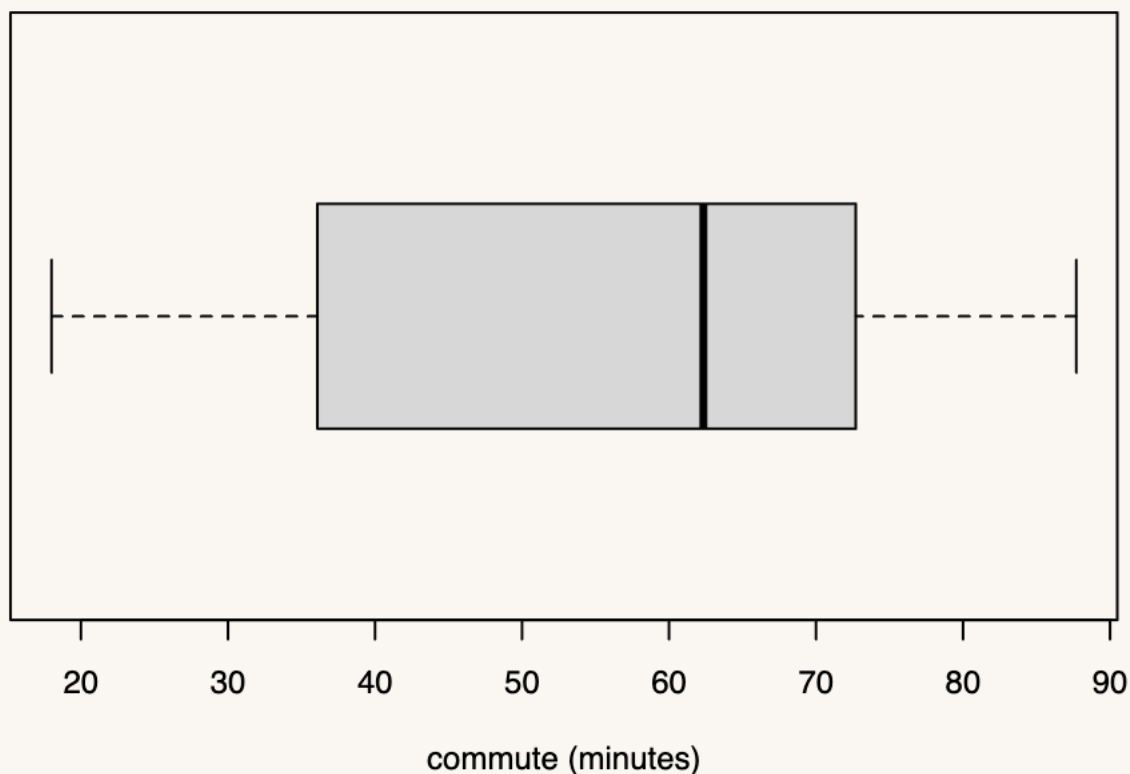


B3. The all-clear on outliers

Task: check the commute data for unusual values before analyzing it.

R Code

```
1 set.seed(6101)
2 grp <- sample(c(0, 1), 400, replace = TRUE)
3 commute <- rnorm(400, mean = ifelse(grp == 1, 72, 34), sd = 6)
4
5 boxplot(commute, horizontal = TRUE, col = "gray85",
6         xlab = "commute (minutes)")
7 summary(commute)
```



The analyst writes: “No outliers, and the distribution is reasonably symmetric around a median of about 62 minutes. Proceeding.”

Two of those three claims are supported by what is on the screen. One of them is contradicted by a number the analyst read straight past—find it. Then say what the analyst missed entirely, what single line of code would have caught it, and what goes wrong downstream if nobody notices.

DISCUSSION



B4. Cleaning by rule

Task: the salary variable has some very large values. Remove the outliers before computing the average.

R Code

```
1 set.seed(6101)
2 salary <- round(rlnorm(240, meanlog = 11, sdlog = 0.55))
3
4 clean <- salary[salary < quantile(salary, 0.95)]
5 c(before = mean(salary), after = mean(clean), removed = length(salary) - length(clean))
```

before	after	removed
70583.40	64618.67	12.00

This is a very common pattern and it has at least three separate problems. Find them. (One is about what the rule is for; one is about what the rule guarantees regardless of the data; one is about what happens to the reported number as a consequence.)

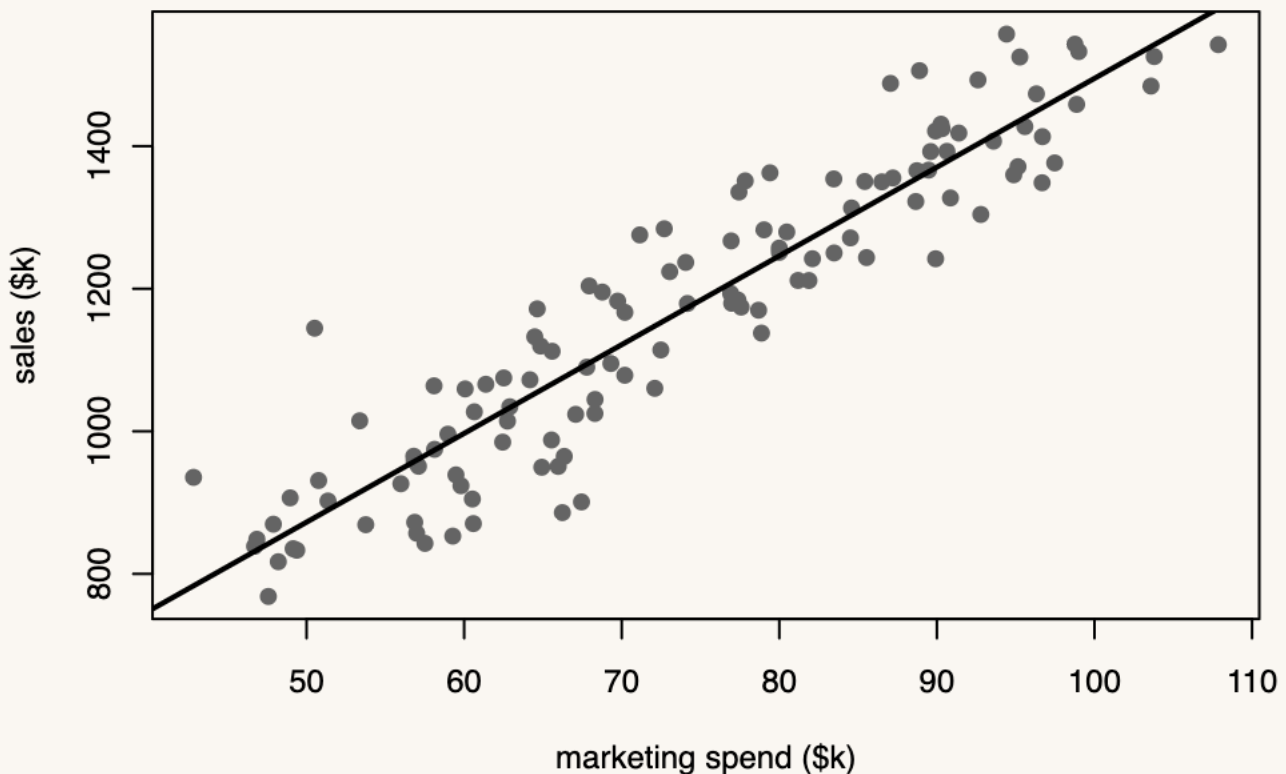
B5. The relationship that is not there

Task: does spending more on marketing increase sales?

R Code

```
1 set.seed(6101)
2 week      <- 1:120
3 marketing <- 50 + 0.4 * week + rnorm(120, 0, 5)
4 sales     <- 800 + 6.0 * week + rnorm(120, 0, 40)
5
6 cor(marketing, sales)
7 plot(marketing, sales, pch = 19, col = "gray40",
8       xlab = "marketing spend ($k)", ylab = "sales ($k)")
9 abline(lm(sales ~ marketing), lwd = 2)
```

[[1] 0.9297237



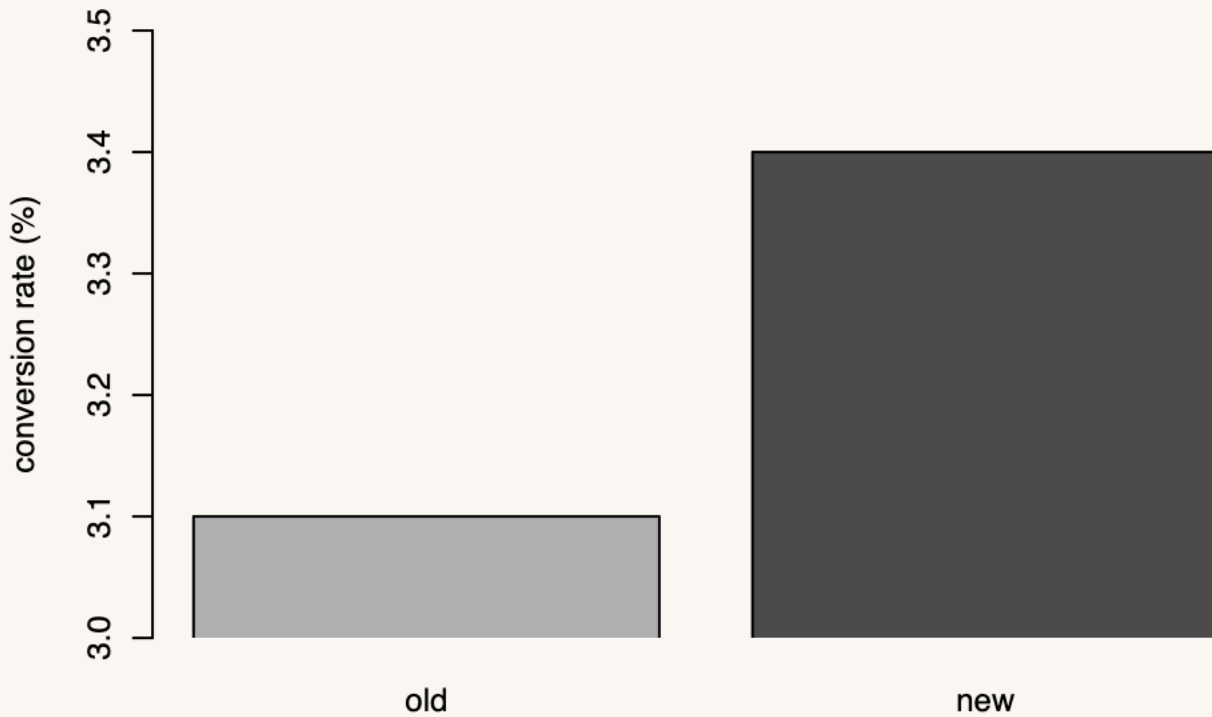
The correlation is strong and the scatterplot looks convincing. Look at the two lines that generated the data. What is actually going on, what is the name for it from [Relationships Between Variables](#), and what would you need in order to answer the task's question honestly?

B6. A defensible chart

Task: show that the new checkout flow improved conversion, which went from 3.1% to 3.4%.

R Code

```
1 conv <- c(old = 3.1, new = 3.4)
2 barplot(conv, ylim = c(3.0, 3.5), xpd = FALSE, col = c("gray70", "gray30"),
3         ylab = "conversion rate (%)")
```



The axis is labeled and both values are correct. Compute the ratio of the drawn bar heights, state the claim a reader takes away, and redraw the figure so that the claim a reader takes away is one you would be willing to write in a sentence.

DISCUSSION



B7. The subgroup that flips

Task: compare pass rates for two tutoring programs.

R Code

```
1 tutoring <- data.frame(
2   program = c("P", "P", "Q", "Q"),
3   prior    = c("strong", "weak", "strong", "weak"),
4   passed   = c(180, 25, 47, 165),
```

```

5   enrolled = c(200, 50, 50, 300)
6   )
7   tutoring$rate <- round(tutoring$passed / tutoring$enrolled, 3)
8
9   pooled <- tapply(tutoring$passed, tutoring$program, sum) /
10              tapply(tutoring$enrolled, tutoring$program, sum)
11   round(pooled, 3)

```

	P	Q
	0.820	0.606

Run it, then look at `tutoring` itself. Which program is better? Write the sentence you would put in a report, and then write the sentence someone advocating for the *other* program would put in theirs—using only correct numbers from this same table. Both sentences should be defensible. That is the uncomfortable part.

DISCUSSION

Judgment

No code. Bring written answers; we will disagree about these out loud.

C1.

A colleague sends you an analysis with the note: “*I dropped the outliers so the mean would be more representative.*” Write the two questions you would ask before you accept the result, and say what a good answer to each would look like.

DISCUSSION

C2.

You are asked for “the average time our support tickets take to resolve”, to put on a public status page. You look at the data: most tickets close in under an hour, but a few sit open for weeks.

Which number do you publish, and what is the argument against your own choice? You must give a real argument against, not a token one.

DISCUSSION

C3.

[Visualization That Does Not Mislead](#) claims that “always disaggregate” is the wrong lesson to take from Simpson’s paradox. Construct a concrete scenario—it may be invented—where disaggregating leads someone to a *worse* conclusion than the pooled figure would have. Then state the principle that distinguishes your scenario from the kidney stone trial.

DISCUSSION

C4.

Everything you computed this week was computed on a sample. Pick any one number from these three chapters—the mean salary, the correlation of 0.716, the 44.5% of coffee drinkers, the pooled cure rate of 78.0%.

Now answer: **if the data collection were repeated from scratch, how different would your number be?** You cannot compute this yet. But write down what you think it depends on, and how you would find out. Bring that list to class; it is the syllabus for the next four weeks, and I would rather you wrote it than that I told you.

Almost no interesting question is about one variable. “How long do patients stay?” is a warm-up; “do patients admitted at night stay longer?” is a question. The moment you have two columns, you can ask whether knowing one tells you anything about the other.

Which tool you reach for depends only on what kinds of variables you have:

	CATEGORICAL	NUMERIC
categorical	two-way table	boxplots by group
numeric	boxplots by group	scatterplot

That is the whole decision. The hard part is what happens after the picture.

Two categorical variables: the two-way table

Three hundred students were asked what they drink in the evening and whether they pulled an all-nighter last week.

R Code	
1	<code>set.seed(6101)</code>
2	<code>n <- 300</code>
3	<code>drink <- sample(c("coffee", "tea", "neither"), n, replace = TRUE,</code>
4	<code> prob = c(0.5, 0.3, 0.2))</code>
5	<code>p <- c(coffee = 0.55, tea = 0.35, neither = 0.30)[drink]</code>
6	<code>allnighter <- ifelse(rbinom(n, 1, p) == 1, "yes", "no")</code>
7	
8	<code>tab <- table(drink, allnighter)</code>
9	<code>tab</code>

	allnighter	
drink	no	yes
coffee	86	69
neither	42	22
tea	48	33

Raw counts are almost never what you want, because the groups are different sizes. There were 155 coffee drinkers and only 64 in the “neither” group, so of course coffee produced more all-nighters in absolute terms. The comparison you want is *within* each row:

R Code	
1	<code>round(prop.table(tab, margin = 1), 3)</code>

	allnighter	
drink	no	yes
coffee	0.555	0.445
neither	0.656	0.344
tea	0.593	0.407

MARGIN = 1 VERSUS MARGIN = 2 VERSUS NEITHER

This one argument changes what question you are answering, and getting it wrong is one of the most common quiet errors in an analysis.

- `prop.table(tab, 1)` —**rows sum to 1**. “Of the coffee drinkers, what fraction pulled an all-nighter?”
- `prop.table(tab, 2)` —**columns sum to 1**. “Of the all-nighter pullers, what fraction drink coffee?”
- `prop.table(tab)` —**everything sums to 1**. “What fraction of all students are coffee-drinking all-nighter-pullers?”

These are three different numbers with three different meanings, and swapping them is the mechanism behind a great many confidently wrong statements. If 90% of people with a rare disease have a certain gene, that tells you very little about your risk given the gene—that would be the other margin.

So: 44.5% of coffee drinkers pulled an all-nighter, against 40.7% of tea drinkers and 34.4% of those who drink neither. Ten percentage points between the extremes, on group sizes of 155 and 64.

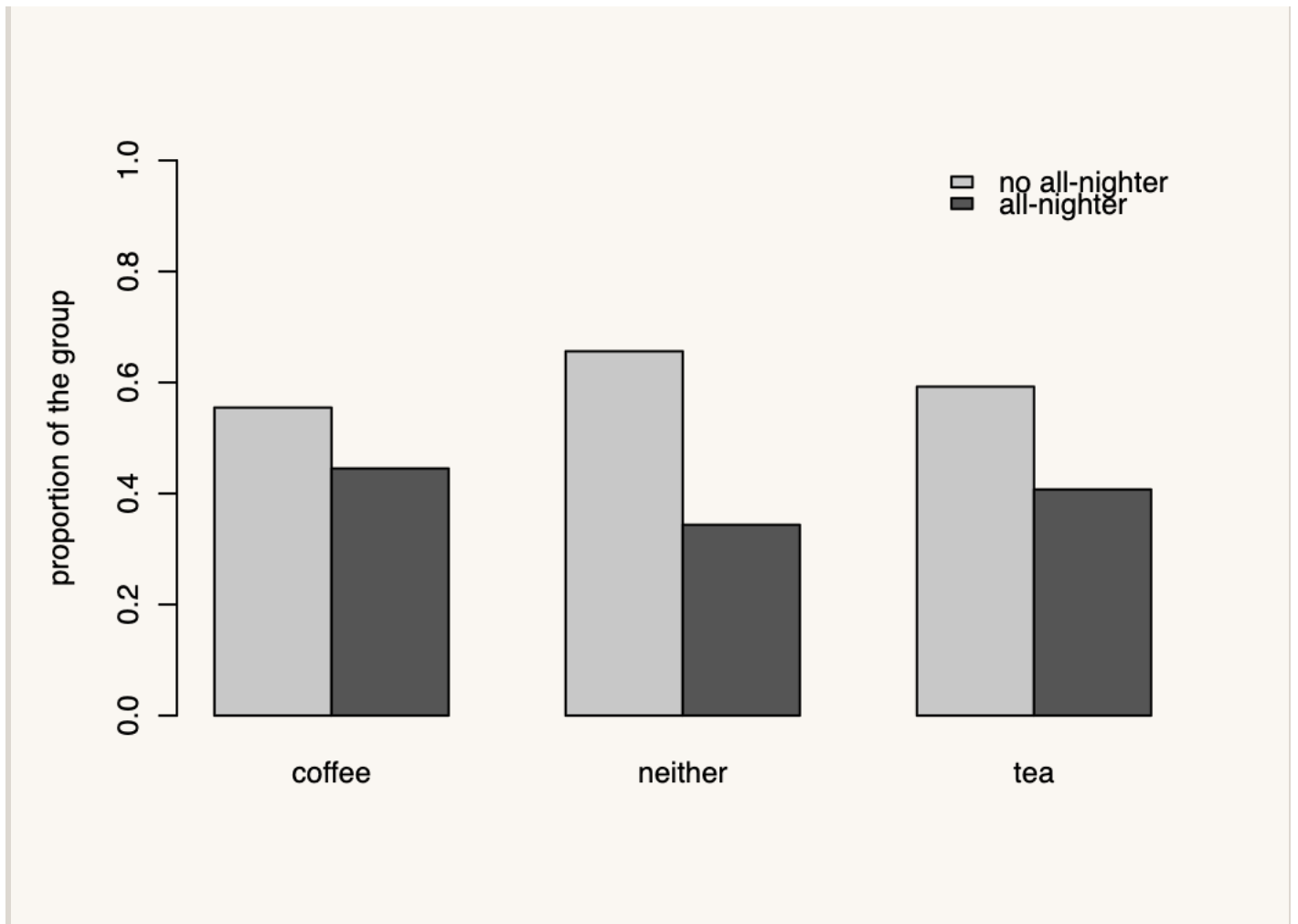
Is ten points a lot? **You cannot answer that from this table.** You know how these 300 students came out; you do not know how much a re-run of the same survey would wobble. That is precisely the gap the next four weeks fill in. For now, note the honest form of the statement: *in this sample*, coffee drinkers reported all-nighters about ten points more often.

Worth noticing: I generated this data with true rates of 55%, 35% and 30%, and the sample returned 44.5%, 40.7% and 34.4%. Every one of those is off, and the tea group is off by nearly six points. Nothing went wrong. That is one draw.

A picture of the same thing:

R Code

```
1 barplot(t(prop.table(tab, 1)), beside = TRUE,  
2         col = c("gray80", "gray35"), ylim = c(0, 1),  
3         ylab = "proportion of the group",  
4         legend.text = c("no all-nighter", "all-nighter"),  
5         args.legend = list(x = "topright", bty = "n"))
```

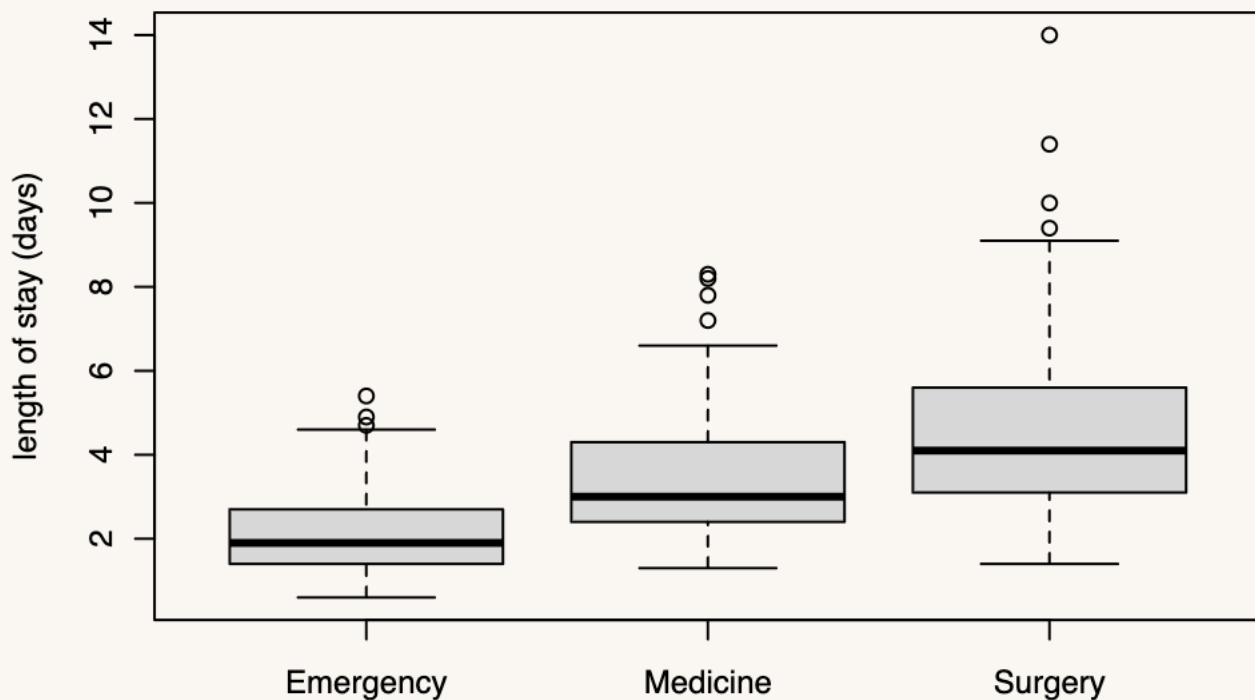


One categorical, one numeric: boxplots by group

When one variable splits the data into groups and the other is a number, put a boxplot in each group. This is where boxplots earn their keep—half a dozen of them line up and are read at a glance in a way half a dozen histograms are not.

R Code

```
1 set.seed(6101)
2 dept <- sample(c("Emergency", "Surgery", "Medicine"), 240, replace = TRUE)
3 stay <- round(rlnorm(240, meanlog = c(Emergency = 0.6, Surgery = 1.4,
4                                     Medicine = 1.1)[dept], sdlog = 0.5), 1)
5
6 boxplot(stay ~ dept, col = "gray85",
7         xlab = "", ylab = "length of stay (days)")
```



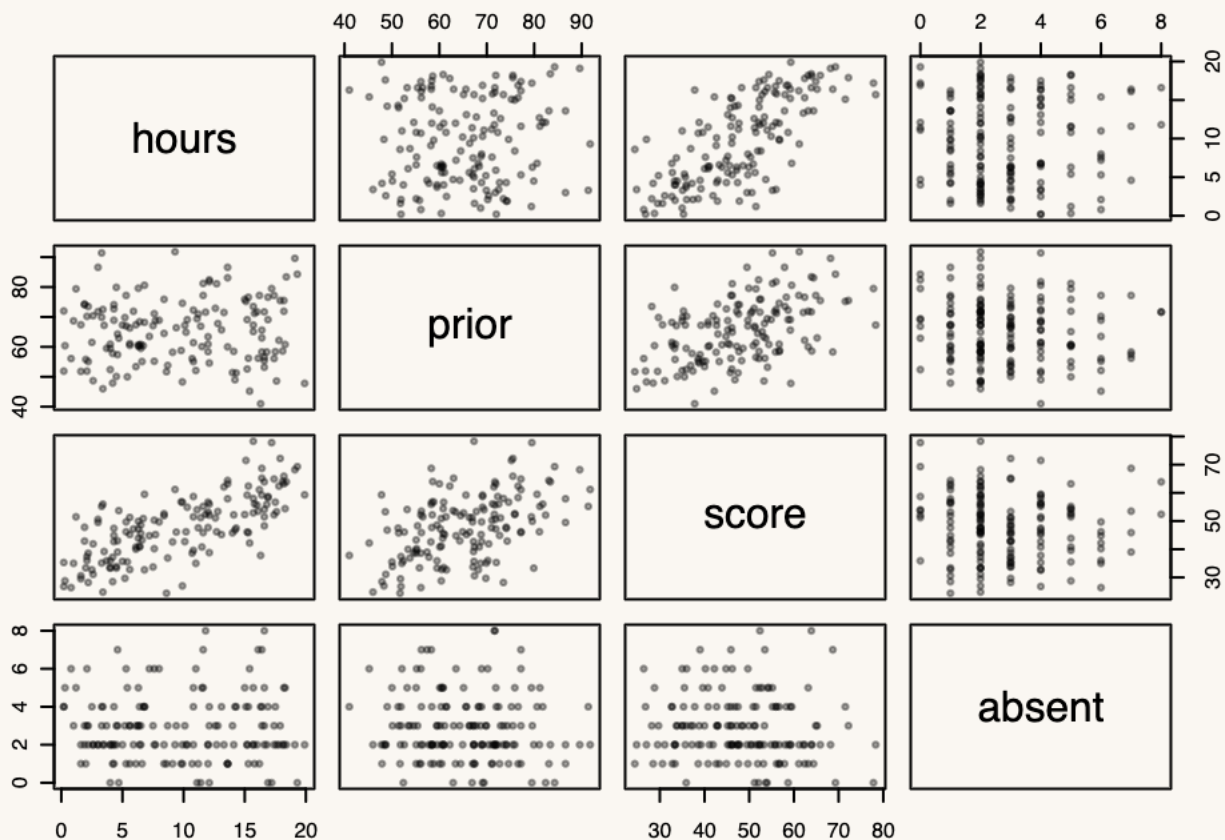
Remember 3.1's warning while you read it: each of those boxes could be hiding a bimodal group. The formula syntax `stay ~ dept` reads as “stay, broken down by dept” and will reappear in every model you fit once we reach modeling, so it is worth getting comfortable with now.

More than two at once, quickly

The rest of this chapter takes variables in pairs, which is the right way to understand them and a slow way to survey them. When a dataset arrives with eight numeric columns you want all twenty-eight pairs at once, and `pairs()` draws them.

R Code

```
1 set.seed(3)
2 n <- 150
3 hours <- round(runif(n, 0, 20), 1)
4 prior <- round(rnorm(n, 65, 10), 1)
5 score <- round(0.5 * prior + 1.5 * hours + rnorm(n, 0, 6), 1)
6 absent <- rpois(n, 3)
7
8 pairs(data.frame(hours, prior, score, absent),
9       pch = 19, col = rgb(0, 0, 0, 0.35), cex = 0.6)
```



You are not reading values here. You are scanning for shape: which panels look like a cloud, which look like a line, which show a curve that a correlation would score as zero, and which show stripes betraying a variable that is really discrete. Then you go back and take the interesting pairs one at a time.

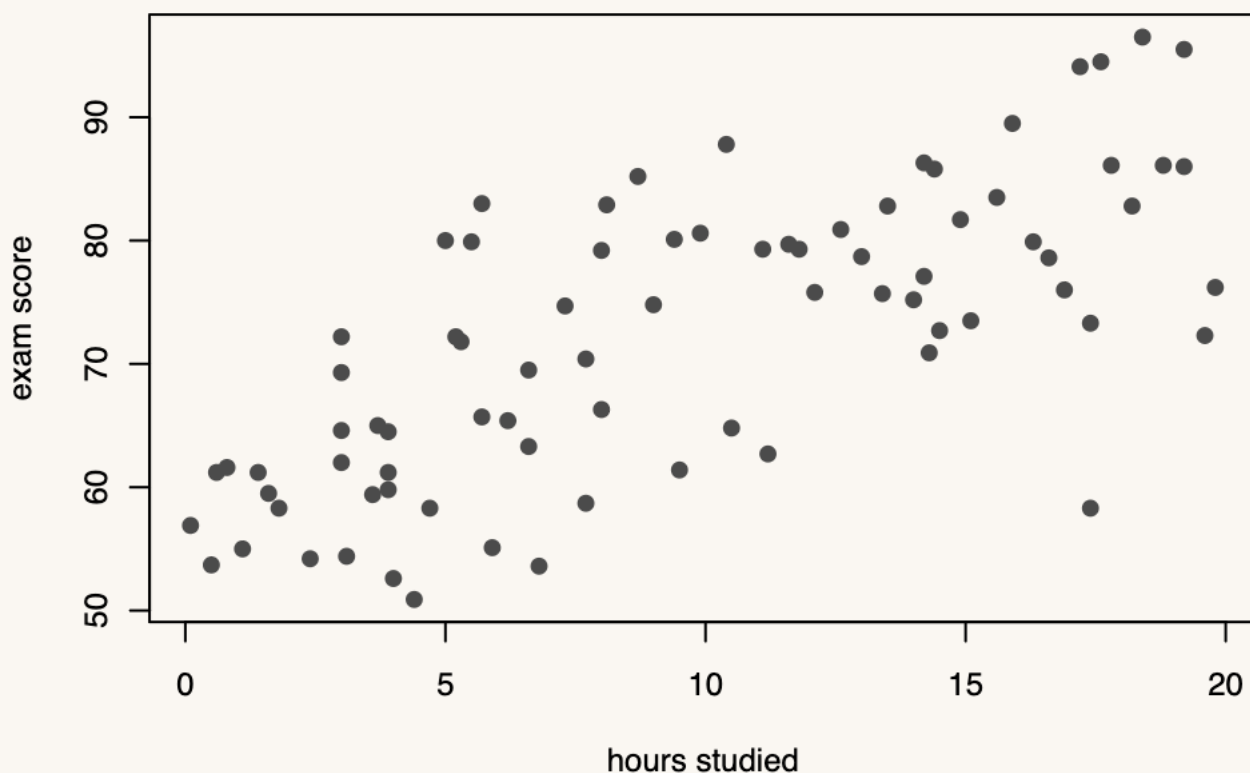
`GGally::ggpairs(df)` is the version to use in your own work: same idea, but it adds the correlation and the marginal distribution to each panel and handles categorical columns sensibly. Install it locally; it is the standard opening move on an unfamiliar dataset.

Two numeric variables: the scatterplot

Eighty students reported hours spent studying for an exam, and we have their scores.

R Code

```
1 set.seed(6101)
2 study <- round(runif(80, 0, 20), 1)
3 score <- round(55 + 1.8 * study + rnorm(80, 0, 8), 1)
4
5 plot(study, score, pch = 19, col = "gray30",
6      xlab = "hours studied", ylab = "exam score")
```



Always plot before you summarize. A scatterplot shows you direction, strength, curvature, clumping, gaps, and outliers all at once. Every numerical summary in the rest of this chapter compresses that picture into one number, and each one of them can be fooled.

Covariance

Covariance asks: when x is above its mean, is y also above its mean?

$$\text{cov}(x, y) = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$$

Each term is positive when both variables are on the same side of their means, negative when they are on opposite sides. Add them up and you get the general tendency. (There is that $n - 1$ again, for exactly the reason from [Describing a Single Variable](#): two means were estimated from this sample.)

R Code

```
1 cov(study, score)
```

```
[1] 48.84057
```

About 48.8. And now the question that kills covariance as a reportable number: 48.8 *what?* Its units are hours $\times$ points. Watch what happens if the survey had recorded study time in minutes instead:

R Code

```
1 c(hours = cov(study, score),
2   minutes = cov(study * 60, score))
```

	hours	minutes
	48.84057	2930.43418

The same relationship, in the same data, is now 2930. Covariance has a meaningful **sign** and a meaningless **magnitude**—you cannot tell from the number alone whether a relationship is strong, because you can make it as large as you like by changing units.

Correlation

Correlation fixes this by dividing out both standard deviations:

$$r = \frac{\text{cov}(x, y)}{s_x s_y}$$

which is the same as computing the covariance after converting both variables to standard deviations from their means. The units cancel. What is left always lies between -1 and $+1$.

R Code

```
1 c(hours = cor(study, score),
2   minutes = cor(study * 60, score),
3   score_rescaled_to_proportion = cor(study, score / 100))
```

	hours	minutes
score_rescaled_to_proportion	0.7162395	0.7162395

Identical to the last decimal place. $r \approx 0.716$ regardless of what units anybody used, which is the entire point.

Reading r :

- **Sign**—does y tend to go up or down with x .
- **Magnitude**—how tightly the points cluster around a *straight line*. $|r| = 1$ means perfectly on a line; $r = 0$ means no *linear* tendency.
- r is **symmetric**: `cor(x, y)` equals `cor(y, x)`. It contains no notion of which variable came first, or which one is the cause. If you were hoping the arithmetic would settle direction for you, it cannot—there is no direction in it.

One consequence of that formula is worth stating plainly, because it saves you checking two things when there is only one. Since s_x and s_y are positive whenever either variable varies at all, r and the covariance **always have the same sign, and they are zero together**. Asking “can the covariance be zero while the relationship is strong?” and asking it of the correlation are the same question with the same answer. Everything the next section says about $r = 0$ applies unchanged to $\text{cov} = 0$.

ROUGH CALIBRATION, WITH A WARNING ATTACHED

In social-science data $|r| > 0.7$ is strong, 0.3–0.7 moderate, under 0.3 weak. In a physics lab, $r = 0.9$ might mean your apparatus is broken.

Do not memorize the thresholds. They are field-specific conventions, not facts, and the next section shows why a single number was never enough anyway.

Exercise: correlation of a perfect relationship

Exercise

```
1 # u runs from -4 to 4. v is EXACTLY u^2—no noise at all, a perfect
2 # deterministic relationship: give me u and I can tell you v with certainty.
3 # Compute the correlation between u and v.
4 # Predict the answer before you press Run.
5 -----
```

Please replace _____ with valid code.

What correlation cannot see

Here are four datasets built to make the point. Compute r for each *before* you look at the pictures.

R Code

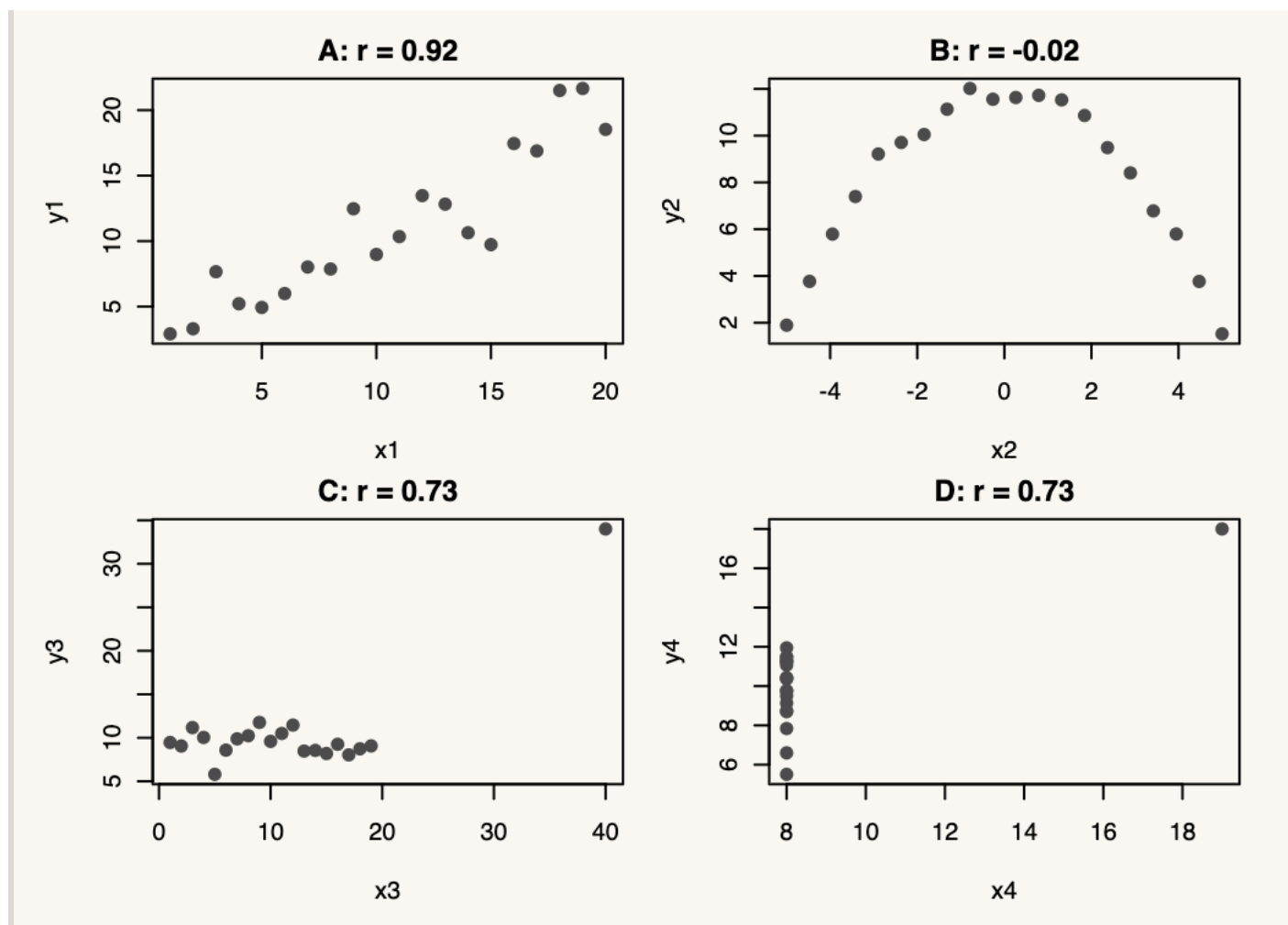
```
1 set.seed(6101)
2 x1 <- 1:20 ; y1 <- 3 + 0.8 * x1 + rnorm(20, 0, 2)
3 x2 <- seq(-5, 5, length.out = 20) ; y2 <- 12 - 0.4 * x2^2 + rnorm(20, 0, 0.4)
4 x3 <- c(1:19, 40) ; y3 <- c(rnorm(19, 10, 1.2), 34)
5 x4 <- c(rep(8, 19), 19) ; y4 <- c(rnorm(19, 10, 2), 18)
6
7 round(c(A = cor(x1, y1), B = cor(x2, y2),
8         C = cor(x3, y3), D = cor(x4, y4)), 2)
```

	A	B	C	D
	0.92	-0.02	0.73	0.73

$r = 0.92$, $r = -0.02$, $r = 0.73$, $r = 0.73$. On the numbers alone, C and D are the same dataset and B has nothing going on. Now look:

R Code

```
1 par(mfrow = c(2, 2), mar = c(4, 4, 2, 1))
2 plot(x1, y1, pch = 19, col = "gray30", main = "A: r = 0.92")
3 plot(x2, y2, pch = 19, col = "gray30", main = "B: r = -0.02")
4 plot(x3, y3, pch = 19, col = "gray30", main = "C: r = 0.73")
5 plot(x4, y4, pch = 19, col = "gray30", main = "D: r = 0.73")
```



- **A** is the honest case. A linear trend with scatter around it, and $r = 0.92$ is a fair one-number summary of it.
- **B** is a clean inverted parabola. Knowing x tells you y to within half a point. Correlation reports -0.02 and would have you conclude there is no relationship at all.
- **C** has *no* relationship among the nineteen ordinary points—and one distant point invented the entire correlation. Drop that one row and see:

R Code

```
1 cor(x3[1:19], y3[1:19])
```

```
[[1]] -0.1704122
```

Without it, r is -0.17 . A single observation out of twenty took the summary from “nothing at all” to “strong positive”.

- **D** is worse. Nineteen points share the identical x value, so among them x does not vary at all and correlation is not even defined. One point sits far to the right. The $r = 0.73$ is manufactured entirely by that point’s position. You have one observation’s worth of evidence about how y changes with x , dressed up as a respectable-looking coefficient.

THE RULE

Never report a correlation you have not plotted. r is a compression of a scatterplot down to a single number, and B, C and D are all cases where the compression is a lie. It costs you one line of code to check.

This is Anscombe’s quartet in spirit: constructed datasets with matching summary statistics and completely different pictures. R ships the original—type `anscombe` at a console someday. The lesson has not aged.

An analyst computes `cor(x, y)` on 40 observations and gets **exactly 0.00**. She has not looked at a scatterplot.

Which conclusion is she entitled to draw?

- ☐ x and y are independent.
- ☐ Knowing x tells her nothing at all about y .
- ☐ There is no *linear* tendency for y to rise or fall with x ; there may still be a strong relationship of another shape.
- ☐ The data must contain an error, since two real variables are never exactly uncorrelated.



Measures that can see a curve

If r is blind to the parabola, is the scatterplot all we have? No—there are numbers that see more than r does. It is worth knowing what they buy, and knowing that none of them buys you out of looking.

Spearman's correlation is the cheapest upgrade and it is already in R. It computes the ordinary correlation of the *ranks* rather than the values, so it asks a weaker question: as x goes up, does y tend to go up—by any amount, in any shape? Watch it earn its keep, and then watch it fail:

R Code

```
1 x <- seq(-3, 3, length.out = 400)
2
3 # A perfect but curved increasing relationship
4 c(pearson = cor(x, exp(x)),
5    spearman = cor(x, exp(x), method = "spearman"))
```

```
pearson spearman
0.8189781 1.0000000
```

Pearson reports 0.82 and undersells a relationship that is *perfect*—give me x and I will give you y exactly. Spearman reports 1, which is the truth, because `exp` never turns around. Now the parabola from the exercise:

R Code

```
1 c(pearson = cor(x, x^2),
2    spearman = cor(x, x^2, method = "spearman"))
```

```
pearson    spearman
-2.108214e-16 -2.328116e-03
```

Both come back at zero—Spearman to within rounding, at -0.002 . **Spearman buys you monotone curves and nothing more.** A relationship that rises and then falls defeats it exactly as it defeats r , because the ranks of y go up and then back down and the two halves cancel just the same. If you take one thing from this section, take that sentence—it is the difference between using Spearman and trusting it.

For relationships that are not monotone, measures do exist. **Distance correlation** and **mutual information** are the two you will hear named; both are zero only when the variables are genuinely independent, which is the property r conspicuously lacks. Neither is in base R, both need more data than r does to be stable, and neither reports a *direction*—you get “there is structure here” without “and it goes up”. They are outside this course, but knowing they exist keeps you from concluding that $r = 0$ settles anything.

WHY THE PLOT STILL WINS

Notice what the whole ladder has in common. Each measure widens the class of relationships it can *detect*, and every one of them still answers with a single number. None of them tells you that C's correlation came from one point, or that D had nineteen tied x values, or that B was a parabola rather than some other curve. Those are facts about the *shape* of the cloud, and a scalar cannot carry a shape.

So the ladder is not “plot, until you learn the better statistic”. It is: plot to see what is there, then choose the number that summarizes it honestly.

Association and cause

Everyone can recite that correlation does not imply causation. Almost everyone then writes a conclusion that quietly assumes it did. It is worth being precise about *what the alternatives are*, because “it might not be causal” is not an analysis—it is a shrug.

Suppose X and Y are associated. At least five things could be going on, and usually more than one is:

1. X **causes** Y . The one you were hoping for.
2. Y **causes** X . Hospitals with more staff have more patients. Which direction is that?
3. **A third variable Z causes both.** The confounder. Below.
4. **Selection.** The association is real in your data and absent in the world, because of how rows got into your file. Among people who were *admitted* to hospital, being young and being sick will look correlated—one or the other had to be true to get you in the door.
5. **Chance.** With 300 students and forty variables, some pairs will correlate at $r = 0.3$ for no reason whatsoever. The inference half of the term is about how to put a number on this one.

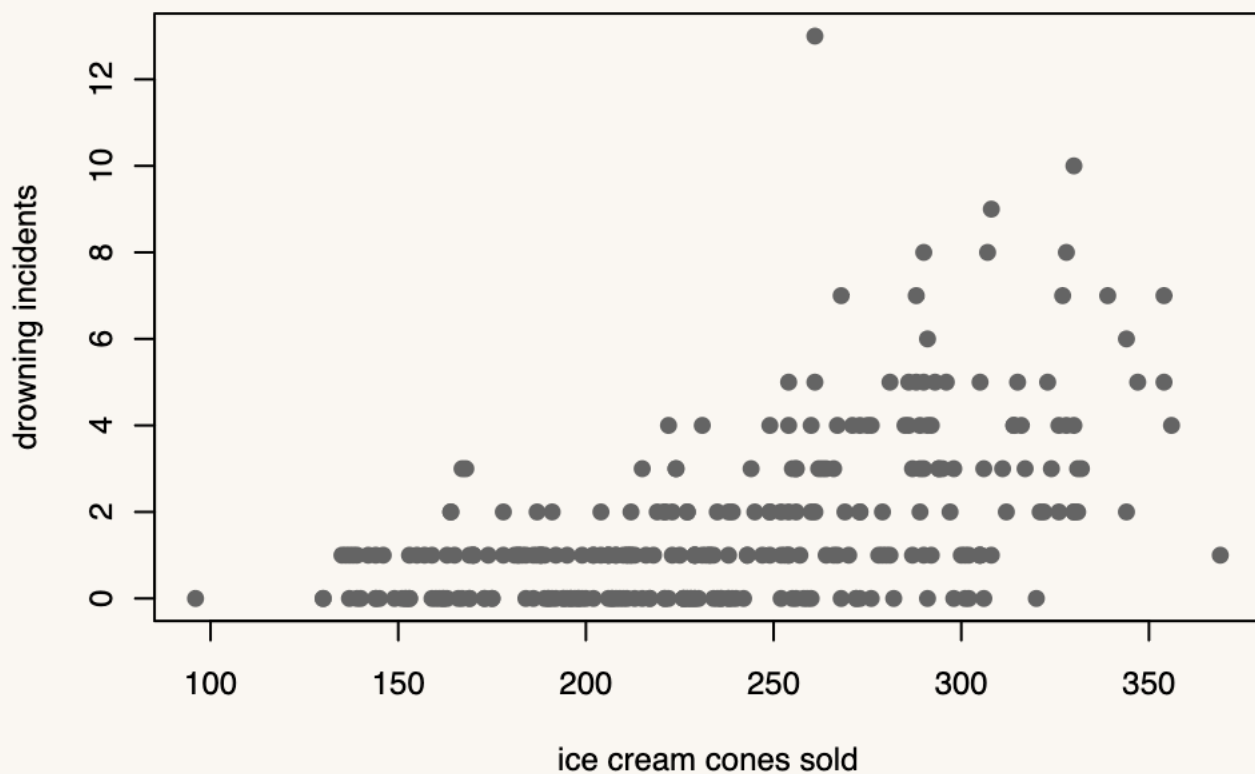
Watching a confounder work

Ice cream sales and drowning deaths, by day, at a coastal town.

R Code

```
1 set.seed(6101)
2 temp <- runif(300, 40, 95) # °F, unobserved for now
3 cones <- round(20 + 3.2 * temp + rnorm(300, 0, 25)) # cones sold
4 drown <- rpois(300, lambda = exp(-3.2 + 0.05 * temp)) # incidents
5
6 cor(cones, drown)
7 plot(cones, drown, pch = 19, col = "gray40",
8      xlab = "ice cream cones sold", ylab = "drowning incidents")
```

```
[1] 0.5474883
```



$r \approx 0.55$, and the scatterplot backs it up—no parabola, no single leverage point, an honest positive association. If all you had were these two columns you would have every technical justification for reporting it.

I built this data, so I can tell you with certainty that ice cream has no effect on drowning: the two lines that generate `cones` and `drown` share only `temp`. Hot days bring people to the ice cream stand and to the water. Now hold temperature roughly fixed and look again:

R Code

```
1 band <- cut(temp, breaks = c(40, 55, 70, 85, 95))
2 table(band)
3 round(sapply(split(data.frame(cones, drown), band),
4             function(d) cor(d$cones, d$drown)), 3)
```

```
band
(40,55] (55,70] (70,85] (85,95]
      87      81      74      58
(40,55] (55,70] (70,85] (85,95]
 0.041  0.050  0.079 -0.014
```

Within each temperature band the correlation collapses to essentially nothing: 0.04, 0.05, 0.08, -0.01 . The association was never between ice cream and drowning. It was temperature, showing up twice.

SAY IT CAREFULLY

The useful version is not “correlation does not imply causation.” It is:

An association is one observation about the data. Attributing it to a cause is an additional claim, and that claim needs support the correlation itself cannot provide.

That support comes from somewhere outside the coefficient: a randomized experiment, a natural experiment, a mechanism you can articulate, measured confounders that you adjusted for, or a design argument about how the rows were selected. If you have none of those, you have not failed—you have an association, which is a real finding. Report it as one.

And here is the practical consequence for your projects. In a write-up, the sentence “*X is associated with a higher Y in this sample*” is defensible and publishable. The sentence “*X increases Y*” is a different claim requiring different evidence. The distance between those two sentences is roughly the distance between a careful analyst and a press release.

Notice, too, what the confounding demo required: I had to **have** the temperature column. Adjusting for a confounder you did not measure is not possible. This is why “what else could explain this?” is a data-collection question, asked at the start, and not something you can repair at the analysis stage.

Exercise: what the coefficient licenses

An observational study of 4,000 adults finds a correlation of $r = 0.42$ between weekly hours of exercise and a self-reported wellbeing score. No randomization; participants chose how much they exercised.

Exercise

```
1 # Which conclusion does this result support? Assign the letter,
2 # for example: answer <- "A"
3 #
4 # A. Exercise raises wellbeing; the large sample rules out coincidence.
5 # B. 42% of the variation in wellbeing is explained by exercise.
6 # C. In this sample, people who exercise more report higher wellbeing on
7 #   average, with substantial scatter; the direction of the effect and the
8 #   role of other variables are not settled by this analysis.
9 # D. There is a weak relationship, since 0.42 is well below 1.
10 answer <- _____
```

Please replace _____ with valid code.

Exercise: reading the right margin

Exercise

```
1 # `tab` is the two-way table of drink by allnighter.
2 # Answer this question, and only this question:
3 #   "Among the students who pulled an all-nighter, what proportion drink tea?"
4 # Return a single number.
5 _____
```

Please replace _____ with valid code.

Summary

- Which tool you use is fixed by the variable types: two-way table, boxplots by group, or scatterplot.

- In a two-way table, **counts mislead when groups differ in size**. Convert to proportions, and be deliberate about which margin—the three versions answer three different questions.
- Covariance has a meaningful sign and a meaningless magnitude; its units are the product of the two variables' units. Correlation divides them out and lands in $[-1, 1]$.
- r measures **linear** clustering only. It is blind to curvature, it is dominated by leverage points, and it is symmetric, so it contains no information about direction of cause.
- **Never report a correlation you have not plotted.**
- An association has at least five possible explanations. Naming which one you are claiming, and what supports it, is the analysis. “Correlation is not causation” on its own is not.

Next: the same data, several honest pictures, and how the choice between them becomes an argument.

There is a comfortable way to think about this topic: there are honest charts and dishonest charts, and if you avoid a short list of tricks—truncated axes, 3D pie charts, misleading pictograms—you are on the right side of the line.

That story is wrong, and believing it is what makes people easy to fool.

Every chart is the result of decisions: what to put on each axis, how far the axes run, how wide the bins are, what to aggregate over, what to leave out. Each decision changes what a reader sees. There is no neutral setting. A chart is not a photograph of the data; it is a **sentence about the data**, and like any sentence it asserts something and can be argued with.

So the question to ask of your own figures is not “did I break a rule.” It is:

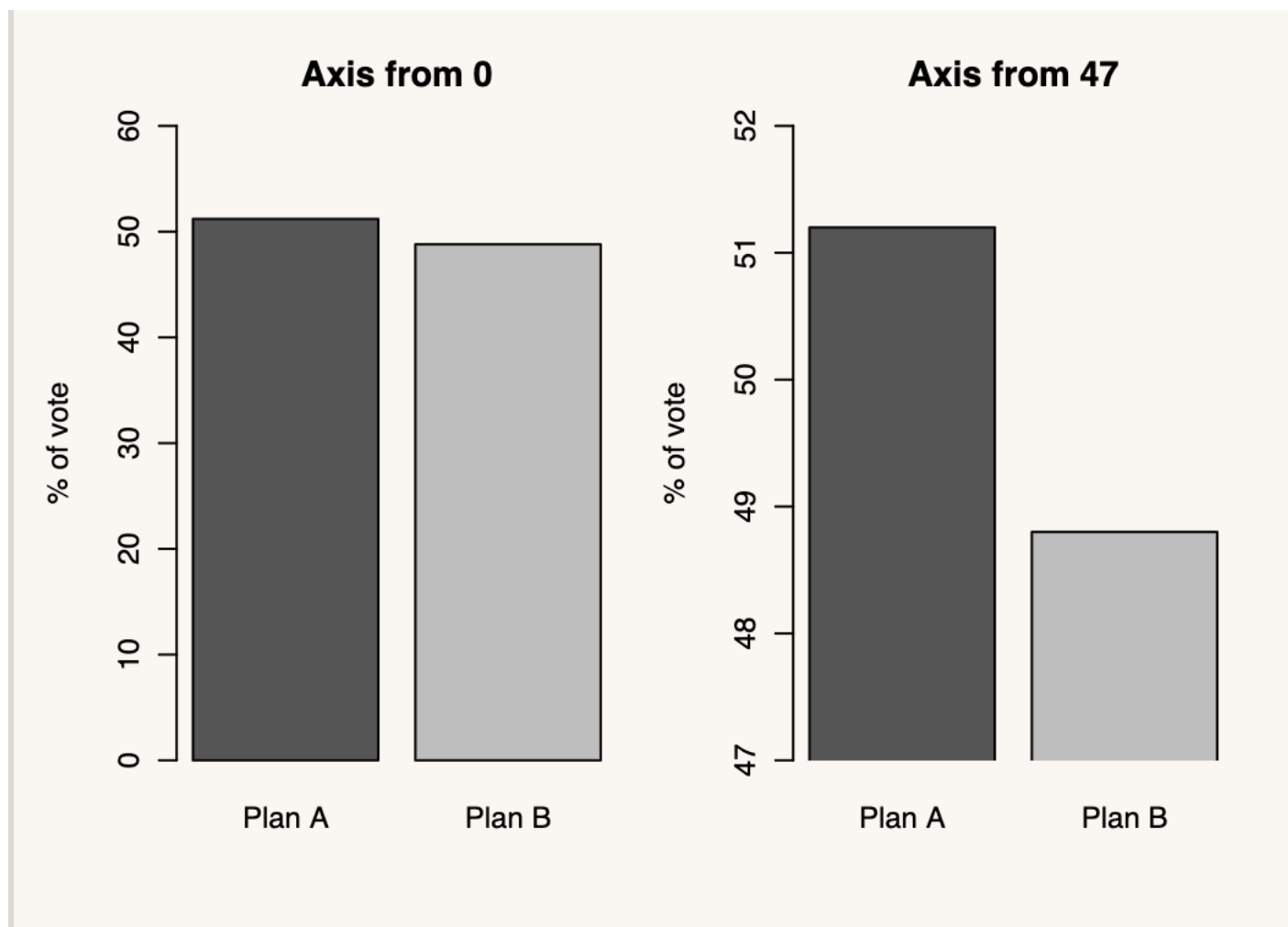
What claim does someone take away from this picture, and is that claim one I would be willing to write down and defend in words?

That reframing does more work than any checklist, because it applies to the choices no checklist anticipated.

Truncated axes

The canonical example, and worth doing properly. Two ballot measures:

R Code	
1	<code>votes <- c("Plan A" = 51.2, "Plan B" = 48.8)</code>
2	
3	<code>par(mfrow = c(1, 2), mar = c(4, 4, 3, 1))</code>
4	<code>barplot(votes, ylim = c(0, 60), col = c("gray35", "gray75"),</code>
5	<code> ylab = "% of vote", main = "Axis from 0")</code>
6	<code>barplot(votes, ylim = c(47, 52), xpd = FALSE, col = c("gray35", "gray75"),</code>
7	<code> ylab = "% of vote", main = "Axis from 47")</code>



Same two numbers. On the left, a near-tie. On the right, a rout.

The reason is arithmetic, and you can put a number on it. A bar chart invites the reader to compare **lengths**, so the ratio of drawn heights is what gets read as the ratio of quantities:

R Code	
1	<code>c(true_ratio = 51.2 / 48.8,</code>
2	<code>drawn_ratio = (51.2 - 47) / (48.8 - 47))</code>
<pre> true_ratio drawn_ratio 1.049180 2.333333 </pre>	

Plan A got 5% more votes than Plan B. The right-hand chart draws its bar **2.3 times** as tall. Nothing in the figure is factually false—the axis is labeled, the values are correct, a careful reader could work it out. But “technically defensible under scrutiny” is a low bar, and the whole point of a chart is that people read it *without* scrutiny.

BARS FROM ZERO. LINES NEED NOT BE.

Bars encode length, so the baseline is part of the encoding and it has to be zero. Truncating a bar chart’s axis breaks the visual metaphor it is built on.

Lines encode change, and forcing a line chart to include zero can be its own distortion—body temperature over a day, plotted from 0°C, is a flat line carrying no information. Choose a range that shows the variation that matters and *say what the range is*.

The general principle underneath: **match the axis to what the reader is being asked to compare**. Comparing magnitudes needs a zero. Comparing movements does not.

Stated as a rule, that is two cases to memorize, and memorized rules fail on the third case. Here is the same thing as a test you can apply to any chart, including ones not listed above:

THE TEST: IS LENGTH CARRYING THE MEANING?

Look at the mark on the page—the bar, the point, the line segment—and ask whether the reader is meant to read a quantity off **how long it is**.

- **Bar chart: yes.** A bar twice as tall is claiming twice as much. That claim is measured from the baseline, so the baseline must be zero or the claim is false. This is also why the fix is never “add a jagged break to the axis”—the bar is still the wrong length.
- **Line chart: no.** Nobody reads the *height* of a line above the axis; they read its slope and where it goes. Length is carrying nothing, so the baseline is free, and you owe the reader a labelled range instead.
- **Dot plot: no.** The dot’s position encodes the value and the dot has no length at all. Truncating is fine, and it is why a dot plot is often the honest way to show small differences between large numbers.

So the answer to “when may I truncate?” is: **whenever no mark’s length is doing the work**. Not “when the difference is too small to see”—that is the reason people truncate, not a licence to.

Exercise: how much does the truncation exaggerate?

Exercise

```
1 # The marketing team plots these two bars with the y-axis running from
2 # 70 to 75, and captions it "satisfaction is surging".
3 # Compute the ratio of the DRAWN bar heights under that axis.
4 -----
```

Please replace _____ with valid code.

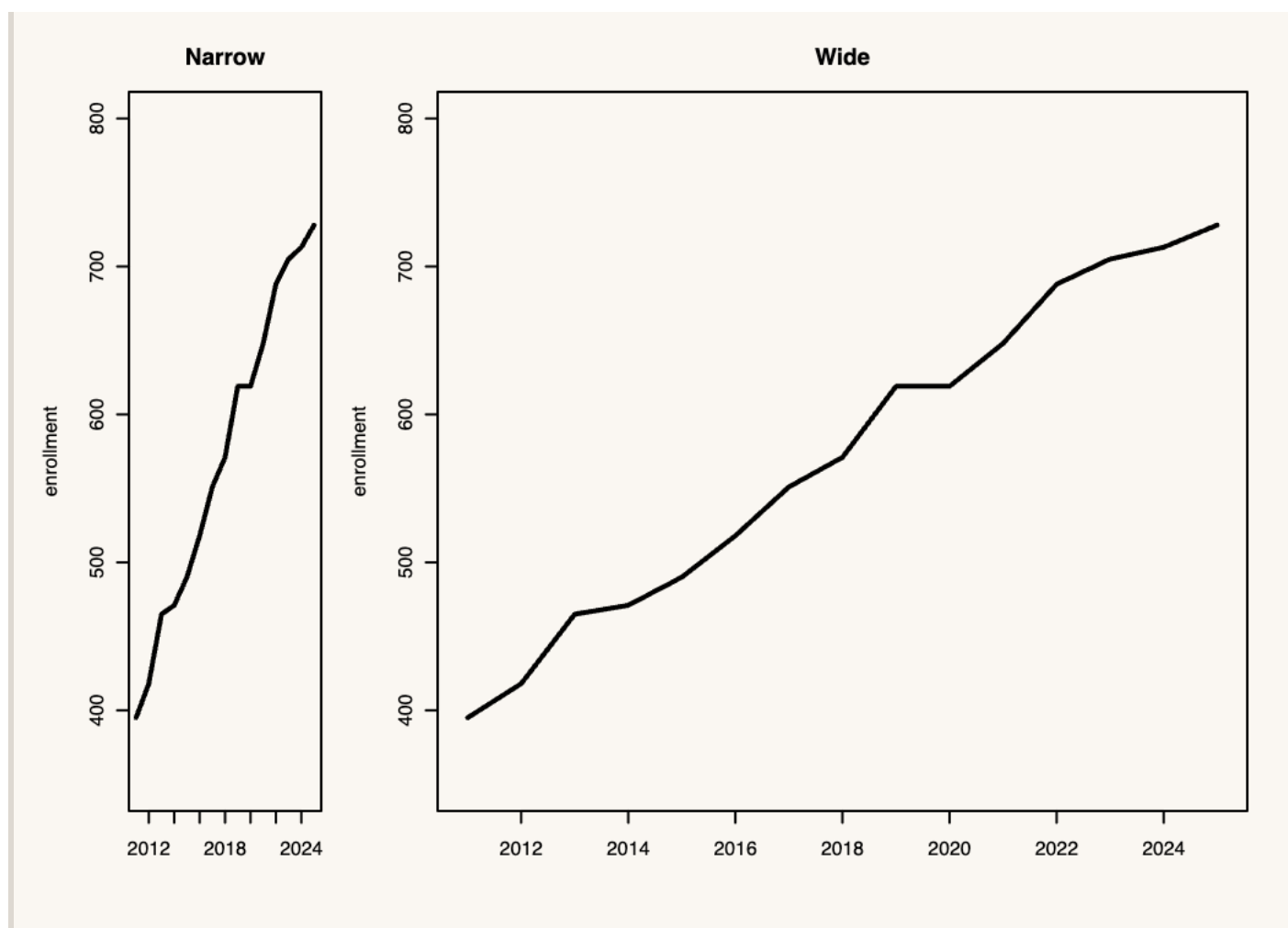
Aspect ratio

Less famous than the truncated axis, and it distorts just as effectively—with the added convenience that nobody can point to anything wrong.

Here is the same enrollment series, same data, same axis limits, drawn in two panels of different widths:

R Code

```
1 year <- 2011:2025
2 set.seed(6101)
3 enroll <- round(seq(400, 760, length.out = 15) + rnorm(15, 0, 12))
4
5 layout(matrix(c(1, 2, 2, 2), nrow = 1))
6 par(mar = c(4, 4, 3, 1))
7 plot(year, enroll, type = "l", lwd = 2, ylim = c(350, 800),
8      xlab = "", ylab = "enrollment", main = "Narrow")
9 plot(year, enroll, type = "l", lwd = 2, ylim = c(350, 800),
10     xlab = "", ylab = "enrollment", main = "Wide")
```



The left panel shows a steep climb. The right panel shows a gentle drift upward. Identical numbers, identical axis ranges, identical everything except the width of the box.

Slope is not a property of your data. It is a property of your data *and* the shape of the plotting region, and the shape of the plotting region is usually decided by whatever default your software had, or by how wide the column was in the report. Nobody chose it on purpose, which means nobody checked it.

THE PRACTICAL FIX

Before you ship a figure whose message is “this is rising fast” or “this is flat”, **resize the window and see whether your sentence survives**. If widening the plot turns your alarming trend into a shrug, the trend was in the aspect ratio, not in the data.

The classical advice is to aim for average slopes near 45°, where the eye judges changes in slope most accurately. Treat it as a starting point, not a rule.

Binning

A histogram has a parameter you must supply, and the shape you see depends on it.

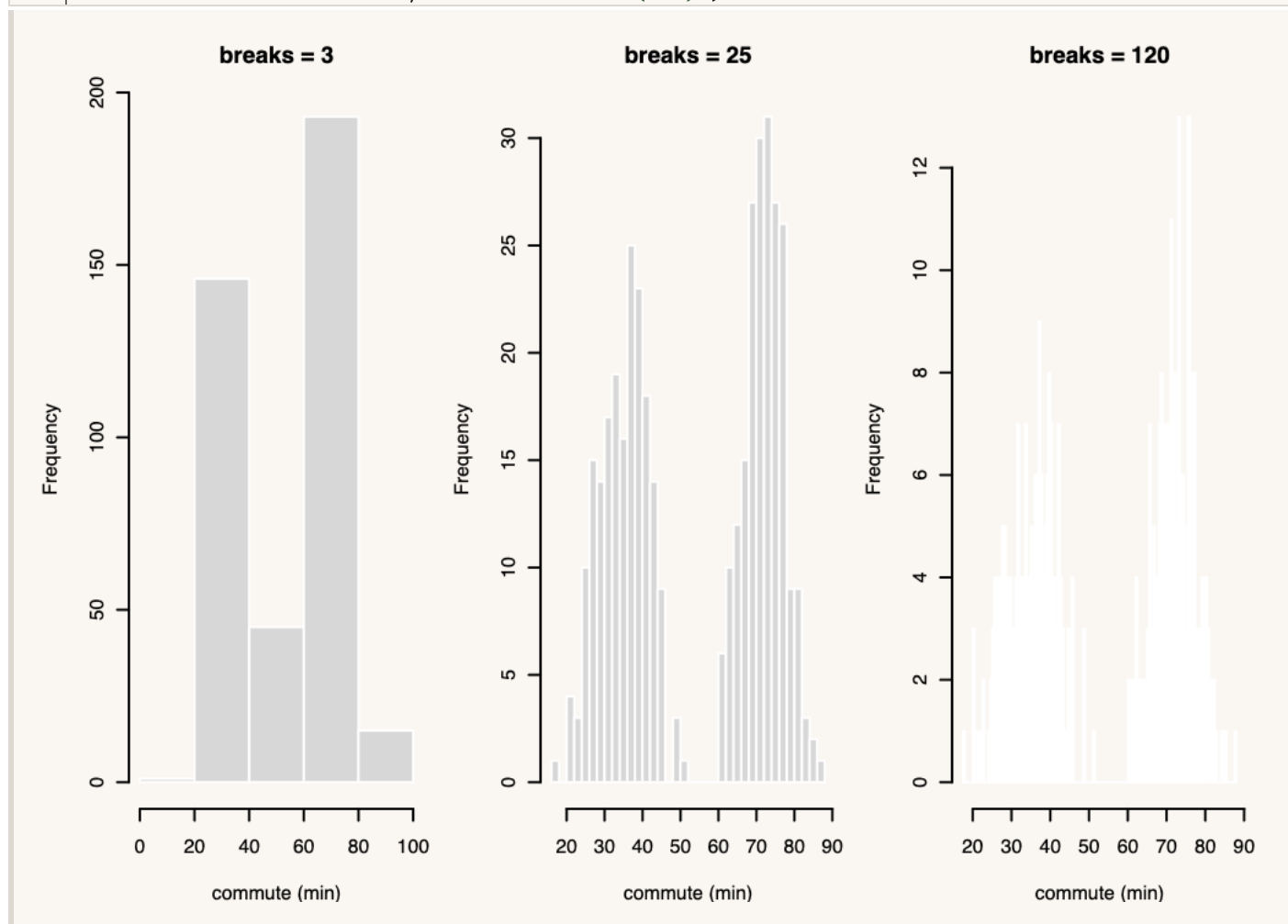
R Code

```
1 set.seed(6101)
2 grp      <- sample(c(0, 1), 400, replace = TRUE)
3 commute  <- rnorm(400, mean = ifelse(grp == 1, 72, 34), sd = 6)
4
5 par(mfrow = c(1, 3), mar = c(4, 4, 3, 1))
```

```

6 hist(commute, breaks = 3, col = "gray85", border = "white",
7     main = "breaks = 3", xlab = "commute (min)")
8 hist(commute, breaks = 25, col = "gray85", border = "white",
9     main = "breaks = 25", xlab = "commute (min)")
10 hist(commute, breaks = 120, col = "gray85", border = "white",
11     main = "breaks = 120", xlab = "commute (min)")

```



Three pictures of the same 400 commutes. The first is coarse enough that you can see *something* uneven but not what: two clumps or one lopsided lump, with a boundary that could be anywhere in a twenty-minute window. The second shows two clean, well-separated groups—this is the truth about the data-generating process, and I know that because I wrote it. The third is a picket fence of noise, and the eye immediately starts inventing structure in the gaps between bars that are one or two observations tall.

(A small R detail worth knowing: `breaks` is a *suggestion*. `hist()` moves the cut points to round numbers, so `breaks = 3` gave you five bars. If you need exact control, pass a vector of cut points instead of a count.)

TOO FEW BINS HIDE STRUCTURE. TOO MANY MANUFACTURE IT.

There is no formula that makes this decision for you. `hist()`'s default (the Sturges rule) is fine as a starting point and known to under-bin large samples.

Try several. Then report the one you would defend, and be aware you chose it. If the finding you are excited about appears at `breaks = 40` and vanishes at `breaks = 20` and at `breaks = 80`, you have not found a bimodality; you have found a bin width.

The same problem, wearing different clothes, shows up everywhere: the smoothing bandwidth in `density()`, the window length in a moving average, the number of categories you collapse a variable into, and the time unit you aggregate to. Daily data plotted monthly is smoother than daily data plotted daily, and it is smoother because you smoothed it.

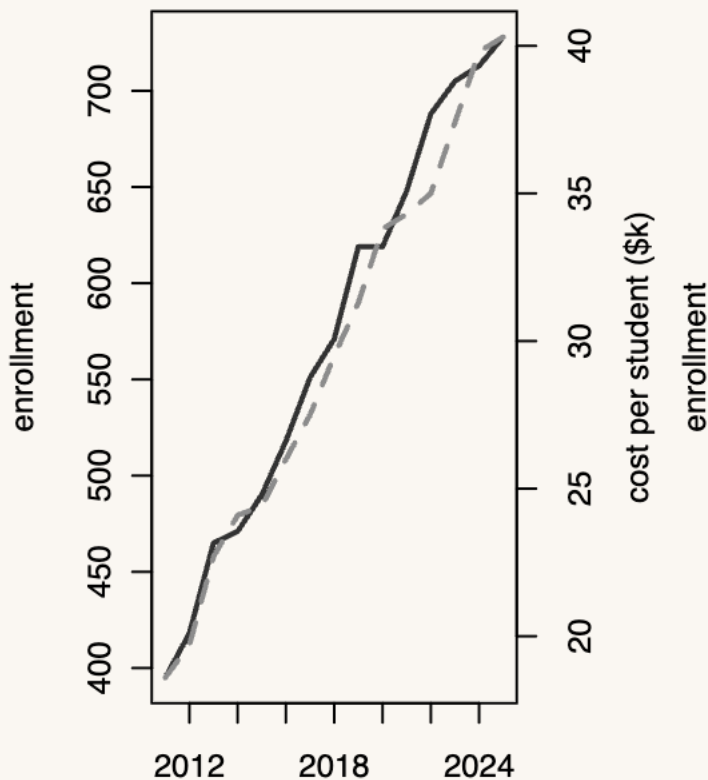
Dual axes

Two series with different units on one plot, one axis on the left and one on the right. Extremely common in business reporting, and almost always a mistake.

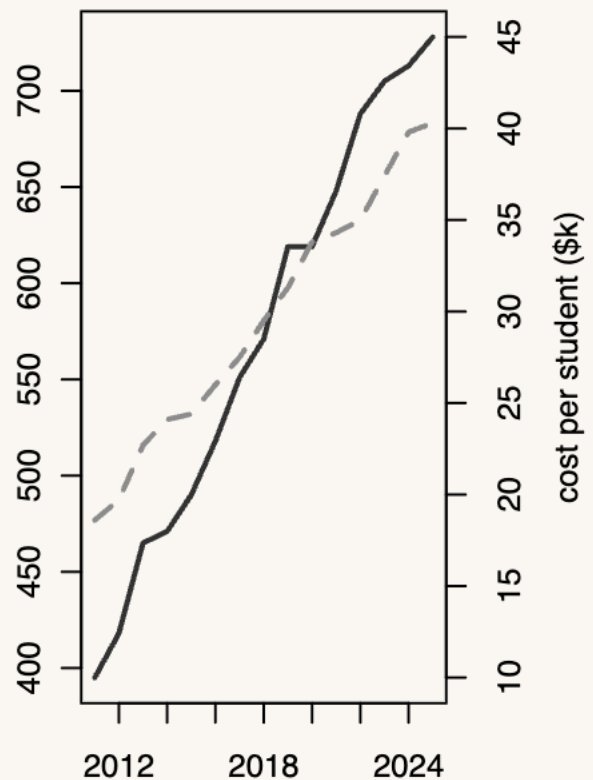
R Code

```
1 year <- 2011:2025
2 set.seed(6101)
3 enroll <- round(seq(400, 760, length.out = 15) + rnorm(15, 0, 12))
4 cost <- round(seq(18, 41, length.out = 15) + rnorm(15, 0, 0.7), 1)
5
6 par(mfrow = c(1, 2), mar = c(4, 4, 3, 4))
7
8 plot(year, enroll, type = "l", lwd = 2, col = "gray20",
9       xlab = "", ylab = "enrollment", main = "Right axis: default")
10 par(new = TRUE)
11 plot(year, cost, type = "l", lwd = 2, lty = 2, col = "gray55",
12       axes = FALSE, xlab = "", ylab = "")
13 axis(4); mtext("cost per student ($k)", side = 4, line = 2.5)
14
15 plot(year, enroll, type = "l", lwd = 2, col = "gray20",
16       xlab = "", ylab = "enrollment", main = "Right axis: 10 to 45")
17 par(new = TRUE)
18 plot(year, cost, type = "l", lwd = 2, lty = 2, col = "gray55",
19       ylim = c(10, 45), axes = FALSE, xlab = "", ylab = "")
20 axis(4); mtext("cost per student ($k)", side = 4, line = 2.5)
```

Right axis: default



Right axis: 10 to 45



The left panel says: enrollment and cost moved in lockstep for fifteen years. The right panel says: cost ran ahead of enrollment for a decade, and enrollment only caught up and overtook it around 2020.

Both are the same two series. The only difference is the range of the right-hand axis—and there is no correct range, because the two variables have no common unit. Where the lines cross, which one is “above”, whether they “diverge” in 2019: every one of those apparent facts is something the analyst chose, usually without noticing.

WHAT TO DO INSTEAD

- **Two stacked panels** sharing an x-axis. The reader compares timing and shape, which is what they wanted, and is never invited to compare heights that are not comparable.
- **Index both series to 100** at a base year and plot them on one axis. Now they share a unit—percentage of the base year—and crossings mean something.
- **Plot the derived quantity you actually care about.** If the story is “cost is outpacing enrollment”, then cost per student is that story; plot it alone.

Aggregation, and the reversal it can hide

Now the most important idea in this chapter, and one of the two or three most important in the course.

Every aggregate—every mean, every rate, every bar in every summary chart—is computed over a group. Choosing the group is a modeling decision made before any modeling starts, and it can reverse your conclusion. Not weaken it. **Reverse** it.

A real trial

Two treatments for kidney stones, on 350 patients each. This is real data from a 1986 study, and it is famous for what it does.

R Code					
1	stones <- data.frame(2 treatment = c("A", "A", "B", "B"), 3 size = c("small", "large", "small", "large"), 4 cured = c(81, 192, 234, 55), 5 treated = c(87, 263, 270, 80) 6) 7 stones\$rate <- round(stones\$cured / stones\$treated, 3) 8 stones				
	treatment	size	cured	treated	rate
1	A	small	81	87	0.931
2	A	large	192	263	0.730
3	B	small	234	270	0.867
4	B	large	55	80	0.688

Read the `rate` column:

- **Small stones:** A cures 93.1%, B cures 86.7%. **A wins.**
- **Large stones:** A cures 73.0%, B cures 68.8%. **A wins.**

A is better for small stones and better for large stones. Every stone is small or large. Now pool the two groups, which is what you do if you have not recorded stone size, or have and did not think it mattered:

R Code					
1	pooled <- tapply(stones\$cured, stones\$treatment, sum) / 2 tapply(stones\$treated, stones\$treatment, sum) 3 round(pooled, 3)				
	A	B			
	0.780	0.826			

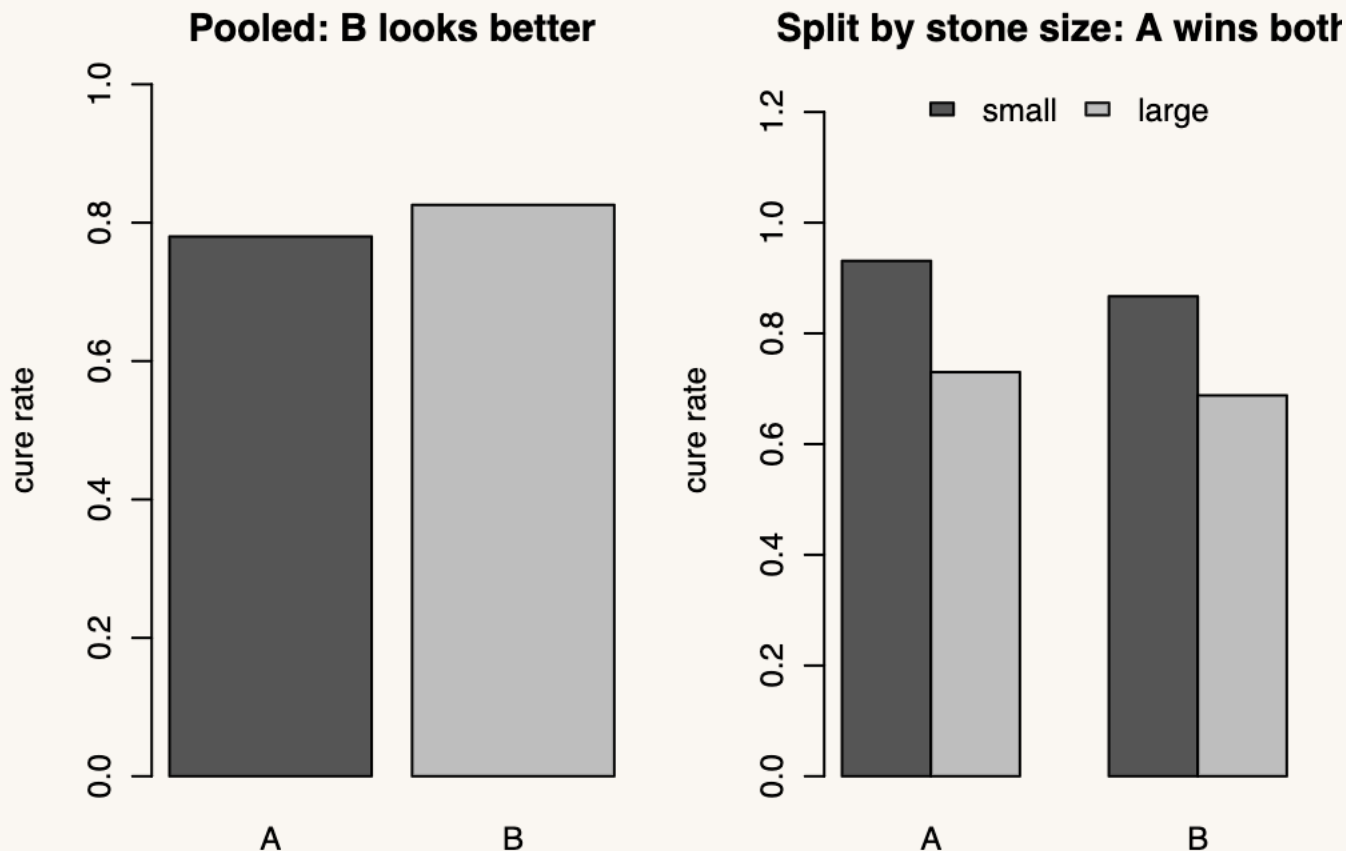
B wins overall, 82.6% to 78.0%. Both statements are arithmetically correct on the same 700 patients. This is **Simpson's paradox**.

R Code

```

1 rates <- matrix(stones$rate, nrow = 2,
2                 dimnames = list(c("small", "large"), c("A", "B")))
3
4 par(mfrow = c(1, 2), mar = c(4, 4, 3, 1))
5 barplot(pooled, ylim = c(0, 1), col = c("gray35", "gray75"),
6         ylab = "cure rate", main = "Pooled: B looks better")
7 barplot(rates, beside = TRUE, ylim = c(0, 1.25), col = c("gray35", "gray75"),
8         ylab = "cure rate", main = "Split by stone size: A wins both",
9         legend.text = TRUE,
10        args.legend = list(x = "top", horiz = TRUE, bty = "n"))

```



Why it happens

There is nothing mystical here, and the resolution is worth working through slowly, because once you see the mechanism you will start spotting it in the wild.

Two things are true at once:

1. **Large stones are harder to cure**, whichever treatment you use (73% and 69%, versus 93% and 87%).
2. **The treatments were not given to comparable patients.**

R Code

```

1 mix <- matrix(stones$treated, nrow = 2,
2               dimnames = list(c("small", "large"), c("A", "B")))
3 mix
4 round(prop.table(mix, margin = 2), 3)

```

	A	B
small	87	270
large	263	80

	A	B
small	0.249	0.771
large	0.751	0.229

Treatment A was given to large stones 75% of the time. Treatment B was given to small stones 77% of the time. Surgeons, reasonably, gave the more invasive treatment A to the harder cases.

So A's pooled rate is mostly the *large-stone* rate, and B's pooled rate is mostly the *small-stone* rate. The pooled comparison is not comparing treatments. It is comparing "A on hard cases" with "B on easy cases", and calling the difference a treatment effect.

Make the mixes match and the reversal disappears. Here is A's cure rate if A had been given to the same mix of stone sizes as B:

R Code					
1	<code>a_small <- 81 / 87</code>				
2	<code>a_large <- 192 / 263</code>				
3	<code>b_mix <- c(small = 270 / 350, large = 80 / 350)</code>				
4					
5	<code>round(c(A_on_Bs_patient_mix = b_mix["small"] * a_small + b_mix["large"] * a_large,</code>				
6	<code> B_actual = 289 / 350), 3)</code>				
<table> <thead> <tr> <th>A_on_Bs_patient_mix.small</th><th>B_actual</th></tr> </thead> <tbody> <tr> <td>0.885</td><td>0.826</td></tr> </tbody> </table>		A_on_Bs_patient_mix.small	B_actual	0.885	0.826
A_on_Bs_patient_mix.small	B_actual				
0.885	0.826				

Given B's patients, A would have cured about **88.5%** against B's 82.6%. The reversal was never about the treatments. It was entirely an artifact of who got which.

WHAT SIMPSON'S PARADOX ACTUALLY TEACHES

The lesson is **not** "always disaggregate". Split finely enough and every subgroup has one patient in it and you learn nothing—and you can usually find *some* split that flips *some* result, which is a recipe for fooling yourself in the other direction.

The lesson is that **the aggregate and the subgroup answer different questions, and you have to know which one you are asking.**

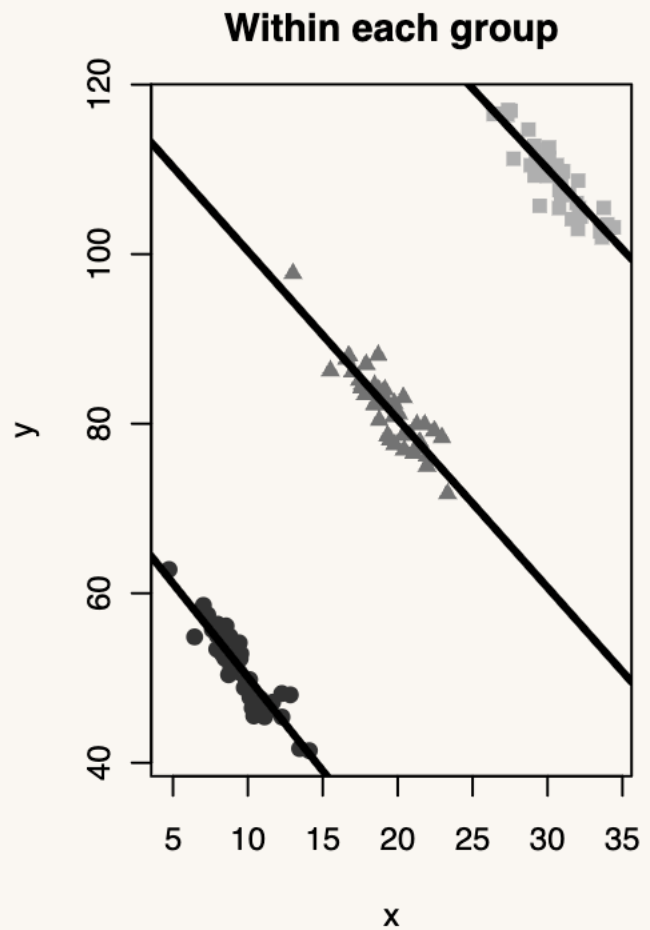
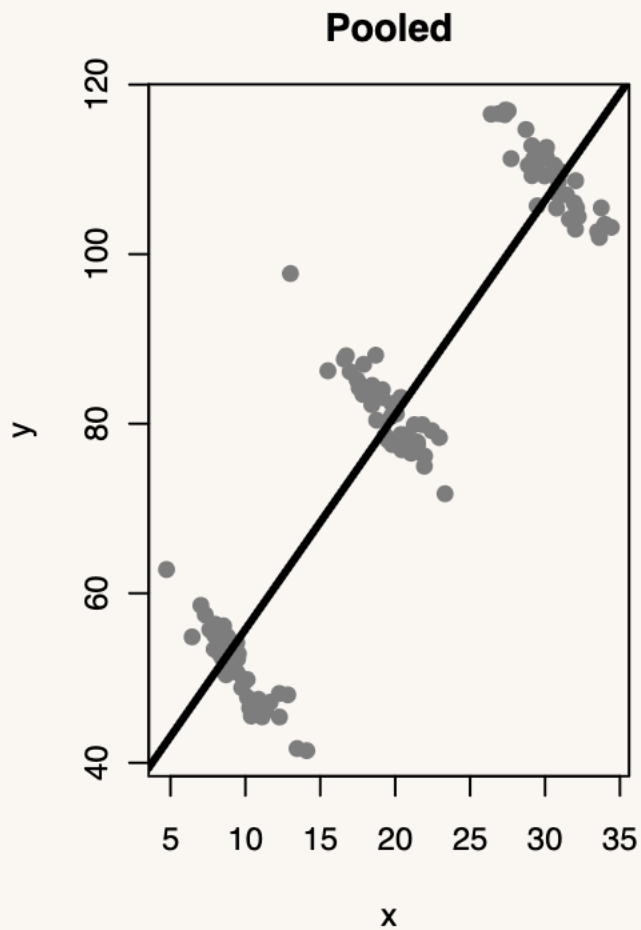
- "If I give this patient treatment A, what happens?"—you want the subgroup rate for that patient's stone size. A is better.
- "If I switch the whole hospital from B to A tomorrow, keeping the current triage practice, what happens to the overall cure rate?"—the pooled number is closer to relevant, precisely because it bakes in the case mix.

The data cannot tell you which question you meant. Only you can, and you must do it before you look, or you will pick the split that gives you the answer you wanted and call it an analysis.

It happens with numeric variables too

The same reversal occurs on continuous data, where it will show up in your regressions later in the term.

R Code	
1	<code>set.seed(6101)</code>
2	<code>grp <- rep(1:3, each = 40)</code>
3	<code>center <- c(10, 20, 30)[grp]</code>
4	<code>x <- center + rnorm(120, 0, 2)</code>
5	<code>y <- 20 + 3 * center - 2 * (x - center) + rnorm(120, 0, 2)</code>
6	
7	<code>par(mfrow = c(1, 2), mar = c(4, 4, 3, 1))</code>
8	<code>plot(x, y, pch = 19, col = "gray50", main = "Pooled")</code>
9	<code>abline(lm(y ~ x), lwd = 3)</code>
10	<code>plot(x, y, pch = c(19, 17, 15)[grp], col = c("gray20", "gray45", "gray70")[grp],</code>
11	<code> main = "Within each group")</code>
12	<code>for (g in 1:3) abline(lm(y[grp == g] ~ x[grp == g]), lwd = 3)</code>



R Code

```
1 slope <- function(xx, yy) unname(coef(lm(yy ~ xx))[2])
2
3 round(c(pooled = slope(x, y),
4         group1 = slope(x[grp == 1], y[grp == 1]),
5         group2 = slope(x[grp == 2], y[grp == 2]),
6         group3 = slope(x[grp == 3], y[grp == 3])), 2)
7 round(cor(x, y), 2)
```

```
pooled group1 group2 group3
2.53 -2.21 -1.98 -1.89
[1] 0.92
```

Pooled, y rises with x at a slope of about $+2.5$, and $r = 0.92$. Within every single group, y falls with x at a slope of about -2 . Neither is a computational error. The groups sit at different heights and at different x positions, so the pooled line is tracking the difference *between* groups while the within-group lines track what happens *inside* one.

If those groups are hospitals, schools, or years, the difference between those two slopes is often the entire finding.

A university reports that men were admitted at a higher rate than women overall. Broken down by department, women were admitted at a higher rate in **every single department**.

Both sets of numbers were computed correctly from the same applications.

Which statement best explains this?

- ☐ One of the two calculations must contain an error, since a comparison cannot reverse itself.
- ☐ The departments have different admission rates, and men and women applied to them in different proportions—so the pooled rates are weighted averages over different mixes of departments.
- ☐ The overall figure is more reliable because it uses every applicant, so the department-level results should be disregarded.
- ☐ The reversal proves that the departments are discriminating against men.



Exercise: undoing the reversal

Exercise

```
1 # Treatment B was given to 270 small-stone and 80 large-stone patients.
2 # Compute what TREATMENT B's overall cure rate would have been if B had
3 # been given to A's mix of patients instead (87 small, 263 large),
4 # keeping B's own size-specific cure rates.
5 # Then compare it in your head with A's actual pooled rate of 0.780.
6 -----
```

Please replace ----- with valid code.

Exercise: which number goes in the recommendation

Same trial. A cures 93.1% of small stones and 73.0% of large ones.
B cures 86.7% of small and 68.8% of large.
Pooled, A cures 78.0% and B cures 82.6%.

A surgeon has a patient in front of her with a large stone,
and asks which treatment to use.

Exercise

```
1 # Which answer is correct? Assign the letter, for example: answer <- "A"
2 #
3 # A. B, because B has the higher overall cure rate across all patients.
4 # B. A, because among large-stone patients A cured 73.0% against B's 68.8%.
5 # C. Either, since the two comparisons contradict each other and the data
6 # cannot settle it.
7 # D. B, because the pooled figure uses all 700 patients and is therefore
8 # based on more evidence.
9 answer <- -----
```

Please replace ----- with valid code.

When there are too many points

Every rule so far has been about a chart that shows too little honestly. This one is about a chart that shows too much and so shows nothing: two hundred thousand points plotted as a scatter, arriving as a solid black blob.

Four fixes get named, and they are all correct. What matters is that **they fail in opposite directions**, so you have to know what you want from the plot before you pick one.

	WHAT IT COSTS
Transparency (<code>alpha</code>)	Saturates. At <code>alpha = 0.01</code> , 500 overlapping points are 0.993 opaque and 5,000 are 1.000—the same colour. It shows you <i>where</i> data is, never <i>how much</i> .
Sampling	Plotting 5,000 of 200,000 keeps roughly 4% of a rare arm of the distribution. Sampling is a rare-event filter; it removes outliers by construction.
Hexbin	Keeps every point, but a bin holding one point is 1/1757 of the busiest bin—invisible on a linear colour ramp.
Faceting	Costs nothing, but only helps if the blob is several groups superimposed. One population facets into the same blob four times.

So: **for the bulk relationship**, any of them works. **For the outliers**, the first three destroy exactly what you are looking for. The usual best answer is a hexbin on a log colour scale *plus* the extreme points drawn as a second layer.

The log colour scale

A hexbin’s counts here run from 1 to about 1757. Because most of that *numeric* range sits at the top, a linear colour ramp spends itself there:

SHARE OF THE COLOUR RAMP GIVEN TO COUNTS 1-10	
linear	0.57%
<code>transform = "log"</code>	31%

```
ggplot(d, aes(x, y)) +
  geom_hex(bins = 60) +
  scale_fill_continuous(transform = "log")
```

The transformation is applied to the **scale**—the mapping from a count to a colour—and not to the data. Note what it does and does not do: $\log(1) = 0$, so a single-point bin still sits at the very bottom of the ramp. Log does not make rare bins *bright*; it makes them *distinguishable from each other*, so 1, 5 and 20 stop being one indistinguishable shade. If you want single-point bins to read as a distinct colour, `transform = "log1p"` is the one that does it.

(The argument was called `trans` before ggplot2 3.5.0. Both still work; `transform` is the current name.)

THE ANSWER ALMOST NOBODY GIVES

If the question is “how does *y* depend on *x*”, **stop plotting raw points**. Bin *x*, and plot the conditional mean with a ribbon for the spread. That is what the regression is doing anyway, and it answers the question the scatterplot was being asked.

And listen for yourself saying “*turn the transparency up until it looks right.*” No value of alpha separates 500 overlapping points from 5,000—the blending is lossy above a certain density. This chapter’s lesson, one more time: **the picture is a summary too**, and this one is hiding a factor of ten.

A short list of the rest

The same reasoning, applied quickly to other common choices:

- **Color.** Roughly one man in twelve has some form of red-green color deficiency. A red-versus-green comparison is invisible to them. Vary lightness as well as hue, or use shape, or label directly.
- **Pie charts.** People compare angles badly and lengths well. Almost any pie chart is a better bar chart. The exception is one slice against the rest, where the whole is genuinely the point.
- **Sorting.** Bars in alphabetical order are almost never useful. Bars sorted by value make the comparison you presumably wanted the reader to make—but note that sorting is itself an argument about what matters.
- **Overplotting.** With 50,000 points a scatterplot is a black rectangle and you cannot see density at all. Use transparency, or smaller points, or bin into a 2D histogram.
- **What is missing.** A time series that starts in 2015 makes a claim by omission if 2014 was the peak. So does a bar chart of six countries out of thirty. Ask what a reader would want to see that is not there, and ask it of your own figures before somebody else does.

🔗 THE CHART AS AN ARGUMENT



You plot monthly server response times for the past two years and the line looks alarmingly steep. You then widen the figure to the full page width and the same line looks nearly flat. The data, the axis limits, and the code are unchanged.

What is the right response?

- ☐ Ship the narrow version—it makes the problem visible, and visibility is the point of a chart.
- ☐ Ship the wide version, because wider figures are more accurate.
- ☐ Neither on its own. Decide in words what you are claiming about the size of the change, check that claim numerically, and then choose an aspect ratio that shows it—stating the numbers alongside the figure.
- ☐ Add a trend line; the fitted slope removes the ambiguity, since a regression coefficient does not depend on the figure's shape.



Summary

- A chart is a sentence about the data, not a photograph of it. Ask what claim a reader will leave with, and whether you would write that claim in words.
- **Bars must start at zero**, because bars encode length. Lines need not, because lines encode change. The test that covers both: ask whether the reader is meant to read a quantity off how long a mark is. If yes, the baseline is zero. If no, choose a range and label it.
- **Aspect ratio changes slope.** Resize the figure and see whether your sentence survives.
- **Bin width changes shape.** Too few bins hide structure; too many manufacture it. Try several, and know that you chose one.
- **Dual axes let you place the crossing point wherever you like.** Use two panels, index both series, or plot the ratio you actually mean.
- **Aggregation can reverse a comparison** when the groups have different compositions. Know whether your question is about an individual case or about a whole population before you choose the split—and consider standardizing.
- None of these is a trick you avoid. They are decisions you make. Make them on purpose, and say what you made.

Next: some problems to argue about before class.

This is the longest section of the week and the one most likely to feel like drudgery. I want to reframe it before we start.

Cleaning is not preparation for the analysis. Cleaning **is** analysis—it is a sequence of decisions about what your data means, each of which changes your answer, and each of which you are responsible for. Deciding that `-999` is missing rather than a very low blood pressure is a decision. Deciding that `north` and `North` are the same site is a decision. Deciding to keep only the rows that matched in a join is a decision. None of them are announced by R, all of them move your final number, and the ones you make without noticing are the ones that end up in a correction notice.

Throughout this section, one small messy table:

R Code					
1	<code>clinic <- data.frame(</code>				
2	<code> patient = c("A", "B", "C", "D", "E", "F"),</code>				
3	<code> site = c("north", "North ", "", "south", "SOUTH", NA),</code>				
4	<code> bp = c(118, 132, -999, 145, 127, -999),</code>				
5	<code> visits = c(2, 3, NA, 1, 4, 2),</code>				
6	<code> stringsAsFactors = FALSE</code>				
7	<code>)</code>				
8	<code>clinic</code>				

	patient	site	bp	visits
1	A	north	118	2
2	B	North	132	3
3	C		-999	NA
4	D	south	145	1
5	E	SOUTH	127	4
6	F	<NA>	-999	2

Six rows. Four separate problems.

Three kinds of nothing

The single most useful idea in this section is that “missing” is not one thing in a data file. There are at least three, and R treats them very differently.

`NA` is R’s actual missing value. It propagates: any arithmetic involving `NA` is `NA`, and functions like `mean` refuse to give an answer unless you tell them what to do about it. That refusal is a feature. It is R declining to make a decision on your behalf.

`"`, the empty string, is a perfectly ordinary character value that happens to have zero characters in it. It is not missing. `is.na("")` is `FALSE`. It counts as a category in `table()`. It is the fifth-most-common “site” in your study.

A **sentinel** like `-999`, `-99`, `9999`, or `1900-01-01` is a real number that somebody’s software wrote to mean “not recorded”, usually because the file format had no way to represent missingness. It is not missing either. It is a number, and R will happily average it.

R Code	
1	<code>sum(is.na(clinic\$bp))</code>
2	<code>mean(clinic\$bp)</code>

<code>[1] 0</code>
<code>[1] -246</code>

Zero missing values in `bp`, according to R, and a mean blood pressure of `-246`. The first number is a lie you were told by your own code, and the second is what happens when you believe it.

The tool that catches this is `summary`, because it prints the range:

R Code	
1	<code>summary(clinic\$bp)</code>

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
-999.0	-719.8	122.5	-246.0	130.8	145.0

A minimum of -999 on a blood pressure variable is not subtle. Once you know, you convert the sentinel to a genuine `NA` and *then* compute, and the honest version reports how many observations it used:

R Code

```
1 bp <- clinic$bp
2 bp[bp == -999] <- NA
3 c(mean = mean(bp, na.rm = TRUE), n = sum(!is.na(bp)))
```

```
mean      n
130.5     4.0
```

130.5, over four patients. Not -246 over six. And notice the second number in that output: “mean 130.5” alone is not a result, because it does not say what it is a mean *of*. `na.rm = TRUE` silently discards observations, and a report that uses it without saying how many were discarded has hidden something.

`NA.rm = TRUE` IS A DECISION, NOT A FIX

It is not wrong—usually it is what you want—but it changes what your number means. `mean(x, na.rm = TRUE)` is the average among those we observed, and it equals the average of everyone only if missingness is unrelated to the value. Blood pressures are often missing precisely for the patients who were too unwell to be measured. Whenever you type `na.rm = TRUE`, ask who is missing and whether they are missing for a reason connected to the thing you are measuring.

EXERCISE: THE HONEST MEAN

`clinic$bp` uses -999 for “not recorded”. Return the mean blood pressure over the real measurements only.

Exercise

```
1 # Convert the -999 sentinels to NA, then take the mean of what is left.
2 bp <- clinic$bp
3 _____
```

Please replace _____ with valid code.

`""` is not missing

Now the `site` column.

R Code

```
1 is.na(clinic$site)
```

```
[1] FALSE FALSE FALSE FALSE FALSE TRUE
```

One `NA`, in row F. Row C, whose site is `""`, is reported as present. That is correct—`""` is a string—and it is also not what you meant. If you tabulate sites, the empty string gets its own column and quietly becomes a category:

R Code

```
1 table(clinic$site, useNA = "ifany")
```

	North	SOUTH	north	south	<NA>
	1	1	1	1	1

Five distinct non-missing values, plus one real `NA`, for what is really two sites. `north` and `North` differ by a capital and a trailing space; `south` and `SOUTH` differ by case; and the empty string sits at the far left of that table masquerading as a site called nothing.

TABLE() HIDES NA UNLESS YOU ASK

By default, `table()` drops `NA` entirely and does not mention that it did. So does `prop.table()` on its output, which means your percentages are percentages of the non-missing—computed on a different denominator than you think. `useNA = "ifany"` is worth typing every single time.

The standard repair for text categories is: trim whitespace, unify case, and promote the empties to real `NA` s.

R Code

```
1 site <- tolower(trimws(clinic$site))
2 site[site == ""] <- NA
3 table(site, useNA = "ifany")
```

site	north	south	<NA>
	2	2	2

Two sites and two missing values, which is the truth. `trimws` removes leading and trailing whitespace; `tolower` unifies case. Notice the order does not matter here but the *combination* does—either one alone leaves you with three categories.

EXERCISE: COUNT THE USABLE SITES

How many rows of `clinic` have a site value that is genuinely usable—not `NA`, and not blank or whitespace-only?

Exercise

```
1 # Count rows whose site is neither NA nor blank.
2 -----
```

Please replace `-----` with valid code.

Type coercion, and the factor trap

R's atomic vectors hold one type. When you combine types, R does not complain; it finds the most permissive type that can hold everything and converts silently.

R Code

```
1 c(class(c(1, 2, 3)), class(c(1, 2, TRUE)), class(c(1, 2, "3")))

[1] "numeric" "numeric" "character"
```

`TRUE` quietly became the number 1, and then a single string turned the whole thing into text. The hierarchy is logical → integer → double → character, and R always slides to the right, never to the left. A single string anywhere in a column turns the whole column into text. That is the mechanism behind the `N/A` problem in [Importing Data](#), and it is also what happens when someone types “n/a” or “see note” into one cell of a spreadsheet with 40,000 rows.

The sharper trap involves factors. A **factor** stores categories as integer codes plus a table of levels. If a numeric column accidentally becomes a factor—which happens with old code, with `stringsAsFactors = TRUE`, and with several data-reading

packages—then converting it back is where people get hurt.

R Code	
1	<code>raw <- factor(c("12", "7", "23", "7"))</code>
2	<code>raw</code>
3	<code>levels(raw)</code>
<pre>[1] 12 7 23 7 Levels: 12 23 7 [1] "12" "23" "7"</pre>	

Two things to notice. The levels are sorted *as text*, so "12" comes before "23" before "7". And the underlying storage is 1, 2, 3—positions in that list. So:

R Code	
1	<code>as.numeric(raw)</code>
<pre>[1] 1 3 2 3</pre>	

1 3 2 3. Not 12 7 23 7. R has given you the level codes, and it has given them to you without a warning, because from R's point of view you asked for the numeric representation of a factor and that is what a factor's numeric representation is.

The values 1 3 2 3 are numbers. They have a mean. They will go into a regression. They will produce coefficients and p-values, and every one of those numbers will be about the alphabetical ordering of your labels.

R Code	
1	<code>as.numeric(as.character(raw))</code>
<pre>[1] 12 7 23 7</pre>	

`as.character` first recovers the labels, and *then* `as.numeric` parses them. This detour is one of the most-repeated pieces of advice in R, and this is why.

EXERCISE: RESCUE THE NUMBERS

`scores` is a factor that should have been numeric. Return the sum of the actual values.

Exercise	
1	<code># The sum of the real values 12 + 7 + 23 + 7.</code>
2	<code>_____</code>

Please replace `_____` with valid code.

Duplicates

Duplicates come in two flavours and they need different treatment.

R Code	
1	<code>orders <- data.frame(</code>
2	<code> order_id = c(101, 102, 102, 103, 104, 104),</code>
3	<code> customer = c("ana", "ben", "ben", "cy", "dee", "dee"),</code>
4	<code> amount = c(20, 35, 35, 12, 50, 55),</code>
5	<code> stringsAsFactors = FALSE</code>

```
6 )
7 orders
```

	order_id	customer	amount
1	101	ana	20
2	102	ben	35
3	102	ben	35
4	103	cy	12
5	104	dee	50
6	104	dee	55

Six rows. Order 102 appears twice, identically—almost certainly an export or join artifact. Order 104 also appears twice, but with **different amounts**, 50 and 55, which is not an artifact; it is a data quality problem, and deleting one of them at random is a decision about which number is true.

R Code

```
1 c(rows = nrow(orders),
2   fully_duplicated = sum(duplicated(orders)),
3   duplicate_ids     = sum(duplicated(orders$order_id)))
```

	rows	fully_duplicated	duplicate_ids
	6	1	2

`duplicated()` returns `TRUE` for every occurrence after the first. On the whole data frame it finds one row (the second 102). On the key alone it finds two, because 104 is repeated as a key even though the rows differ.

R Code

```
1 unique(orders)
```

	order_id	customer	amount
1	101	ana	20
2	102	ben	35
4	103	cy	12
5	104	dee	50
6	104	dee	55

`unique()` on the whole frame removes exact repeats and leaves five rows, which is safe: nothing was lost. Deduplicating on the key is different:

R Code

```
1 orders[!duplicated(orders$order_id), ]
```

	order_id	customer	amount
1	101	ana	20
2	102	ben	35
4	103	cy	12
5	104	dee	50

Four rows—and the 104 that survived is the one with `amount = 50`, purely because it appeared first in the file. Nobody decided that. The file's row order decided it.

THE RIGHT MOVE WHEN KEYS REPEAT WITH DIFFERENT DATA

Do not silently take the first. Find out why. Are these two genuine orders that were assigned the same id? Two versions of one order, of which the later is correct? A join upstream that inflated the table? Each has a different fix, and `!duplicated(key)` implements none of them—it implements “trust the file's sort order”, which is a decision you would never make deliberately.

EXERCISE: HOW MANY DISTINCT ORDERS?

Return the number of rows left after keeping only the first row for each `order_id`.

Exercise	
1	# Number of rows after deduplicating on order_id.
2	-----

Please replace ----- with valid code.

Wide and long

The same data can be laid out two ways, and which one you want depends entirely on what you are about to do.

Wide puts one row per unit and one column per time point or measurement:

R Code	
1	wide <- data.frame(
2	city = c("Austin", "Boston", "Chicago"),
3	y2024 = c(3.1, 2.8, 4.0),
4	y2025 = c(3.4, 2.9, 4.2),
5	stringsAsFactors = FALSE
6	)
7	wide

```

      city y2024 y2025
1  Austin   3.1   3.4
2  Boston   2.8   2.9
3 Chicago   4.0   4.2

```

Long puts one row per observation, with the thing that used to be a column name now a value in its own column:

R Code	
1	long <- reshape(
2	wide,
3	direction = "long",
4	varying = c("y2024", "y2025"),
5	v.names = "rate",
6	timevar = "year",
7	times = c(2024, 2025),
8	idvar = "city"
9	)
10	row.names(long) <- NULL
11	long

```

      city year rate
1  Austin 2024   3.1
2  Boston 2024   2.8
3 Chicago 2024   4.0
4  Austin 2025   3.4
5  Boston 2025   2.9
6 Chicago 2025   4.2

```

Same six numbers, six rows instead of three. `reshape` is base R's tool for this and its argument names take some getting used to: `varying` names the columns that are really repeated measurements, `v.names` is what the stacked values should be called, `timevar` and `times` name and fill the new column that records which measurement each row is, and `idvar` says what identifies a unit. (`row.names(long) <- NULL` is cosmetic—`reshape` builds row names like `Austin.2024`, which are harmless and ugly.)

Going back is the same function with a different `direction`:

R Code	

```

1 back <- reshape(long, direction = "wide", idvar = "city",
2                 timevar = "year", v.names = "rate")
3 row.names(back) <- NULL
4 back

```

	city	rate.2024	rate.2025
1	Austin	3.1	3.4
2	Boston	2.8	2.9
3	Chicago	4.0	4.2

Why the shape matters

Wide is for humans. It is compact, it fits on a slide, and comparing 2024 to 2025 for one city is a glance across a row.

Long is for computers, and specifically for anything that has to treat “year” as a variable. Group by year, plot rate against year, fit a model with year as a predictor—every one of those needs a column called `year`, and in the wide layout there isn’t one, because the year is hiding in the column *names*, where no function can reach it.

The tell that you need long format: you find yourself writing the same operation once per column, or naming columns in a formula. In long format:

R Code

```

1 tapply(long$rate, long$year, mean)

```

	2024	2025
	3.3	3.5

One call, and it would still be one call with forty years instead of two.

THE TIDYVERSE VERSION

You will hear the terms **pivot longer** and **pivot wider**, from the `tidyr` package, which does the same job with much friendlier arguments (`pivot_longer(wide, cols = c(y2024, y2025), names_to = "year", values_to = "rate")`). We will meet it later in the course. It is genuinely nicer, and it does not do anything `reshape` cannot—so the concept you are learning here transfers unchanged, which is the point of learning it here.

EXERCISE: A SUMMARY THAT NEEDS THE LONG SHAPE

`long` is already built for you. Return the mean rate for each year.

Exercise

```

1 # Mean rate per year. One call.
2 -----

```

Please replace `-----` with valid code.

Joining two tables

Almost every real project needs data from more than one file, and joining them is where rows go to die quietly.

R Code

```

1 sales <- data.frame(
2   store = c("S1", "S2", "S3", "S4"),
3   units = c(10, 25, 7, 14),
4   stringsAsFactors = FALSE

```

```

5   )
6   regions <- data.frame(
7     store = c("S1", "S2", "S5"),
8     region = c("east", "west", "north"),
9     stringsAsFactors = FALSE
10  )
11  merge(sales, regions, by = "store")

```

```

  store units region
1    S1     10  east
2    S2     25  west

```

Two rows. You started with four stores and you have two, and R said nothing at all, because `merge` defaults to an **inner join**: keep a row only if its key appears in both tables. `S3` and `S4` have no region; `S5` has no sales; all three are gone.

Now the part that should make you uneasy:

R Code

```

1  j <- merge(sales, regions, by = "store")
2  c(units_before = sum(sales$units), units_after = sum(j$units))

```

```

units_before  units_after
           56           35

```

Total units fell from 56 to 35. If the next line of your report is “total units sold were 35,000”, you have just published a number that is 37% too small, and nothing in your session told you so.

The other join types are one argument away:

R Code

```

1  merge(sales, regions, by = "store", all.x = TRUE)

```

```

  store units region
1    S1     10  east
2    S2     25  west
3    S3       7 <NA>
4    S4     14 <NA>

```

`all.x = TRUE` is a **left join**: keep every row of the first table, fill `region` with `NA` where there is no match. All four stores survive, and the `NA`s are visible—they are the point. This is usually what you want when one table is your data and the other is a lookup.

R Code

```

1  merge(sales, regions, by = "store", all = TRUE)

```

```

  store units region
1    S1     10  east
2    S2     25  west
3    S3       7 <NA>
4    S4     14 <NA>
5    S5      NA north

```

`all = TRUE` is a **full join**: every key from either side, five rows, with `NA` where a side had nothing. `S5` appears with `NA` units, which tells you your region table knows about a store your sales table has never heard of—often a genuine finding about your pipeline.

The other direction: rows multiplying

The opposite failure is less famous and just as damaging. If the key repeats in the *second* table, rows multiply.

R Code

```

1  prices <- data.frame(
2    store = c("S1", "S1", "S2"),

```

```

3 plan = c("basic", "premium", "basic"),
4 price = c(5, 9, 6),
5 stringsAsFactors = FALSE
6 )
7 merge(sales, prices, by = "store")

```

```

  store units plan price
1    S1    10 basic     5
2    S1    10 premium    9
3    S2    25 basic     6

```

S1 now appears twice, and units = 10 appears twice with it. Sum units on this table and you have counted store S1's sales once for each price plan it offers. The rule: merge gives you every combination of matching rows, so a key that is unique on the left and duplicated on the right inflates the left table's numbers.

Keys that do not match for stupid reasons

R Code

```

1 a <- data.frame(store = c("S1", "S2 "), units = c(1, 2), stringsAsFactors = FALSE)
2 b <- data.frame(store = c("S1", "S2"), region = c("e", "w"), stringsAsFactors = FALSE)
3 merge(a, b, by = "store")

```

```

  store units region
1    S1     1     e

```

One row. "S2 " and "S2" are different strings. The same happens with case ("s2"), with a zip code that is "02134" in one file and 2134 in the other, and with an id read as numeric in one file and character in the other. Clean your keys— trimws , tolower , and a deliberate as.character —before you join, not after you notice.

THE JOIN CHECKLIST

Every single time you join, in three lines:

1. nrow() of each input, and sum(duplicated(x\$key)) on each—if the key is not unique on at least one side, you will inflate.
2. The join.
3. nrow() of the output, compared with what you expected. For a left join it should equal the left table's row count. If it is smaller, something went wrong; if it is larger, the right-hand key was not unique.

Then colSums(is.na(result)) to see how many rows failed to match. Ten seconds, and it converts the most common silent failure in data science into something you can see.

EXERCISE: COUNT THE CASUALTIES

How many rows of sales are lost by the default merge with regions?

Exercise

```

1 # Rows of `sales` that do not survive merge(sales, regions, by = "store").
2 -----

```

Please replace ----- with valid code.

complete.cases and the shrinking sample

One last tool, and one last warning about it.

R Code

```
1 complete.cases(clinic)

[1] TRUE TRUE FALSE TRUE TRUE FALSE
```

Rows C and F are incomplete—C is missing `visits`, F is missing `site`. But look at what `complete.cases` says about rows C and F's blood pressure:

R Code

```
1 sum(complete.cases(clinic[, c("bp", "visits")]))

[1] 5
```

Five of six rows are “complete” on `bp` and `visits`, even though two of those five have a blood pressure of `-999`. `complete.cases` looks for `NA` and nothing else. It cannot see a sentinel, an empty string, or an impossible value.

That is the shape of this whole section. R's missingness tools are excellent, and they only work on missingness you have already told R about. Everything before this point—the sentinel conversion, the empty strings, the type fixes—exists so that by the time you reach `complete.cases`, it is telling you the truth.

🏆 CHECK YOURSELF



A clinic file records blood pressure, and the extraction tool wrote `-999` wherever the reading was not taken. You did not know that. Which single check, run before you computed anything, would most likely have caught it?

- ☐ `sum(is.na(clinic$bp))`
- ☐ `complete.cases(clinic)`
- ☐ `summary(clinic$bp)`
- ☐ `nrow(clinic)`



🏆 AND ONE MORE



`sales` has one row for each of four stores, `S1` through `S4`. `regions` has three rows, for `S1`, `S2`, and `S5`. You run `merge(sales, regions, by = "store")`. What do you get, and what does R tell you?

- ☐ Four rows; `S3` and `S4` are kept with `NA` in the `region` column.
- ☐ Two rows; R prints a warning that two rows were dropped.
- ☐ Two rows; R prints nothing at all.
- ☐ Five rows; every store from either table appears once.



A cleaning log

The last thing, and it takes two minutes per project. Keep a list, in your README or at the top of your script, of every decision you made in this section:

- *bp*: -999 treated as missing (142 rows affected)
- *site*: trimmed, lower-cased; 38 empty strings converted to NA
- *orders*: 4 exact duplicate rows removed; 2 ids repeat with conflicting amounts, kept the later timestamp
- joined to *regions* on *store*; 2 of 4 stores unmatched, kept via left join

Every line of that is a judgment call that changed your answer, and a reader who disagrees with one of them can now say so. That is what separates an analysis from an assertion—and in this course it is also a substantial part of what your project report is graded on.

Where this goes next

Practice Problems is a set of problems to bring to class. Most of them are code that runs perfectly and gives the wrong answer.

Importing looks like the least interesting thing in this course. One function call, one line, done. In practice it is where a surprising share of wrong analyses are born, because `read.csv` makes a series of *guesses* about your file—where the header is, what separates the fields, what each column's type is, what counts as missing—and it does not tell you what it guessed. It just hands you a data frame that looks fine.

The discipline of this section is short and it is not negotiable: **after every import, look at what you got, before you use it.**

ABOUT THE CODE ON THIS PAGE

The cells here run R in your browser, and the browser has no access to your hard drive. So instead of reading a file that already exists, each example *writes* a small file into a temporary location and then reads it back. That keeps the `read.csv` calls completely honest—they really are parsing text out of a file—while letting you edit the file's contents and re-run to see what changes. `tempfile()` invents a filename nobody else is using; `writeLines()` writes the lines you give it.

The simplest possible import, and what it already got wrong

Here is a small file and a plain `read.csv`.

R Code	
1	<code>f <- tempfile(fileext = ".csv")</code>
2	<code>writeLines(c(</code>
3	<code> "id,city,population,median_income",</code>
4	<code> "007,Austin,961855,86556",</code>
5	<code> "008,Boston,653833,89212",</code>
6	<code> "009,Chicago,2664452,71673"</code>
7	<code>), f)</code>
8	
9	<code>d <- read.csv(f)</code>
10	<code>d</code>
<pre> id city population median_income 1 7 Austin 961855 86556 2 8 Boston 653833 89212 3 9 Chicago 2664452 71673</pre>	

That looks perfect. It is not.

R Code	
1	<code>str(d)</code>
<pre>'data.frame': 3 obs. of 4 variables: \$ id : int 7 8 9 \$ city : chr "Austin" "Boston" "Chicago" \$ population : int 961855 653833 2664452 \$ median_income: int 86556 89212 71673</pre>	

`id` came in as `int`, and the values are `7`, `8`, `9`. The leading zeros are gone, permanently, and R did not consider this worth mentioning. If those ids are keys you later join on, or zip codes, or account numbers, or anything else where `007` and `7` are different strings in some other system, the damage is done at line one and shows up in week four as a join that silently drops rows.

The general shape of the problem: `read.csv` **infers the type of each column from its contents**. If every value in a column parses as a number, you get a number, and any information carried by the *text form*—leading zeros, a distinction between `01` and `1`, a phone number's formatting—is discarded.

You fix this by telling it what you want:

R Code

```
1 d <- read.csv(f, colClasses = c(id = "character"))
2 str(d)
3 d$id
```

```
'data.frame':  3 obs. of  4 variables:
 $ id      : chr  "007" "008" "009"
 $ city    : chr  "Austin" "Boston" "Chicago"
 $ population: int   961855 653833 2664452
 $ median_income: int   86556 89212 71673
[1] "007" "008" "009"
```

`colClasses` accepts a named vector, so you only have to name the columns you care about; everything else is still inferred. It is worth using for every identifier column in every file, as a reflex.

What `read.csv` actually does

`read.csv` is `read.table` with some defaults pre-set. The ones worth knowing:

ARGUMENT	DEFAULT IN <code>read.csv</code>	WHAT IT CONTROLS
<code>header</code>	<code>TRUE</code>	Is the first line column names?
<code>sep</code>	<code>","</code>	Field separator
<code>quote</code>	<code>"\""</code>	What wraps a field containing a comma
<code>dec</code>	<code>."</code>	Decimal mark
<code>na.strings</code>	<code>"NA"</code>	Which text becomes <code>NA</code>
<code>stringsAsFactors</code>	<code>FALSE</code>	Do text columns become factors?
<code>strip.white</code>	<code>FALSE</code>	Trim spaces around unquoted fields
<code>comment.char</code>	<code>" "</code>	Text after this is ignored
<code>check.names</code>	<code>TRUE</code>	Make column names syntactically valid
<code>colClasses</code>	<code>NA</code>	Force column types
<code>skip</code>	<code>0</code>	Lines to discard before the header
<code>nrows</code>	<code>-1</code>	Stop after this many rows
<code>fileEncoding</code>	<code>" "</code>	Character encoding of the file

Two of these deserve their own paragraphs.

`stringsAsFactors`

Before R 4.0.0, released in April 2020, this defaulted to `TRUE`, and every text column silently became a **factor**—R’s type for a categorical variable, stored internally as integer codes plus a lookup table of levels. This caused so much grief that changing the default was one of the headline items in that release.

You are on a modern R, so text stays text and you may reasonably wonder why this matters. It matters for two reasons. First, a great deal of code and a great many Stack Overflow answers on the internet were written before 2020, and an AI assistant trained on that corpus will occasionally hand you code that assumes the old behavior. Second, factors have not gone anywhere—you will create them deliberately, and `read.csv(f, stringsAsFactors = TRUE)` still exists—and the classic factor trap in [Cleaning and Reshaping](#) is one of the sharpest silent failures in R.

`check.names`

R makes column names syntactically valid by default, which means anything that is not a letter, digit, or dot becomes a dot, and names starting with a digit get an **X** in front.

```
R Code
1 h <- tempfile(fileext = ".csv")
2 writeLines(c(
3   "Median Income ($),% Uninsured,2020 Population",
4   "86556,12.1,961855",
5   "89212,3.5,653833"
6 ), h)
7
8 names(read.csv(h))

[1] "Median.Income...." "X..Uninsured"      "X2020.Population"
```

`Median Income ($)` has become `Median.Income....`, which is ugly but usable. The reason to know about it is that `check.names = FALSE` preserves the originals, and you then have to refer to them with backticks, as in `d$`Median Income ($)``, which is why most people accept the mangling and rename the columns explicitly instead:

```
R Code
1 d3 <- read.csv(h)
2 names(d3) <- c("median_income", "pct_uninsured", "population_2020")
3 str(d3)

'data.frame':  2 obs. of  3 variables:
 $ median_income  : int  86556 89212
 $ pct_uninsured  : num  12.1 3.5
 $ population_2020: int  961855 653833
```

Renaming to short, lower-case, underscore-separated names on import is a habit worth forming. It costs one line and it makes the next three weeks pleasant.

What a messy file actually looks like

Real exports are not four tidy lines. Here is a file of the sort that arrives from a survey platform or an agency portal: a couple of provenance lines above the header, a missing value written as `N/A`, and an empty cell.

```
R Code
1 g <- tempfile(fileext = ".csv")
2 writeLines(c(
3   "# City satisfaction survey, export v3.1",
4   "# generated 2026-08-14 by portal-svc",
5   "city,respondents,mean_score",
6   "Austin,142,4.1",
7   "Boston,98,N/A",
8   "Chicago,201,3.7",
9   "Denver,,4.4"
10 ), g)
11
12 bad <- read.csv(g)
13 dim(bad)
14 names(bad)

[1] 6 2
[1] "X..City.satisfaction.survey" "export.v3.1"
```

Read that again. No error. No warning. A data frame with six rows and two columns, and a straight face.

Here is what happened. `comment.char` is `"` in `read.csv`, so `#` is not special—it is just a character. The first line therefore became the header, and because that line happens to contain a comma (“survey, export”), R concluded the file has **two** columns. The data rows have three fields each, so R decided the extra leading field must be row names, and quietly used the city names for that purpose.

R Code	
1	<code>head(bad)</code>
<pre>X..City.satisfaction.survey export.v3.1 # generated 2026-08-14 by portal-svc city respondents mean_score Austin 142 4.1 Boston 98 N/A Chicago 201 3.7 Denver 4 4.4</pre>	

Every value in the file is still on screen somewhere, which is what makes this so dangerous: nothing looks *empty*. It looks like data. If your next line had been `nrow(bad)` you would have got 6, which is even close to the truth.

This is the whole argument for the inspection ritual below. Skip the two junk lines and the file parses properly:

R Code	
1	<code>m <- read.csv(g, skip = 2)</code>
2	<code>str(m)</code>
<pre>'data.frame': 4 obs. of 3 variables: \$ city : chr "Austin" "Boston" "Chicago" "Denver" \$ respondents: int 142 98 201 NA \$ mean_score : chr "4.1" "N/A" "3.7" "4.4"</pre>	

Now look carefully at what you have.

- `respondents` is `int`, and Denver’s empty cell became `NA`. Correct: an entirely empty field in a numeric column is read as missing.
- `mean_score` is `chr`. It is text, because `N/A` is not in the default `na.strings` and so the column is not entirely numeric.

And here is the silent failure:

R Code	
1	<code>mean(m\$mean_score, na.rm = TRUE)</code>
R Warning in <code>`mean.default(m\$mean_score, na.rm = TRUE)`</code> Warning: argument is not numeric or logical: returning NA [[1] NA	

`NA`, with a warning. Trace it: there are no `NA` values in `mean_score`—it is text throughout—so `na.rm = TRUE` removes nothing. `mean` receives three strings, declines, and returns `NA`. The `na.rm = TRUE` you carefully typed was pure decoration.

Fix it at the point of import, which is where it belongs:

R Code	
1	<code>m2 <- read.csv(g, skip = 2, na.strings = c("NA", "N/A", "n/a", "", " "))</code>
2	<code>str(m2)</code>
3	<code>mean(m2\$mean_score, na.rm = TRUE)</code>

```
'data.frame': 4 obs. of 3 variables:
 $ city      : chr  "Austin" "Boston" "Chicago" "Denver"
 $ respondents: int   142  98  201  NA
 $ mean_score: num   4.1  NA  3.7  4.4
[1] 4.066667
```

Now the column is numeric, the missing value is genuinely `NA`, and `na.rm` has something to do. Note that I passed a *vector* of strings: real files frequently use several spellings of missing in the same column, because several different people exported them.

Delimiters that are not commas

CSV is a family, not a format. European exports often use `;` because the comma is the decimal mark; tab-separated files are everywhere in bioinformatics; and pipe-delimited files turn up in banking.

R Code

```
1 k <- tempfile(fileext = ".csv")
2 writeLines(c("city;pop", "Austin;961855", "Boston;653833"), k)
3 read.csv(k)
```

```
      city.pop
1 Austin;961855
2 Boston;653833
```

One column named `city.pop`, with values like `"Austin;961855"`. Nothing errored. If you had not looked, you would have a data frame with the right number of rows and one useless column.

R Code

```
1 read.csv(k, sep = ";")
```

```
      city      pop
1 Austin 961855
2 Boston 653833
```

R also ships `read.delim` (tab-separated) and `read.csv2` (semicolon separator, comma decimal mark) as pre-configured wrappers, but `read.csv` with an explicit `sep` and `dec` is clearer about what you assumed.

Encodings, briefly

A text file is bytes; an encoding is the agreement about which bytes mean which characters. When the agreement breaks you get `Ã©` where `é` should be, or a column name that starts with an invisible character and never matches `names(d) == "city"`.

Three things worth knowing.

UTF-8 is the modern default and what you should ask for whenever you control the export. `read.csv(f, fileEncoding = "UTF-8")` states it explicitly.

Excel on Windows writes a BOM. Files exported from Excel as “CSV UTF-8” begin with three invisible bytes marking the encoding. R reads them as part of the first column name, so `names(d)[1]` looks like `city` and is not equal to `"city"`. The cure is `fileEncoding = "UTF-8-BOM"`. If a name comparison is mysteriously failing on the very first column, this is why—check with `nchar(names(d)[1])`.

Older exports are often Latin-1. `fileEncoding = "latin1"` handles most Western European legacy files.

The diagnostic habit: if any text in your data frame looks wrong on screen, the problem is almost certainly encoding and almost certainly at import, not something to patch downstream with `gsub`.

The inspection ritual

Five calls, every time, before you trust anything.

R Code

```
1 big <- data.frame(  
2   id      = sprintf("%03d", 1:8),  
3   region = c("east", "west", "east", "north", "west", "east", "north", "west"),  
4   score  = c(4.1, 3.8, 4.4, 2.9, 3.3, 5.0, 3.6, -99),  
5   n      = c(142L, 98L, 201L, 55L, 130L, 88L, 74L, 160L),  
6   stringsAsFactors = FALSE  
7 )  
8  
9 dim(big)
```

```
[1] 8 4
```

R Code

```
1 str(big)
```

```
'data.frame':  8 obs. of  4 variables:  
 $ id   : chr  "001" "002" "003" "004" ...  
 $ region: chr  "east" "west" "east" "north" ...  
 $ score : num  4.1 3.8 4.4 2.9 3.3 5 3.6 -99  
 $ n     : int  142 98 201 55 130 88 74 160
```

R Code

```
1 head(big, 3)
```

```
   id region score  n  
1 001  east   4.1 142  
2 002  west   3.8  98  
3 003  east   4.4 201
```

R Code

```
1 summary(big)
```

id	region	score	n
Length:8	Length:8	Min. : -99.000	Min. : 55.0
Class :character	Class :character	1st Qu.: 3.200	1st Qu.: 84.5
Mode :character	Mode :character	Median : 3.700	Median :114.0
		Mean : -8.988	Mean :118.5
		3rd Qu.: 4.175	3rd Qu.:146.5
		Max. : 5.000	Max. :201.0

R Code

```
1 sapply(big, class)
```

```
   id      region      score      n  
"character" "character" "numeric" "integer"
```

What each one is for:

- `dim`—do you have the number of rows and columns the source claimed? A file that should have 50 states and has 51 has a “United States” total row in it, and every mean you compute will be wrong.
- `str`—the single most useful function in R. Types, and the first few values of each column. This is where you catch `chr` where you expected `num`.
- `head`—do the first rows look like data, or like a second header line?
- `summary`—ranges. This is where `-99` in the `score` column is visible as a minimum, and where you notice an age of 999 or a price of 0.
- `sapply(d, class)`—a compact list of types when there are too many columns for `str` to be readable.

`summary(big)` above shows `Min. :-99.00` for `score`, on a variable that is otherwise between 2.9 and 5.0. That is a sentinel value, it is Cleaning and Reshaping's subject, and you found it in four seconds because you looked.

Exercises

EXERCISE: GET PAST THE METADATA

The file `f` has two provenance lines above the header. Read it correctly and return its dimensions.

Exercise

```
1 # Fill in the argument that discards the two lines above the header.
2 d <- read.csv(f, _____)
3 dim(d)
```

Please replace `_____` with valid code.

EXERCISE: MAKE MISSING ACTUALLY MISSING

The file `f` writes missing scores as `N/A`. Read it so that `mean_score` comes back as a numeric column, and return its mean over the non-missing values.

Exercise

```
1 # Fill in the argument that makes "N/A" count as missing.
2 d <- read.csv(f, _____)
3 mean(d$mean_score, na.rm = TRUE)
```

Please replace `_____` with valid code.

EXERCISE: KEEP THE LEADING ZEROS

`f` holds a zip code column. Read it so the zip codes survive as written, and return the first one.

Exercise

```
1 # Fill in the argument that keeps `zip` as text.
2 d <- read.csv(f, _____)
3 d$zip[1]
```

Please replace `_____` with valid code.

A CSV has a column `score` whose three values are `4.1`, `N/A`, and `3.7`. You read it with `read.csv(f)` using every default, then run `mean(d$score, na.rm = TRUE)`. What happens?

- ☐ It returns `3.9`, silently and correctly.
- ☐ It returns `3.9` together with a warning that one value was dropped.
- ☐ It returns `NA` together with a warning that the argument is not numeric.
- ☐ It stops with an error, because a character column cannot be averaged.



Beyond CSV, in one paragraph

Base R reads delimited text and `.RData`; everything else needs a package. Excel files: **readxl**. Stata, SPSS and SAS: **haven**. JSON: **jsonlite**. Databases: **DBI** with a driver. Large CSVs where `read.csv` is too slow: **data.table**'s `fread`, or **readr**'s `read_csv`, which also declines to mangle your column names and prints the types it guessed. We will meet `readr` when we get to the tidyverse; for now, everything you need is in base R, and knowing what `read.csv` does with a file is the foundation for understanding what the fancier readers are doing differently.

Where this goes next

You have the file in R and you have looked at it. [Cleaning and Reshaping](#) is about everything you found when you looked.

Read this part after Monday, August 24. It carries no deadline and no reading questions—post whenever something does not come right, and it gets picked up in office hours or at the start of a session.

The [R Primer](#) covers the machinery—R, RStudio, Git, and what this field is for—and runs at your own pace alongside the course. This part is about the two things that actually decide whether a data science project is any good, and neither of them is a model.

The first is the question. Almost every project that goes badly went badly here, in the first week, when somebody wrote down something that sounded like a research question but was not one—because it could not be measured, or because no achievable dataset contained the thing it asked about, or because it had no particular answer that would count as *the* answer. You cannot fix that later with a better regression. [S.M.A.R.T. Questions](#) and [The Data Science Life Cycle](#) are about writing a question you can be wrong about, and about the fact that the work loops back and rewrites that question rather than marching forward from it.

The second is the data, in the condition it actually arrives in. Real files have a stray metadata line above the header. Real columns hold `-999` where a measurement was not taken, and `N/A` and `n/a` and an empty cell in the same column meaning three different things. Real categories are `north`, `North`, and `NORTH`. Real joins drop a third of your rows and tell you nothing. [Importing Data](#) and [Cleaning and Reshaping](#) are about getting a file into R and finding out what you actually got, before you trust it. [Where the Data Came From](#) then asks the question the mechanics cannot: who never made it into the file at all.

What is here

S.M.A.R.T. Questions	Turning an interest into a question data can answer
The Data Science Life Cycle	The loop, and where the time really goes
Importing Data	<code>read.csv</code> , types, encodings, and what a messy file looks like
Cleaning and Reshaping	Missingness, duplicates, coercion, shape, and joins
Where the Data Came From	Selection, survivorship, and missingness as a mechanism
Practice Problems	For discussion in class—not auto-graded

Why this part carries more weight than it looks like it does

A fifth of your grade is a team project, proposed and defended on exactly what [S.M.A.R.T. Questions](#) asks you to do. The proposal is one page and it is mostly a question. When a project comes apart, the autopsy nearly always ends at that page.

The other half of the week is the part nobody puts in the highlight reel. Surveys of working data scientists put the share of project time spent finding, importing, and cleaning data somewhere between half and three quarters. That number is repeated so often it has become a cliché, and it is still roughly right. If you feel, in [Cleaning and Reshaping](#), that you are doing tedious work rather than data science, the honest response is that this *is* data science, and the tedium is where most published mistakes come from.

A theme for the whole week

Watch for the **silent failure**: the code that runs, returns a number, prints no warning, and is wrong. A join that dropped half your rows. A mean computed over a column of text. A sentinel value averaged in as though `-999` were a blood pressure. R will not stop you. Errors are the easy case—an error is R doing you a favor. What you are training here is the instinct to check the things that do not announce themselves.

Textbooks draw the data science process as a row of boxes with arrows between them, left to right, and it looks like a factory line:

question → acquire → clean → explore → model → communicate

That picture is useful in exactly one way: it names the stages, and the stages are real. It is misleading in every other way, because the arrows go both directions and most of them go backwards. What actually happens is that you write a question, go looking for data, discover the data is not what you assumed, rewrite the question, clean what you have, discover during exploration that a whole subgroup is missing, rewrite the question again, model, find the model is answering something slightly different from what you asked, and write up the version of the question you ended with rather than the one you started with.

That is not a description of a project going wrong. That is a description of a project going *right*. The loop is the method.

The stages, honestly

Question

[S.M.A.R.T. Questions](#). The thing to add here is that the question is the only stage you will revisit deliberately rather than by accident. Every other stage feeds information back into it: acquisition tells you what exists, cleaning tells you how much of it is usable, exploration tells you whether the pattern you were going to test is even plausible.

Acquire

Getting the data. Downloading a CSV is the easy version; the realistic versions involve an API with rate limits, a government portal that hands you 51 separate files with slightly different column names, a PDF that a human was clearly expected to read, or a colleague's spreadsheet with the units in a merged cell.

Two things to record at this stage, always, and both take one minute: **where it came from** (URL, date accessed, version) and **what one row means**. The second sounds trivial until you have a file where you assumed one row was one patient and it turns out to be one patient-visit, at which point every count you have produced is wrong by a factor you cannot recover after the fact.

Clean

[Importing Data](#) and [Cleaning and Reshaping](#), and the majority of your time. Missing values, sentinel values, duplicates, inconsistent categories, types that came in wrong, two tables that need joining and disagree about their keys.

Explore

Summaries and pictures, before any modeling. What is the distribution of each variable? Which pairs move together? Is anything impossible—negative ages, prices of zero, a date in 1900? This is [exploratory analysis](#)'s subject, and it is also where cleaning problems that survived the cleaning stage announce themselves, which is one of the backward arrows.

Model

Fitting something: a test, a regression, a classifier. This is the part that looks like data science in the recruiting brochure, and it is usually one afternoon. It is also the part an AI assistant can now do for you in about fifteen seconds, which is exactly why the rest of the loop is where your value is.

Communicate

A report, a plot, a presentation, a number in a meeting. The stage where you say what you found *and* what you did not find, and where the honest sentence about the 78% missing variable goes.

Where the time actually goes

Ask a working data scientist how their week broke down and the modeling is a rounding error. The widely quoted figure is that something like 50–80% of project time goes to finding, importing, and cleaning data. That statistic has been repeated so often it has become a cliché, and the specific number is softer than people pretend, but the shape of it is right and matches what you will experience.

A rough and honest allocation for a five-week course project:

STAGE	SHARE OF YOUR TIME	WHAT IT FEELS LIKE
Question and re-question	10%	Frustrating, and the highest-leverage 10%
Acquire	10%	Tedious, occasionally impossible
Clean	40%	Tedious, and where the mistakes come from
Explore	20%	Enjoyable, and where you learn the most
Model	10%	Fast, and the part everyone budgets for
Communicate	10%	Always underestimated

If your team’s plan allocates four weeks to modeling and three days to cleaning, the plan is wrong, and it will be wrong in the specific way that produces a presentation on the Sunday night before it is due.

The week-three discovery

Here is the situation this section exists for.

It is week three. Your question is fixed, your teammates have divided the work, and you finally sit down with the file to fit the model. You need three variables: `price`, `sqft`, and `built`. Individually each one looks fine—each is missing for only two or three rows out of twelve. Then you ask how many rows have *all three*.

R Code

```
1 housing <- data.frame(  
2   id    = 1:12,  
3   price = c(310, 425, NA, 275, 500, 380, NA, 610, 295, 440, 350, NA),  
4   sqft  = c(1200, 1800, 1500, NA, 2400, 1650, 1300, 2900, 1100, 1950, NA, 1400),  
5   built = c(1998, NA, 2005, 1975, 2012, NA, 1988, 2018, 1960, 2001, 1994, 2007)  
6 )  
7 colSums(is.na(housing))
```

id	price	sqft	built
0	3	2	2

Three missing prices, two missing areas, two missing build years, out of twelve rows. Nothing alarming. Now the number that matters:

R Code

```
1 sum(complete.cases(housing[, c("price", "sqft", "built"))))
```

```
[1] 5
```

Five. Not eleven, not nine—five, out of twelve. Missingness in different columns lands on different rows, so it accumulates, and it accumulates faster than anyone’s intuition. On a real file with eight predictors this is how a 50,000-row dataset becomes a 4,000-row dataset without anybody deciding that it should.

EXERCISE: HOW BAD IS THE WORST SINGLE COLUMN?

Return the largest number of missing values in any one column of `housing`.

Exercise	
1	# The largest per-column count of NAs, as a single number.
2	-----

Please replace ----- with valid code.

EXERCISE: WHAT FRACTION OF THE DATA SURVIVES?

Return the proportion of rows that are complete on `price`, `sqft`, and `built`—a number between 0 and 1.

Exercise	
1	# Proportion of rows complete on price, sqft and built.
2	-----

Please replace ----- with valid code.

What to do when the data cannot answer the question

You will hit this. Somebody in your team will hit it in week three of Project 1. The variable is not there, or it is there and 80% empty, or it turns out to be self-reported when the question needed it measured, or the years you need are the years the agency stopped publishing.

The instinct is to feel that the project has failed and to start hiding it. Both halves of that instinct are wrong.

It is a finding. “The 2019–2023 county-level data cannot support a comparison of rural and urban outcomes, because rural counties are suppressed for privacy whenever the count is under ten, and 62% of rural counties are suppressed” is a genuine, reportable, useful result. Someone reading it now knows not to spend their own five weeks on the same road. Negative results about *data* are, if anything, more reusable than negative results about hypotheses.

Revising the question is the process, not an admission. The loop exists precisely so that what you learn in the cleaning stage can rewrite the question stage. What is *not* allowed is revising silently—presenting the new question as though it had always been the plan, with no mention of what you found. That is where a normal course correction turns into a misrepresentation.

A workable move, in order:

1. **Quantify it.** How many rows, exactly? Missing for whom? Is the missingness related to the outcome you care about?
2. **Ask what question the surviving data *can* answer.** Often it is a narrower version: one state instead of fifty, one year instead of a decade, an association instead of an effect.
3. **Check that the narrower question is still Relevant.** Sometimes it is not, and you should change datasets. That is allowed too.
4. **Write down both.** The report says what you asked, what you found out about the data, and what you asked instead. One paragraph. It costs you nothing and it is the difference between a project that reads as honest and one that reads as lucky.

IN THIS COURSE SPECIFICALLY

A project that reports “we could not answer our original question, here is precisely why, and here is the narrower question we answered instead” can earn full marks. A project that reports a clean result and turns out to have quietly dropped 60% of its rows to get there cannot. Every presentation includes individual questions, and *how many rows did you start with and how many did you model on?* is one I ask often.

Three weeks into a project you discover that the variable your question depends on is missing for 78% of the rows, and that the rows where it is present are mostly from one large city. Which response is the honest one?

- ☐ Impute the missing 78% from the observed 22% and proceed with the original question as written.
- ☐ Delete the incomplete rows, analyze the remaining 22%, and report the result as a finding about the full population.
- ☐ Revise the question to one the surviving data can actually answer, and report both the missingness and what it rules out.
- ☐ Quietly switch to a different dataset and present the project as though the first attempt never happened.



A note on version control

Because the loop revisits stages, the state of your analysis changes in ways that are hard to reconstruct from memory three weeks later. This is what Git is for, and it is why last week set it up before there was anything to commit. Commit after each stage, with a message that says what changed and why—“drop rows with suppressed rural counts; n falls 4,812 → 1,840” is a useful message, “update” is not. When you write the report, your commit history is the record of what you actually did, and it is much more reliable than anyone’s recollection.

Where this goes next

The next two sections are the cleaning stage in detail: getting a file into R ([Importing Data](#)), and finding out what is wrong with it ([Cleaning and Reshaping](#)).

These are not graded and nothing here is auto-checked. They are the raw material for Monday's reasoning block, where you will argue about them in pairs and one of you will be asked to defend an answer to the room.

HOW TO USE THIS PAGE

Write your prediction down before you run anything. On paper, in a comment, anywhere—but committed, so that you cannot revise it retroactively. The value of these problems is entirely in the gap between what you expected and what happened, and that gap disappears if you press Run first.

Every problem has a collapsed discussion note. Open it *after* you have predicted and run.

Predict the output

A1. Four ways to count

R Code	
1	<code>x <- c(1, NA, 3, 4)</code>
2	
3	<code>mean(x)</code>
4	<code>mean(x, na.rm = TRUE)</code>
5	<code>sum(x > 2)</code>
6	<code>sum(x > 2, na.rm = TRUE)</code>
7	<code>length(x[x > 2])</code>
<pre>[1] NA [1] 2.666667 [1] NA [1] 2 [1] 3</pre>	

Predict all five before running. In particular: are the last two the same number, and if not, why not?

DISCUSSION

>

A2. The phantom row

R Code	
1	<code>d <- data.frame(</code>
2	<code> x = c(1, NA, 3, 4),</code>
3	<code> g = c("a", "b", "a", "b"),</code>
4	<code> stringsAsFactors = FALSE</code>
5	<code>)</code>
6	
7	<code>d[d\$x > 2,]</code>
8	<code>subset(d, x > 2)</code>
<pre> x g NA NA <NA> 3 3 a 4 4 b x g 3 3 a 4 4 b</pre>	

Two ways of doing the same thing. Do they return the same data frame?

DISCUSSION



A3. Percentages of what?

R Code

```
1 v <- c("y", "n", "y", NA, NA)
2
3 table(v)
4 prop.table(table(v))
```

```
v
n y
1 2
v
      n      y
0.3333333 0.6666667
```

Five responses. What denominator do those proportions use?

DISCUSSION



A4. A perfectly reasonable number

R Code

```
1 f <- factor(c("12", "7", "23", "7"))
2
3 mean(as.numeric(f))
```

```
[1] 2.25
```

What comes out, and what is it the mean of?

DISCUSSION



A5. `ifelse` and the third possibility

R Code

```
1 ifelse(c(1, NA, 3, 4) > 2, "high", "low")
```

```
[1] "low" NA "high" "high"
```

`ifelse` takes a condition, a value for `TRUE`, and a value for `FALSE`. There are only two branches. What happens to the `NA`?

DISCUSSION



A6. Sorting

R Code

```
1 sort(c("10", "9", "100"))
```

```
[1] "10" "100" "9"
```

DISCUSSION



This runs and is still wrong

Every cell in this part executes without error. Some print a warning; several do not. In each case, say what the output should have been and what the reader of a report containing this number would be misled about.

B1. Which summaries survive a sentinel?

R Code

```
1 temps <- c(-999, 71, 68, 74, -999, 70)
2
3 c(mean = mean(temps), median = median(temps), max = max(temps))
```

```
      mean      median      max
-285.8333    69.0000    74.0000
```

The mean is absurd on sight. Are the other two safe?

DISCUSSION



B2. Stacking two files

R Code

```
1 p <- data.frame(id = 1:2, v = c(1.5, 2.5))
2 q <- data.frame(id = 3:4, v = c("3.5", "oops"), stringsAsFactors = FALSE)
3
4 r <- rbind(p, q)
5 str(r)
6 mean(as.numeric(r$v), na.rm = TRUE)
```

```
'data.frame':  4 obs. of  2 variables:
 $ id: int  1 2 3 4
 $ v : chr  "1.5" "2.5" "3.5" "oops"
```

R Warning in `mean(as.numeric(r\$v), na.rm = TRUE)`

Warning: NAs introduced by coercion

```
[1] 2.5
```

Two months of data, stacked. The mean comes out at 2.5. What is wrong with it?

DISCUSSION



B3. A join that made you richer

R Code			
1	sales <- data.frame(2 store = c("S1", "S2", "S3", "S4"), 3 units = c(10, 25, 7, 14), 4 stringsAsFactors = FALSE 5) 6 prices <- data.frame(7 store = c("S1", "S1", "S2", "S3", "S4"), 8 plan = c("basic", "premium", "basic", "basic", "basic"), 9 price = c(5, 9, 6, 4, 8), 10 stringsAsFactors = FALSE 11) 12 13 j <- merge(sales, prices, by = "store") 14 c(rows = nrow(j), units_before = sum(sales\$units), units_after = sum(j\$units))		
	rows	units_before	units_after
	5	56	66

Every store matched. Nothing was dropped. Total units went from 56 to 66.

DISCUSSION



B4. The average of the averages

R Code			
1	schools <- data.frame(2 school = c("A", "B"), 3 mean_score = c(80, 60), 4 n = c(10, 200) 5) 6 7 mean(schools\$mean_score) 8 weighted.mean(schools\$mean_score, schools\$n)		
	[1]	70	
	[1]	60.95238	

A district reports its average score as 70.

DISCUSSION



B5. The zip code join

R Code			
1	a <- data.frame(zip = c(2134, 20052, 7030), pop = c(1000, 2000, 3000)) 2 b <- data.frame(3 zip = c("02134", "20052", "07030"), 4 region = c("northeast", "south", "northeast"), 5 stringsAsFactors = FALSE 6)		

```

7
8 merge(a, b, by = "zip")

```

```

      zip pop region
1 20052 2000 south

```

Three zips on each side, all three the same places. How many rows come back?

DISCUSSION



B6. Dates that are not dates

R Code

```

1 dates <- c("3/1/2026", "11/2/2026", "7/15/2026")
2
3 sort(dates)
4 max(dates)

```

```

[1] "11/2/2026" "3/1/2026"  "7/15/2026"
[1] "7/15/2026"

```

DISCUSSION



What does this license you to claim?

No code here. For each, decide what the analyst may write in the report, and what they may not. Bring your answer.

C1.

A team joins 4,812 sales rows to a store lookup table with `merge`, gets 3,104 rows, does not notice, and reports: “Average units per store were 17.5.”

What, precisely, is the claim they are entitled to make instead? Note that “average units per store among stores appearing in both tables” is technically accurate and probably still misleading—why?

DISCUSSION



C2.

From [S.M.A.R.T. Questions](#): eight commuters, four by transit and four by car, and transit commutes averaged 17.5 minutes longer.

Which of these are supported? (a) In this sample, transit commuters averaged 17.5 more minutes. (b) Transit commuting takes about 17.5 minutes longer. (c) Switching from car to transit would add about 17.5 minutes to your commute. (d) Transit commuters live further from work.

DISCUSSION



C3.

A housing dataset has 12 rows. Three variables are needed; each is missing for two or three rows; five rows are complete on all three. The team fits a model and reports an R^2 of 0.91.

What has to appear in the report alongside that number, and what would you ask the team at the presentation?

DISCUSSION



C4.

A team cleans a `site` column by lower-casing, trimming whitespace, and converting empty strings to `NA`. They now have two sites where the raw file had five distinct values.

Under what circumstances is that cleaning step *wrong*? Give a concrete example of a dataset where `North` and `north` are genuinely different things.

DISCUSSION



Bring to class

1. Take one of the four “before” questions in [S.M.A.R.T. Questions](#) and write a *fifth* pair of your own: a vague question you have actually been curious about, and a S.M.A.R.T. version of it. Be ready to have the room attack the second one.
2. Find a real CSV—any open data portal, any Kaggle dataset—load it, and run the five-call inspection ritual from [Importing Data](#). Come with one thing you found that you would not have found by opening it in Excel, and one thing Excel showed you that R did not.
3. For the same file: how many rows, how many columns, what does one row mean, and what proportion of rows are complete on the three variables you would most want to use? If any of those four questions is hard to answer, that is the interesting part—bring the difficulty rather than a number.
4. Argue one of these, either side:

“Dropping rows with missing values is almost always the wrong default, and the fact that R does it automatically inside modeling functions—without printing how many it dropped—is a design mistake.”

You will be assigned a side in class, so prepare both.

Every data science project starts with somebody being interested in something. That is a good place to start and a terrible place to stay. “I’m interested in housing affordability” is an interest. It is not a question, because there is nothing you could compute that would settle it, and there is no result that would count as being wrong.

The gap between an interest and a question is where most student projects are won or lost. I want to be blunt about this, because it is not what people expect. When a project in this course goes badly, it is very rarely because the regression was misspecified or the plot used the wrong geometry. It is because in the first week somebody wrote a sentence that sounded like a research question, everybody nodded, and six weeks of work went into an activity that could not have produced an answer no matter how well it was executed.

The five words

S.M.A.R.T. is a checklist borrowed from project management, and like most checklists it is worth roughly what you put into it. Used lazily it is five words you attach to a question you had already decided on. Used properly it is five separate ways of catching yourself.

Specific. Who or what is being described, and over what population? “Do students who attend office hours do better?” leaves open which students, at which institution, in which course, and in which term. Each of those has to be decided before anyone can compute anything, and if you do not decide them, the data will decide for you and you will not notice.

Measurable. Every noun in the question has to correspond to something you can actually put in a column. “Better” is not measurable. “Final course grade out of 100” is. Notice that making a thing measurable is a *substantive* decision, not a clerical one: choosing final grade as your measure of “doing better” commits you to a particular and arguable claim about what doing better means, and an honest project says so rather than pretending the choice was obvious.

Achievable. Can you get the data, in the time you have, with the access you have? This is where most ambitious questions die, and it is worth separating two different failures. One is practical: the data exists but is behind an IRB, a paywall, or a six-week request queue. The other is deeper: *no* dataset of that kind could answer the question. If nobody randomly assigned people to use social media, then no amount of survey data will tell you what social media *causes*, and a question phrased in causal language is unachievable in principle rather than in practice.

Relevant. Would anyone act differently, or believe something different, depending on the answer? A question whose answer is already known, or whose answer nobody would use, is a fine exercise and a poor project. The test I like: write down both possible answers, and check that they are genuinely different worlds.

Time-bound. Two meanings, and both matter. The question needs a time window in it—“in Fall 2026”, “between 2010 and 2020”—because a relationship that holds in one decade need not hold in another, and a question without a window is quietly claiming to hold in all of them. And the *project* needs a time budget: you have until the proposal deadline, which is roughly five working weeks, most of which will go on things that are not modeling.

Before and after

Checklists are easier to believe once you have watched one do some work. Here are questions students have actually proposed, and what they became.

EXAMPLE 1

Before. *Is Twitter bad for mental health?*

After. *Among the 1,247 adult respondents to the 2023 wave of the National Health Interview Survey, is self-reported daily social media use (in minutes) associated with the PHQ-2 depression screening score, and how large is the association?*

Look at what changed. “Twitter” became a variable that exists in a dataset somebody actually collected. “Bad for mental health” became a specific instrument with a known scale. The population and the year are named. And—this is the part students resist—“is bad for” became “is associated with”, because the data is observational and the honest verb is the weaker one. You have not made the question smaller by doing this. You have made it answerable, which the original was not.

EXAMPLE 2

Before. *We want to predict housing prices.*

After. *Using the 21,613 King County home sales from May 2014 to May 2015, how much of the variation in sale price is explained by square footage, number of bedrooms, and year built—and does adding location (zip code) improve prediction enough to matter?*

The “before” is not a question, it is a genre. Every third project proposal in every data science course in the country is that sentence. The “after” commits to a dataset, a time window, a specific set of predictors, and a comparison that will have an answer. The word “matter” is still soft, and a good proposal would pin it down further: *reduce out-of-sample RMSE by more than \$15,000*, say.

EXAMPLE 3

Before. *Does the minimum wage cause unemployment?*

After. *In the 12 months after New Jersey raised its minimum wage in April 1992, did fast-food employment in New Jersey change differently from fast-food employment in neighboring Pennsylvania counties, where the wage was unchanged?*

This one is instructive because the “before” is a real and important question that serious economists have argued about for forty years. It fails **Achievable** for you, this semester—not because it is stupid but because answering it requires a design that lets you say something causal. The “after” narrows to one policy change, one industry, one comparison region, and one window. That is roughly the design of a famous 1994 paper by Card and Krueger, and it is still argued about, which should tell you something about how hard the “before” version is.

EXAMPLE 4

Before. *We will analyze the Titanic dataset and see what we find.*

After. *Among the 891 passengers in the Kaggle Titanic training set, was passenger class associated with survival after accounting for sex and age, and is the class effect large enough to be visible within each sex separately?*

“See what we find” is not a question, and it is the most common thing on a first draft. It is not that exploration is wrong—[The Data Science Life Cycle](#) has a whole stage for it—but exploration is what you *do*, not what you *ask*. If your proposal contains no sentence with a question mark that names two variables, you do not have a project yet.

The tell-tale signs of a bad question

Over the years the failures repeat, and they are recognizable from the outside.

A question with the word “**best**” in it usually has no answer: *what is the best predictor of income?* depends entirely on what you put in the candidate set, and the answer changes when you add a variable. A question with “**impact**”, “**effect**”, or “**drives**” in it is making a causal claim; ask yourself whether anything in the data design earns that verb, and if not, weaken the verb rather than hoping nobody notices. A question that could be answered by a single number with no comparison in it—*what is the average salary of data scientists?*—is a lookup, not an analysis. And a question you already know the answer to is not worth five weeks, however comfortable it feels.

The most useful move I know is this: **write down the table you would need**. Not the data you have—the table. One row per what? Which columns? Then go and see whether such a table exists. Half of all bad questions do not survive that exercise, and they die in ten minutes rather than in week four.

Kicking the tires on a question

A question is a claim about what your data contains, and claims can be checked. Before you commit to one, check it against the actual file.

Suppose the question is: *Among students enrolled in Fall 2026, is the number of office-hour visits associated with the final course grade?* It sounds respectable. Here is the roster it would have to be answered from.

--

R Code

```
1 roster <- data.frame(
2   student = paste0("s", 1:10),
3   term     = c("2025F", "2025F", "2025F", "2026S", "2026S",
4               "2026F", "2026F", "2026F", "2026F", "2026F"),
5   visits   = c(3, 0, 1, 2, NA, 4, NA, 0, NA, NA),
6   grade     = c(91, 78, 85, 88, 72, 95, 81, 69, 90, 77),
7   stringsAsFactors = FALSE
8 )
9 str(roster)
10 table(roster$term)
```

```
'data.frame':  10 obs. of  4 variables:
 $ student: chr  "s1" "s2" "s3" "s4" ...
 $ term   : chr  "2025F" "2025F" "2025F" "2026S" ...
 $ visits : num  3 0 1 2 NA 4 NA 0 NA NA
 $ grade  : num  91 78 85 88 72 95 81 69 90 77
```

```
2025F 2026F 2026S
      3      5      2
```

Five rows are from Fall 2026, which already makes the question thinner than it sounded. But the question needs `visits` as well, and `visits` is missing for much of the file. The number that matters is not how many Fall 2026 rows there are; it is how many Fall 2026 rows have a visit count at all.

EXERCISE: IS THE QUESTION ACHIEVABLE?

Count the rows that could actually contribute to the question—Fall 2026 *and* a non-missing `visits` value.

Exercise

```
1 # How many rows are from term "2026F" AND have a non-missing visits value?
2 -----
```

Please replace `-----` with valid code.

Specific means naming the comparison

The other habit worth building is noticing when a question quietly fails to say what is being compared to what. “Do transit commuters have long commutes?” has no comparison in it, so the natural thing to compute is an average, and an average of one group answers nothing—long compared to what?

R Code

```
1 survey <- data.frame(
2   respondent = 1:8,
3   commute    = c("transit", "car", "transit", "car",
4                 "transit", "car", "transit", "car"),
5   minutes    = c(38, 27, 44, 31, 52, 22, 41, 25),
6   stringsAsFactors = FALSE
7 )
8 mean(survey$minutes)
```

```
[[1]] 35
```

Thirty-five minutes. Thirty-five minutes is not an answer to anything, because the question never said what the alternative was. Sharpen it—*do transit commuters spend more time commuting than car commuters?*—and the quantity you want changes.

EXERCISE: COMPUTE WHAT THE SHARPENED QUESTION ASKS FOR

Return the difference in mean commute time, transit minus car, as a single number.

Exercise

```
1 # Mean commute time for transit riders minus mean for car commuters,  
2 # as a single number. `tapply(x, group, mean)` will give you both means.  
3 -----
```

Please replace ----- with valid code.

How this shows up in your grade

The project—a fifth of the course—is proposed long before it is presented. The proposal is due Oct 19 and the presentation is in December, which gives you weeks to discover that the question you wrote is not answerable. The proposal is short, and it is mostly the question.

What I read for, in that order:

1. **Is it a question?** Does the document contain a sentence, with a question mark, naming a population, a time window, and at least two variables?
2. **Does the data exist, and have you looked at it?** Not “we plan to obtain”—have you loaded it, run `str()`, and counted the rows that survive the conditions your question imposes? The exercise above is the whole of this check.
3. **What answer would count as no?** If you cannot describe a result that would make you say “no, that is not there”, the question is not falsifiable and the project will end in a paragraph of hedging.
4. **Is the verb honest?** Causal language needs a design that earns it.

At the presentation, every team member answers questions individually, and one of the questions is always some version of *what would have to be true for your conclusion to be wrong?* A team that framed its question well finds that question easy. A team that did not spends the whole answer explaining what they meant.

THE UNCOMFORTABLE PART

You will be tempted to pick the question that fits the dataset you already downloaded. Sometimes that is fine—it is called secondary analysis and half of published research works that way. It stops being fine when the question drifts silently to fit whatever the data happened to support, and the report is written as though that had been the plan all along. Changing your question is allowed and normal. Changing it without saying so is not.

CHECK YOURSELF



A team submits this project proposal: “*We will study whether social media use affects mental health.*” They have one semester and a single public survey dataset. Which revision is the strongest S.M.A.R.T. question?

- ☐ Does social media use cause depression?
- ☐ Among the 1,247 adult respondents in the 2023 wave of this survey, is self-reported daily screen time associated with the PHQ-2 depression score, and how large is that association?
- ☐ Is social media bad for teenagers?
- ☐ We will explore every variable in the survey and report whatever patterns emerge.



Where this goes next

You now have a question. The next section is about what happens to it—which is that it does not survive contact with the data unchanged, and was never supposed to.

Every table you will analyze is a subset of something larger. Some rows made it in and some did not, and every method in the rest of this course assumes—quietly, and usually without anyone checking—that the rows which made it in resemble the rows which did not.

When that assumption fails, nothing downstream repairs it. A larger sample does not help. A better model does not help. The confidence interval you compute in [The Standard Error of a Mean](#) will be narrow and wrong, which is worse than wide and wrong.

So there is one question to ask of a dataset before any of the others, and it is the question this chapter is about:

Who is missing, and does their absence push the answer in a predictable direction?

The aircraft that came back

During the Second World War, the U.S. military studied the bullet holes on aircraft returning from missions over Europe. The damage was concentrated on the wings and fuselage; the engines were comparatively clean. The apparent conclusion was to add armour where the holes were.

Abraham Wald, working for the Statistical Research Group, pointed out that the sample consisted entirely of aircraft that had returned. Planes hit in the engines were not underrepresented in the data because engines were rarely hit. They were underrepresented because they were at the bottom of the Channel.

Armour the engines.

The story survives because the error is so easy to make and so hard to see from inside the data. Nothing in the table is wrong. Every bullet hole was recorded accurately. The table simply answers a different question from the one being asked—not *where are aircraft hit*, but *where are aircraft hit given that they survived being hit*.

Three ways a sample goes wrong

Survivorship. The sample was filtered by the outcome you care about. The aircraft is the classic case, but the modern ones are everywhere: *companies that have lasted ten years share these three habits*, which cannot distinguish a habit that causes survival from a habit that survivors adopt; *our users love the redesign*, measured among the users who stayed.

Selection. The process that put a row into the table is related to the thing you are measuring. Volunteers for a trial are not like non-volunteers. People who click a feedback link are people with an opinion strong enough to click.

Convenience. Whoever was easy to reach—the lobby at two o'clock on a Tuesday, the students in your own program, the first thousand rows the API returned. Convenience samples are the most common and the hardest to argue about, because the bias is real but its direction is unknown. You cannot say which way the answer is wrong, only that you have no grounds to say it is right.

THE DIAGNOSTIC

Describe, in one sentence, the process that decided whether a row exists in this table.

If you cannot write that sentence, you do not yet know what your sample is, and every number you compute from it is provisional.

Missing data is a mechanism, not a gap

The same reasoning applies one level down. A row can be present while a *value* is absent, and the reason the value is absent matters far more than the fact that it is.

Statisticians name three mechanisms, and the names are worth learning because they tell you what you are allowed to do next.

MCAR—missing completely at random. Whether a value is missing has nothing to do with anything, observed or not. A sensor dropped packets at random; a page of forms was lost in the post. Dropping the incomplete rows costs you precision and nothing else. This is the case everyone assumes and almost nobody has.

MAR—missing at random. Whether a value is missing depends on something you *did* observe. Younger respondents skip the income question more often, and you recorded age. The name is unfortunate—nothing here is random in the ordinary sense—but the situation is the hopeful one: because the thing driving the missingness is in your data, you can use it to recover the answer.

MNAR—missing not at random. Whether a value is missing depends on the missing value itself. The highest earners decline to state their income; the sickest patients drop out of the study. No amount of cleverness recovers this from the data alone, because the information you would need is precisely the information that is absent. You need an assumption from outside the dataset, and you need to say out loud what it is.

Seeing MAR happen

Below, twelve employees in three departments. `salary` is the truth, which you would not normally have. `reported` is what the survey collected: two engineers declined, and nobody else did.

R Code	
1	<code>staff <- data.frame(</code>
2	<code> dept = rep(c("eng", "ops", "sales"), each = 4),</code>
3	<code> salary = c(120, 130, 140, 150, 90, 95, 100, 105, 70, 75, 80, 85),</code>
4	<code> reported = c(120, NA, NA, 150, 90, 95, 100, 105, 70, 75, 80, 85)</code>
5	<code>)</code>
6	
7	<code># The truth, and the number the survey appears to give you.</code>
8	<code>c(truth = mean(staff\$salary),</code>
9	<code> complete_case = mean(staff\$reported, na.rm = TRUE))</code>
	<code>truth complete_case</code>
	<code>103.3333 97.0000</code>

The survey says 97.0 when the truth is 103.3—a shortfall of six, entirely because the best-paid department is the one that answered least. Nothing was mistyped. `na.rm = TRUE` did exactly what it promised, and the promise was the problem.

This is MAR, not MNAR: within engineering the two who declined were not the two highest earners, so knowing somebody's department tells you everything about their chance of answering. That is the opening the next exercise uses.

🔗 EXERCISE: RECOVER WHAT DROPPING DESTROYED

Estimate the overall mean salary by averaging the three **department** means computed from `reported`. Return a single number.

Exercise	
1	<code># Average the three department means of `reported`.</code>
2	<code>_____</code>

Please replace `_____` with valid code.

🔗 EXERCISE: NAME THE MECHANISM

A survey asks for household income. People with very high incomes decline to answer noticeably more often than everyone else. The survey records nothing else that predicts income.

Assign the letter of the correct answer, for example `answer <- "A"`.

Exercise	
1	# A. MCAR—the non-responses are scattered at random.
2	# B. MAR—the missingness depends on a variable you observed.
3	# C. MNAR—the missingness depends on the missing value itself.
4	# D. MAR, because the other respondents can be used to impute the gaps.
5	answer <- _____

Please replace _____ with valid code.

Imputation, and why it is not free

Filling gaps in is a large subject and this course does not teach it. Two things are worth knowing now.

Mean imputation—replacing every gap with the column average—is almost always the wrong move. It does not merely fail to add information; it actively destroys some, because every imputed row now sits exactly at the center. The variance shrinks, the standard error shrinks with it, and the analysis reports more confidence than the data can support. A method that makes you *more* certain by discarding information should be treated with suspicion.

Principled methods exist—multiple imputation is the standard one—and they work by imputing repeatedly and letting the disagreement between the versions carry the uncertainty forward. They assume MAR. Under MNAR they are as stuck as everything else.

The honest minimum for this course, and for the project:

1. **Report how many rows you lost**, not just the ones you kept.
2. **Say what you believe the mechanism was**, and why.
3. **Say which direction it would push your answer** if you are wrong.

The third is the one that separates an analyst from a technician. You will often not be able to quantify the bias. You can nearly always sign it.

Finding a filter nobody told you about

Everything above assumes you know what was excluded. Usually you do not, and nothing in the file announces it. Here is how you look, cheapest first, run against `quakes`—1,000 seismic events near Fiji, built into R.

1. Scan every column's boundary, not just the interesting one. A filter leaves a round number at an extreme with observations piled on it.

R Code	
1	<code>data(quakes)</code>
2	<code>sapply(quakes, function(x)</code>
3	<code> c(min = min(x), max = max(x), n_at_min = sum(x == min(x))))</code>

	lat	long	depth	mag	stations
min	-38.59	165.67	40	4.0	10
max	-10.72	188.13	680	6.4	132
n_at_min	1.00	1.00	12	46.0	20

`mag` stops dead at 4.0. So does `stations` at **10**, with twenty events sitting on it, and `depth` at **40**. An event had to be reported by at least ten stations to be in this file. Nothing in the documentation mentions it, and almost nobody looks, because `stations` is not the variable anyone came for.

2. Check the shape against what the subject says it must be. Small earthquakes vastly outnumber large ones, so counts must *fall* as magnitude rises:

--	--

R Code

```
1 table(quakes$mag)[1:9]
```

```
4 4.1 4.2 4.3 4.4 4.5 4.6 4.7 4.8  
46 55 90 85 101 107 101 98 65
```

They rise—46, 55, 90, 85, 101, 107—to 4.5, and only then fall. Earthquakes do not do that. **Detection networks do.** The catalogue is incomplete up to about 4.6, so the filter you can see at 4.0 is the advertised one rather than the operating one. This is the most useful check on the list, and the only one that needs you to know something about the subject rather than about data.

3. Look at two variables at once. A cut can be invisible in its own histogram and obvious as a straight edge in a scatterplot.

4. Read the provenance. From `?quakes`: these 1,000 rows are “a subsample from a larger dataset containing 5000 observations.”

SOME FILTERS CANNOT BE FOUND FROM THE INSIDE

A one-in-five uniform subsample changes no shape, moves no boundary, and leaves no trace anywhere in the file. No amount of looking at these numbers can recover it. It exists in one place only—the sentence in the documentation that almost nobody reads.

That is the argument for this whole chapter. **Where the data came from is not recoverable from the data.** Four of the five things wrong with `quakes` were findable by looking; the fifth was findable only by asking, and you would never have known to ask.

Four situations

Take these one at a time. For each, name who is missing, and say whether their absence pushes the answer up, down, or in a direction you cannot determine.

1. “We surveyed the users who clicked the feedback link.”
2. “We studied the habits of companies that have lasted ten years.”
3. “We dropped every row with a missing income value.”
4. “We asked people in the lobby at two o’clock on a Tuesday.”

Post your answers in the comment box below. Disagreement is the useful outcome here—two of these have a clearly signed direction and two do not, and arguing about which is which is the whole exercise.

CHECK YOURSELF



Where this goes next

[Importing Data](#) and [Cleaning and Reshaping](#) are about the mechanics—getting a file open, getting the types right, deciding what `na.rm = TRUE` should mean in a particular line. Read them with this chapter in mind, because the mechanics will happily produce a clean, tidy, fully documented table that answers the wrong question.

The idea returns twice more. When we reach modeling, **confounding** is this same problem wearing different clothes: a comparison contaminated by a difference between the groups being compared. And the project asks you to state what your analysis does not support, which is usually a sentence about who was not in your data.

Almost every question anyone actually asks is conditional.

Not “what is the probability of churn” but “what is the probability of churn *given* that they contacted support”. Not “what is the probability of default” but “given this credit history”. Not “does the patient have the disease” but “given that the test came back positive”. The unconditional version is a curiosity; the conditional version is the job.

The mechanics are one line long. The reasoning is where careers are made, and this chapter contains the single most-asked probability question in data science interviews, along with the reason most candidates get it wrong.

Conditioning is subsetting

Here is the entire idea, before any notation.

To condition on B is to throw away every outcome where B did not happen, and then compute as usual on what is left. The world shrinks to B , and probabilities are recomputed inside that smaller world.

Two dice again:

R Code	
1	<code>set.seed(6101)</code>
2	<code>d1 <- sample(1:6, 100000, replace = TRUE)</code>
3	<code>d2 <- sample(1:6, 100000, replace = TRUE)</code>
4	
5	<code>big <- (d1 + d2) >= 8 # event A</code>
6	<code>first_six <- d1 == 6 # event B</code>

Unconditionally, a total of 8 or more happens on about 42% of rolls:

R Code	
1	<code>mean(big)</code>
<code>[1] 0.41635</code>	

Now restrict to the rolls where the first die came up 6, and ask again:

R Code	
1	<code>sum(first_six) # how many rolls survive the restriction</code>
2	<code>mean(big[first_six]) # the proportion of THOSE that were 8 or more</code>
<code>[1] 16345</code>	
<code>[1] 0.8368308</code>	

Of the 16,345 rolls that had a 6 on the first die, 84% totaled 8 or more, against 42% overall. Learning about the first die doubled the probability. That is a conditional probability, and the R for it is `mean(A[B])`.

THE DEFINITION, AND WHY IT HAS A DENOMINATOR

$$P(A | B) = \frac{P(A \cap B)}{P(B)}, \quad P(B) > 0$$

Read the two pieces. The numerator counts the outcomes where **both** happened. The denominator counts the outcomes where B happened at all. You are computing a proportion *within* B , so B becomes the new denominator.

Nearly every conditional-probability mistake is a denominator mistake: someone divides by the wrong population. Whenever you meet a conditional probability in the wild, your first question should be “**out of what?**”

The subsetting version and the formula version are the same computation:

R Code					
1	<code>round(c(by_subsetting = mean(big[first_six]),</code>				
2	<code>by_formula = mean(big & first_six) / mean(first_six)), 4)</code>				
<table><tr><td>by_subsetting</td><td>by_formula</td></tr><tr><td>0.8368</td><td>0.8368</td></tr></table>		by_subsetting	by_formula	0.8368	0.8368
by_subsetting	by_formula				
0.8368	0.8368				

The multiplication rule

Rearranging the definition gives the **multiplication rule**:

$$P(A \cap B) = P(B) P(A | B)$$

which is how you build up the probability of a sequence of things happening: the first, times the second given the first, and so on.

When A and B are independent, $P(A | B) = P(A)$ and this collapses to $P(A)P(B)$ —the familiar “just multiply”. That collapse is a *special case*, not the rule.

Draw two cards from a deck, without replacement, and ask for the probability that both are aces:

R Code

```
1 set.seed(6101)
2 n <- 200000
3 first <- sample(1:52, n, replace = TRUE)      # cards 1-4 are the aces
4 second <- sample(1:51, n, replace = TRUE)
5 second <- ifelse(second >= first, second + 1, second)  # skip the card already taken
6
7 ace1 <- first <= 4
8 ace2 <- second <= 4
9
10 round(c(P_ace1          = mean(ace1),
11         P_ace2_given_ace1 = mean(ace2[ace1]),
12         P_both          = mean(ace1 & ace2),
13         naive_product    = mean(ace1) * mean(ace2)), 5)
```

P_ace1	P_ace2_given_ace1	P_both	naive_product
0.07676	0.05745	0.00441	0.00589

$P(\text{ace}) = 4/52 \approx 0.0769$, but once one ace is gone, $P(\text{ace} | \text{ace}) = 3/51 \approx 0.0588$. The correct joint probability is $\frac{4}{52} \times \frac{3}{51} \approx 0.00452$; treating the draws as independent gives about 0.0059, roughly 31% too high.

Sampling without replacement is the most common place independence quietly fails, and it is worth noticing that it fails *less* the larger the population is. Drawing two people from a city of a million, you may multiply and nobody will ever know. Drawing two from a team of eight, you may not.

Bayes' rule

Now the reversal. You know $P(B | A)$ and you want $P(A | B)$.

These are not the same number and they are not close. $P(\text{wet} | \text{raining})$ is nearly 1; $P(\text{raining} | \text{wet})$ is not, because sprinklers exist. Getting from one to the other requires knowing how common rain and wetness are in the first place—and that requirement is Bayes' rule.

Write the joint probability two ways using the multiplication rule:

$$P(A \cap B) = P(B | A)P(A) = P(A | B)P(B)$$

and divide:

$$P(A | B) = \frac{P(B | A) P(A)}{P(B)}$$

The denominator is usually expanded by the **law of total probability**— B can arrive either with A or without it:

$$P(B) = P(B | A)P(A) + P(B | A^c)P(A^c)$$

That is the whole of Bayes' rule: two lines of algebra from the definition. Nobody finds the derivation hard. What people find hard is believing the answer.

🔍 WHICH WAY ROUND IS IT?



An analyst reports: “80% of the customers who canceled last month had contacted support in the 30 days before canceling.”

The head of product proposes acting on it: flag every customer who contacts support as a churn risk and route them to a retention offer.

What is the problem with that reasoning?

- ☐ There is no problem—80% is a strong signal and acting on it is exactly what the number is for.
- ☐ The report gives $P(\text{contacted} | \text{canceled})$, but the proposal needs $P(\text{canceled} | \text{contacted})$, and converting between them requires knowing how common contacting support is among *all* customers.
- ☐ The number is only valid if contacting support and canceling are independent, which has not been checked.
- ☐ One month of data is too small a sample to support any conclusion.



The medical test problem

This is the centerpiece of the chapter. If you take one thing from [Probability](#), take this.

THE SETUP

A disease affects **5 in 1,000** people—a prevalence of 0.5%.

A screening test is good:

- **Sensitivity 99%**. If you have the disease, it comes back positive 99% of the time.
- **Specificity 95%**. If you do not have the disease, it comes back negative 95% of the time.

Your test comes back **positive**. You have no symptoms; you were screened because it was on the routine panel.

How worried should you be?

Write your answer down before you read on. Most people—including most doctors, in studies that have been repeated for forty years—say something in the region of 95%.

We are not going to argue about it. We are going to build the population and count.

R Code

```
1 set.seed(6101)
2 N <- 100000
3
4 # 1. Who actually has the disease? 0.5% of the population.
5 disease <- rbinom(N, 1, 0.005)
6
7 # 2. What does the test say? 99% positive if diseased, 5% positive if not.
8 positive <- rbinom(N, 1, prob = ifelse(disease == 1, 0.99, 0.05))
9
10 table(disease = disease, positive = positive)
```

	positive	
disease	0	1
0	94413	5073
1	4	510

Read the table before doing any arithmetic.

- **514** of the 100,000 simulated people have the disease. The test caught 510 of them and missed 4. It is, as advertised, an excellent test.
- **99,486** do not have the disease. The test correctly cleared 94,413 of them—and wrongly flagged **5,073**.

So 5,583 people got a positive result, and only 510 of them are ill.

R Code

```
1 mean(disease[positive == 1])
```

```
[[1]] 0.09134874
```

About 9%.

Nine percent. Not 95%. A positive result on a 99%-sensitive test for this disease leaves you more than ten times more likely to be healthy than sick.

Why, in one sentence

The healthy group is **200 times larger** than the sick group. Its 5% error rate is applied to a vastly bigger population than the test's 99% success rate is. Five percent of a huge number beats 99% of a tiny one, and it is not close.

The number doing the work here is the one nobody was given prominently: the **prevalence**, 0.5%. The two impressive-sounding percentages describe the test. The prevalence describes the world, and the world wins.

Now the algebra, and watch it agree

With D = has the disease and $+$ = tests positive:

$$P(D | +) = \frac{P(+ | D) P(D)}{P(+ | D) P(D) + P(+ | D^c) P(D^c)} = \frac{0.99 \times 0.005}{0.99 \times 0.005 + 0.05 \times 0.995}$$

R Code	
1	prev <- 0.005 # P(D)
2	sens <- 0.99 # P(+ D)
3	spec <- 0.95 # P(- not D), so P(+ not D) = 1 - spec
4	
5	ppv <- sens * prev / (sens * prev + (1 - spec) * (1 - prev))
6	round(ppv, 4)
[1] 0.0905	

0.0905, against the simulation's 0.0913. They agree to within simulation noise, which is what should happen—the algebra is a shortcut for the counting, not a separate source of truth.

The same thing in whole people, per 100,000 screened, which is the version to use when explaining this to somebody:

R Code										
1	<code>rbind(2 "has disease" = c(positive = 100000 * prev * sens, 3 negative = 100000 * prev * (1 - sens)), 4 "does not have it" = c(positive = 100000 * (1 - prev) * (1 - spec), 5 negative = 100000 * (1 - prev) * spec) 6)</code>									
	<table><tr><th></th><th>positive</th><th>negative</th></tr><tr><td>has disease</td><td>495</td><td>5</td></tr><tr><td>does not have it</td><td>4975</td><td>94525</td></tr></table>		positive	negative	has disease	495	5	does not have it	4975	94525
	positive	negative								
has disease	495	5								
does not have it	4975	94525								

495 true positives against 4,975 false positives. Ten false alarms for every real case, from a test that is right 99% of the time when it matters and 95% of the time when it does not.

THE INTERVIEW QUESTION



A disease affects 1 person in 1,000. A screening test is very good: it flags 99% of the people who have the disease, and it correctly clears 99% of the people who do not.

A patient with no symptoms is screened as part of a routine check-up and tests **positive**. Roughly how likely is it that this patient has the disease?

- ☐ About 99%, since that is the accuracy of the test.
- ☐ About 90%.
- ☐ About 9%.
- ☐ It cannot be estimated without knowing how many people were screened in total.



Base rates, and why the intuition fails

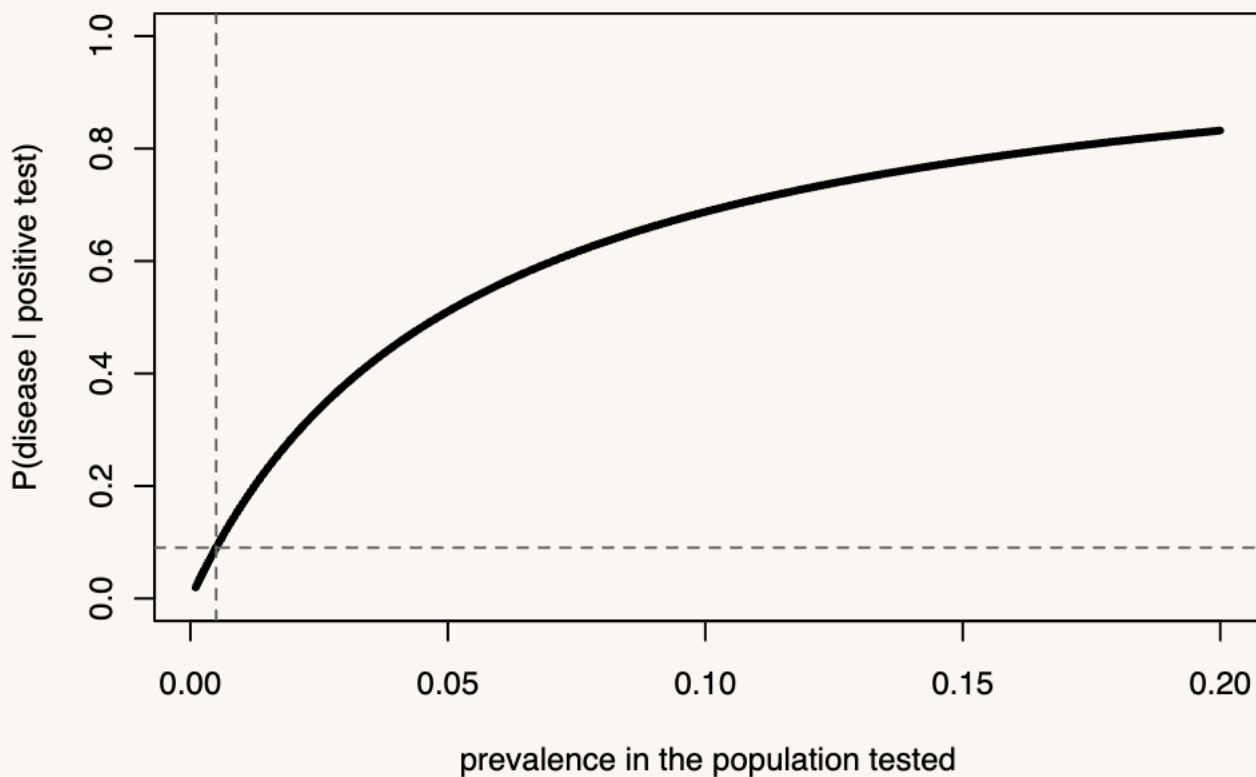
The technical name for what just happened is **base-rate neglect**: when given vivid, specific evidence, people weigh it heavily and quietly discard the background rate.

It is not stupidity, and knowing about it does not cure it. The pull is structural: the test result is *about you*, and the prevalence is about a faceless population, so the mind treats one as evidence and the other as trivia. But the prevalence is your **prior**, and Bayes' rule says the answer is a compromise between the prior and the evidence, weighted by how strong each one is.

Watch how completely the same test changes character as the base rate moves:

R Code

```
1 p <- seq(0.001, 0.2, length.out = 200)
2 ppv_curve <- sens * p / (sens * p + (1 - spec) * (1 - p))
3
4 plot(p, ppv_curve, type = "l", lwd = 3, ylim = c(0, 1),
5      xlab = "prevalence in the population tested",
6      ylab = "P(disease | positive test)")
7 abline(v = 0.005, lty = 2, col = "gray40")
8 abline(h = ppv, lty = 2, col = "gray40")
```



R Code

```
1 tab <- data.frame(prevalence = c(0.001, 0.005, 0.01, 0.05, 0.10, 0.50))
2 tab$P_disease_given_positive <-
3   round(sens * tab$prevalence /
4         (sens * tab$prevalence + (1 - spec) * (1 - tab$prevalence)), 3)
5 tab
```

```
prevalence P_disease_given_positive
1      0.001      0.019
2      0.005      0.090
3      0.010      0.167
4      0.050      0.510
5      0.100      0.687
6      0.500      0.952
```

The test never changed. Its sensitivity and specificity are properties of the assay and they are fixed. But a positive result means 2% at a prevalence of 1 in 1,000, 9% at 5 in 1,000, 51% at 5%, and 95% at 50%.

THE PRACTICAL CONSEQUENCE, WHICH IS NOT ABOUT MEDICINE

A test's accuracy is not a property you can quote on its own. What a positive result *means* depends on who you tested.

This is why mass screening of healthy populations is genuinely controversial: at low prevalence the false positives swamp the true ones, and the resulting biopsies, anxiety, and follow-up procedures are real harms borne mostly by healthy people.

And it is why the same argument transfers, unchanged, to everything you will build:

- **Fraud detection.** Fraud is rare. A 99%-accurate model flooding an investigations team with false positives is the normal outcome, not a bug.
- **Rare-disease and rare-event models.** Same arithmetic.
- **Alerting and monitoring.** An alert that fires on a rare condition with a 1% false-positive rate will be ignored within a fortnight, and then it will be ignored on the day it is right.
- **Hiring screens, security screening, content moderation, spam filters.** Every one of them is this problem.

When someone reports a classifier as “99% accurate”, the only useful reply is: *out of the ones it flagged, how many were real?* That number is called **precision**, and it is the same quantity as $P(D \mid +)$.

The same four numbers, under two sets of names

This is worth pinning down now, because you will meet these quantities again in DATS 6103 and in every interview that touches classification, and the two fields that use them most do not use the same words.

CONDITIONAL PROBABILITY	MEDICINE AND SCREENING	MACHINE LEARNING
$P(+ \mid D)$	sensitivity	recall , true positive rate
$P(- \mid \text{no } D)$	specificity	true negative rate
$P(D \mid +)$	positive predictive value	precision
$P(\text{no } D \mid -)$	negative predictive value	—

They are the same four numbers. Read the table by which side of the bar the test result sits on: **sensitivity and recall condition on the truth** and describe the test, so they do not move when the disease becomes rarer. **Precision and PPV condition on the result** and describe what a flag is worth, so they collapse when the base rate falls—which is the whole content of the previous section.

That is also why a model can be reported as having 99% recall and be nearly useless. Recall is a property of the classifier; precision is a property of the classifier *and* the population you deployed it into, and only one of those is in the paper.

The other conditional, for completeness

A negative result, on the other hand, is enormously reassuring—precisely because the disease is rare:

R Code

```
1 round(mean(disease[positive == 0]), 6)
```

```
[1] 4.2e-05
```

About 4 in 100,000. The same low prevalence that made the positive result nearly meaningless makes the negative result almost conclusive. Both facts come from the same base rate, and if only one of them surprises you, that is worth sitting with.

Exercises

Exercise: the conditional, from the simulated population

Exercise	
1	# `disease` and `positive` are 0/1 indicators for 100,000 screened people.
2	# Estimate $P(\text{disease} \mid \text{positive})$: among the people who tested positive,
3	# what proportion actually have the disease?
4	-----

Please replace ----- with valid code.

Exercise: the same table, the other way round

Exercise	
1	# Same two vectors. Now estimate $P(\text{positive} \mid \text{disease})$ -- the test's
2	# sensitivity -- from the simulated population.
3	-----

Please replace ----- with valid code.

Exercise: Bayes without the simulation

Exercise	
1	# Using only `prev`, `sens` and `spec`, compute $P(\text{disease} \mid \text{positive})$
2	# directly from Bayes' rule. No simulation.
3	# Remember: $P(\text{positive} \mid \text{no disease})$ is $1 - \text{spec}$.
4	-----

Please replace ----- with valid code.

Exercise: the same test, a different clinic

Exercise	
1	# The identical test is now used in a specialist clinic, where 5% of the
2	# patients walking in have the disease -- ten times the general-population
3	# prevalence, because of who gets referred there.
4	#
5	# Compute $P(\text{disease} \mid \text{positive})$ for a patient at this clinic.
6	-----

Please replace ----- with valid code.

Exercise: what do you tell the patient?

Routine screening. Prevalence 0.5%, sensitivity 99%, specificity 95%.
The patient's test is positive and they ask what it means.

Exercise

```
1 # Assign the letter of the best answer, e.g. answer <- "A"
2 #
3 # A. "The test is 99% accurate, so it is very likely you have it."
4 # B. "About a 9% chance you have it. That is roughly 18 times your risk
5 #     before the test, which is why we follow up--but 9 out of 10 people
6 #     with this result turn out not to have it."
7 # C. "The test is meaningless at this prevalence, so ignore it."
8 # D. "There is a 9% chance the test is wrong."
9 answer <- _____
```

Please replace _____ with valid code.

Summary

- **Conditioning is subsetting.** $P(A | B)$ is computed by discarding everything outside B and counting what remains:
`mean(a[b == 1])`.
- $P(A | B) = P(A \cap B) / P(B)$. Most errors are denominator errors, so always ask “out of what?”
- **Multiplication rule:** $P(A \cap B) = P(B)P(A | B)$. Plain multiplication is the independent special case, and sampling without replacement is not it.
- **Bayes' rule** reverses a conditional, and it needs the base rate to do so. $P(A | B) \neq P(B | A)$, and they can differ by an order of magnitude.
- A 99%-sensitive, 95%-specific test for a disease with 0.5% prevalence gives a positive predictive value of **9%**. That is not a paradox; it is what happens when a small error rate is applied to a much larger group.
- **Base-rate neglect** is the habit of weighing the vivid evidence and forgetting the prior. It is why “99% accurate” is not a reportable property of a classifier and why “precision” exists as a separate word.
- The same arithmetic governs fraud detection, alerting, screening, and every rare-event classifier you will ever build.

Next: random variables, which is what you get when you stop asking whether an event happened and start asking for a number.

There are hundreds of named probability distributions. You will use about five.

They earn their place not because they are mathematically convenient—though they are—but because each one is the answer to a *kind of question about a process*. A named distribution is a compressed description of a mechanism: “independent yes/no trials with a fixed success rate” is a binomial, and once you have said that, every probability about it is already determined.

That is the skill this chapter trains, and it is why “which distribution would you use for X ?” is such a common interview question. The bad answer squints at a histogram. The good answer describes the process that generated the data and reads off the distribution that matches.

R’s four-letter alphabet

Before the distributions, the interface. Every distribution in R comes with four functions, distinguished by a single prefix letter. Learn the pattern once and you know it for all of them.

PREFIX	GIVES YOU	NORMAL	BINOMIAL	POISSON
d	density or mass at a point, $f(x)$ or $P(X = x)$	dnorm	dbinom	dpois
p	cumulative probability, $P(X \leq x)$	pnorm	pbinom	ppois
q	quantile: the x with $P(X \leq x) = p$	qnorm	qbinom	qpois
r	random draws from the distribution	rnorm	rbinom	rpois

`p` and `q` are inverses of each other: `p` takes a value and returns a probability, `q` takes a probability and returns a value. Mixing them up is the most common error in this chapter, and the way to catch it is to ask what the units of your answer should be. A cutoff has units of the data; a probability does not.

Every one of these takes the distribution’s parameters as further arguments— `pnorm(x, mean, sd)`, `pbinom(q, size, prob)`, `ppois(q, lambda)`. If you forget them, R silently uses defaults (`mean = 0`, `sd = 1`), which is a real source of wrong answers that look fine.

Bernoulli: one yes/no trial

The simplest random variable there is. $X = 1$ with probability p and $X = 0$ otherwise.

The generating process: one trial with two outcomes. A coin flip, a click, a conversion, a default, a patient recovering.

R Code

```
1 set.seed(6101)
2 b <- rbinom(20, size = 1, prob = 0.3)      # size = 1 is a Bernoulli
3 b
4 mean(b)
```

[1] 0 1 0 0 1 1 0 0 0 1 0 0 0 0 0 1 1 1 0 1
[1] 0.4

Twenty draws with a true rate of 0.3 gave a sample mean of 0.4. Nothing went wrong—that is [Probability](#)’s whole theme in one line, and quantifying how far off it is allowed to be is [The Standard Error of a Mean](#)’s job.

The Bernoulli is not a curiosity. Every logical vector you have taken the mean of in these notes was a sample of Bernoullis, and every proportion you will ever estimate—conversion rate, click rate, mortality rate, accuracy—is the mean of one.

Binomial: how many successes in n trials

Add up n independent Bernoulli trials that all share the same p , and you have a **binomial**.

THE THREE CONDITIONS, ALL OF WHICH CAN FAIL

$X \sim \text{Binomial}(n, p)$ requires:

1. a **fixed** number of trials n , decided before you start;
2. the trials are **independent**;
3. the success probability p is the **same** for every trial.

When someone hands you count data and asks whether it is binomial, these are the three questions. In real data, condition 3 is the one that usually fails, and we will simulate exactly that failure at the end of the chapter.

An email campaign sends 20 messages to each recipient list, and each message has a 30% chance of being opened. How many opens per list?

R Code

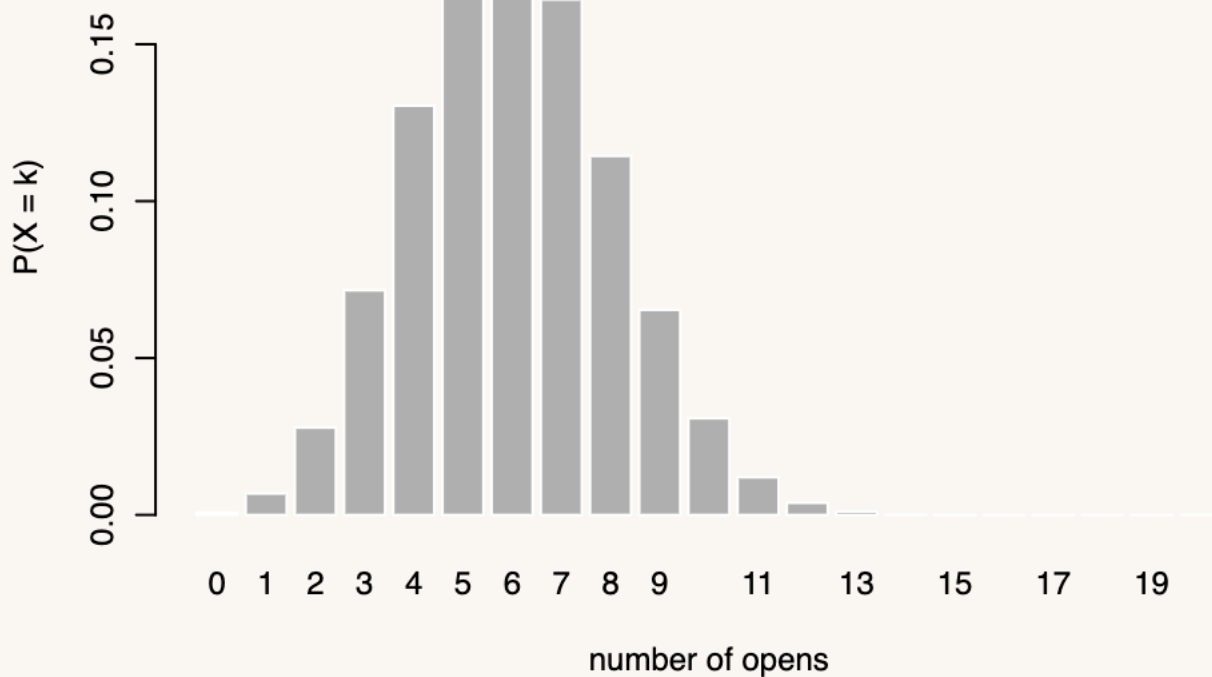
```
1 set.seed(6101)
2 opens <- rbinom(100000, size = 20, prob = 0.3)
3 head(opens, 15)
```

```
[1] 5 8 5 7 8 8 5 3 4 9 4 5 5 4 4
```

The parameters mean what they say: $n = 20$ is how many chances there are, and $p = 0.3$ is the chance each one comes off. The distribution is centered at $np = 6$.

R Code

```
1 k <- 0:20
2 barplot(dbinom(k, size = 20, prob = 0.3), names.arg = k,
3         col = "gray70", border = "white",
4         xlab = "number of opens", ylab = "P(X = k)")
```



Simulation against formula, as always:

R Code

```
1 round(c(simulated = mean(opens == 6),
2       dbinom      = dbinom(6, size = 20, prob = 0.3)), 4)
```

simulated	dbinom
0.1925	0.1916

R Code

```
1 round(c(simulated = mean(opens <= 4),
2       pbinom      = pbinom(4, size = 20, prob = 0.3)), 4)
```

simulated	pbinom
0.2398	0.2375

0.1925 against 0.1916; 0.2398 against 0.2375. The `d` and `p` functions are doing in closed form what 100,000 simulated lists did by brute force.

And `q` runs it backwards—what open count is the 95th percentile?

R Code

```
1 qbinom(0.95, size = 20, prob = 0.3)
```

```
[[1]] 9
```

Nine. So a list that produced 12 opens is genuinely unusual under this model, and that sentence—*this outcome is unusual under this model*—is the seed of every hypothesis test you will meet later in the term.

Exercise: a tail probability, and the off-by-one

A fair coin is flipped 20 times.

Exercise

```
1 # Compute P(X >= 15): the probability of getting 15 OR MORE heads
2 # in 20 flips of a fair coin. Use pbinom(), not a simulation.
3 # Watch the endpoint: pbinom(q, ...) gives P(X <= q).
4 -----
```

Please replace ----- with valid code.

Exercise: simulate it, then check it

Exercise

```
1 # `opens` holds 100,000 simulated lists of 20 emails at a 30% open rate.
2 # Estimate P(8 or more opens) from the simulation.
3 -----
```

Please replace ----- with valid code.

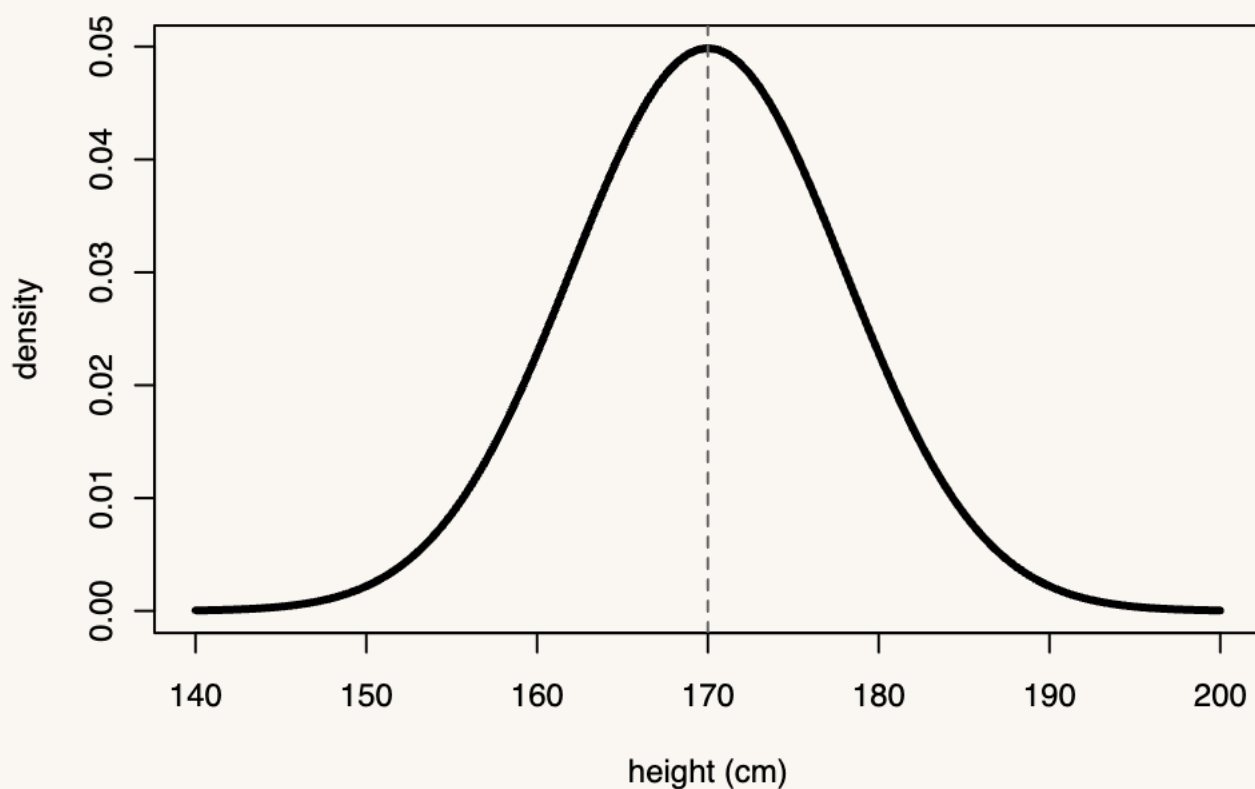
Normal: the one everybody has heard of

The Normal (or Gaussian) distribution is continuous, symmetric, and defined by two parameters that are exactly what you would want them to be: μ the center and σ the spread.

The generating process: a quantity built from **many small, independent contributions added together**. Adult height is genetics plus nutrition plus childhood illness plus a hundred other things, none dominant. Measurement error is the accumulation of many small imperfections. That is the mechanism, and it is why the Normal turns up as often as it does.

R Code

```
1 curve(dnorm(x, mean = 170, sd = 8), from = 140, to = 200, lwd = 3,
2       xlab = "height (cm)", ylab = "density")
3 abline(v = 170, lty = 2, col = "gray40")
```



THE 68-95-99.7 RULE

For any Normal distribution, the proportion within 1, 2, and 3 standard deviations of the mean is fixed:

R Code

```
1 round(c(within_1_sd = pnorm(1) - pnorm(-1),
2         within_2_sd = pnorm(2) - pnorm(-2),
3         within_3_sd = pnorm(3) - pnorm(-3)), 4)
```

```
within_1_sd within_2_sd within_3_sd
0.6827      0.9545      0.9973
```

68%, 95%, 99.7%. It does not depend on μ or σ —rescaling a Normal gives another Normal, which is why one table used to serve for all of them.

Worth memorizing because it makes you fast: an observation two SDs out is noticeably unusual, three SDs out is rare. And the multiplier for a 95% interval is not exactly 2 but `qnorm(0.975) = 1.96`, a number you will type roughly a hundred times between here and December.

The three non-random functions, on a test scored with mean 100 and SD 15:

R Code

```
1 round(c(
2   density_at_115 = dnorm(115, mean = 100, sd = 15), # d: height of the curve
```

```

3 | P_below_115 = pnorm(115, mean = 100, sd = 15), # p: cumulative probability
4 | P_above_130 = pnorm(130, mean = 100, sd = 15, lower.tail = FALSE),
5 | cutoff_top_2.5 = qnorm(0.975, mean = 100, sd = 15) # q: the value at a quantile
6 | ), 4)

```

	density_at_115	P_below_115	P_above_130	cutoff_top_2.5
	0.0161	0.8413	0.0228	129.3995

Read them as four different kinds of question. The density is a curve height and means little on its own. `pnorm` answers “what fraction score below 115?”—84%. `lower.tail = FALSE` answers “what fraction above 130?”—2.3%. And `qnorm` answers “what score puts you in the top 2.5%?”—129.4, which is in points, not in probability.

Exercise: a cutoff, not a probability

A standardized test is Normal with mean 100 and standard deviation 15.

Exercise

```

1 | # A program admits the top 10% of scorers.
2 | # What is the minimum score that gets in?
3 | -----

```

Please replace `-----` with valid code.

Poisson: counts of events in a window

The generating process: events arriving **independently at a constant average rate** over a fixed interval of time or space, with no fixed number of trials and no natural upper bound.

Support tickets per hour. Emergency calls per shift. Typos per page. Website visits per minute. Defects per square meter. Earthquakes per decade.

One parameter, λ , which is the expected number of events per window.

R Code

```

1 | set.seed(6101)
2 | tickets <- rpois(100000, lambda = 3.2) # 3.2 tickets per hour, on average
3 |
4 | round(rbind(simulated = table(factor(tickets, levels = 0:10)) / length(tickets),
5 |           exact      = dpois(0:10, lambda = 3.2)), 4)

```

	0	1	2	3	4	5	6	7	8	9
simulated	0.0408	0.1322	0.2084	0.2227	0.1765	0.1145	0.0613	0.0275	0.0104	0.004
exact	0.0408	0.1304	0.2087	0.2226	0.1781	0.1140	0.0608	0.0278	0.0111	0.004

	10
simulated	0.0013
exact	0.0013

THE POISSON'S FINGERPRINT: MEAN EQUALS VARIANCE

R Code

```
1 round(c(mean = mean(tickets), variance = var(tickets), lambda = 3.2), 3)
```

	mean	variance	lambda
	3.192	3.188	3.200

For a Poisson, $E[X] = \text{Var}(X) = \lambda$. Both come out at about 3.19 here.

This is a genuinely useful diagnostic. Given count data, compute the mean and the variance. If the variance is much **larger** than the mean—which happens constantly—your counts are **overdispersed**, the constant-rate assumption is wrong, and a Poisson model will understate your uncertainty. That usually means the rate is not constant: some hours are busier, some pages have more typos, some users click far more than others.

The relationship to the binomial is worth one line: a binomial with huge n and tiny p , holding $np = \lambda$, converges to a Poisson. That is why the Poisson is sometimes called the law of rare events, and it is why “many chances, each unlikely” and “a steady arrival rate” describe the same data.

Uniform: no value preferred over any other

The generating process: every value in a range is equally likely, and nothing in the mechanism favors any part of it.

R Code

```
1 set.seed(6101)
2 u <- runif(100000, min = 0, max = 1)
3 round(c(mean = mean(u), P_below_0.3 = mean(u < 0.3), punif = punif(0.3)), 4)
```

	mean	P_below_0.3	punif
	0.4988	0.3019	0.3000

Genuinely uniform real-world data is rarer than students expect—“I have no information, so everything is equally likely” is a statement about you, not about the world. Where the uniform earns its keep is as the **raw material of simulation**: `runif` is the engine underneath `sample()`, and under the hood every other `r`-function is a transformation of uniform draws. You will also meet it later in the term as the distribution of a p-value when the null hypothesis is true, which is a fact worth arriving pre-warned about.

A continuous uniform on $[0, 0.5]$ has density 2 everywhere inside the interval—`dunif(0.3, 0, 0.5)` returns 2. Densities above 1 are ordinary, as [Random Variables](#) insisted.

Exponential: the time until the next event

The Poisson counts how many events fall in a window. The **exponential** measures the gap between consecutive events. Same process, two questions—if arrivals are Poisson at rate λ , the waiting times are $\text{Exponential}(\lambda)$.

R Code

```
1 set.seed(6101)
2 # Support tickets arrive at 3 per hour. How long between them?
3 gaps <- rexp(100000, rate = 3)
4
5 round(c(mean_gap = mean(gaps), theory = 1 / 3,
```

6	sd_gap = sd(gaps),				
7	P_under_10min = mean(gaps < 1 / 6), pexp = pexp(1 / 6, rate = 3)), 4)				
	mean_gap	theory	sd_gap	P_under_10min	pexp
	0.3340	0.3333	0.3332	0.3920	0.3935

Two features are worth carrying away. The mean and the standard deviation are both $1/\lambda$, so an exponential is always as spread out as it is large—a distribution with a mean of twenty minutes routinely produces gaps of an hour. And it is **memoryless**: having already waited ten minutes tells you nothing about how much longer you will wait.

R Code				
1	set.seed(6101)			
2	gaps <- rexp(200000, rate = 3)			
3				
4	# Among gaps that already exceeded 20 minutes, how many exceed 40?			
5	already <- gaps > 1/3			
6	round(c(P_extra_20min_given_waited = mean(gaps[already] > 2/3),			
7	P_20min_from_scratch = mean(gaps > 1/3)), 4)			
	P_extra_20min_given_waited	P_20min_from_scratch		
	0.3677	0.3697		

The two numbers agree. This is the formal version of the intuition that a bus which is already late is not therefore “due”—and it is the assumption that fails most often in practice, because real machines wear out and real customers give up.

Log-normal: positive, skewed, and everywhere

The Normal arises when many small effects **add**. When they **multiply**, you get the log-normal—because the logarithm of a product is a sum, and a sum of many small independent things is Normal.

That covers a great deal of the data you will actually be handed: incomes, revenue per customer, response latency, session durations, file sizes, claim amounts. All are positive, all have a long right tail, and none of them are Normal.

R Code				
1	set.seed(6101)			
2	# Response times in milliseconds.			
3	ms <- rlnorm(100000, meanlog = 5, sdlog = 0.8)			
4				
5	round(c(mean = mean(ms), median = median(ms),			
6	p95 = quantile(ms, 0.95), max = max(ms)), 1)			
	mean	median	p95.95%	max
	205.1	148.6	556.5	4686.4

The mean sits well above the median, which is the signature you met in [Describing a Single Variable](#)—now with a mechanism attached rather than just a shape.

There is a trap here that catches people who ought to know better. `meanlog` is the mean **of the logarithm**, not of the variable:

R Code			
1	set.seed(6101)		
2	ms <- rlnorm(100000, meanlog = 5, sdlog = 0.8)		
3			
4	round(c(exp_meanlog = exp(5), # what people assume the mean is		
5	actual_mean = mean(ms), # what it actually is		
6	actual_median = median(ms)), 1)		
	exp_meanlog	actual_mean	actual_median
	148.4	205.1	148.6

`exp(meanlog)` is the **median**, not the mean; the mean is larger by a factor of $e^{\sigma^2/2}$. Reporting the wrong one understates an average revenue or latency by more than a quarter here—148.6 against 205.1—and by more as the spread grows.

The practical move is to **work on the log scale**—take logs, do the ordinary Normal-theory analysis, and transform back while remembering that the back-transformed center is a median. Nearly every “our average response time is fine but users are complaining” story is a log-normal being summarized by its mean.

Choosing a distribution: ask what made the data

Here is the decision, stated as a set of questions about the **process**, not about the shape of a histogram:

IF THE PROCESS IS...	REACH FOR
a single yes/no outcome	Bernoulli (p)
a count of successes out of a fixed number of independent trials sharing the same p	Binomial (n, p)
a count of events in a fixed window, arriving independently at a steady rate, with no fixed n	Poisson (λ)
a continuous quantity built from many small independent contributions that add	Normal (μ, σ)
a positive quantity built from many small independent contributions that multiply	Log-normal (μ, σ)
the waiting time between events arriving at a steady rate	Exponential (λ)
every value in a range equally likely, <i>by the mechanism</i>	Uniform (a, b)

🔑 WHICH DISTRIBUTION, AND WHY

You are modeling **the number of emergency calls a dispatch center receives in a given 10-minute window**. Calls arrive independently from a large population at a roughly steady average rate, and there is no fixed upper limit on how many could arrive.

Which distribution is the natural starting point, and why?

- ☐ Binomial, because each call either happens or does not, so the count is a sum of successes.
- ☐ Poisson, because the quantity is a count of independent events arriving at a constant rate in a fixed window, with no fixed number of trials and no natural maximum.
- ☐ Normal, because counts of many small independent contributions are approximately Normal.
- ☐ Uniform, because with no information favouring any particular count, all counts should be treated as equally likely.



When the assumption fails: overdispersion

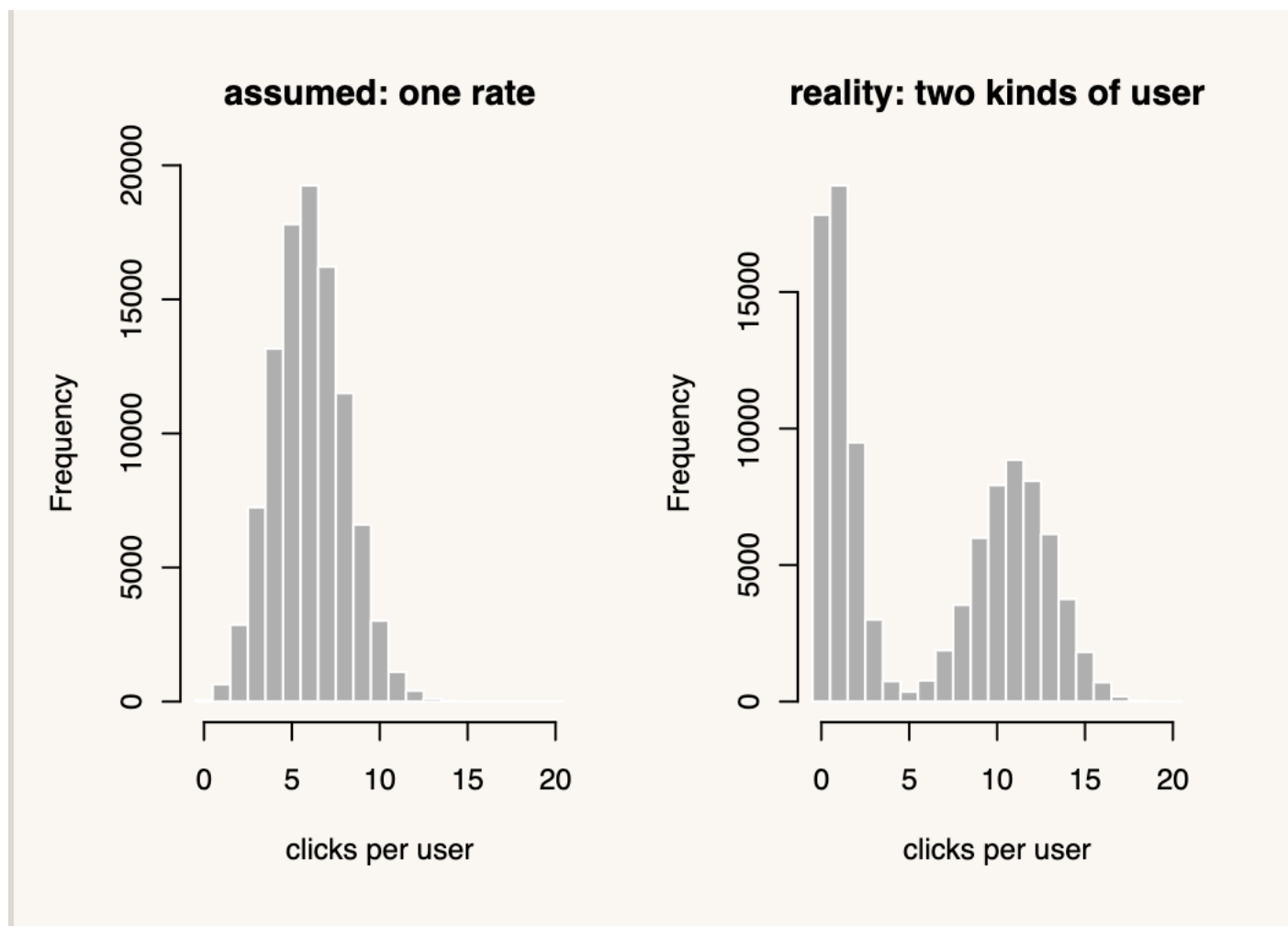
The binomial’s third condition—the same p for every trial—is the one that breaks in real data, and it breaks quietly.

Twenty ad impressions per user. Suppose you are told the overall click rate is 30% and you model clicks as $\text{Binomial}(20, 0.3)$. But users are not interchangeable: half are casual browsers who click 5% of the time, half are engaged users who click 55%. The **overall** rate is still exactly 30%.

R Code				
1	<code>set.seed(6101)</code>			
2	<code>segment <- sample(c("casual", "engaged"), 100000, replace = TRUE)</code>			
3	<code>p_user <- ifelse(segment == "engaged", 0.55, 0.05)</code>			
4	<code>clicks <- rbinom(100000, size = 20, prob = p_user)</code>			
5				
6	<code>set.seed(6101)</code>			
7	<code>pure <- rbinom(100000, size = 20, prob = 0.3) # the model you assumed</code>			
8				
9	<code>round(c(mean_real = mean(clicks), mean_model = mean(pure),</code>			
10	<code>sd_real = sd(clicks), sd_model = sd(pure)), 3)</code>			
	<code>mean_real</code>	<code>mean_model</code>	<code>sd_real</code>	<code>sd_model</code>
	6.000	5.990	5.285	2.045

Identical means—6.00 against 5.99. And a standard deviation of **5.28** against the model's **2.05**, more than two and a half times too small.

R Code	
1	<code>par(mfrow = c(1, 2))</code>
2	<code>hist(pure, breaks = seq(-0.5, 20.5, 1), col = "gray70", border = "white",</code>
3	<code>main = "assumed: one rate", xlab = "clicks per user")</code>
4	<code>hist(clicks, breaks = seq(-0.5, 20.5, 1), col = "gray70", border = "white",</code>
5	<code>main = "reality: two kinds of user", xlab = "clicks per user")</code>



One distribution has a single hump; the other has two, and the model's center sits in the valley between them—the same failure mode as the commute data in [Describing a Single Variable](#), arriving now with a probability model attached.

The practical damage:

R Code					
1	<code>round(c(P_at_most_1_real = mean(clicks <= 1),</code>				
2	<code> P_at_most_1_model = mean(pure <= 1)), 4)</code>				
<table> <tr> <th>P_at_most_1_real</th><th>P_at_most_1_model</th></tr> <tr> <td>0.3673</td><td>0.0071</td></tr> </table>		P_at_most_1_real	P_at_most_1_model	0.3673	0.0071
P_at_most_1_real	P_at_most_1_model				
0.3673	0.0071				

The model says 0.7% of users click at most once. The reality is **37%**. If you built an alerting rule on “fewer than two clicks is anomalous”, you would have flagged more than a third of your users as anomalies and had no idea why.

THE LESSON, WHICH IS THE POINT OF THE WHOLE CHAPTER

Every number you would have reported about the *average* was right. The mean was exactly correct. What was wrong was the **spread**, and therefore every statement about how surprising an observation is—which is to say, every p-value, every confidence interval, every threshold.

Choosing a distribution is not a formality you perform before the real analysis. It is a claim about the mechanism, and when the claim is wrong the analysis is confidently, quietly wrong in exactly the direction that makes you overconfident.

The diagnostic is cheap: **compare the variance you observe with the variance the model implies**. For a binomial that is $np(1 - p)$ = 4.2; we observed 27.9. For a Poisson it is λ . When the observed spread is much larger, some parameter you assumed constant is not.

Exercise: which process is which

Four quantities:

1. Out of 500 patients in a trial, how many recovered.
2. How many cars pass a sensor between 9:00 and 9:05 on a quiet road.
3. The systolic blood pressure of one adult drawn at random.
4. Whether a single visitor to a page converts.

Exercise

```
1 # Assign the letter of the correct assignment, e.g. answer <- "A"
2 #
3 # A. 1 binomial, 2 Poisson, 3 Normal, 4 Bernoulli
4 # B. 1 Poisson, 2 binomial, 3 Normal, 4 Bernoulli
5 # C. 1 binomial, 2 Poisson, 3 uniform, 4 binomial
6 # D. All four are Normal, because with enough data everything is Normal.
7 answer <- _____
```

Please replace _____ with valid code.

Summary

- A named distribution is a **compressed description of a generating process**. Choose one by describing the mechanism, not by looking at a histogram.
- R gives every distribution four functions: **d** (density or mass), **p** (cumulative, $P(X \leq x)$), **q** (quantile, the inverse of **p**), and **r** (random draws). Check the units of your answer to catch **p / q** confusion.
- **Bernoulli(p)**: one yes/no trial. Every proportion you estimate is a mean of these.
- **Binomial(n, p)**: successes in a **fixed** number of **independent** trials with a **constant** p . All three conditions can fail.
- **Normal(μ, σ)**: a continuous quantity built from many small independent contributions. 68–95–99.7, and `qnorm(0.975) = 1.96`.
- **Poisson(λ)**: counts of independent events at a steady rate in a fixed window. Its fingerprint is **mean = variance**, and a variance much larger than the mean is evidence the rate is not constant.
- **Exponential(λ)**: the gap between Poisson arrivals. Mean and standard deviation are both $1/\lambda$, and it is memoryless—waiting longer does not make the event more due.
- **Log-normal**: positive quantities built from effects that multiply—income, revenue, latency, durations. `exp(meanLog)` is the **median**, not the mean. Work on the log scale.
- **Uniform(a, b)**: every value equally likely *by the mechanism*—not as a default for ignorance. Mostly seen as the raw material of simulation.
- When a distributional assumption fails, the **mean often survives and the spread does not**. A mixture of two click rates matched the binomial's mean exactly and had a standard deviation two and a half times larger, turning a 0.7% event into a 37% one.

Next: the practice problems, and then [The Standard Error of a Mean](#) turns these distributions into two numbers apiece and shows you what those two numbers buy.

Monday, September 14

Three weeks of describing data end here. From this point the question changes.

Everything you computed in [Describing a Single Variable](#) was a fact about the rows in front of you. The mean salary of those 240 alumni is \$70,600—there is nothing uncertain about it. But nobody hired you to describe 240 rows. They want to know about the population those rows came from, and the honest answer is that a different 240 people would have given you a different number.

To say anything about that, you need a language for “a different draw”. That language is probability, and this week builds it.

THE ONE HABIT THIS WEEK IS TRYING TO INSTALL

Simulate it before you solve it.

Probability is the branch of mathematics with the worst intuition-to-difficulty ratio: the algebra is short, and people who can do the algebra still get the answers wrong, because the wrong model felt obvious. A simulation is a check that does not care what felt obvious. You write down the *process*—flip this, draw that, condition on the other—and count.

So the pattern in every chapter this week is the same: build the process in code, count the outcomes, and only then write the formula and watch it agree. When the two disagree, one of them is a mistake you would otherwise have shipped.

What this week covers

PROBABILITY

Outcomes, events, and what a probability actually is. Watching a long-run frequency settle down as you run a simulation more times. The addition rule and complements. And **independence versus mutual exclusivity**, which are opposites that students routinely treat as synonyms—with a simulation where assuming independence is off by a factor of eight.

CONDITIONAL PROBABILITY AND BAYES

Conditioning is just restricting attention to a sub-population, and once you see it that way `mean(a[b == 1])` is the entire definition. Then the false-positive problem: a 99%-accurate test, a rare disease, and a positive result that means far less than everyone in the room assumes. We simulate a population of 100,000, count, and only then check the algebra.

RANDOM VARIABLES

A random variable is a *function* of a random outcome, and that one sentence resolves most of the confusion around the term. Discrete versus continuous, mass versus density, and the CDF—the one object that works for both.

THE DISTRIBUTIONS WE ACTUALLY USE

Bernoulli and binomial, Normal, Poisson, uniform. For each: the generating process that produces it, what its parameters mean, and R's `d / p / q / r` family. The skill being trained is choosing a distribution by asking what made the data, not by squinting at a histogram.

PRACTICE PROBLEMS

Predict-before-you-run, simulations that answer a different question from the one asked, and interpretations to argue about. Not graded—bring your answers and your disagreements.

Where this is heading

Next week takes the random variables you meet here and asks two questions about them: what value do they take on average, and how far do they typically stray from it. Those are **expectation** and **variance**, and [The Standard Error of a Mean](#) will use them to

derive the result the rest of the term rests on—that the variance of a sample mean is σ^2/n , so the noise in your estimate shrinks like $1/\sqrt{n}$.

That derivation needs exactly two things from this week: the idea of a random variable as a function of a random draw, and the idea of independence. If you finish this week solid on those two, next week is short. If you finish shaky on independence, next week will feel like magic, and magic is not a thing you can check.

DUE THIS WEEK

HW 1 is due Sunday at 11:59 p.m., the night before class. And post your questions on these pages by **Monday at 12:00 noon**, the morning of class—that afternoon’s session is built from them.

Probability is the topic where I most expect the questions to be good ones, and where “I followed every step and still do not believe it” is the most useful thing you can write.

Everything in the last three weeks was a statement about data you had. From here on, every statement is about data you *might have had*, and probability is the vocabulary for that.

The awkward thing about probability is that it is easy to compute and hard to believe. The algebra fits on a postcard. The mistakes are almost never algebraic: they come from writing down the wrong model of the situation, confidently, and then doing correct arithmetic on it. That is why this course does simulation first. A simulation forces you to state the process—what gets flipped, what gets drawn, what depends on what—and then it counts, without any opinion about what the answer ought to be.

Outcomes, events, and probability

Three words, used precisely from here on.

An **outcome** is one complete result of a random process. Roll two dice and the outcome is a pair, like $(3, 4)$. There are 36 of them, and the collection of all possible outcomes is the **sample space**.

An **event** is a *set* of outcomes—something you can ask a yes/no question about. “The dice sum to 7” is an event containing six outcomes. “At least one die shows a 6” is an event containing eleven. Events are the things probability assigns numbers to.

A **probability** is a number between 0 and 1 attached to an event, obeying three rules that are less a discovery than a decision about what the word should mean:

1. $P(A) \geq 0$ for every event.
2. $P(\text{something happens}) = 1$.
3. If A and B cannot both occur, $P(A \text{ or } B) = P(A) + P(B)$.

Everything else in this chapter follows from those three lines.

What the number means

There are two respectable answers, and it is worth knowing that the argument exists rather than being ambushed by it later.

The **frequentist** reading: $P(A)$ is the proportion of times A happens if you repeat the process forever. A probability is a property of a repeatable experiment.

The **Bayesian** reading: $P(A)$ is a degree of belief, which you may hold about things that happen only once—“the probability this model ships on time”.

This course is mostly frequentist in its machinery, because confidence intervals and p-values are frequentist objects and you need to be able to say precisely what they claim. But the frequentist reading has a real cost: it makes “probability” meaningless for one-off events, and people quietly use it that way constantly. Notice when you are doing it.

The useful thing about the frequentist reading is that it is *literally what a simulation does*. So let us watch it happen.

Watching a probability settle

Here are 100,000 coin flips. Nothing is being computed from a formula; we are just flipping.

R Code	
1	<code>set.seed(6101)</code>
2	<code>flips <- sample(c("H", "T"), 100000, replace = TRUE)</code>
3	<code>head(flips, 12)</code>
[1] "T" "T" "H" "T" "T" "T" "T" "T" "T" "H" "T" "T"	

Now the proportion of heads in the first 10, the first 1,000, and all 100,000:

--

R Code

```

1 round(c(first_10   = mean(flips[1:10] == "H"),
2         first_1000 = mean(flips[1:1000] == "H"),
3         all_100000 = mean(flips == "H")), 4)

```

```

first_10 first_1000 all_100000
0.2000   0.4980   0.5004

```

The first ten flips gave **two heads**. Not 5, not “about 5”—2. If you had stopped there you would have concluded the coin lands heads 20% of the time, and you would have been wrong by a mile with a perfectly correct calculation on perfectly real data.

By a thousand flips it is 0.498. By a hundred thousand, 0.5004.

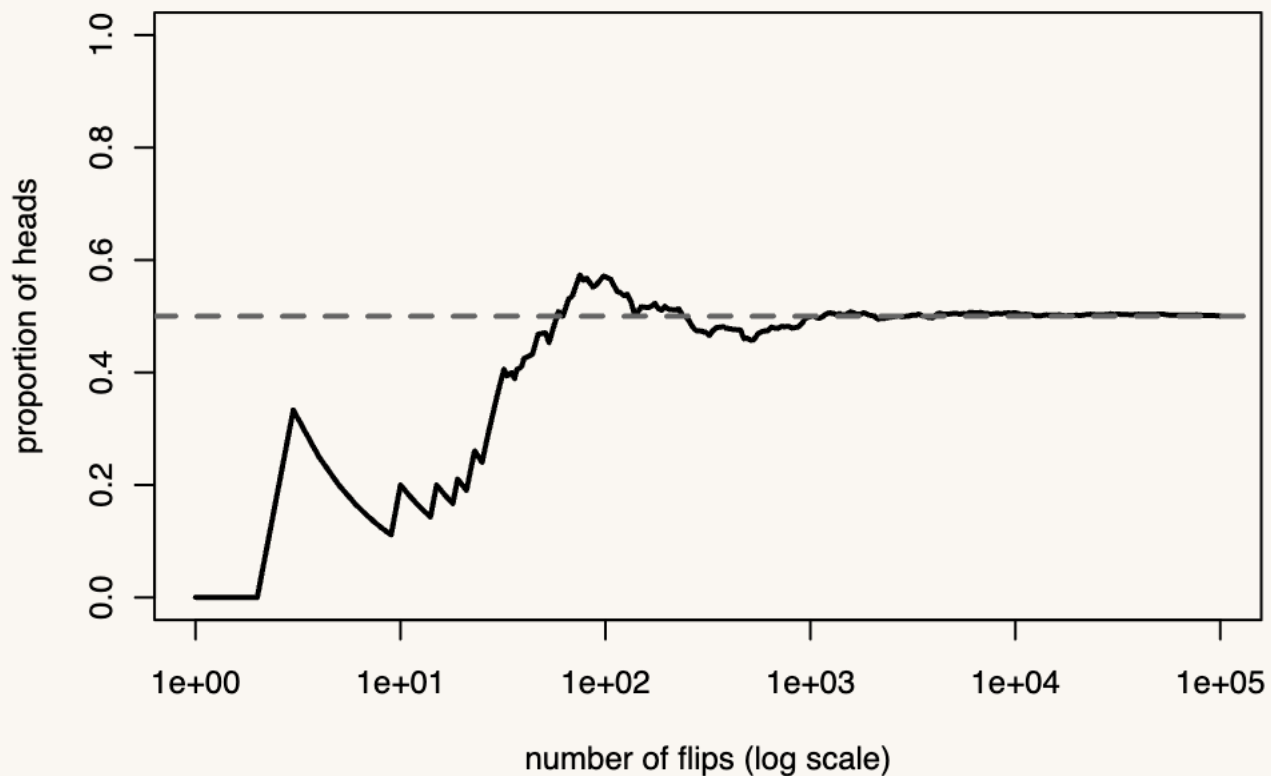
Plot the running proportion, on a log scale so that the early chaos and the late calm are both visible:

R Code

```

1 run <- cumsum(flips == "H") / seq_along(flips)
2 idx <- unique(round(10^seq(0, 5, length.out = 300)))
3
4 plot(idx, run[idx], type = "l", log = "x", ylim = c(0, 1), lwd = 2,
5       xlab = "number of flips (log scale)", ylab = "proportion of heads")
6 abline(h = 0.5, lty = 2, lwd = 2, col = "gray40")

```



WHAT THAT PICTURE IS AND IS NOT SAYING

The curve settles toward 0.5. That is the **law of large numbers**, and it is the whole justification for using simulation as a tool.

What it does **not** say is that the coin “corrects itself”. After a run of ten heads, the next flip is still 50/50—the coin has no memory and no obligation. The proportion converges because the early results get diluted by an ever-growing denominator, not because later flips compensate. Nothing pushes back; the imbalance just stops mattering.

That distinction is the gambler’s fallacy, and it survives in professional settings dressed as “we’re due for a good quarter”.

The same picture is also a warning about sample size. At $n = 10$ the estimate was off by 0.30. At $n = 1,000$ it was off by 0.002. **How fast** that error shrinks is exactly the question [The Standard Error of a Mean](#) answers, and the answer— $1/\sqrt{n}$ —is the single most load-bearing fact in the rest of this course.

Events, and combining them

Take a concrete process: roll two fair dice, 100,000 times.

R Code

```
1 set.seed(6101)
2 d1 <- sample(1:6, 100000, replace = TRUE)
3 d2 <- sample(1:6, 100000, replace = TRUE)
4
5 seven <- (d1 + d2) == 7           # event A: the dice sum to 7
6 six   <- d1 == 6 | d2 == 6       # event B: at least one die shows a 6
```

Notice what `seven` and `six` are: **logical vectors**, one entry per roll, `TRUE` where the event occurred. An event really is a subset of the outcomes, and in R a subset is a logical vector. The probability of an event is then the mean of that vector, because `mean()` of `TRUE / FALSE` counts the proportion of `TRUE`s—the same trick you used in [Describing a Single Variable](#).

R Code

```
1 round(c(P_seven = mean(seven), P_six = mean(six)), 3)
```

```
  P_seven  P_six
  0.165    0.301
```

About 0.165 and 0.301, against exact answers of $6/36 = 0.1667$ and $11/36 = 0.3056$. Close, but not equally close. With 100,000 rolls an estimate of $11/36$ usually lands within about 0.0015 of it, and this one is 0.005 away: this particular seed rolled noticeably few sixes on the first die. That shortfall comes back below, and it is worth knowing it is there.

The complement rule

The event “**not** A ” is the complement A^c , and

$$P(A^c) = 1 - P(A).$$

This is not a triviality—it is the single most useful computational trick in elementary probability, because “at least one” is usually painful to count directly and “none” is usually easy.

R Code

```
1 round(c(direct      = mean(six),
2         complement = 1 - mean(!six)), 3)
```

```
  direct complement
  0.301    0.301
```

Same number, which it must be. The payoff shows up when the direct count is hard: the probability of at least one 6 in ten rolls is a nuisance to enumerate, but the probability of *no* 6 is $(5/6)^{10}$, so the answer is $1 - (5/6)^{10}$.

The addition rule

For any two events,

$$P(A \cup B) = P(A) + P(B) - P(A \cap B).$$

You subtract the overlap because adding the two counts it twice. When A and B are **mutually exclusive**—they cannot both happen—the overlap is empty and the rule collapses to $P(A) + P(B)$, which is axiom 3.

Check it against the simulation:

R Code			
1	round(c(P_either_sim = mean(seven six),		
2	P_A_plus_P_B = mean(seven) + mean(six),		
3	addition_rule = mean(seven) + mean(six) - mean(seven & six)), 3)		
	P_either_sim	P_A_plus_P_B	addition_rule
	0.411	0.466	0.411

Simply adding gives 0.466. The correct answer is 0.411. The gap is the 5.5% of rolls that are *both* a seven and a six—(1, 6) and (6, 1)—counted twice.

Two dice, two events, one of the two most basic rules in the subject, and naive addition is off by 13%. This is the whole argument for checking yourself by counting.

A cross-tabulation makes the overlap visible:

R Code	
1	<code>table(sums_to_seven = seven, has_a_six = six)</code>
	<pre> has_a_six sums_to_seven FALSE TRUE FALSE 58915 24597 TRUE 11021 5467</pre>

Exercise: at least one six

Exercise	
1	<code># `d1` and `d2` are 100,000 rolls of two dice.</code>
2	<code># Using the COMPLEMENT rule, estimate P(at least one die shows a 6):</code>
3	<code># compute the proportion of rolls with NO six, and subtract from 1.</code>
4	<code>-----</code>

Please replace `-----` with valid code.

Exercise: the union, honestly

Exercise	
1	<code># `seven` and `six` are logical vectors marking the two events.</code>
2	<code># Estimate P(the roll sums to 7 AND shows at least one six)</code>
3	<code># -- that is, the OVERLAP that the addition rule has to subtract.</code>
4	<code>-----</code>

Please replace _____ with valid code.

Independence versus mutual exclusivity

Now the section that this chapter exists for.

These two terms describe *opposite* situations, and they are confused so routinely that “are mutually exclusive events independent?” is a standard screening question in data science interviews. It is asked because the wrong answer is fast and confident.

THE TWO DEFINITIONS, SIDE BY SIDE

Mutually exclusive (disjoint): A and B cannot both happen.

$$P(A \cap B) = 0$$

Independent: knowing whether B happened tells you nothing about A .

$$P(A \cap B) = P(A)P(B), \quad \text{equivalently } P(A | B) = P(A)$$

These are not variations on a theme. If A and B are mutually exclusive and both have positive probability, they are **maximally dependent**: the moment you learn B occurred, $P(A)$ drops to zero. Knowing B told you everything.

Two mutually exclusive events can be independent only if one of them has probability zero, which is a degenerate case with nothing in it.

Confirm it on the dice. “Sums to 7” and “sums to 10” are mutually exclusive:

R Code

```
1 ten <- (d1 + d2) == 10
2
3 round(c(P_seven = mean(seven),
4         P_ten    = mean(ten),
5         P_both   = mean(seven & ten),      # 0: mutually exclusive
6         product  = mean(seven) * mean(ten)), # what independence would require
7         4)
```

P_seven	P_ten	P_both	product
0.1649	0.0827	0.0000	0.0136

$P(A \cap B) = 0$ but $P(A)P(B) \approx 0.0136$. Zero is not 0.0136, so the events are not independent—they are as dependent as two events get.

Meanwhile, “the first die is a 6” and “the second die is a 6” *are* independent, because the dice do not communicate:

R Code

```
1 round(c(P_both = mean(d1 == 6 & d2 == 6),
2         product = mean(d1 == 6) * mean(d2 == 6)), 4)
```

P_both	product
0.0282	0.0270

0.0282 against 0.0270, with the exact answer $1/36 = 0.0278$ between them.

They are not equal, and no simulation would make them equal: independence is a statement about the true probabilities, and both numbers here are estimates. But be honest about this run. A gap of 0.0012 is wider than usual. With 100,000 rolls, two

genuinely independent dice produce a gap with a standard deviation of about 0.0004, so this one is about 2.7 standard deviations out. The cause is the same shortfall of sixes: the first die came up six 16.3% of the time instead of 16.7%, which drags the product down.

What separates independence from dependence is not that one run matches. It is that the gap shrinks as you roll more, and keeps shrinking. The mutually exclusive pair above shows the opposite: 0 against 0.0136, a gap no amount of rolling closes.

EXCLUSIVE OR INDEPENDENT



A single card is drawn from a well-shuffled deck. Let A be “the card is a heart” and B be “the card is a spade”.

Which statement is correct?

- ☐ A and B are independent, because there is only one draw and nothing about it can influence anything else.
- ☐ A and B are mutually exclusive, and that makes them **dependent**: learning that B occurred changes $P(A)$ from $1/4$ to 0.
- ☐ A and B are both mutually exclusive and independent, since $P(A \cap B) = 0$ and disjoint events cannot affect each other.
- ☐ Independence cannot be judged here, because independence is a property of repeated trials and this is a single draw.



When independence is assumed and is not true

Here is why this matters outside a quiz.

Two servers sit in the same rack. Each fails on about 5% of days. You are asked: what is the probability that **both** are down at once, taking the service offline?

The instant answer is $0.05 \times 0.05 = 0.0025$, one day in 400. That answer assumes independence. Nobody stated independence; it got assumed because the multiplication was available.

But the servers share a rack, and the rack shares a power supply. Model it honestly: on 2% of days the rack power fails and both servers go down together. On the other days, each fails on its own with a small probability, tuned so that each server's overall failure rate is still exactly the 5% you were told.

R Code

```
1 set.seed(6101)
2 n <- 200000
3
4 rack <- rbinom(n, 1, 0.02)      # shared cause: the whole rack goes down
5 own_a <- rbinom(n, 1, 0.0306)   # server A's own faults
6 own_b <- rbinom(n, 1, 0.0306)   # server B's own faults
7
8 a <- pmax(rack, own_a)          # A is down if the rack died OR A died
9 b <- pmax(rack, own_b)
10
11 round(c(P_A = mean(a), P_B = mean(b)), 4)
```

P_A	P_B
0.0497	0.0497

Each server is down about 5% of the time, exactly as reported. The marginal numbers are not in dispute. Now the joint:

R Code		
1	<code>round(c(P_both_simulated = mean(a & b),</code>	
2	<code> P_A_times_P_B = mean(a) * mean(b),</code>	
3	<code> ratio = mean(a & b) / (mean(a) * mean(b))), 5)</code>	
	<code>P_both_simulated</code>	<code>P_A_times_P_B</code>
	0.02051	0.00247
		<code>ratio</code>
		8.31289

The real answer is about **0.0205**, roughly one day in 49. The independence answer was 0.0025, one day in 400. **The assumption was wrong by a factor of eight**, and every marginal number that went into it was correct.

Look at it as a conditional and the mechanism is obvious:

R Code	
1	<code>round(mean(a[b == 1]), 3)</code>
[[1]] 0.413	

Given that server B is down, server A is down **41%** of the time—not 5%. B being down is strong evidence that the rack lost power, and the rack losing power takes A with it.

THE GENERAL SHAPE OF THIS ERROR

Independence fails whenever two things share a hidden common cause. The pattern recurs everywhere:

- Two servers in one rack, one data center, one cloud region.
- Two loans that default independently—until a recession arrives, which is precisely what happened in 2008 to models that had priced mortgage pools assuming independence.
- Two “independent” A/B test users who are actually the same person on two devices.
- Repeated measurements on the same patient, treated as separate observations.

Multiplying probabilities is the most-used move in applied probability and the most-abused. Before you multiply, say out loud: *what would have to be true for these to be independent, and is it?*

You will meet this again in [The Standard Error of a Mean](#) in a friendlier form. Variances add for independent quantities—which is the fact that makes $\text{Var}(X) = \sigma^2/n$ work—and if the observations are not independent, that formula is wrong too, in the same direction and for the same reason.

Exercise: check independence by counting

Exercise	
1	<code># `a` and `b` are 0/1 indicators of the two servers being down.</code>
2	<code># Estimate P(A is down B is down): among the days when B was down,</code>
3	<code># what proportion had A down too?</code>
4	<code>-----</code>

Please replace `-----` with valid code.

Exercise: which is which

A single fair die is rolled once.	
A = "the roll is even"	(2, 4, 6)
B = "the roll is 5 or 6"	
C = "the roll is odd"	(1, 3, 5)

Exercise	
1	<code># Assign the letter of the ONE correct statement, e.g. answer <- "A"</code>

```

2 | #
3 | # A. A and C are independent, because the die has no memory.
4 | # B. A and B are mutually exclusive, since an even roll cannot be 5.
5 | # C. A and C are mutually exclusive, and therefore NOT independent:
6 | #   P(A and C) = 0 while P(A)P(C) = 1/4.
7 | # D. A and B are independent AND mutually exclusive, which is the usual
8 | #   situation for events defined on the same roll.
9 | answer <- _____

```

Please replace _____ with valid code.

Summary

- An **outcome** is one result; an **event** is a set of outcomes; a **probability** is a number on an event obeying three short rules. In R, an event is a logical vector and its probability is `mean()` of that vector.
- The **frequentist** reading of a probability is a long-run frequency, and that is exactly what a simulation computes. Ten flips told us 0.2; a hundred thousand told us 0.5004.
- The proportion settles because early results get diluted, **not** because later results compensate. Nothing is ever “due”.
- **Complement rule:** $P(A^c) = 1 - P(A)$. Use it whenever the phrase “at least one” appears.
- **Addition rule:** $P(A \cup B) = P(A) + P(B) - P(A \cap B)$. Forgetting the overlap overstated a two-dice answer by 13%.
- **Mutually exclusive** means $P(A \cap B) = 0$. **Independent** means $P(A \cap B) = P(A)P(B)$. Events that are mutually exclusive and have positive probability are the *most dependent* events there are.
- Assuming independence because multiplication was convenient made a correlated-failure answer wrong by a factor of eight, with every input correct.

Next: what `mean(a[b == 1])` was really doing, and the problem that most interview candidates get wrong.

None of these are graded and none of them are auto-checked. They are here so you arrive on Monday with something to argue about.

Work them in order. Most of the first section comes in two parts: **predict**, then **run**. The prediction is where the learning happens. Probability is the topic where skipping straight to the answer is most tempting and most useless, because every output looks reasonable once you have seen it.

Write your prediction down. On paper, in a comment, in a note on your phone. Then run it.

WHAT TO BRING TO CLASS

For each problem, bring **an answer you are prepared to be wrong about**. In probability, being wrong is the normal case and it is how the intuition gets built—several of these are designed so that the confident answer is the wrong one.

The in-class discussion is graded on whether you said something substantive and could defend it, not on whether it was correct.

Predict before you run

A1. Twenty flips

Predict all four numbers before running.

R Code				
1	set.seed(6101)			
2	x <- sample(c(0, 1), 20, replace = TRUE)			
3				
4	c(prop_first_5 = mean(x[1:5]),			
5	prop_all = mean(x),			
6	n_ones = sum(x),			
7	longest_run = max(rle(x)\$lengths))			
	prop_first_5	prop_all	n_ones	longest_run
	0.8	0.8	16.0	6.0

Then answer: the fair coin came up 1 sixteen times out of twenty. Is this evidence that `sample()` is broken? Write down what you would need to compute to decide, without computing it—you will have the tools in three weeks. Then say what the `longest_run` figure would have to be before you started to worry, and why “that looks non-random” is a bad instinct.

DISCUSSION



A2. Two events on one die

Predict all four numbers, and in particular predict whether the last two will be equal.

R Code				
1	<code>set.seed(6101)</code>			
2	<code>d <- sample(1:6, 100000, replace = TRUE)</code>			
3				
4	<code>A <- d %% 2 == 0 # the roll is even</code>			
5	<code>B <- d >= 5 # the roll is 5 or 6</code>			
6				
7	<code>round(c(P_A = mean(A), P_B = mean(B),</code>			

```

8 | P_A_and_B = mean(A & B),
9 | P A times P B = mean(A) * mean(B)), 4)

```

	P_A	P_B	P_A_and_B	P_A_times_P_B
	0.4977	0.3311	0.1635	0.1648

Then answer: A and B are events on a *single roll of a single die*, and they are independent. Explain how that is possible when they are computed from the same number. Then construct a third event C on the same die that is **not** independent of A , and say how you would know before simulating.

DISCUSSION

A3. The screen, at a different prevalence

Do not run this yet. Write down your guess for the second number first.

```

R Code
1 | set.seed(6101)
2 | N <- 100000
3 | disease <- rbinom(N, 1, 0.02) # 2% prevalence
4 | positive <- rbinom(N, 1, ifelse(disease == 1, 0.95, 0.10)) # sens 95%, spec 90%
5 |
6 | round(c(P_positive_given_disease = mean(positive[disease == 1]),
7 |         P_disease_given_positive = mean(disease[positive == 1])), 4)

```

	P_positive_given_disease	P_disease_given_positive
	0.9465	0.1552

Then answer: state, in one sentence each, which of the two numbers a test manufacturer would print on the box, which one a patient is asking for, and which one a hospital deciding whether to run the screen at all should care about. They are not all the same.

DISCUSSION

A4. The Normal, checked

Predict all four before running.

```

R Code
1 | set.seed(6101)
2 | y <- rnorm(100000, mean = 50, sd = 10)
3 |
4 | round(c(within_1_sd = mean(abs(y - 50) <= 10),
5 |         within_2_sd = mean(abs(y - 50) <= 20),
6 |         empirical_97.5th = quantile(y, 0.975),
7 |         qnorm_97.5th = qnorm(0.975, mean = 50, sd = 10)), 4)

```

	within_1_sd	within_2_sd	empirical_97.5th	qnorm_97.5th
	0.6808	0.9542	69.6782	69.5996

Then answer: the last two numbers do not match exactly. Which one is “right”? Say precisely what each of them estimates, and which you would report if these 100,000 values were real measurements rather than draws from a distribution you chose yourself.

A5. Same mean, different world

Predict which pair of numbers will match and which will not.

R Code

```
1 set.seed(6101)
2 n1 <- rbinom(50000, size = 10, prob = 0.5)      # everyone the same
3
4 p <- sample(c(0.1, 0.9), 50000, replace = TRUE) # two kinds of user
5 n2 <- rbinom(50000, size = 10, prob = p)
6
7 round(c(mean_1 = mean(n1), mean_2 = mean(n2),
8         var_1 = var(n1), var_2 = var(n2)), 3)
```

```
mean_1 mean_2 var_1 var_2
4.992  5.019  2.489 16.880
```

Then answer: the variances differ by a factor of about seven. Name a business decision that would be identical under both models and one that would be badly wrong under the first. Then say which single plot would have revealed the difference in two seconds.

This runs, and it answers a different question

Every cell below executes cleanly and prints a correct number. Each one is attached to a stated task that it does **not** answer. Find the mismatch, say what question the code actually answers, and write the code that answers the one asked.

This is the harder skill and the more valuable one. Code that crashes tells you it is broken.

B1. At least one six

Task: you roll four dice. What is the probability of getting **at least one** six?

R Code

```
1 set.seed(2026)
2 n <- 100000
3 r1 <- sample(1:6, n, replace = TRUE)
4 r2 <- sample(1:6, n, replace = TRUE)
5 r3 <- sample(1:6, n, replace = TRUE)
6 r4 <- sample(1:6, n, replace = TRUE)
7
8 sixes <- (r1 == 6) + (r2 == 6) + (r3 == 6) + (r4 == 6)
9 mean(sixes == 1)
```

```
[1] 0.38742
```

Fix it, then compute the exact answer with the complement rule and check that the two agree.

B2. The reassuring number

Task: a patient tests positive on the screen in problem A3. Report the probability that they have the disease.

R Code	
1	<code>set.seed(6101)</code>
2	<code>N <- 100000</code>
3	<code>disease <- rbinom(N, 1, 0.02)</code>
4	<code>positive <- rbinom(N, 1, ifelse(disease == 1, 0.95, 0.10))</code>
5	
6	<code>mean(positive[disease == 1])</code>
[1] 0.9465128	

The number printed is correct and it is not the answer. Say which conditional probability it is, compute the one that was asked for, and then write the sentence you would actually say to the patient—including both the posterior probability and the size of the update from the prior.

DISCUSSION



B3. Two flights

Task: you have a connection with a two-hour layover. Flight A and flight B each run late about 20% of the time. What is the probability that **both** legs are late?

R Code	
1	<code>set.seed(6101)</code>
2	<code>n <- 100000</code>
3	<code>weather <- rbinom(n, 1, 0.15) # a bad weather day at the hub</code>
4	<code>late_a <- pmax(weather, rbinom(n, 1, 0.06))</code>
5	<code>late_b <- pmax(weather, rbinom(n, 1, 0.06))</code>
6	
7	<code>round(c(P_late_a = mean(late_a),</code>
8	<code> P_late_b = mean(late_b),</code>
9	<code> answer = mean(late_a) * mean(late_b)), 4)</code>
P_late_a P_late_b answer	
0.2012 0.2000 0.0402	

Every marginal probability here is correct. The final line is not. Compute the right answer from the same three vectors, state the ratio between the two, and name the variable in the code that is the reason. Then give two more examples from your own field where the same shared-cause structure would break a multiplication.

DISCUSSION



B4. Two children

Task: a family has two children. You learn that **at least one** of them is a girl. What is the probability that both are girls?

R Code

```

1 set.seed(6101)
2 n <- 100000
3 child1 <- sample(c("B", "G"), n, replace = TRUE)
4 child2 <- sample(c("B", "G"), n, replace = TRUE)
5
6 both_girls <- child1 == "G" & child2 == "G"
7 mean(both_girls[child1 == "G"])

```

```
[[1]] 0.4990692
```

The answer printed is about 0.5. The answer to the task is not 0.5.

Find the line that conditions on the wrong event, fix it, and explain the gap in terms of **denominators**: how many families are in each of the two conditioning sets, and which families are in one but not the other. This is a famous problem and the argument about it is usually about the wording, not the arithmetic—so also state one plausible way of learning “at least one is a girl” that would genuinely make 0.5 the right answer.

DISCUSSION

B5. The alert threshold

Task: an error-logging service sees about 7 errors an hour. Set an alert threshold so that it fires on roughly 1% of normal hours.

R Code

```

1 set.seed(6101)
2 busy <- sample(c(0.5, 1), 100000, replace = TRUE) # quiet hours and busy hours
3 lam <- ifelse(busy == 1, 12, 2)
4 errors <- rpois(100000, lambda = lam)
5
6 threshold <- qpois(0.99, lambda = mean(errors))
7 round(c(mean_errors = mean(errors),
8         var_errors = var(errors),
9         threshold = threshold,
10        actual_alert_rate = mean(errors > threshold)), 4)

```

mean_errors	var_errors	threshold	actual_alert_rate
6.9846	31.8162	14.0000	0.1123

The threshold was computed correctly from a Poisson with the correct mean. The alert fires on 11% of hours, not 1%.

Diagnose it using a comparison the chapter told you to run on every count variable. Then propose a fix, and say what the fix costs—there is more than one defensible answer here, and “use the empirical quantile instead” is only one of them.

DISCUSSION

B6. The classifier that is 97% accurate

Task: a fraud model flags transactions. Report whether it is any good.

R Code

```

1 set.seed(6101)
2 n <- 100000
3 fraud <- rbinom(n, 1, 0.003)

```

```

4 flagged <- rbinom(n, 1, ifelse(fraud == 1, 0.90, 0.03))
5
6 accuracy <- sum(flagged == fraud) / n
7 accuracy
8 [1] 0.9692

```

Accuracy of 0.97 sounds excellent. Compute two further numbers from the same vectors: the proportion of flagged transactions that are actually fraud, and the proportion of fraud that gets flagged. Then compute the accuracy of the model that flags **nothing at all**, and say what that comparison tells you about accuracy as a metric for rare events.

DISCUSSION

Judgment

No code. Bring written answers; we will disagree about these out loud.

C1.

You are asked in an interview: “*Are mutually exclusive events independent?*”

Write the answer you would give out loud, in under thirty seconds, including one concrete example. Then write the follow-up question you would expect next, and answer that too.

DISCUSSION

C2.

A public health agency is deciding whether to screen the whole adult population for a disease with a prevalence of 1 in 2,000, using a test with 99% sensitivity and 98% specificity.

Make the strongest case **for** the program and the strongest case **against** it, each in three sentences, using numbers you compute yourself. Then say which single additional piece of information would settle it for you.

DISCUSSION

C3.

Your A/B test assigns visitors to a variant and records whether they convert. The analysis treats the visitors as independent.

List three concrete ways that assumption can fail in a real product, and for each one say whether it makes your reported uncertainty too small, too large, or unclear. (You will be able to prove the direction after [The Standard Error of a Mean](#); for now, argue it.)

DISCUSSION

C4.

[Conditional Probability and Bayes](#) showed that a positive result on an excellent test can leave you at 9% probability of illness, and also that the same result raises your risk eighteenfold.

Both statements are true of the same number. Write a short paragraph—the kind you would put in a report—that conveys both without leaving the reader with a false impression in either direction. Then write the version a journalist would print from it, and say what you would do differently to prevent that.

C5.

Everything this week was about a process you *chose*: you knew the true probability, so you could check the answer.

Real data does not come with the true probability attached. Pick any one quantity from these chapters—the 9% posterior, the 0.30 chance of at least one six, the click rate of 0.3—and answer: **if you only had the data and not the generating process, how would you know your distributional assumption was wrong?**

Bring your list of checks to class. Some of them are standard-error and hypothesis-testing material, and I would rather you wrote them down first.

So far, events. An event happened or it did not, and we counted. But data is not made of yes/no answers—it is made of numbers, and to reason about numbers we need one more object.

THE DEFINITION, WHICH IS LESS MYSTERIOUS THAN THE NAME

A **random variable** is a **function** that assigns a number to each outcome of a random process.

It is not a variable in the programming sense, and it is not random in the sense of “arbitrary”. It is a rule: *outcome in, number out*. The randomness lives entirely in which outcome occurs; the function itself is completely deterministic.

Once you read the phrase as “function of a random outcome”, most of the confusion around the term disappears.

Roll two dice. The outcome is a pair like (6, 1). From that one outcome you can compute many different numbers, and each rule is a different random variable:

X = the sum, Y = how many sixes appeared

R Code

```
1 set.seed(6101)
2 d1 <- sample(1:6, 100000, replace = TRUE)
3 d2 <- sample(1:6, 100000, replace = TRUE)
4
5 X <- d1 + d2           # random variable: the sum
6 Y <- (d1 == 6) + (d2 == 6) # random variable: the number of sixes
7
8 head(data.frame(die1 = d1, die2 = d2, X = X, Y = Y), 8)
```

```
  die1 die2  X Y
1     6    1  7 1
2     4    5  9 0
3     1    6  7 1
4     4    5  9 0
5     4    6 10 1
6     4    6 10 1
7     2    1  3 0
8     4    1  5 0
```

One column of outcomes, two columns of numbers computed from them. $d1 + d2$ is the function; X is what it returns. Notice that $(d1 == 6) + (d2 == 6)$ adds two TRUE / FALSE values as 1 and 0—an event is a random variable that only takes the values 0 and 1, which is why the `mean()` trick has been working all along.

Notation. Capital letters (X, Y) are the random variable; lower-case (x, y) is a particular value it might take. So $P(X = 7)$ is the probability that the sum comes out 7, and $P(X \leq 7)$ is the probability it comes out 7 or less. Getting the case right is not pedantry; it is the difference between “the rule” and “a number”.

Discrete and continuous

Random variables come in two kinds, and which one you have determines what questions are answerable.

A **discrete** random variable takes values you can list: 0, 1, 2, . . . Counts are discrete. Number of sixes, number of clicks, number of defective units, number of support tickets today.

A **continuous** random variable takes any value in a range. Measurements are continuous. Height, time, temperature, concentration. Between any two possible values there are infinitely many others.

The line is blurrier in practice than in theory—money is technically discrete (there are only so many cents) and heights are recorded to the nearest centimetre—and that is fine. The question is never “what is it really?” but “which set of tools describes it usefully?” Salary in dollars has so many possible values that treating it as continuous is more useful than listing them.

Discrete: the probability mass function

For a discrete variable, each value carries a lump of probability. The **probability mass function** is the rule that says how much:

$$p(x) = P(X = x), \quad \sum_x p(x) = 1$$

You already know how to estimate one—it is a table of proportions:

R Code	
1	<code>pmf <- table(X) / length(X)</code>
2	<code>round(pm, 4)</code>

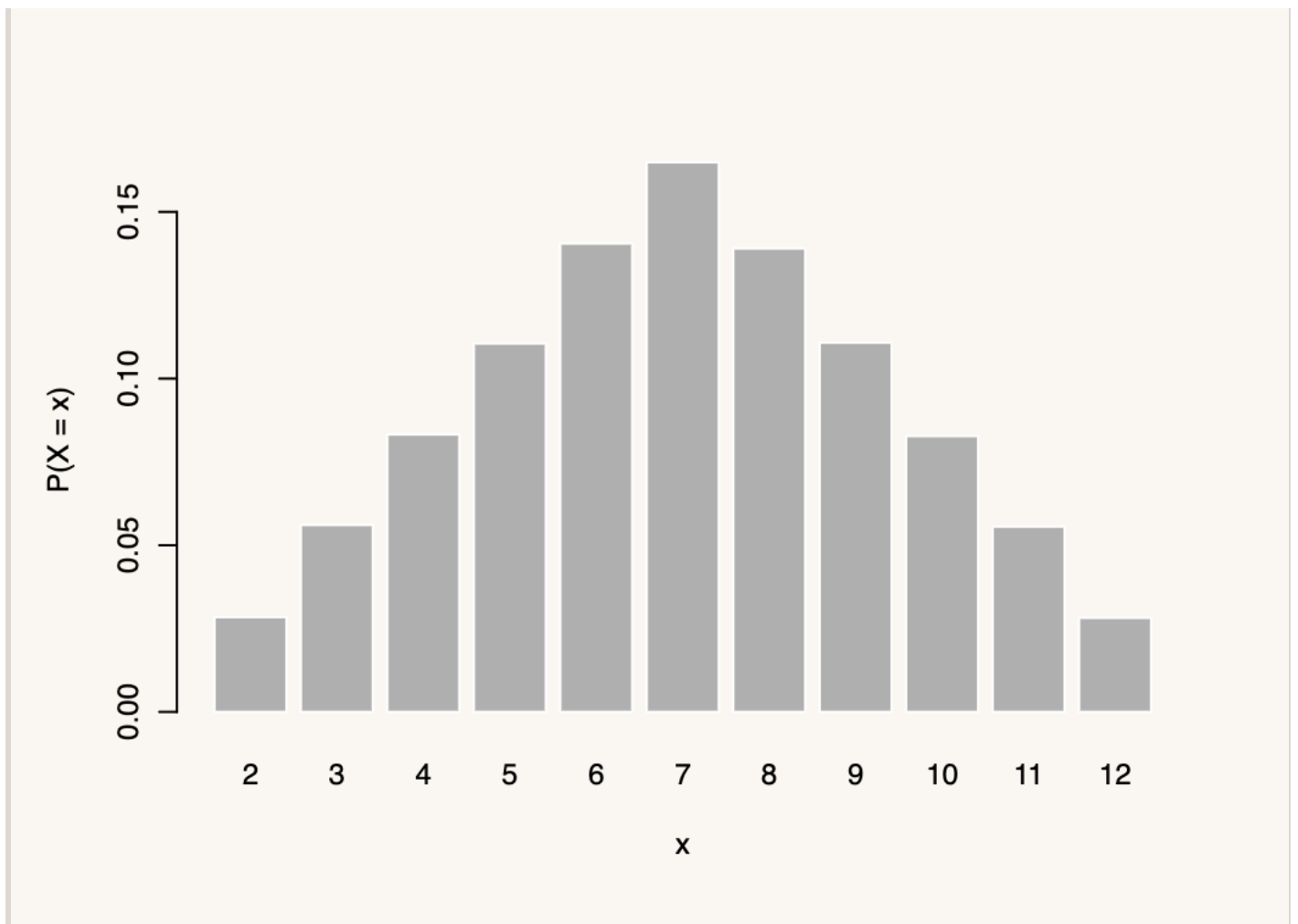
X	2	3	4	5	6	7	8	9	10	11	12
	0.0284	0.0561	0.0832	0.1105	0.1405	0.1649	0.1390	0.1108	0.0827	0.0556	0.0282

R Code	
1	<code>sum(pm)</code>

[[1]]	1
-------	---

It sums to 1, because every roll produced exactly one value and they all got counted. That is not a coincidence to be checked; it is what “probability distribution” means.

R Code	
1	<code>barplot(pm, col = "gray70", border = "white",</code>
2	<code> xlab = "x", ylab = "P(X = x)")</code>



The familiar triangle. Seven is the most likely total because it has the most outcomes producing it—six of the 36 pairs—while 2 and 12 have one each. The *shape* of a distribution comes from the process that generated it, and this is the smallest example of that principle.

Compare the pmf of Y , the number of sixes:

R Code	
1	<code>round(table(Y) / length(Y), 4)</code>
<pre> Y 0 1 2 0.6994 0.2724 0.0282 </pre>	

Three values, and their exact probabilities are $25/36$, $10/36$, $1/36$ —0.694, 0.278, 0.028. The simulation gives 0.699, 0.272, 0.028. Same 100,000 rolls, a completely different distribution, because it is a different function of the same outcomes.

The cumulative distribution function

The pmf breaks down for continuous variables, for a reason we will get to. The **CDF** does not, which makes it the more fundamental object:

$$F(x) = P(X \leq x)$$

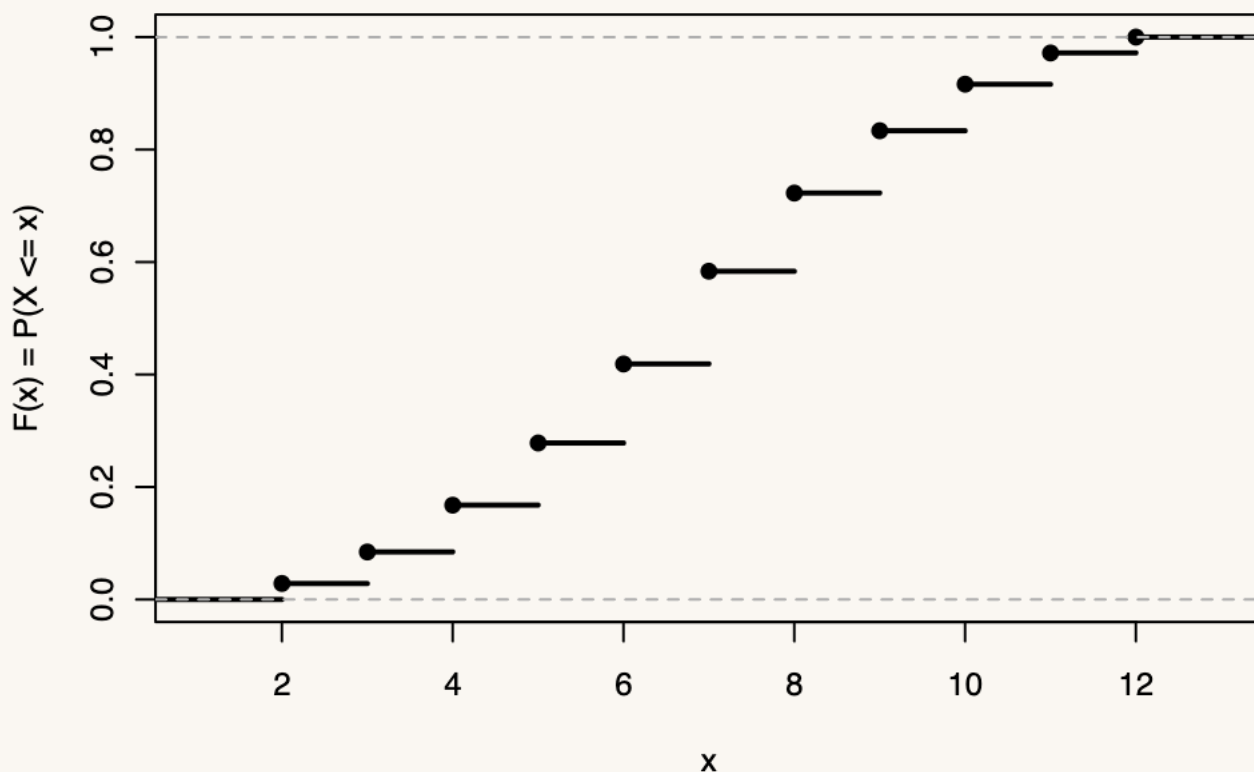
It is defined for every random variable of either kind, it starts at 0, it never decreases, and it ends at 1.

R Code

```

1 Fx <- ecdf(X) # the EMPIRICAL cdf of our simulated values
2 plot(Fx, main = "", xlab = "x", ylab = "F(x) = P(X <= x)", lwd = 2)

```



For a discrete variable the CDF is a **staircase**: flat between the possible values, jumping at each one, and the height of each jump is exactly that value's probability mass. The picture and the pmf carry identical information.

The reason to care is that the CDF answers interval questions by subtraction:

$$P(a < X \leq b) = F(b) - F(a)$$

R Code

```

1 round(c(F_9      = Fx(9),
2         F_4      = Fx(4),
3         difference = Fx(9) - Fx(4),
4         direct    = mean(X > 4 & X <= 9)), 4)

```

	F_9	F_4	difference	direct
	0.8335	0.1677	0.6657	0.6657

0.6657, against the exact $24/36 = 0.6667$. Every `p`-function you meet in the next chapter—`pnorm`, `pbinom`, `ppois`—is a CDF, and this subtraction is how you get an interval probability out of one.

Watch the inequality signs. $F(9) - F(4)$ excludes 4 and includes 9. For a discrete variable that matters: $P(X \leq 9) - P(X \leq 4)$ is not the same as $P(X \leq 9) - P(X \leq 5)$, and off-by-one errors here are the most common source of quietly wrong probabilities in applied work. For a continuous variable it makes no difference at all, for reasons that are about to become clear.

Continuous: density instead of mass

Now the part that trips people up.

Take heights of adults, in centimetres:

R Code	
1	<code>set.seed(6101)</code>
2	<code>h <- rnorm(100000, mean = 170, sd = 8)</code>
3	<code>round(head(h, 6), 2)</code>
[1] 166.49 164.84 179.06 166.08 161.75 162.79	

Ask for the probability that someone is exactly 180 cm tall:

R Code	
1	<code>mean(h == 180)</code>
[1] 0	

Zero. And not “zero because our sample is finite”—**exactly zero, in the true distribution**. There are infinitely many possible heights, and if every one of them carried a positive lump of probability the total would be infinite. So each individual value must carry none.

That sounds like a defeat, but it is only a redefinition. Probability for a continuous variable lives in **intervals**, not points:

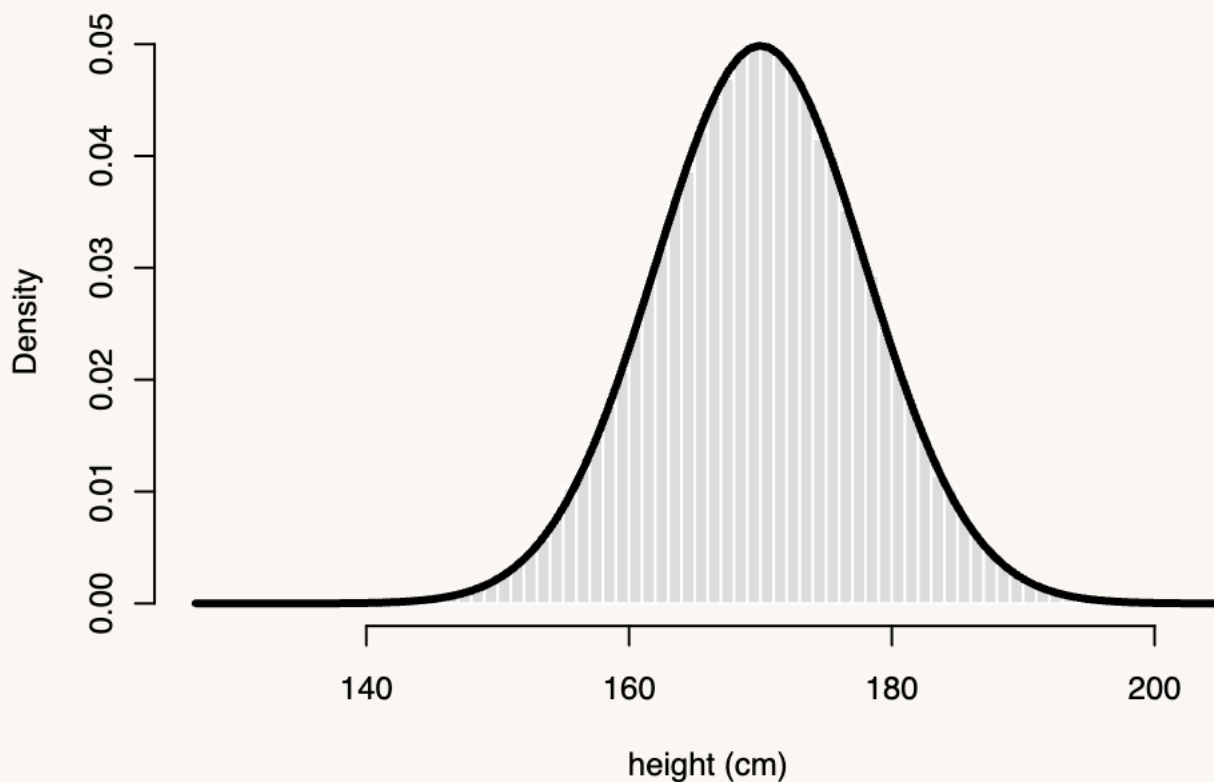
R Code	
1	<code>round(c(within_half_a_cm = mean(h > 179.5 & h < 180.5),</code>
2	<code>between_162_178 = mean(h > 162 & h < 178)), 4)</code>
within_half_a_cm between_162_178 0.0229 0.6808	

The **probability density function** $f(x)$ is what you integrate over an interval to get its probability:

$$P(a < X < b) = \int_a^b f(x) dx$$

so $f(x)$ is probability *per unit of x*, not probability. Its units are “probability per centimetre”. A histogram with `freq = FALSE` is an estimate of it—bar heights scaled so that total **area** is 1:

R Code	
1	<code>hist(h, breaks = 60, freq = FALSE, col = "gray88", border = "white",</code>
2	<code>main = "", xlab = "height (cm)")</code>
3	<code>curve(dnorm(x, mean = 170, sd = 8), add = TRUE, lwd = 3)</code>



A DENSITY IS NOT A PROBABILITY, AND IT CAN EXCEED 1

`dnorm(0, mean = 0, sd = 0.1)` returns about **3.99**. Nothing is broken. Squeeze a distribution into a narrow range and the curve must get tall to keep the area at 1. A uniform distribution on $[0, 0.5]$ has density exactly 2 everywhere inside it.

The practical version: **read heights of a density as relative, and read areas as probabilities.** Density $\times$ width $\approx$ probability, for a narrow enough width:

R Code

```
1 round(c(density_at_180 = dnorm(180, mean = 170, sd = 8),
2         times_width_1cm = dnorm(180, mean = 170, sd = 8) * 1,
3         actual_P_179.5_to_180.5 = mean(h > 179.5 & h < 180.5)), 5)
```

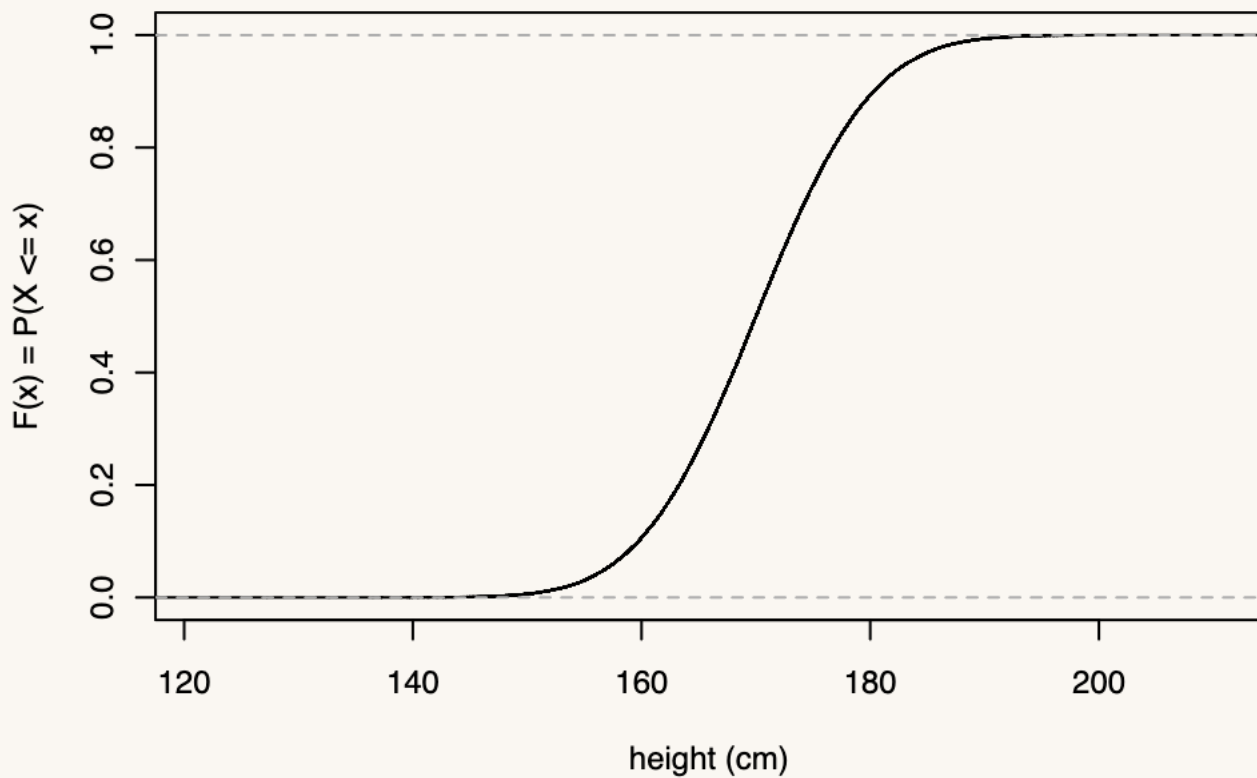
density_at_180	times_width_1cm	actual_P_179.5_to_180.5
0.02283	0.02283	0.02289

0.02283 predicted against 0.02289 counted. The density did its job; it just was not itself a probability.

The CDF, meanwhile, needs no such apology. It means the same thing for continuous variables as for discrete ones—it is simply smooth instead of a staircase:

R Code

```
1 plot(ecdf(h), main = "", xlab = "height (cm)", ylab = "F(x) = P(X <= x)")
```



And the interval rule is unchanged:

R Code

```
1 round(c(simulated = mean(h > 180),  
2       exact      = pnorm(180, mean = 170, sd = 8, lower.tail = FALSE)), 4)
```

simulated	exact
0.1068	0.1056

0.1068 against 0.1056. That is the entire relationship between a simulation and a formula: same question, two ways of answering it, and if they disagree by more than noise you have found a bug in one of them.

A continuous random variable X has probability density $f(0.3) = 2.4$.

Which statement is correct?

- ☐ Something is wrong: a probability cannot exceed 1, so f is not a valid density.
- ☐ Nothing is wrong. $f(0.3)$ is a density, not a probability. $P(X = 0.3) = 0$; probabilities are **areas** under f , and an area can stay below 1 while the curve above it is tall and narrow.
- ☐ It means X takes the value 0.3 about 2.4 times as often as any other value in the sample.
- ☐ It means $P(X \leq 0.3) = 2.4$, which is impossible, so X must be discrete.



What comes next, and why

You now have a distribution: a complete description of what values a random variable takes and how often. A complete description is often *too much* description. Eleven numbers for the dice sum, a whole curve for heights—you cannot put that in a sentence, and you cannot compare two of them at a glance.

So next week compresses a distribution into two numbers:

- **Expectation**, $E[X]$ —the long-run average value, the balance point of the distribution. It is the mean from [Describing a Single Variable](#), promoted from a fact about a sample to a property of the process that generated it.
- **Variance**, $\text{Var}(X)$ —how far the variable typically strays from its expectation.

WHY THE STANDARD ERROR OF A MEAN IS THE HINGE OF THIS COURSE

Those two summaries would be mildly useful on their own. What makes them the center of the term is a single fact about how they combine:

For independent random variables, variances add.

Everything downstream follows from that one line. Apply it to a sample mean $\bar{X}$ and you get $\text{Var}(\bar{X}) = \sigma^2/n$, so the standard deviation of a sample mean is $\sigma/\sqrt{n}$ —the **standard error**. That formula is why bigger samples are better, exactly how much better, and by what rate. Confidence intervals are built on it. So are p-values, sample-size calculations, and every A/B test you will ever run.

Notice the word **independent** in the statement. That is the concept from [Probability](#), and it is load-bearing: when observations are not independent—repeat visits from the same user, students within a classroom, days within a week— $\sigma/\sqrt{n}$ is wrong, your interval is too narrow, and you are more confident than the data entitles you to be. This is one of the most common real errors in applied statistics, and by [The Standard Error of a Mean](#) you will be able to say exactly which step it breaks.

We are not deriving any of that here. Arrive next week solid on two things and it will be a short evening: a random variable is a function of a random outcome, and independence means $P(A \cap B) = P(A)P(B)$.

Exercises

Exercise: a different function of the same rolls

Exercise

```
1 # Define the random variable D = the ABSOLUTE DIFFERENCE between the two dice,  
2 # then estimate P(D = 0) -- the probability that the two dice match.  
3 -----
```

Please replace ----- with valid code.

Exercise: reading a CDF

Exercise

```
1 # `Fx` is the empirical CDF of the dice sum: Fx(x) gives P(X <= x).  
2 # Using Fx ONLY -- no counting, no mean() of X -- compute P(5 <= X <= 9).  
3 # Careful with the endpoints: 5 must be INCLUDED.  
4 -----
```

Please replace ----- with valid code.

Exercise: a tail probability for a continuous variable

Exercise

```
1 # `h` holds 100,000 simulated heights in centimetres.  
2 # Estimate P(h > 185), the probability of being taller than 185 cm.  
3 -----
```

Please replace ----- with valid code.

Exercise: which kind of variable

Four quantities recorded by a hospital:

1. Number of patients admitted on a given day
2. Time from admission to discharge, in hours
3. Whether a readmission occurred within 30 days (yes / no)
4. Blood pressure, in mmHg

Exercise

```
1 # Assign the letter of the correct statement, e.g. answer <- "A"  
2 #  
3 # A. 1 and 3 are discrete; 2 and 4 are continuous.  
4 # B. Only 1 is discrete, since it is the only one that is a count.  
5 # C. All four are discrete, because a computer stores every number to  
6 #   finite precision.  
7 # D. 2 and 4 are discrete because they are recorded rounded, and 1 and 3  
8 #   are continuous because they have no fixed upper limit.  
9 answer <- -----
```

Please replace ----- with valid code.

Summary

- A **random variable** is a function from outcomes to numbers. The rule is fixed; only the outcome is random. Several different random variables can be defined on the same outcomes, and the same simulation supports all of them.
- **Discrete** variables take listable values (counts, categories, 0/1); **continuous** ones take any value in a range (times, measurements).
- The **probability mass function** $p(x) = P(X = x)$ describes a discrete variable, and `table(x) / length(x)` estimates it.
- For a continuous variable $P(X = x) = 0$ for every x . Probability lives in intervals, described by a **density** $f(x)$ whose *area* is the probability. Densities are not probabilities and may exceed 1.
- The **CDF** $F(x) = P(X \leq x)$ works for both kinds, and interval probabilities are $F(b) - F(a)$. Mind the endpoints for discrete variables.
- Next week: **expectation** and **variance**, and the fact that variances of independent quantities add—which gives $\text{Var}(\bar{X}) = \sigma^2/n$ and everything built on it.

Next, though: the handful of distributions that describe most of the data you will ever meet, and how to tell which one you are looking at.

Last week you met random variables and the distributions we actually use. A distribution is a complete description of a random quantity: every value it can take and how often. That is more information than anyone can hold in their head, so we do to distributions exactly what [Describing a Single Variable](#) did to columns of data—replace them with a couple of numbers.

The first of those numbers is the **expectation**, $E[X]$. It is the center of the distribution in the same sense the mean was the center of a dataset: the balance point.

But the definition worth carrying around is the operational one.

WHAT $E[X]$ ACTUALLY MEANS

$E[X]$ is the number the average of your observations settles down to as you collect more and more of them.

Not “the value you expect to see”—for a fair die $E[X] = 3.5$, which is not a face. Not “the most likely value”. It is the long-run average, and everything else in this chapter follows from taking that seriously.

Watching it settle

The cleanest way to believe this is to watch it happen. Roll a fair die ten thousand times and track the running average after every roll.

R Code

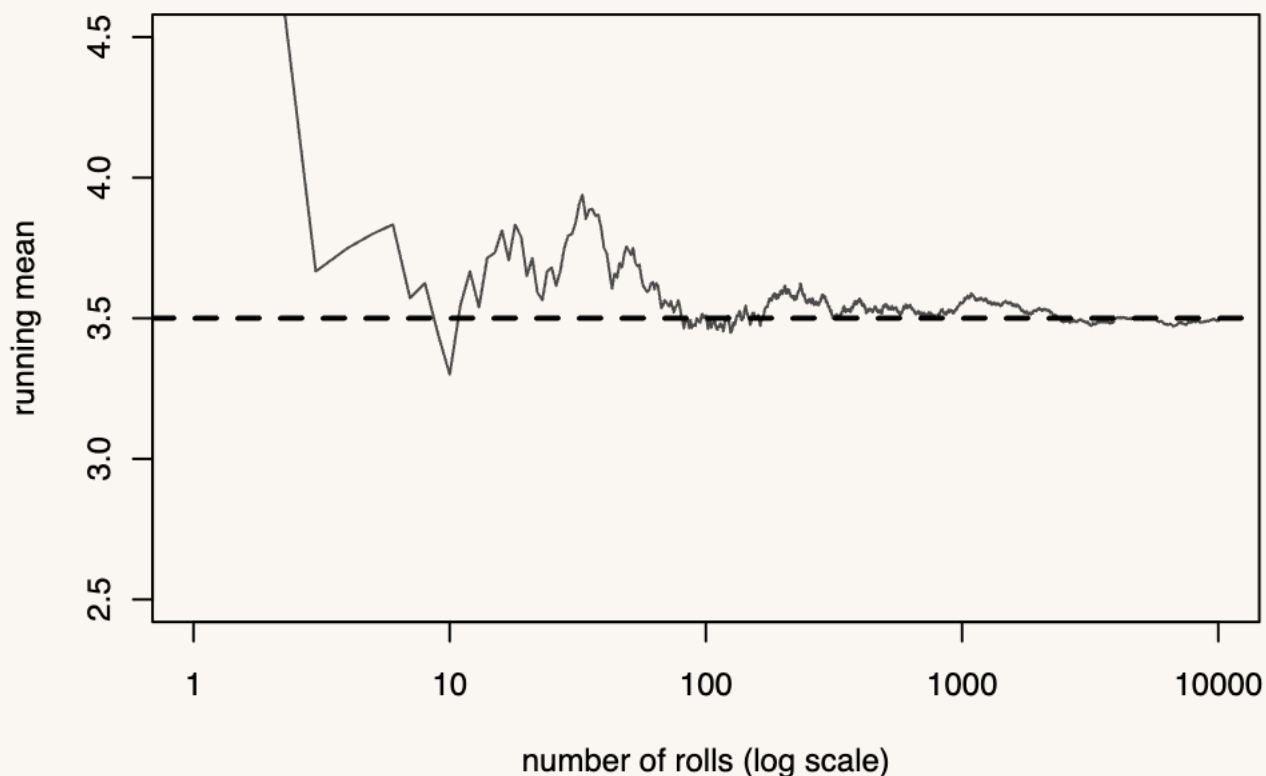
```
1 set.seed(6101)
2 rolls <- sample(1:6, 10000, replace = TRUE)
3 head(rolls, 20)
4
5 running <- cumsum(rolls) / seq_along(rolls)
6 round(c(after_10 = running[10], after_100 = running[100],
7         after_1000 = running[1000], after_10000 = running[10000]), 4)
```

```
[1] 6 4 1 4 4 4 2 4 2 2 6 5 2 6 4 5 2 6 3 1
    after_10   after_100  after_1000 after_10000
      3.3000      3.5100      3.5560      3.4909
```

After 10 rolls the running mean is 3.30. After 100 it is 3.51, after 1,000 it is 3.556, and after 10,000 it is 3.4909. Now look at the whole path, with $E[X] = 3.5$ drawn as a dashed line. The horizontal axis is on a log scale, because otherwise the first thousand rolls are squeezed into an invisible sliver at the left:

R Code

```
1 plot(seq_along(running), running, type = "l", log = "x", col = "gray30",
2       xlab = "number of rolls (log scale)", ylab = "running mean",
3       ylim = c(2.5, 4.5))
4 abline(h = 3.5, lwd = 2, lty = 2)
```



Two things are worth noticing, and the second one is the one people miss.

It settles. The early wobbling is violent—after five rolls the running mean can easily be 2 or 5—and then it calms down and hugs the line. That convergence has a name, the **law of large numbers**, and it is the reason a casino with enough customers has no risk at all while any individual customer has plenty.

It settles slowly. Look at the axis: it took a *log scale* to make the picture readable, because the improvement from 100 rolls to 1,000 rolls looks about the same as the improvement from 1,000 to 10,000. Getting one more decimal place costs a hundred times the data. You have just seen $1/\sqrt{n}$ without being told about it, and in [The Standard Error of a Mean](#) we will pin the rate down exactly.

Computing $E[X]$

The discrete case

For a discrete random variable, weight each value by its probability and add:

$$E[X] = \sum_x x \cdot P(X = x)$$

That is a weighted average, and the weights are the probabilities. Here is a scratch card. It costs \$2, and the prize table is:

PRIZE	\$0	\$1	\$5	\$20
Probability	0.70	0.20	0.08	0.02

R Code

```

1 prize <- c(0, 1, 5, 20)
2 p      <- c(0.70, 0.20, 0.08, 0.02)
3
4 sum(p)          # a sanity check you should never skip
5 sum(prize * p)   # E[prize]

```

```

[1] 1
[1] 1

```

The expected prize is exactly \$1.00. The card costs \$2. So the expected profit per card, from your side, is $-\$1.00$: every card you buy costs you a dollar on average, forever, regardless of how you feel about the one that paid \$20.

Believe the arithmetic, then check it anyway:

R Code

```

1 set.seed(6101)
2 draws <- sample(prize, 500000, replace = TRUE, prob = p)
3 round(c(mean_prize = mean(draws), mean_profit = mean(draws - 2)), 4)

```

```

mean_prize mean_profit
0.9967      -1.0033

```

Half a million cards give a mean prize of 0.9967 and a mean profit of -1.0033 . The formula and the simulation are computing the same thing by two routes, which is exactly the habit this week is trying to build: **when a formula surprises you, simulate it**.

Note the scale at which the expectation becomes a *fact* rather than a tendency. Sell 100,000 of these cards and the operator's take is close to \$100,000 with very little uncertainty. Buy one, and the expectation tells you almost nothing about your evening. Same number, two completely different kinds of usefulness.

The continuous case

For a continuous random variable the sum becomes an integral, weighted by the density $f(x)$ instead of by point probabilities:

$$E[X] = \int_{-\infty}^{\infty} x f(x) dx$$

You will not be asked to evaluate integrals in this course. What you do need is that it is **the same idea**—every possible value, weighted by how much probability sits near it—and that for the distributions you met last week the answer is already known:

DISTRIBUTION	$E[X]$
Bernoulli(p)	p
Binomial(n, p)	np
Normal(μ, σ)	μ
Uniform(a, b)	$(a + b)/2$
Exponential(rate λ)	$1/\lambda$
Poisson(λ)	λ

Check one of them. An exponential with rate 0.4 should have expectation $1/0.4 = 2.5$:

R Code

```

1 set.seed(6101)

```

```
2 | w <- rexp(200000, rate = 0.4)
3 | round(c(simulated = mean(w), theory = 1/0.4), 4)

simulated    theory
2.5029       2.5000
```

2.5029 against 2.5. Close, and not exact—which is the subject of the next section.

$E[X]$ is not $\bar{x}$

This distinction is worth more to you than any formula in the chapter, and it is the one that quietly confuses people for years.

Draw 30 observations from that same exponential, six separate times, and compute the sample mean each time:

```
R Code

1 | set.seed(6101)
2 | round(replicate(6, mean(rexp(30, rate = 0.4))), 3)

|[1] 2.869 3.329 2.356 3.123 2.298 2.147
```

2.869, 3.329, 2.356, 3.123, 2.298, 2.147. Six numbers, all of them the “average” of an exponential with expectation 2.5, none of them 2.5, and the spread between the largest and the smallest is over a full unit.

TWO OBJECTS, TWO KINDS		
	$E[X]$, OR μ	$\bar{x}$
What it belongs to	the distribution	a sample
Is it random?	no—it is a fixed number	yes—a different sample gives a different value
Do you know it?	usually not	always, you computed it
How do you get it?	from the model, by formula	from data, with <code>mean()</code>
$\bar{x}$ is an estimate of μ . It is right on average and wrong in every particular sample. The question “how wrong, typically?” is not answerable from anything in this chapter—it needs the variance, from Variance and Standard Deviation , and then the derivation in The Standard Error of a Mean .		

Notation helps and is worth adopting now: Greek letters (μ, σ) for population quantities you do not know, Latin letters ($\bar{x}, s$) for things you computed from data. When you see a formula with both in it, it is telling you something about how the second relates to the first.

You simulate 30 draws from an exponential distribution whose expectation is exactly 2.5, and `mean()` of your draws returns 2.869.

What is the relationship between the 2.5 and the 2.869?

- ☐ The 2.869 is the better number, since it came from actual data rather than from a formula.
- ☐ Something is wrong with the simulation: with a correct random number generator the sample mean should reproduce 2.5.
- ☐ $E[X] = 2.5$ is a fixed property of the distribution and does not change. The 2.869 is a statistic computed from one sample; a different 30 draws would give a different value, and the whole of this week is about how different.
- ☐ They are the same quantity with different names, so either may be reported.



Exercise: reading a payout table

Exercise	
1	# A game has five outcomes. `payout` holds the dollar amounts (one of them
2	# is a loss) and `prob` holds their probabilities.
3	# Compute the expected payout. One line; do not simulate.
4	-----

Please replace ----- with valid code.

Exercise: what is a fair price?

Exercise	
1	# The scratch card from above: `prize` and `p` describe the prize distribution.
2	#
3	# A "fair" game is one where the expected profit to the player is exactly zero.
4	# Return the price, in dollars, at which this card would be fair.
5	-----

Please replace ----- with valid code.

Linearity

Now the property that makes expectation genuinely useful, as opposed to merely meaningful.

Constants pass straight through

$$E[aX + b] = aE[X] + b$$

If you double every value and add ten, the balance point doubles and moves by ten. This is not deep; it is what “balance point” means. But it is worth verifying once so you trust it later:

R Code

```
1 set.seed(6101)
2 x <- rnorm(200000, mean = 5, sd = 2)
3 round(c(mean_x      = mean(x),
4         direct      = mean(3 * x + 10),
5         from_linearity = 3 * mean(x) + 10), 4)
```

mean_x	direct	from_linearity
4.9992	24.9976	24.9976

`mean(x)` is 4.9992, and both routes to $E[3X + 10]$ give 24.9976. The same number, not approximately the same: the rule holds for the sample mean exactly as it holds for the expectation, and the two differ only by floating-point rounding in the fifteenth decimal place.

The practical use is unit conversion. If waiting times average 14 minutes, then in seconds they average $60 \times 14 = 840$, and nobody has to re-derive anything.

Sums add, and independence is not required

This rule and the next one turn on whether two random variables are **independent**, so pin the word down first.

INDEPENDENT RANDOM VARIABLES

Probability defined independence for two **events**: $P(A \cap B) = P(A)P(B)$. Two **random variables** are independent when every event about one is independent of every event about the other. For discrete X and Y that means

$$P(X = x, Y = y) = P(X = x)P(Y = y) \quad \text{for every pair of values } x, y,$$

and for continuous variables the same statement holds with densities.

In words: learning the value of X does not change the distribution of Y . Two separate dice are independent. Tomorrow's temperature and a seaside kiosk's ice-cream sales are not: a hot day moves the whole distribution of sales up.

Independence is stronger than having **zero covariance**. Independent variables always have zero covariance, but not the other way round: if X is symmetric about zero, X and X^2 have covariance zero and are completely dependent. Some rules below need full independence and some need only zero covariance, and the text says which.

$$E[X + Y] = E[X] + E[Y]$$

This one deserves your attention. It holds **always**. It does not care whether X and Y are independent, correlated, or one is a deterministic function of the other.

Most people, having been told all term that independence matters, assume it must be lurking here. It is not. Watch: let Y be X^2 , which is about as dependent on X as anything can be.

R Code

```
1 set.seed(6101)
2 x <- rnorm(200000, mean = 5, sd = 2)
3 y <- x^2 # completely determined by x
4
5 round(c(mean_x      = mean(x),
6         mean_y      = mean(y),
7         mean_of_sum  = mean(x + y),
8         sum_of_means = mean(x) + mean(y)), 4)
```

mean_x	mean_y	mean_of_sum	sum_of_means
4.9992	29.0087	34.0079	34.0079

$E[X] \approx 4.999$, $E[Y] \approx 29.009$, and both the mean of the sum and the sum of the means come out at 34.0079. To the last printed digit.

Now the contrast that shows independence *does* matter somewhere. Try the same thing with a product:

R Code					
1	<code>round(c(mean_of_product = mean(x * y),</code>				
2	<code>product_of_means = mean(x) * mean(y)), 4)</code>				
	<table><tr><td>mean_of_product</td><td>product_of_means</td></tr><tr><td>185.1951</td><td>145.0203</td></tr></table>	mean_of_product	product_of_means	185.1951	145.0203
mean_of_product	product_of_means				
185.1951	145.0203				

185.20 against 145.02. Not close, and not noise— $E[XY] = E[X]E[Y]$ is a **different rule**, and it holds only when the two are uncorrelated. Independence is the usual way to guarantee that, and X and X^2 are spectacularly not.

THE LINE TO REMEMBER

Sums are free. Products are not.

$E[X + Y] = E[X] + E[Y]$ needs no assumptions at all. $E[XY] = E[X]E[Y]$ needs independence (strictly, only zero covariance).

This is why you can compute the expected value of a total—revenue across regions, total delay across the legs of a journey, the sum of n observations in a sample—without knowing anything whatever about how the pieces move together. In [The Standard Error of a Mean](#) that is precisely what lets us find $E[\bar{X}]$ in one line.

And then in [Variance and Standard Deviation](#), variance will turn out to behave like the *product* rule, not the sum rule. That asymmetry is the whole reason independence is a condition worth checking.

🔗 SUMS VERSUS PRODUCTS



Let X be tomorrow's high temperature and Y the number of ice creams a seaside kiosk sells tomorrow. These two are about as dependent as two variables get.

Which of the following are you entitled to write down?

- ☐ Neither, because X and Y are dependent and both rules assume independence.
- ☐ $E[X + Y] = E[X] + E[Y]$ always holds, dependence or not. $E[XY] = E[X]E[Y]$ does not, and here it will fail badly.
- ☐ Both, because expectation is linear and linearity covers products as well as sums.
- ☐ $E[XY] = E[X]E[Y]$ always holds; it is the sum rule that needs independence.



Exercise: linearity without re-simulating

Exercise	
1	<code># `x` holds 200,000 draws. Suppose these are temperatures in degrees Celsius</code>
2	<code># and you need the expected temperature in degrees Fahrenheit,</code>
3	<code># F = 1.8 * C + 32.</code>
4	<code>#</code>
5	<code># Use linearity: build the answer from mean(x) alone.</code>

```

6 | # Do NOT write mean(1.8 * x + 32) -- that recomputes it from scratch,
7 | # which is exactly what linearity saves you from.
8 | -----

```

Please replace _____ with valid code.

Exercise: a sum of dependent variables

Exercise

```

1 | # `u` is uniform on [0, 10]. `v` is 100 - 2*u, so v is a perfect
2 | # (and perfectly negative) function of u -- as dependent as two variables get.
3 | #
4 | # Predict E[U + V] on paper FIRST, using only E[U] = 5 and linearity.
5 | # Then compute mean(u + v) here and see whether the rule survived
6 | # the dependence.
7 | -----

```

Please replace _____ with valid code.

A worked fair-game problem

Expectation questions in interviews are almost always one of these two shapes, and both are the same shape underneath.

Shape one: is this bet worth taking? A friend offers: roll a fair die, they pay you the face value in dollars, and you pay \$4 to play.

$E[X] = 3.5$ for a fair die, so the expected profit is $3.5 - 4 = -0.50$ per roll. Decline.

R Code

```

1 | set.seed(6101)
2 | g <- sample(1:6, 200000, replace = TRUE)
3 | round(c(expected_face = mean(1:6),
4 |         simulated_profit = mean(g - 4)), 4)

```

```

| expected_face simulated_profit
| 3.5000      -0.5027

```

-0.5027 over 200,000 rolls, against the exact -0.50 .

Shape two: what is the edge? American roulette, straight-up bet on a single number. There are 38 pockets. You stake \$1; if your number comes up you win \$35 in profit, otherwise you lose your \$1.

R Code

```

1 | outcomes <- c(35, -1)
2 | probs    <- c(1/38, 37/38)
3 | round(sum(outcomes * probs), 6)

```

```

| [1] -0.052632

```

-0.052632 per dollar staked. That is the famous 5.26% house edge, and it is the entire business model: a number small enough that no individual player notices it and large enough that, multiplied by the law of large numbers you watched in the first figure, it builds a building.

Notice the structure both problems share. **List the outcomes, attach probabilities, multiply, sum, compare to the price.**

Insurance premiums, warranty pricing, the value of a coupon campaign, and whether to run an expensive experiment are all the

same computation wearing different clothes.

Summary

- $E[X]$ is the number a running average settles down to. Watch it settle, and notice that it settles **slowly**—that slowness is $1/\sqrt{n}$, and it is what [The Standard Error of a Mean](#) is about.
- Discrete: $E[X] = \sum xP(X = x)$, a weighted average with probabilities as weights. Continuous: the same idea with an integral. For the standard distributions the answer is a known formula.
- $E[X]$ **is not** $\bar{x}$. The first is a fixed property of a distribution; the second is a random quantity computed from one sample. Six samples gave six different means and none of them was the truth.
- $E[aX + b] = aE[X] + b$. Constants pass straight through, which is why unit conversion is free.
- $E[X + Y] = E[X] + E[Y]$ **always**, independent or not. $E[XY] = E[X]E[Y]$ requires independence. Sums are free; products are not.
- Fair value equals expectation. Every “should I take this bet” problem is: list outcomes, weight by probability, sum, compare with the price.

Next: the second summary number. If the expectation says where a distribution sits, the variance says how far it typically strays—and it is the variance, not the expectation, that will make independence matter.

Monday, September 21

Post your questions on any of these pages by **Monday at 12:00 noon**, the morning of class.

This is the most important week of the course, and I would rather say so plainly than let you discover it in November.

Everything from confidence intervals onward, hypothesis tests, A/B tests, the standard errors printed next to every regression coefficient you will ever read—rests on one result, and that result is derived here, in [The Standard Error of a Mean](#), in about four lines. If those four lines are solid, the rest of the term is bookkeeping around them. If they are not, you will spend the next ten weeks manipulating formulas whose origin you cannot reconstruct, and you will not be able to tell when software has handed you a number that does not mean what its label says.

The result is this. Take a sample of size n from a population with standard deviation σ , and compute the sample mean. That mean is itself random—a different sample would give a different value. How much would it move?

$$\text{SE}(\bar{X}) = \frac{\sigma}{\sqrt{n}}$$

That is the whole of inference in one line. It says the wobble in your estimate is knowable, that it shrinks as you collect more data, and—the part that costs money—that it shrinks only like $\sqrt{n}$.

[Practice Problems](#) ended by asking: *if the data collection were repeated from scratch, how different would your number be?* This is the week you can answer it.

What is here

EXPECTATION

$E[X]$ as the number a long-run average settles down to, watched settling. The discrete and continuous cases. Linearity, including the fact that $E[X + Y] = E[X] + E[Y]$ **whether or not** X and Y are independent. And the distinction that trips people for years: expectation is a property of a distribution, the sample mean is a statistic computed from data.

VARIANCE AND STANDARD DEVIATION

$\text{Var}(X) = E[(X - E[X])^2]$ as average squared distance. Why we square, and why we then take a root. $\text{Var}(aX + b) = a^2 \text{Var}(X)$, and the b vanishing. And the key fact: for **independent** X and Y , variances add—standard deviations do not. We simulate a case where the rule holds and a case where it fails spectacularly, so that “independent” reads as a condition rather than as a word in the way.

THE STANDARD ERROR OF A MEAN

The payoff. $\text{Var}(\bar{X}) = \sigma^2/n$ derived from the two rules in [Expectation](#) and [Variance and Standard Deviation](#), then demonstrated by simulation at $n = 4, 16, 64, 256$. What $1/\sqrt{n}$ costs you in practice. And the difference between the **standard deviation of the data** and the **standard error of the mean**, which is the most reliably muddled pair of numbers in applied statistics.

PRACTICE PROBLEMS

Predict-before-you-run, code that executes cleanly and is still wrong, and questions with no code at all. Not graded and not auto-checked—bring answers and disagreements to class.

How to read this week

Simulation before formula, every time. Nearly every rule in these three chapters is stated, then checked by generating data and looking. That is not decoration. A formula you have verified yourself, on numbers you watched appear, is a formula you can reconstruct in an interview or a meeting three years from now. A formula you accepted is one you will misremember.

So when a cell is about to demonstrate a rule, **predict the output first**. If the chapter says variances add, write down what you think `var(x + y)` will print before you press Run. Being wrong once, deliberately and cheaply, is worth more than reading the correct statement five times.

DUE THIS WEEK

HW 2 is due Sunday at 11:59 p.m., the night before class, and Monday's lab is *Watching $\sigma/\sqrt{n}$ emerge by simulation*—so arrive having read [The Standard Error of a Mean](#).

Post your reading questions by **Monday noon**. This is the week to ask small, embarrassing questions: why the square, why the square root, why independence, why $\sqrt{n}$ and not n . Every one of those is a good question, and the session is built out of what you post.

None of these are graded and none of them are auto-checked. They exist so that you arrive on Monday with something to argue about.

Work them in order. Most of the first section comes in two halves: **predict**, then **run**. The prediction is the half that teaches you anything. Pressing Run first and nodding at the output feels like learning and is not—output you expected and output you did not expect look identical after the fact.

Write the prediction down somewhere before you run the cell. On paper, in a comment, in a note on your phone.

WHAT TO BRING TO CLASS

For each problem, bring **an answer you are prepared to be wrong about**. The in-class discussion is graded on whether you said something substantive and could defend it, not on whether it was correct.

This week that matters more than usual. Almost every mistake in *This runs, and it is still wrong* is one that a competent analyst makes at least once, and several of them survive peer review regularly. Being able to name the error out loud is the skill.

Predict before you run

A1. Shifting and scaling

Predict all six numbers. You need `mean(q)` and `sd(q)` from the first two, and the rules from [Expectation](#) and [Variance and Standard Deviation](#) for the rest—do not run it and then explain the output.

R Code						
1	<code>set.seed(6101)</code>					
2	<code>q <- rnorm(10000, mean = 50, sd = 10)</code>					
3						
4	<code>round(c(mean_q = mean(q), sd_q = sd(q),</code>					
5	<code>mean_2q_5 = mean(2*q + 5), sd_2q_5 = sd(2*q + 5),</code>					
6	<code>mean_q_less = mean(q - 20), sd_q_less = sd(q - 20)), 4)</code>					
	mean_q	sd_q	mean_2q_5	sd_2q_5	mean_q_less	sd_q_less
	49.8572	10.0894	104.7143	20.1789	29.8572	10.0894

Then answer: the mean responded to both transformations and the SD responded to only one. State the two rules, and then explain—in terms of what variance *is*—why subtracting 20 cannot possibly change it.

DISCUSSION



A2. Two dice

Predict all four numbers, at least to the nearest whole number.

R Code						
1	<code>set.seed(6101)</code>					
2	<code>n <- 100000</code>					
3	<code>d1 <- sample(1:6, n, replace = TRUE)</code>					
4	<code>d2 <- sample(1:6, n, replace = TRUE)</code>					
5						
6	<code>round(c(var_d1 = var(d1),</code>					
7	<code>var_d2 = var(d2),</code>					
8	<code>var_sum = var(d1 + d2),</code>					

9	<code>var_double = var(2 * d1), 4)</code>			
	<code>var_d1</code>	<code>var_d2</code>	<code>var_sum</code>	<code>var_double</code>
	2.9155	2.9003	5.8606	11.6619

Then answer: `d1 + d2` and `2 * d1` both have twice the expectation of a single die, and their variances differ by a factor of two. Explain the difference in one sentence that a non-statistician would understand. Then say which of the two is a better model for “the total score after two rolls” and which is a better model for “a game that pays double on one roll”, and why the distinction is not pedantic.

DISCUSSION

A3. The two numbers again

R Code	
1	<code>set.seed(6101)</code>
2	<code>h <- rnorm(2500, mean = 175, sd = 7)</code>
3	
4	<code>round(c(sd_of_data = sd(h), se_of_mean = sd(h)/sqrt(2500)), 4)</code>
	<code>sd_of_data se_of_mean</code>
	7.0217 0.1404

Then answer, without running anything else: you collect 22,500 more heights from the same population, for 25,000 in total.

1. What will `sd(h)` be, roughly?
2. What will the standard error be, roughly?
3. Which of the two changed, and what is the one-sentence reason?

DISCUSSION

A4. Does independence matter here?

Two of these four numbers match and two do not. Predict which before running.

R Code	
1	<code>set.seed(6101)</code>
2	<code>n <- 100000</code>
3	<code>a <- rnorm(n, mean = 10, sd = 4)</code>
4	<code>b <- 3 * a - 5</code> # perfectly dependent on a
5	
6	<code>round(c(mean_of_sum = mean(a + b),</code>
7	<code>sum_of_means = mean(a) + mean(b),</code>
8	<code>var_of_sum = var(a + b),</code>
9	<code>sum_of_vars = var(a) + var(b)), 3)</code>
	<code>mean_of_sum sum_of_means var_of_sum sum_of_vars</code>
	35.037 35.037 257.459 160.912

Then answer: one rule survived the dependence and one did not. Which is which, why, and what is the general rule that covers the case that failed?

DISCUSSION

A5. Where the $\sqrt{n}$ comes from

R Code

```
1 set.seed(6101)
2 se_of <- function(n) sd(replicate(3000, mean(rnorm(n, mean = 0, sd = 10))))
3
4 round(c(n_9 = se_of(9), n_36 = se_of(36), n_144 = se_of(144)), 3)
```

```
      n_9  n_36 n_144
3.397 1.668 0.813
```

Then answer: the sample size quadrupled twice. Write down the ratio of consecutive standard errors, and then answer this: if the standard error went like σ/n instead of $\sigma/\sqrt{n}$, what would these three numbers have been? Which step of the derivation in [The Standard Error of a Mean](#) would have to be different for that to happen?

DISCUSSION



This runs, and it is still wrong

Every cell below executes without an error or a warning, and every one produces a number that is wrong, misleading, or answers a question nobody asked. Find the problem, then fix it.

B1. The impressive precision

Task: 900 customers rated a product from 1 to 5. Report the average rating and how precisely it has been estimated.

R Code

```
1 set.seed(6101)
2 w <- rnorm(900, mean = 3.2, sd = 0.9)
3
4 c(mean = mean(w), precision = sd(w))
```

```
      mean precision
3.2509814 0.8901807
```

The analyst writes: “Mean rating 3.25, ± 0.89 .”

Which number did they report, which one did the task ask for, and what is the factor between them? Then write the two sentences that use each number correctly, and say which one belongs in an executive summary and which in a methods appendix.

DISCUSSION



B2. The diversified portfolio

Task: two assets, each with an annual return standard deviation of about 0.99. You hold half your money in each. Report the standard deviation of the portfolio return.

R Code

```
1 set.seed(6101)
2 n <- 100000
```

```

3 mkt <- rnorm(n, 0, 1) # the market, unobserved
4 r1 <- 0.02 + 0.9 * mkt + rnorm(n, 0, 0.4)
5 r2 <- 0.02 + 0.9 * mkt + rnorm(n, 0, 0.4)
6
7 portfolio_var_assuming_independence <- (var(r1) + var(r2)) / 4
8
9 round(c(sd_r1 = sd(r1), sd_r2 = sd(r2),
10        claimed_portfolio_sd = sqrt(portfolio_var_assuming_independence)) / 4)

```

sd_r1	sd_r2	claimed_portfolio_sd
0.9880	0.9881	0.6986

The analyst reports a portfolio SD of 0.699 and calls it a 29% risk reduction from diversification. Now compute the truth:

R Code

```

1 round(c(claimed = sqrt(portfolio_var_assuming_independence),
2        actual = sd((r1 + r2) / 2)), 4)

```

claimed	actual
0.6986	0.9466

Then answer: where exactly in the first cell was independence assumed, and what is it about the two lines generating `r1` and `r2` that makes the assumption false? By what factor was the risk understated? Finally: give the general formula that would have produced the right answer, and say which single extra quantity the analyst would have needed to compute.

DISCUSSION

>

B3. The standard error that is too good

Task: 200 students sat an exam. Report the standard error of the mean score.

R Code

```

1 set.seed(6101)
2 score <- rnorm(200, mean = 72, sd = 11)
3
4 c(se = sd(score) / 200)

```

se
0.05606155

The number is 0.056. Two questions: what is the correct value, and—more usefully—**what should have made you suspicious of 0.056 before you checked the formula?** Answer the second one in terms of what the number is claiming about the world.

DISCUSSION

>

B4. The average of the averages

Task: four laboratories each measured the same compound. Report the overall mean concentration and its standard error.

R Code

```

1 lab_mean <- c(4.1, 4.4, 3.9, 4.3)
2 lab_n    <- c(200, 12, 8, 5)
3
4 c(overall_mean = mean(lab_mean),

```

5	se	= sd(lab_mean) / sqrt(4))
	overall_mean	se
	4.1750000	0.1108678

Two independent errors here, one in each line.

For the first line, compute the correct overall mean and say why it differs. For the second, the analyst used $n = 4$ in the standard error. Say what population that standard error is describing, whether that is the population the task asked about, and what you would need in order to do it properly.

DISCUSSION

B5. More data, same problem

Task: estimate the mean household income in a city. A web panel returns 250,000 responses.

R Code		
1	set.seed(6101)	
2	income <- rlnorm(250000, meanlog = 11.1, sdlog = 0.6)	
3		
4	round(c(mean = mean(income),	
5	se = sd(income) / sqrt(250000)), 2)	
	mean	se
	79271.74	104.55

The standard error is about \$105 on a mean of about \$79,300—roughly an eighth of one percent. The analyst reports the city’s mean household income as “\$79,300, with a margin of error of $\pm \$210$ ”.

The arithmetic is correct. Say what the standard error is and is not measuring, and construct a specific, plausible way in which this estimate could be wrong by \$15,000 while the standard error stays at \$105. Then answer the practical question: what would you spend the next \$10,000 of project budget on—more responses, or something else?

DISCUSSION

B6. The repeated measurement

Task: estimate the mean resting heart rate of a population. Ten volunteers each wore a monitor that recorded their heart rate once a minute for 100 minutes.

R Code		
1	set.seed(6101)	
2	person <- rep(1:10, each = 100)	
3	true_hr <- rnorm(10, mean = 68, sd = 8)[person] # each person's own level	
4	reading <- true_hr + rnorm(1000, mean = 0, sd = 2)	
5		
6	c(n = length(reading),	
7	mean = mean(reading),	
8	se = sd(reading) / sqrt(length(reading)))	
	n	mean
	1000.0000000	65.4749322
		se
		0.2070255

The reported standard error is about 0.21 beats per minute, from $n = 1000$.

Then answer: which step of the derivation in [The Standard Error of a Mean](#) does this violate, and where in the data-generating code can you point to the violation? Roughly what is the effective sample size here—10, 1000, or something in between—and why? Then compute a defensible standard error from these same data with **one line of code**. (Hint: what are the ten quantities you actually have independent information about?)

R Code

```
1 # your one-line fix
```

DISCUSSION

Judgment

No code. Bring written answers; we will disagree about these out loud.

C1.

A colleague says: “Our A/B test needs to detect a 0.5% lift. We ran it for a week and got 40,000 users per arm, and the confidence interval is still too wide. Let’s run it another week.”

Using only $\sigma/\sqrt{n}$, explain what a second week buys them, as a factor. Then say how long they would need to run it to halve the width of the interval, and give one reason why running an A/B test for that long is a bad idea for reasons that have nothing to do with statistics.

DISCUSSION

C2.

You read a paper reporting “mean tumor volume $340 \text{ mm}^3 \pm 25$ ”. The methods section does not say whether the 25 is an SD or an SE, and the sample size is $n = 9$.

Work out what the *other* quantity would be in each case. Then say which interpretation makes the paper’s conclusion look stronger, and why an author with a result to sell has an incentive here that a reader should be aware of.

DISCUSSION

C3.

[Variance and Standard Deviation](#) showed three sums with identical marginal distributions whose variances came out 0, 18, and 36 depending only on the dependence structure.

Invent a **real** situation—not a mathematical one—where two quantities you might add together are strongly *negatively* dependent, so that the total is far less variable than either part. Then explain why someone would deliberately build such a pairing. (Every hedging strategy, every redundant system design, and a good deal of portfolio theory is an answer to this question.)

DISCUSSION

C4.

Here is a claim you will hear: “*With big data, statistics doesn’t matter any more. The standard errors are so small that everything is significant.*”

The first sentence of that is wrong and the second is nearly right. Separate them. Say what genuinely does become easy with a very large n , what does not improve at all, and what new problem the second sentence is actually pointing at. Then give one concrete example of a question that a billion rows cannot answer and a randomized experiment on two hundred people can.

DISCUSSION



C5.

Go back to the list you wrote at the end of [Practice Problems](#)—*if the data collection were repeated from scratch, how different would my number be?*—and answer it, for the number you chose then.

You now have the machinery. Write the actual sentence you would put in a report: the estimate, the give-or-take, and the assumption you had to make to be entitled to the give-or-take. That last clause is the part most people leave out, and it is the part this week was for.

DISCUSSION



This is the chapter the course is built on.

You have a sample. You computed its mean, and R printed a number to seven figures with complete confidence. But that number came from the particular rows that happened to end up in your file, and a different sample would have given a different answer. The practice problems in [Practice Problems](#) ended on exactly this discomfort, and [Expectation](#) sharpened it: six samples from the same distribution gave six different means.

The question is: **how far off is $\bar{x}$ likely to be?**

It sounds like an impossible question. You have one sample. You cannot see the others. And yet the answer takes four lines of algebra, uses nothing beyond the two rules you just met, and is the reason every confidence interval, every p -value, and every standard error printed beside a regression coefficient exists.

The set-up

Take n observations $X_1, X_2, \dots, X_n$, drawn **independently** from a population with expectation μ and standard deviation σ . The sample mean is

$$\bar{X} = \frac{1}{n}(X_1 + X_2 + \dots + X_n)$$

Notice the capital letter. $\bar{X}$ is written as a **random variable**, because before you collect the data it *is* one—it has a distribution, with an expectation and a variance of its own. That distribution has a name, the **sampling distribution of the mean**, and everything below is about it.

First, where is it centered?

By linearity from [Expectation](#), and remembering that linearity asks for nothing at all:

$$E[\bar{X}] = \frac{1}{n}(E[X_1] + \dots + E[X_n]) = \frac{1}{n} \cdot n\mu = \mu$$

The sample mean is centered on the truth. It is not systematically too big or too small. In the language of [Describing a Single Variable](#), it is **unbiased**.

That is reassuring and it is not enough. Unbiased says the errors average out over infinitely many repetitions; it says nothing about the size of the error in the one sample you actually have. For that we need the variance.

Now, how much does it move?

THE DERIVATION

$$\text{Var}(\bar{X}) = \text{Var}\left(\frac{1}{n} \sum_{i=1}^n X_i\right)$$

Step 1—pull the constant out. By the scaling rule $\text{Var}(aX) = a^2 \text{Var}(X)$ from [Variance and Standard Deviation](#), with $a = 1/n$:

$$= \frac{1}{n^2} \text{Var}\left(\sum_{i=1}^n X_i\right)$$

Step 2—split the sum. The observations are independent, so variances add:

$$= \frac{1}{n^2} \sum_{i=1}^n \text{Var}(X_i)$$

Step 3—every term is the same. Each X_i comes from the same population, so each has variance σ^2 , and there are n of them:

$$= \frac{1}{n^2} \cdot n\sigma^2 = \frac{\sigma^2}{n}$$

Step 4—square root, to get back to the data's units:

$$\text{SD}(\bar{X}) = \frac{\sigma}{\sqrt{n}}$$

This is the **standard error of the mean**, written $\text{SE}(\bar{X})$.

Four lines. Look at what each one used:

STEP	WHAT IT NEEDED
$E[\bar{X}] = \mu$	linearity— no assumptions
Step 1	$\text{Var}(aX) = a^2 \text{Var}(X)$ —no assumptions
Step 2	variances add—requires independence (strictly, zero covariance)
Step 3	identical distribution

Exactly one step needed an assumption about how the observations relate to each other, and it is Step 2. That is why [Variance and Standard Deviation](#) spent so long breaking the addition rule on purpose. If your observations are not independent—repeated measures on the same patient, students clustered in classrooms, daily readings from a sensor that drifts—Step 2 is wrong, the covariance terms are usually positive, and $\sigma/\sqrt{n}$ is an *underestimate* of the real wobble. Software will print it anyway. It has no way of knowing.

Notice too where the $\sqrt{\cdot}$ comes from. There is no square root anywhere in the derivation until the last line, and its only job is to convert squared units back into readable ones. **The n divides the variance; the $\sqrt{n}$ is what that looks like after the units are fixed.**

Watching it happen

Algebra is a claim. Here is the check.

We will pretend to be a population with $\mu = 100$ and $\sigma = 12$. For each sample size n , we draw 5,000 separate samples, compute the mean of each, and then measure how spread out those 5,000 means are. If the derivation is right, that spread should come out

at $12/\sqrt{n}$.

R Code

```
1 set.seed(6101)
2 sigma <- 12
3 reps <- 5000
4 ns <- c(4, 16, 64, 256)
5
6 observed <- sapply(ns, function(n) {
7   sample_means <- replicate(reps, mean(rnorm(n, mean = 100, sd = sigma)))
8   sd(sample_means)
9 })
10
11 predicted <- sigma / sqrt(ns)
12
13 round(rbind(n = ns, observed = observed, predicted = predicted), 4)
```

```
      [,1]      [,2]      [,3]      [,4]
n      4.0000 16.0000 64.0000 256.0000
observed 5.9903 2.9748 1.4892 0.7592
predicted 6.0000 3.0000 1.5000 0.7500
```

n	4	16	64	256
observed	5.9903	2.9748	1.4892	0.7592
predicted $12/\sqrt{n}$	6.0000	3.0000	1.5000	0.7500

Within about 1% at every sample size, and the residual gap is simulation noise—5,000 replications is a lot but it is not infinity. Nothing was fitted. The predicted row was computed from σ and n alone, without looking at the simulation.

R Code

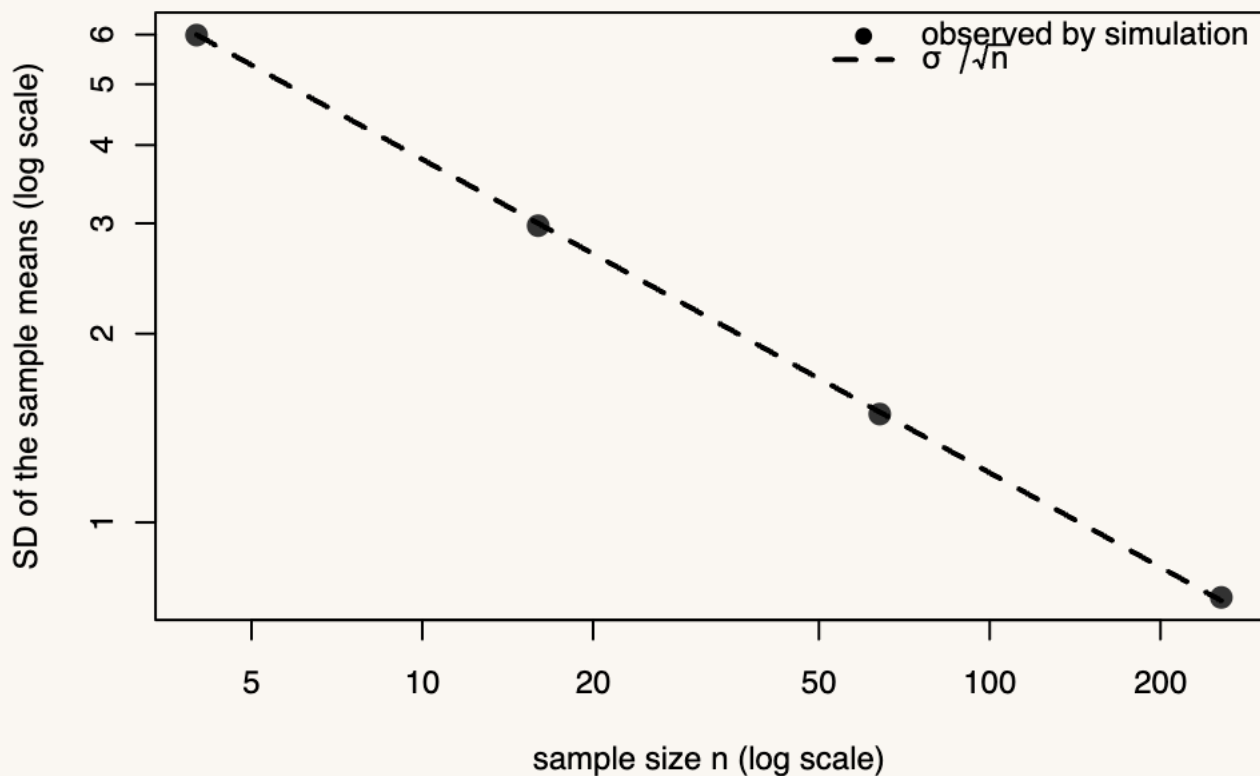
```
1 round(observed / predicted, 4)
```

```
[1] 0.9984 0.9916 0.9928 1.0122
```

Plot it on log-log axes, where $\sigma/\sqrt{n}$ becomes a straight line of slope $-1/2$:

R Code

```
1 plot(ns, observed, log = "xy", pch = 19, col = "gray20", cex = 1.4,
2       xlab = "sample size n (log scale)",
3       ylab = "SD of the sample means (log scale)")
4 lines(ns, predicted, lwd = 2, lty = 2)
5 legend("topright", bty = "n", lty = c(NA, 2), pch = c(19, NA), lwd = c(NA, 2),
6       legend = c("observed by simulation", expression(sigma / sqrt(n))))
```

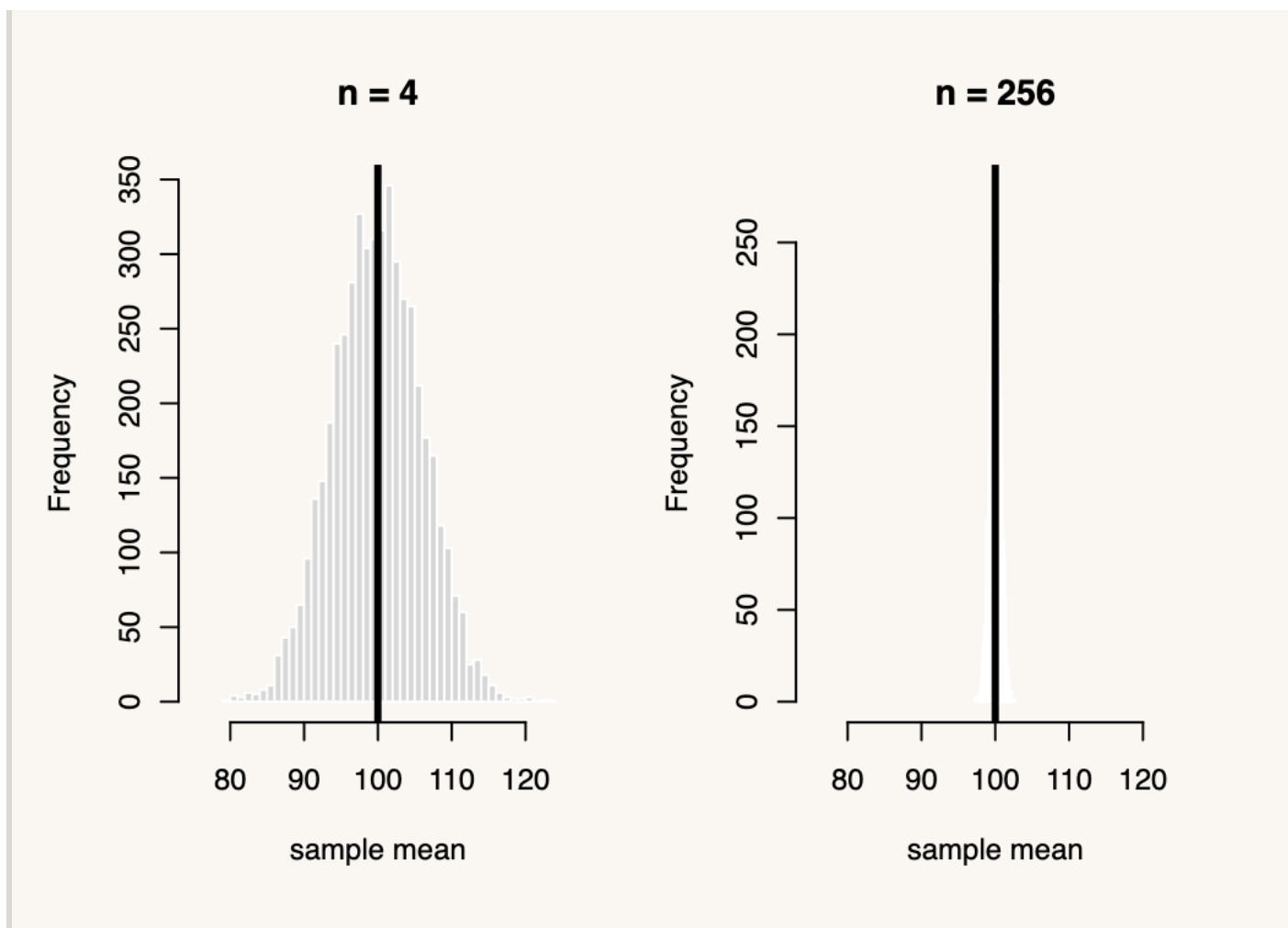


What the sampling distribution looks like

The same 5,000-sample experiment, drawn, at the smallest and largest n :

R Code

```
1 set.seed(6101)
2 m4 <- replicate(5000, mean(rnorm(4, mean = 100, sd = 12)))
3 m256 <- replicate(5000, mean(rnorm(256, mean = 100, sd = 12)))
4
5 par(mfrow = c(1, 2))
6 hist(m4, breaks = 40, xlim = c(75, 125), col = "gray85", border = "white",
7     main = "n = 4", xlab = "sample mean")
8 abline(v = 100, lwd = 3)
9 hist(m256, breaks = 40, xlim = c(75, 125), col = "gray85", border = "white",
10    main = "n = 256", xlab = "sample mean")
11 abline(v = 100, lwd = 3)
```



R Code

```
1 round(rbind(sd      = c(n4 = sd(m4), n256 = sd(m256)),
2           lowest    = c(min(m4), min(m256)),
3           highest    = c(max(m4), max(m256))), 2)
```

	n4	n256
sd	5.99	0.75
lowest	79.18	97.25
highest	123.57	102.65

Both are centered on 100—that is $E[\bar{X}] = \mu$, visible. But at $n = 4$ the sample means ranged from 79.18 to 123.57, and at $n = 256$ from 97.25 to 102.65. Same population, same estimator, and one of those two pictures would let you report a useful number while the other would not.

This is the diagram that becomes a confidence interval. Once you know how wide the sampling distribution is, and roughly what shape it has, you can say how far from μ your one sample mean plausibly landed—and run the statement backwards to get an interval around $\bar{x}$.

It does not need a Normal population

A reasonable worry: all of that used `rnorm`, so perhaps $\sigma/\sqrt{n}$ is a fact about Normal distributions.

It is not. Nothing in the derivation mentioned the shape of the distribution—only its variance. Here is the same experiment on a heavily right-skewed exponential population, rate 0.4, whose standard deviation is $1/0.4 = 2.5$:

R Code

```
1 set.seed(6101)
2 pop_sd <- 2.5
3 obs_exp <- sapply(ns, function(n) sd(replicate(5000, mean(arexp(n, rate = 0.4))))))
4
5 round(rbind(n = ns, observed = obs_exp, predicted = pop_sd / sqrt(ns)), 4)
```

	[,1]	[,2]	[,3]	[,4]
n	4.0000	16.0000	64.0000	256.0000
observed	1.2697	0.6292	0.3147	0.1549
predicted	1.2500	0.6250	0.3125	0.1562

1.2697, 0.6292, 0.3147, 0.1549 against 1.25, 0.625, 0.3125, 0.15625. The rule holds for a distribution that looks nothing like a bell curve.

What *does* depend on the population shape is the **shape** of the sampling distribution, not its width. That is the central limit theorem, and it is next week's chapter.

What $1/\sqrt{n}$ costs

The derivation is done. Now the consequence, which is the part that will follow you into every job you have.

R Code

```
1 n_values <- c(100, 400, 1600, 6400)
2 round(rbind(n = n_values, SE = 12 / sqrt(n_values)), 4)
```

	[,1]	[,2]	[,3]	[,4]
n	100.0	400.0	1600.0	6400.00
SE	1.2	0.6	0.3	0.15

Each column has **four times** the data of the one before it, and each SE is **half** the one before it: 1.20, 0.60, 0.30, 0.15.

THE RULE TO CARRY AROUND

To halve the standard error, quadruple the sample. To divide it by k , multiply the sample by k^2 .

Data collection typically costs roughly linearly in n . Precision improves like $\sqrt{n}$. So every additional decimal place of precision costs a hundred times what the previous one did.

This is not an abstraction. It is the number that decides whether a proposal is a week of work or a year of it:

R Code

```
1 sigma <- 12
2 target_se <- c(2, 1, 0.5, 0.25)
3 rbind(target_SE = target_se, n_required = (sigma / target_se)^2)
```

	[,1]	[,2]	[,3]	[,4]
target_SE	2	1	0.5	0.25
n_required	36	144	576.0	2304.00

To pin the mean down to ± 0.25 you need 2,304 observations; to get to ± 0.5 , only 576. The last halving cost you 1,728 extra observations—three times everything you had collected up to that point.

Two practical readings of the same fact:

- **Early data is cheap and valuable.** Going from 10 observations to 40 is transformative. This is why a pilot study is worth running.

- **Late data is expensive and marginal.** Going from 10,000 to 40,000 buys you the same *factor* of improvement, at 1,000 times the cost of the first jump. When someone asks for “just a bit more precision” at the end of a project, this is the arithmetic you show them.

And the one that matters most for reading other people’s work: **an enormous sample makes the standard error tiny, and does nothing whatever about bias.** A survey of two million people who all opted in has a beautiful standard error and may be wrong by twenty points. $\sigma/\sqrt{n}$ measures the wobble from sampling, and only that. It has no opinion about whether you sampled the right people.

🔍 QUADRUPLING THE SAMPLE



Your current study has $n = 400$ and a standard error of 3.0. Your manager wants the standard error down to 1.0, and asks how much more data that will take.

- ☐ $n = 1,200$. The standard error has to be divided by three, so the sample has to be tripled.
- ☐ $n = 3,600$. The standard error falls like $1/\sqrt{n}$, so cutting it to a third means nine times the data.
- ☐ $n = 1,600$. Quadrupling the sample halves the error, and one quadrupling gets close enough.
- ☐ It cannot be done by collecting more data—the standard error is a property of the population.



The proportion is a mean in disguise

Before leaving this, the special case you will use more than any other.

A conversion rate, a click rate, a defect rate, a survey’s “yes” share—each is a **proportion**, and a proportion is the mean of a column of zeros and ones. So everything above applies with nothing changed.

Write $X_i = 1$ if the i th visitor converts and 0 otherwise. Then $\hat{p} = \bar{X}$, and a Bernoulli variable has variance $p(1 - p)$, so the derivation you just did hands you the answer:

$$SE(\hat{p}) = \sqrt{\frac{p(1 - p)}{n}}$$

No new theorem. The same $\sigma/\sqrt{n}$, with $\sigma = \sqrt{p(1 - p)}$ substituted.

R Code

```
1 set.seed(6101)
2 # 4,000 visitors, a true conversion rate of 3%.
3 visits <- rbinom(4000, size = 1, prob = 0.03)
4
5 p_hat <- mean(visits)
6 se <- sqrt(p_hat * (1 - p_hat) / length(visits))
7
8 round(c(p_hat = p_hat, se = se,
9         by_the_general_formula = sd(visits) / sqrt(length(visits))), 5)
```

p_hat	se	by_the_general_formula
0.02725	0.00257	0.00257

The last two agree to four decimals, which is the point: $sd(visits)/sqrt(n)$ and $\sqrt{p(1 - p)/n}$ are the same calculation, except that $sd()$ divides by $n - 1$ where $p(1 - p)$ divides by n .

Two consequences worth carrying.

The spread depends on the rate. $p(1 - p)$ is largest at $p = 0.5$ and shrinks toward either end, so a rate near 50% is the hardest to pin down and a rate near 1% is comparatively easy in absolute terms—but not in *relative* terms, which is what anyone actually cares about:

R Code			
1	n <- 4000		
2	p <- c(0.01, 0.03, 0.10, 0.50)		
3	se <- sqrt(p * (1 - p) / n)		
4			
5	round(data.frame(p, se, relative_error = se / p), 4)		
	p	se	relative_error
1	0.01	0.0016	0.1573
2	0.03	0.0027	0.0899
3	0.10	0.0047	0.0474
4	0.50	0.0079	0.0158

A 3% conversion rate measured on 4,000 visitors carries a standard error of 0.0027—which sounds tiny until you notice it is **9% of the thing being measured**. That is why rare-event experiments need so much traffic, and it is the arithmetic behind every “we need to run this test for another three weeks”.

You need p to plan for p . Rearranging for n requires the rate you have not measured yet, so in practice you either use a pilot estimate or take the worst case $p = 0.5$, which gives the largest n and cannot be an underestimate.

The SD of the data versus the SE of the mean

Now the confusion that outlives every course. It is worth slowing down for, because it is asked in interviews, it appears in published papers, and the two numbers can differ by a factor of thirty in the same analysis.

R Code			
1	set.seed(6101)		
2	s <- rnorm(400, mean = 100, sd = 12)		
3			
4	round(c(sample_mean = mean(s),		
5	sd_of_data = sd(s),		
6	se_of_mean = sd(s) / sqrt(400)), 4)		
	sample_mean	sd_of_data	se_of_mean
	100.2270	12.2321	0.6116

The mean is 100.2270. The SD is 12.2321. The SE is 0.6116. Twenty times apart, computed from the same 400 numbers, and both are correct—they answer different questions.

TWO NUMBERS, TWO QUESTIONS

	SD OF THE DATA, s	SE OF THE MEAN, $s/\sqrt{n}$
Answers	how much do individuals differ?	how much would my estimate move on a re-run?
Describes	the population	the estimate
As n grows	settles down to σ — does not shrink	shrinks like $1/\sqrt{n}$ — goes to zero
Goes in the sentence	“a typical adult is within about 12 of 100”	“the population mean is 100.2, give or take 0.6”

People do not become more alike because you measured more of them. Your *estimate of their average* does get sharper.

Watch both behave, at four sample sizes from the same population:

R Code

```
1 set.seed(6101)
2 round(sapply(c(25, 100, 400, 1600), function(n) {
3   d <- rnorm(n, mean = 100, sd = 12)
4   c(n = n, sd_of_data = sd(d), se_of_mean = sd(d) / sqrt(n))
5 })), 4)
```

	[,1]	[,2]	[,3]	[,4]
n	25.0000	100.0000	400.0000	1600.0000
sd_of_data	12.9408	12.0599	11.8842	12.0447
se_of_mean	2.5882	1.2060	0.5942	0.3011

n	25	100	400	1600
SD of the data	12.94	12.06	11.88	12.04
SE of the mean	2.59	1.21	0.59	0.30

The top row wanders around 12 and stays there. The bottom row falls by half every time n quadruples. Different rows, different jobs.

WHERE THIS GOES WRONG IN THE WILD

- **Error bars on a chart with no caption saying which they are.** SE bars are about $\sqrt{n}$ times shorter than SD bars, so the same data can be drawn to look tight or scattered depending on a choice the reader cannot see. If a figure does not say, it does not mean anything. In your own work, say.
- **“ $\pm$ ” in a report.** “Mean satisfaction 7.2 ± 1.8 ” is ambiguous and the two readings support entirely different conclusions.
- **Dividing by n instead of $\sqrt{n}$.** It produces a number small enough to look impressive and it is simply wrong. Check the units of your own claim: if you would not believe the precision, you probably made this mistake.
- **The n that goes under the root is the number of *independent* observations.** Fifty measurements on ten patients is not $n = 50$. Step 2 of the derivation is where that breaks.

A survey of 900 adults reports a mean daily screen time of **4.2 hours** with a **standard deviation of 2.4 hours**.

You are writing the sentence that says *how precisely the population mean has been pinned down*. Which number belongs in that sentence?

- ☐ 2.4 hours—the standard deviation is the spread of the data, and spread is what precision means.
- ☐ $2.4/900 = 0.0027$ hours, because dividing by the sample size is what accounts for having a lot of data.
- ☐ $2.4/\sqrt{900} = 0.08$ hours—the standard error. The SD says how much individual people differ from each other; the SE says how much the sample *mean* would move if the survey were run again.
- ☐ Neither. Precision of an estimate cannot be assessed without knowing the true population standard deviation.



Exercise: compute a standard error

Exercise

```
1 # `bp` holds systolic blood pressure for 150 patients.
2 # Return the STANDARD ERROR OF THE MEAN -- a single number.
3 -----
```

Please replace ----- with valid code.

Exercise: check the derivation by simulation

Exercise

```
1 # Do NOT use the formula here. Build the sampling distribution and measure it.
2 #
3 # Draw 4,000 samples, each of size 36, from a Normal population with
4 # mean 50 and sd 9. Take the mean of each sample. Return the standard
5 # deviation of those 4,000 sample means.
6 #
7 # Predict the answer from sigma/sqrt(n) first, then see whether the
8 # simulation agrees.
9 -----
```

Please replace ----- with valid code.

Exercise: how much data do you need?

A pilot study suggests the population standard deviation is about 18.
Your team needs the standard error of the mean to be 1.5 or better.

Exercise

```
1 # Return the sample size n required to achieve a standard error of exactly 1.5,
2 # given sigma = 18. A single number.
```

3 | -----

Please replace ----- with valid code.

Exercise: what happens to the SE

A study has $n = 100$ and some standard error.
A second study, same population and same measurement, has $n = 900$.

Exercise

```
1 # Return the RATIO SE(n = 900) / SE(n = 100) as a single number.
2 # Note that sigma is unknown -- and that you do not need it.
3 # Predict the answer before you compute it.
4 -----
```

Please replace ----- with valid code.

Exercise: which number does the sentence need?

A city measures commute times for a random sample of 2,500 residents.
sample mean = 31.4 minutes
sd of the data = 14.0 minutes
se of the mean = 0.28 minutes

The transit director asks: "How much do commutes vary across residents?
We are deciding whether to run one bus schedule or several."

Exercise

```
1 # Which number answers the director's question? Assign the letter,
2 # for example: answer <- "A"
3 #
4 # A. 0.28 minutes, the standard error—it is the more precise number, and
5 # precision is what a decision needs.
6 # B. 14.0 minutes, the standard deviation of the data—the director is asking
7 # how residents differ from each other, not how well the mean is estimated.
8 # C. 31.4 minutes, the mean—variation is best summarized by the center.
9 # D. Neither 14.0 nor 0.28; with  $n = 2,500$  the variation across residents is
10 # negligible.
11 answer <- -----
```

Please replace ----- with valid code.

Where this goes

You now have the result the rest of the course quotes:

$$SE(\bar{X}) = \frac{\sigma}{\sqrt{n}}, \quad \text{estimated in practice by } \frac{s}{\sqrt{n}}$$

In real work you do not know σ , so you substitute the sample standard deviation s . That substitution introduces extra uncertainty of its own, which is small when n is large and not small when n is small—and it is exactly why the t distribution exists, with its $n - 1$ degrees of freedom, the same $n - 1$ [Describing a Single Variable](#) argued about.

- **Confidence intervals.** You know the sampling distribution is centered at μ with width $\sigma/\sqrt{n}$; the central limit theorem will tell you it is roughly Normal. Put those together and $\bar{x} \pm 2 SE$ becomes an interval with a stateable success rate. There is

nothing else in it.

- **Hypothesis tests.** A test statistic is (estimate — claim)/SE—“how many standard errors away is what I saw from what you claimed?” The SE is the denominator of every test in this course.
- **A/B testing.** How long to run the test is the sample-size calculation you just did, with a difference of two means in place of one mean.
- **Regression.** Every coefficient in every model output comes with a standard error beside it, and it means precisely what it means here: how much would this number move if I collected the data again.

If you take one thing from this chapter, take this: **every “give or take” in statistics is a standard error, and every standard error is a variance divided by n with a square root taken at the end.**

Summary

- $E[\bar{X}] = \mu$: the sample mean is centered on the truth, by linearity, with no assumptions required.
- $\text{Var}(\bar{X}) = \sigma^2/n$, and so $\text{SE}(\bar{X}) = \sigma/\sqrt{n}$. Four lines, using only the scaling rule and the addition rule.
- **Independence is used exactly once**, in the step where variances add. When observations are clustered or repeated, that step fails, the covariances are usually positive, and the reported SE is too small.
- Simulation confirms it: at $n = 4, 16, 64, 256$ the spread of 5,000 sample means came out 5.99, 2.97, 1.49, 0.76 against a predicted 6, 3, 1.5, 0.75. It works for skewed populations too—the derivation never mentioned shape.
- **Quadruple the sample to halve the error.** $n = (\sigma/\text{SE})^2$. Early data is cheap and valuable; late data is expensive and marginal.
- A tiny standard error says nothing about bias. Two million self-selected respondents give a beautiful SE and can still be badly wrong.
- **SD of the data** describes people and does not shrink with n . **SE of the mean** describes your estimate and shrinks like $1/\sqrt{n}$. Say which one your error bars are.
- A **proportion is a mean of zeros and ones**, so $\text{SE}(\hat{p}) = \sqrt{p(1-p)/n}$ with no new theory. Rare rates have small absolute errors and large relative ones, which is why they need so much traffic.

Next: practice problems, and then—next week—what shape that sampling distribution has, and what you are allowed to claim once you know.

The expectation tells you where a distribution sits. It tells you nothing about how far the values stray from there, and two distributions with the same expectation can be nothing alike—a bet that pays \$1.00 every time and a bet that pays \$0 or \$100 with the right probabilities both have $E[X] = 1$, and only one of them will end your month badly.

The second summary number fixes that.

$$\text{Var}(X) = E[(X - E[X])^2]$$

Read it inside out and it is a sentence you already understand: take the distance from each value to the center, square it, and average. **Variance is the average squared distance from the expectation.**

[Describing a Single Variable](#) computed s^2 from a column of data. This chapter is about $\text{Var}(X)$, a property of a distribution—the same relationship the last chapter drew between $\bar{x}$ and $E[X]$. The formulas look alike on purpose.

Why squared, and why then a root

The obvious first attempt is to average the raw deviations $X - E[X]$. It fails instantly, and it is worth seeing it fail:

R Code	
1	<code>set.seed(6101)</code>
2	<code>x <- rnorm(200000, mean = 5, sd = 2)</code>
3	
4	<code>round(c(sum_of_deviations = sum(x - mean(x)),</code>
5	<code>mean_of_deviations = mean(x - mean(x))), 10)</code>
<pre>sum_of_deviations mean_of_deviations 0 0</pre>	

Zero, to ten decimal places, and not by luck. The mean is the balance point, so the positive and negative deviations cancel exactly, by construction, for every dataset that has ever existed. An average deviation of zero is not a measurement of anything.

You need to stop the canceling, and there are two ways to do it: take absolute values, or square. Both are used. Squaring won, for three reasons:

1. **It penalizes large deviations more.** A point twice as far away contributes four times as much. That is often what you want—being off by 20 once is usually worse than being off by 10 twice, and only squaring says so: 400 against 200, where absolute values call it a tie.
2. **It is smooth.** $|x|$ has a corner at zero, which makes calculus awkward. x^2 does not, and every optimization you meet later (least squares, maximum likelihood, gradient descent) leans on that.
3. **It adds.** This is the real reason, and it is the subject of the second half of this chapter. Variances of independent quantities add; mean absolute deviations do not. Without that additivity there is no $\sigma/\sqrt{n}$ and there is no inference at all.

Here is the calculation, all three quantities side by side:

R Code	
1	<code>round(c(mean_absolute_deviation = mean(abs(x - mean(x))),</code>
2	<code>mean_squared_deviation = mean((x - mean(x))^2),</code>
3	<code>variance = var(x),</code>
4	<code>standard_deviation = sd(x)), 4)</code>
<pre>mean_absolute_deviation mean_squared_deviation variance 1.5982 4.0168 4.0168 standard_deviation 2.0042</pre>	

The mean squared deviation is 4.0168, and `var(x)` is also 4.0168.

A NOTE ON THE DIVISOR, NOT A REPEAT OF IT

Strictly, `var()` divides by $n - 1$ and `mean((x - mean(x))^2)` divides by n , so those two numbers are not identical—they differ by a factor of $(n - 1)/n = 199999/200000$, which does not survive rounding to four places.

[Describing a Single Variable](#) explained *why* the $n - 1$ is there: the sample mean was fitted to this very sample, so deviations measured from it are systematically too small. That argument is about **estimating** a variance from data. This chapter is about the variance itself, $\text{Var}(X) = E[(X - \mu)^2]$, which is defined with the true μ and has no divisor to argue about at all.

Two different objects, and the $n - 1$ belongs only to the first.

Back to the original units

The variance here is 4.0168 *what?* The data are in some unit—say millimeters—so the variance is in **squared millimeters**, which is not a quantity anyone has an intuition about. So we take the square root:

$$\text{SD}(X) = \sigma = \sqrt{\text{Var}(X)}$$

`sd(x)` returns 2.0042, and that number is in millimeters. It is the one you put in a sentence: *values are typically about 2 mm from the center.*

So why not work with the SD throughout and skip the variance? Because the SD does not obey nice rules and the variance does. Every derivation in this course happens in variance, and the square root is taken at the very end, purely so that a human can read the answer. Watch for that pattern in [The Standard Error of a Mean](#)—it is exactly what happens there.

Rescaling: $\text{Var}(aX + b)$

$$\text{Var}(aX + b) = a^2 \text{Var}(X)$$

Two claims in one line, and students distrust both.

The b vanishes entirely. Adding a constant to every value slides the whole distribution sideways. No distance between any two points changes. Variance is built out of distances, so it cannot possibly notice.

The a is squared. Variance lives in squared units, so a factor applied to the data is applied twice to the variance.

Predict all five of these before you run it:

R Code					
1	<code>set.seed(6101)</code>				
2	<code>x <- rnorm(200000, mean = 5, sd = 2)</code>				
3					
4	<code>round(c(var_x = var(x),</code>				
5	<code>var_x_plus_100 = var(x + 100),</code>				
6	<code>var_3x = var(3 * x),</code>				
7	<code>var_3x_plus_100 = var(3 * x + 100),</code>				
8	<code>var_minus_3x = var(-3 * x)), 4)</code>				
	<code>var_x</code>	<code>var_x_plus_100</code>	<code>var_3x</code>	<code>var_3x_plus_100</code>	<code>var_minus_3x</code>
	4.0168	4.0168	36.1513	36.1513	36.1513

4.0168, 4.0168, 36.1513, 36.1513, 36.1513.

Adding 100 changed nothing—not approximately nothing, *nothing*. Multiplying by 3 multiplied the variance by 9, and $9 \times 4.01681 = 36.1513$ exactly. Multiplying by -3 gave the identical answer, because $(-3)^2 = 9$: flipping a variable's sign does not make it less variable.

In standard deviations the rule loses the square:

$$SD(aX + b) = |a| SD(X)$$

R Code

```
1 round(c(sd_x = sd(x), sd_3x = sd(3 * x), sd_x_plus_100 = sd(x + 100)), 4)
```

sd_x	sd_3x	sd_x_plus_100
2.0042	6.0126	2.0042

2.0042 and 6.0126, which is exactly three times it.

🔗 SHIFTING AND SCALING



A variable X has $\text{Var}(X) = 25$.

What is $\text{Var}(4X + 10)$?

- ☐ 110, because the 4 multiplies the variance and the 10 is then added on.
- ☐ 100, because the 4 multiplies the variance and adding a constant does nothing.
- ☐ 410, because the 4 is squared and the 10 is still added on.
- ☐ 400, because the 4 is squared and adding a constant does nothing at all.



Exercise: rescaling without recomputing

Exercise

```
1 # `g` holds 50,000 measurements. A colleague needs the variance of
2 # 5 * g - 3.
3 #
4 # Compute it FROM var(g), using the rule. Do not write var(5 * g - 3);
5 # the point of the exercise is to get the rule into your hands.
6 -----
```

Please replace _____ with valid code.

Exercise: changing units

Exercise

```
1 # `h` holds 5,000 heights recorded in inches. One inch is 2.54 cm.
2 #
3 # Return the STANDARD DEVIATION of these heights in centimeters,
4 # computed from sd(h) rather than by converting the data.
5 -----
```

Please replace _____ with valid code.

The key fact: variances add

Here is the result the whole week is built on.

VARIANCES OF INDEPENDENT QUANTITIES ADD

If X and Y are **independent**, then

$$\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y)$$

Standard deviations do **not** add. You must go to the variance, add there, and come back with a square root.

Two independent variables, one with SD 3 and one with SD 4:

R Code

```
1 set.seed(6101)
2 n <- 200000
3 a <- rnorm(n, mean = 0, sd = 3)      # variance 9
4 b <- rnorm(n, mean = 0, sd = 4)      # variance 16
5
6 round(c(var_a      = var(a),
7         var_b      = var(b),
8         var_of_sum  = var(a + b),
9         sum_of_vars = var(a) + var(b)), 3)
```

var_a	var_b	var_of_sum	sum_of_vars
9.038	15.975	25.003	25.013

25.003 against 25.013. The rule holds, up to the noise you get from 200,000 draws.

Now look at what that means in standard deviations:

R Code		
1	<code>round(c(sd_of_sum</code>	<code>= sd(a + b),</code>
2	<code>sd_a_plus_sd_b</code>	<code>= sd(a) + sd(b)), 3)</code>
	<code>sd_of_sum</code>	<code>sd_a_plus_sd_b</code>
	5.000	7.003

5.000 versus 7.003. The SD of the sum is 5, not 7. If you had added the standard deviations you would have overstated the spread of the sum by 40%.

The 3-4-5 triangle is not a coincidence: $\sqrt{3^2 + 4^2} = 5$. Adding independent variability is Pythagorean, not arithmetic. And it is always in this direction— $\sqrt{a + b} < \sqrt{a} + \sqrt{b}$ —so **adding SDs always overstates the answer**. The reason is cancellation. Some of the time a is high while b is low, and the two partially cancel; the 5 is the arithmetic of that cancellation.

Two measurement errors are independent, and each has a standard deviation of 5 units. You add the two measurements together.

What is the standard deviation of the sum?

- ☐ About 7.07, because the variances add: $25 + 25 = 50$, and $\sqrt{50} \approx 7.07$.
- ☐ Exactly 10, because standard deviations add just as variances do.
- ☐ Exactly 5, because averaging two errors of the same size leaves the size unchanged.
- ☐ Exactly 50, because the variances add and the answer is that sum.



Independence is a condition, not decoration

The word “independent” in that box is doing real work, and the fastest way to see it is to break it deliberately.

Here are three sums. In every one of them, both variables have **identical marginal distributions**—Normal, mean 0, SD 3. Nothing about either variable on its own differs between the three cases. Only the relationship between them changes.

R Code

```
1 set.seed(6101)
2 n <- 200000
3 x <- rnorm(n, mean = 0, sd = 3)
4 z <- rnorm(n, mean = 0, sd = 3) # independent of x, same distribution
5
6 round(c(marginal_var_x = var(x), marginal_var_z = var(z)), 3)
7
8 round(c(independent = var(x + z),          # x + z
9         identical   = var(x + x),          # x + x, i.e. Y is a copy of X
10        opposite    = var(x + (-x))), 3) # x + (-x), Y is X flipped
```

```
marginal_var_x marginal_var_z
          9.038          8.986
independent   identical   opposite
       18.017        36.151         0.000
```

Read the three numbers: **18.017, 36.151, 0.000**.

- **Independent:** $9 + 9 = 18$. The rule works.
- $Y = X$: the sum is $2X$, so the variance is $4 \times 9 = 36$, twice what the rule predicts. When the two always move together nothing ever cancels, and the swings compound.
- $Y = -X$: the sum is identically zero. Variance zero. The two cancel perfectly, every single time.

Same marginals in all three cases. Answers of 0, 18 and 36. **The variance of a sum is not determined by the variances of the parts**—it is determined by those *and* by how the parts move together, and only independence pins it down.

The general rule

The complete statement uses the covariance from [Relationships Between Variables](#):

$$\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y) + 2 \text{Cov}(X, Y)$$

Independence forces $\text{Cov}(X, Y) = 0$, and the last term disappears. The three cases above are the covariance term at zero, at its maximum, and at its minimum.

Here is an intermediate case, two variables correlated at 0.6:

R Code

```
1 set.seed(6101)
2 n <- 200000
3 x <- rnorm(n, 0, 3)
4 e <- rnorm(n, 0, 3)
5 y <- 0.6 * x + sqrt(1 - 0.6^2) * e      # SD 3, correlated 0.6 with x
6
7 round(c(var_x = var(x), var_y = var(y), correlation = cor(x, y)), 3)
8
9 round(c(actual_var_of_sum = var(x + y),
10       naive_rule      = var(x) + var(y),
11       with_covariance  = var(x) + var(y) + 2 * cov(x, y)), 3)
```

	var_x	var_y	correlation	
	9.038	9.001	0.601	
actual_var_of_sum		naive_rule	with_covariance	
	28.879	18.039	28.879	

The naive rule says 18.039. The truth is 28.879. Adding the covariance term recovers 28.879 exactly.

WHERE THIS BITES IN PRACTICE

The true variance was 60% larger than the naive rule said, and understating risk is the direction that ends careers.

- **A portfolio of ten tech stocks** is not ten independent bets. They all move with the market, the covariance terms are large and positive, and a risk model that adds variances alone will report a portfolio far safer than it is. This is not a hypothetical failure mode; it is roughly what happened in 2008.
- **Two sensors on the same rig** share vibration, temperature drift, and a power supply. Averaging them removes less noise than the arithmetic promises.
- **Repeated measurements on the same patient** are not independent observations, and treating n measurements on 10 people as n independent data points will produce a standard error that is far too small. [The Standard Error of a Mean](#) says exactly why.

The practical question is never “is this exactly independent?” It is: **where did the independence assumption enter, and what is the covariance likely to be?** If you cannot answer the first half of that, you have an assumption you did not know you had made.

Exercise: the SD of a sum

Two independent sources of measurement error.
Error A has standard deviation 6.
Error B has standard deviation 8.

Exercise	
1	<code># The total error is A + B, and the two are independent.</code>
2	<code># Return the STANDARD DEVIATION of the total.</code>
3	<code># No simulation, no data -- just the numbers 6 and 8.</code>
4	<code>-----</code>

Please replace `-----` with valid code.

Exercise: a difference, not a sum

Exercise	
1	<code># `p1` and `p2` are two INDEPENDENT variables, with SDs 3 and 4.</code>

```

2 #
3 # Predict var(p1 - p2) before you run anything. Many people expect the minus
4 # sign to make it smaller than var(p1 + p2). Then compute it here.
5

```

Please replace _____ with valid code.

Exercise: where did independence get smuggled in?

A logistics analyst writes:

```

"A delivery has three legs. Each leg's delay has a standard deviation of
10 minutes. So the total delay has a standard deviation of
sqrt(3 * 100) = 17.3 minutes."

```

The delays on the three legs are all driven by the same weather system, the same depot's staffing, and the same day's traffic.

Exercise

```

1 # What is wrong with the analyst's figure? Assign the letter,
2 # for example: answer <- "A"
3 #
4 # A. Nothing—sqrt(3 * 100) is the correct application of the addition rule.
5 # B. The arithmetic is wrong: it should be 10 + 10 + 10 = 30 minutes, since
6 #   the three delays accumulate along the route.
7 # C. The rule used requires the three delays to be independent, and they
8 #   plainly are not. The positive covariances between legs make the true
9 #   SD LARGER than 17.3, so the figure understates the risk.
10 # D. The rule used requires independence, and since the legs are dependent
11 #   the true SD must be SMALLER than 17.3.
12 answer <- _____

```

Please replace _____ with valid code.

Summary

- $\text{Var}(X) = E[(X - E[X])^2]$: the average squared distance from the expectation.
- Raw deviations sum to exactly zero, so they measure nothing. We square to stop the canceling—and specifically square rather than take absolute values because squares **add** and absolute values do not.
- The square root brings you back to the data's own units. Derive in variance, report in standard deviation.
- $\text{Var}(aX + b) = a^2 \text{Var}(X)$. The additive constant changes nothing at all; the multiplier enters twice. In SDs the rule is $|a| \text{SD}(X)$.
- **For independent X and Y** , $\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y)$. **Standard deviations do not add**— $\sqrt{3^2 + 4^2} = 5$, not 7.
- The same holds for a **difference**: the minus sign is squared away, so $\text{Var}(X - Y) = \text{Var}(X) + \text{Var}(Y)$ too.
- Independence is a **condition**. With identical marginals, the variance of a sum came out 0, 18 or 36 depending only on the dependence structure. The general rule carries a $2 \text{Cov}(X, Y)$ term, and ignoring it usually understates the risk.

Next: put the two rules together—linearity from [Expectation](#) and additivity from this chapter—point them at the sample mean, and get the result the rest of this course is built on.

The Standard Error of a Mean told you where the sampling distribution of $\bar{X}$ sits and how wide it is: centered at μ , with spread $\sigma/\sqrt{n}$. Two numbers, derived in four lines, with no assumption about the population beyond having a finite variance.

What it did not tell you is the **shape**. And without the shape you cannot turn those two numbers into a statement, because “the estimate is 31.4 with a standard error of 0.28” does not yet say how often the truth is within half a point.

The central limit theorem supplies the shape, and it supplies it in a form so convenient that it is worth being suspicious of.

What it says

*For independent observations from a population with finite mean μ and finite variance σ^2 , the sampling distribution of $\bar{X}$ approaches a **Normal** distribution with mean μ and standard deviation $\sigma/\sqrt{n}$, as n grows.*

Read what is not in that sentence. There is **no condition on the shape of the population**. It may be skewed, bimodal, discrete, bounded, or something with no name. Provided the variance is finite and the draws are independent, the means go Normal.

That is a genuinely surprising claim, so watch it happen rather than believing it.

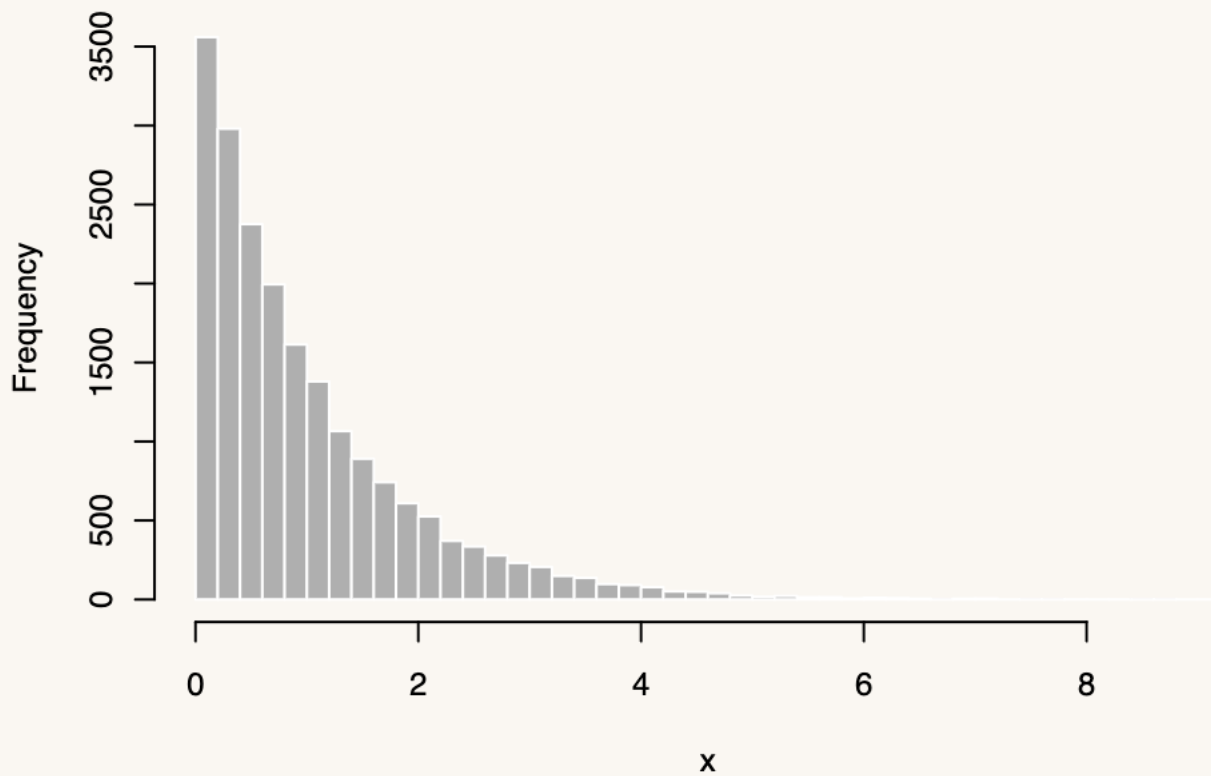
Watching it happen

The exponential distribution is about as un-Normal as a common distribution gets: it starts at zero, falls away immediately, and has a long right tail. Its mean and standard deviation are both 1 when the rate is 1.

R Code

```
1 set.seed(6101)
2 x <- rexp(20000, rate = 1)
3 hist(x, breaks = 50, col = "gray70", border = "white",
4       main = "The population: exponential(1)", xlab = "x")
```

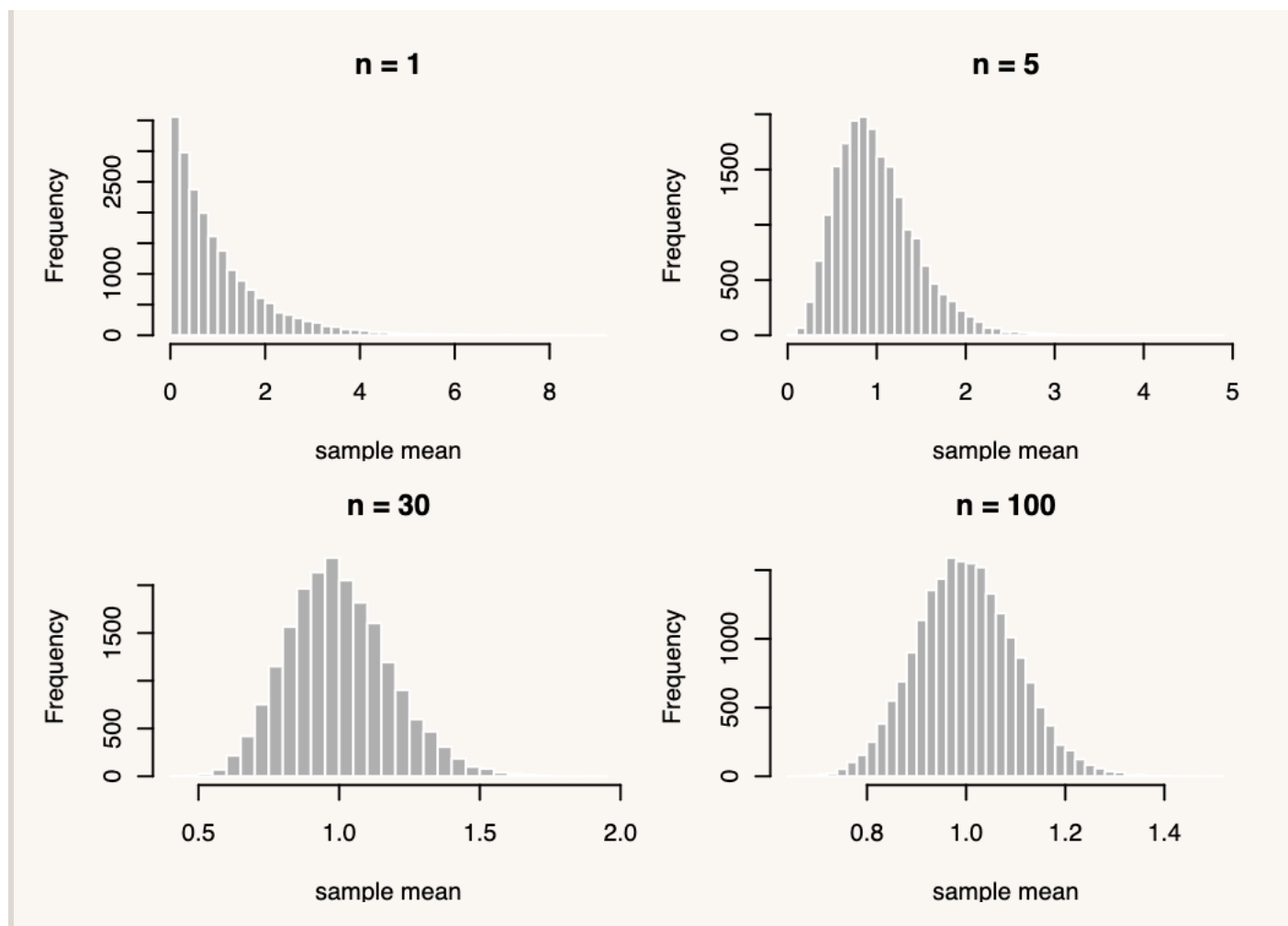
The population: exponential(1)



Now take samples of size n , average each one, and look at the distribution of those averages.

R Code

```
1 set.seed(6101)
2
3 par(mfrow = c(2, 2), mar = c(4, 4, 3, 1))
4 for (n in c(1, 5, 30, 100)) {
5   means <- replicate(20000, mean(rexp(n, rate = 1)))
6   hist(means, breaks = 50, col = "gray70", border = "white",
7        main = paste("n =", n), xlab = "sample mean")
8 }
```



At $n = 1$ you are looking at the population itself. By $n = 5$ the worst of the skew is gone. By $n = 30$ it is nearly symmetric, and at $n = 100$ you would not guess the parent distribution from the picture.

Put numbers on it. **Skewness** measures asymmetry: zero for a symmetric distribution, positive for a long right tail.

R Code					
1	<code>set.seed(6101)</code>				
2	<code>skew <- function(v) mean((v - mean(v))^3) / sd(v)^3</code>				
3					
4	<code>out <- t(sapply(c(1, 5, 30, 100), function(n) {</code>				
5	<code> m <- replicate(20000, mean(rexp(n, rate = 1)))</code>				
6	<code> c(n = n, mean = mean(m), sd = sd(m), predicted_sd = 1 / sqrt(n), skew = skew(m))</code>				
7	<code>}))</code>				
8	<code>round(out, 4)</code>				
	<code>n</code>	<code>mean</code>	<code>sd</code>	<code>predicted_sd</code>	<code>skew</code>
[1,]	1	1.0105	1.0017	1.0000	1.9068
[2,]	5	0.9998	0.4458	0.4472	0.9185
[3,]	30	0.9993	0.1832	0.1826	0.3952
[4,]	100	1.0003	0.1000	0.1000	0.1927

Two things at once. The **center and spread** are exactly what the previous chapter predicted—0.1 at $n = 100$, against $1/\sqrt{100}$. And the **skew** falls from 1.91 to 0.19, roughly like $1/\sqrt{n}$, which is the central limit theorem doing its work.

How large is large enough?

Textbooks say $n = 30$. Treat that as folklore rather than a theorem.

The honest answer is that the required n depends on how skewed the population is. A symmetric population is close to Normal at $n = 5$; the exponential above is respectable by 30; a distribution with a rare enormous value—insurance claims, revenue per customer, time on site—may still be visibly skewed at $n = 1,000$.

R Code		
1	set.seed(7)	
2	skew <- function(v) mean((v - mean(v))^3) / sd(v)^3	
3		
4	# A log-normal, of the kind you meet in revenue and duration data.	
5	pop_skew <- skew(rlnorm(200000, meanlog = 0, sdlog = 1.2))	
6		
7	res <- t(sapply(c(30, 100, 1000), function(n) {	
8	c(n = n, skew_of_means = skew(replicate(8000, mean(rlnorm(n, 0, 1.2)))))	
9	}))	
10	round(rbind(c(n = 1, skew_of_means = pop_skew), res), 3)	
	n	skew_of_means
[1,]	1	8.921
[2,]	30	1.627
[3,]	100	1.117
[4,]	1000	0.320

The population's skew is about 8.9. At $n = 30$ the means are still about as skewed as the exponential population itself was to begin with—1.6 against 1.9—and it takes $n = 1,000$ to reach the symmetry the exponential had at $n = 30$. **The rule is not a sample size; it is to check.** Simulating from a plausible population takes one line, and it is the only defensible way to answer the question for data you actually have.

What it does not say

Three misreadings, in order of how often they appear.

It does not say your data become Normal. Nothing happens to the data. The 20,000 exponential values in the first histogram are just as skewed as they ever were. It is the distribution of the *mean* that goes Normal, and the mean is a different object.

It does not say every statistic goes Normal. The theorem is about sums, and the mean is a sum divided by n . A median, a maximum, a correlation, a ratio—none of them are covered by it. This is the gap [Why Resample](#) exists to fill.

It does not license a Normal model for individual observations. If you need to say where the next single value will fall, the CLT is silent. It describes the estimate, not the world.

Where it fails

The theorem has one condition worth respecting: the population must have a **finite variance**. Distributions without one exist, and they are not merely pathological—financial returns, network traffic and file sizes are all modeled with heavy tails.

The Cauchy distribution is the standard example. It looks like a bell curve with fat tails, and it has no mean and no variance at all.

R Code		
1	set.seed(1)	
2	# Average 10 Cauchy draws, and 1000 of them. Compare the spread of the averages.	
3	m10 <- replicate(5000, mean(rcauchy(10)))	
4	m1000 <- replicate(5000, mean(rcauchy(1000)))	
5		
6	round(c(IQR_of_means_n10 = IQR(m10), IQR_of_means_n1000 = IQR(m1000)), 3)	

IQR_of_means_n10	IQR_of_means_n1000
2.006	1.951

A hundredfold increase in sample size and the spread does not move. For a Cauchy population, the mean of 1,000 observations is no more precise than the mean of 10—averaging accomplishes nothing, because the occasional enormous draw dominates whatever the other 999 do.

You will not meet a true Cauchy. You will meet data with tails heavy enough that convergence is slow and the sample mean is a poor summary, and the practical signal is the same: **if the mean of your data lurches around when you add observations, averaging is not working and no interval built on it will be honest.**

🔗 EXERCISE: WHAT THE THEOREM CLAIMS

A dataset of 5,000 customer transaction amounts is strongly right-skewed. Assign the letter of the correct statement, for example `answer <- "A"`.

Exercise

```
1 # A. With n = 5,000 the central limit theorem says the transaction amounts
2 #   are approximately Normal, so Normal-based methods can be used on them.
3 # B. The CLT says the sampling distribution of the MEAN transaction amount is
4 #   approximately Normal; the amounts themselves stay just as skewed.
5 # C. The CLT does not apply, because the population is not Normal.
6 # D. The CLT guarantees the median is also approximately Normal, since it is
7 #   another measure of center.
8 answer <- _____
```

Please replace _____ with valid code.

🔗 EXERCISE: HOW SKEWED ARE THE MEANS?

The population is `rexp(n, rate = 1)`. Return the **skewness of 5,000 sample means** at $n = 25$, using the function `skew()` provided.

Exercise

```
1 # 5,000 replications, each the mean of 25 exponential(1) draws.
2 _____
```

Please replace _____ with valid code.

Summary

- The CLT supplies the **shape** that The Standard Error of a Mean was missing: $\bar{X}$ is approximately Normal, centered at μ , with spread $\sigma/\sqrt{n}$.
- It requires **independence and finite variance**, and nothing at all about the shape of the population.
- Convergence was visible by $n = 30$ for an exponential population and incomplete at $n = 1,000$ for a heavily skewed log-normal. **The number 30 is folklore; simulate instead.**
- Skew of the sampling distribution falls like $1/\sqrt{n}$.
- It is about the **mean**, not the data and not every statistic. Medians, maxima and correlations are outside it.
- With infinite variance it fails outright: averaging 1,000 Cauchy draws is no better than averaging 10.

Next: what these two numbers and a shape let you actually say—the confidence interval, and the careful sentence that goes with it.

Monday, September 28

Post your questions on any of these pages by **Monday at 12:00 noon**, the morning of class.

Everything so far has described a sample. This is the week the description turns into a claim about the world.

You already have the two numbers that make it possible. [The Standard Error of a Mean](#) established that the sampling distribution of $\bar{X}$ is centered at μ with spread $\sigma/\sqrt{n}$. What was missing was its **shape**, because without a shape you cannot convert “give or take 0.28” into “and here is how often that works”.

The central limit theorem supplies the shape, and the confidence interval is what you get by rearranging one line of algebra. The difficulty of this week is not the arithmetic—the interval is a mean, a standard error, and a $\pm$. The difficulty is saying precisely what the resulting pair of numbers claims, and resisting the three or four more satisfying things it does not.

What is here

THE CENTRAL LIMIT THEOREM

Why the mean of almost anything goes Normal, watched happening rather than asserted. How large “large enough” really is—not 30, but whatever your simulation says. And the case where it fails outright: with infinite variance, averaging a thousand observations is no better than averaging ten.

CONFIDENCE INTERVALS

The construction, and then the careful part. A hundred studies produce a hundred different intervals, a few of which miss the truth and none of which look wrong from the inside. What 95% attaches to, what it does not, and the four misreadings worth being able to name.

WHEN σ IS UNKNOWN

We have been substituting s for σ and pretending nothing happened. At $n = 5$ that pretense turns a 95% interval into an 88% one. Student's t , why its degrees of freedom are $n - 1$, and the one thing it does not fix.

INTERVALS FOR A PROPORTION

Conversion rates, click rates, defect rates. The obvious interval covers 80% of the time at a 3% rate, which is exactly where commercial data lives—and the fix is one function call.

PRACTICE PROBLEMS

Predict-before-you-run, code that runs and is still wrong, and six statements to judge against a single interval. Not graded—bring your disagreements.

A habit for this week

Check the coverage. Every claim in these chapters is of the form “this procedure works $x\%$ of the time”, and every one of them can be verified in about four lines: simulate a population where you know the answer, run the procedure many times, count how often it was right.

That is not a pedagogical device. It is what you should do whenever you are unsure whether a method applies to your data—far more useful than remembering whether the rule was 30 or 10, and it is how the two failures in this week's material were found.

DUE THIS WEEK

HW 3 is due Monday at 11:59 p.m., the night of class. And as always, post your questions on these pages by Monday noon—that afternoon's session is built out of them.

You have everything you need. The sampling distribution of $\bar{X}$ is centered at μ , has standard deviation $\sigma/\sqrt{n}$, and is approximately Normal. A Normal distribution puts 95% of its mass within 1.96 standard deviations of its center. Therefore:

$$P\left(|\bar{X} - \mu| < 1.96 \cdot \frac{\sigma}{\sqrt{n}}\right) \approx 0.95$$

Rearranged so that μ sits in the middle, that is the **confidence interval**:

$$\bar{x} \pm 1.96 \cdot \frac{\sigma}{\sqrt{n}}$$

That is the entire derivation. There is nothing else in it, and the rest of this chapter is about what the resulting number does and does not entitle you to say.

Building one

R Code

```
1 set.seed(6101)
2 # A sample of 200 commute times, from a population we happen to know.
3 commute <- rnorm(200, mean = 31, sd = 14)
4
5 xbar <- mean(commute)
6 se <- sd(commute) / sqrt(length(commute))
7
8 round(c(estimate = xbar, se = se,
9         lower = xbar - 1.96 * se, upper = xbar + 1.96 * se), 3)
```

estimate	se	lower	upper
31.301	1.009	29.323	33.279

Two numbers you already knew how to compute, and a $\pm$. The difficulty was never the arithmetic.

What “95% confident” means

Here is the sentence that costs students marks, and practitioners credibility:

The interval is random. The parameter is not.

μ is a fixed number. It does not have a distribution, it is not more likely to be here than there, and it is either inside your particular interval or it is not. What varies from study to study is the **interval**—a different sample gives a different center and a different width.

So “95% confidence” is a statement about the **procedure**, not about the interval in front of you:

If you repeated the whole study many times and built an interval each time, about 95% of those intervals would contain μ .

That is testable, which is the good news. Build a hundred and count.

The coverage experiment

R Code

```
1 set.seed(6101)
2 mu <- 31 # the truth, which in real work you never have
3 reps <- 100
4
```

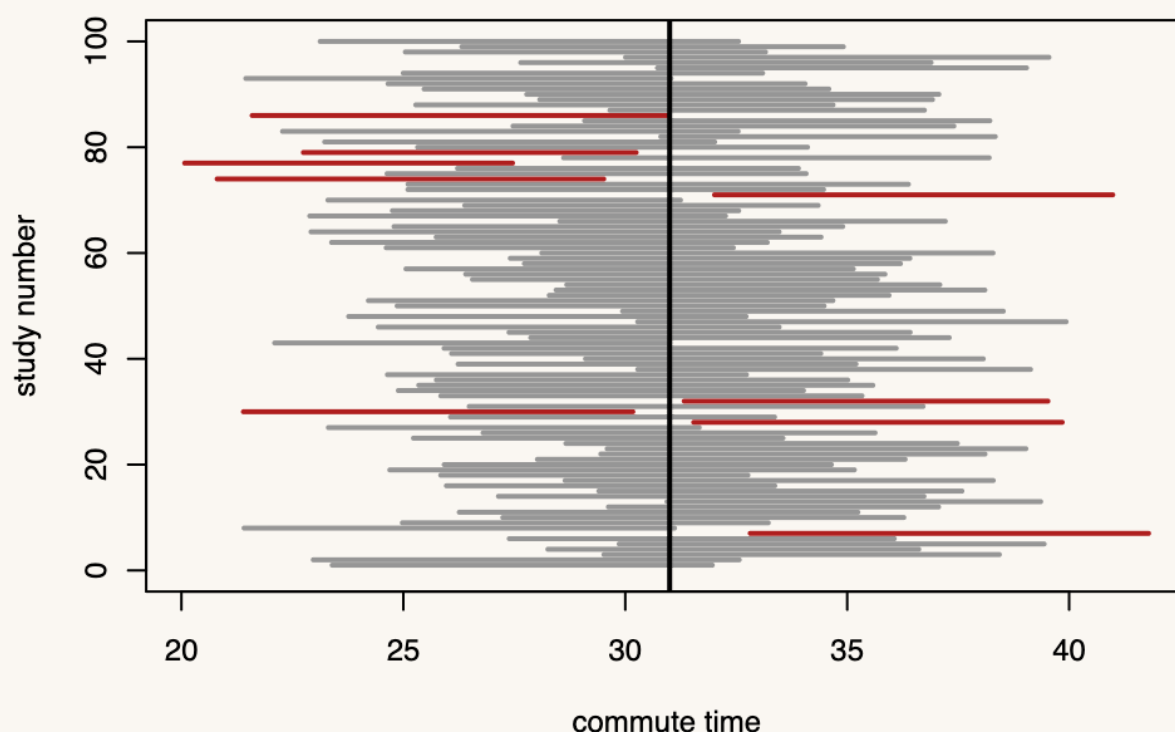
```

5 ints <- t(replicate(reps, {
6   x   <- rnorm(40, mean = mu, sd = 14)
7   se  <- sd(x) / sqrt(40)
8   mean(x) + c(-1, 1) * qt(0.975, df = 39) * se
9 })
10
11 covers <- ints[, 1] <= mu & mu <= ints[, 2]
12 cat("intervals containing the truth:", sum(covers), "out of", reps, "\n")
13
14 plot(NA, xlim = range(ints), ylim = c(0, reps), xlab = "commute time",
15      ylab = "study number", main = "100 studies, 100 intervals")
16 segments(ints[, 1], 1:reps, ints[, 2], 1:reps,
17          col = ifelse(covers, "gray60", "firebrick"), lwd = 2)
18 abline(v = mu, lwd = 2)

```

intervals containing the truth: 91 out of 100

100 studies, 100 intervals



Every interval is a different length and sits in a different place. The vertical line—the truth—does not move. A handful of intervals miss it entirely, and **nothing about those intervals looks wrong from the inside**. They were built by the same correct procedure as the others; they simply came from unlucky samples.

That is what a confidence level is: the long-run fraction of the gray ones.

R Code

```

1 set.seed(6101)
2 # The same experiment, 20,000 times, to see the rate settle.
3 mu <- 31
4 covered <- replicate(20000, {
5   x <- rnorm(40, mean = mu, sd = 14)
6   ci <- mean(x) + c(-1, 1) * qt(0.975, 39) * sd(x) / sqrt(40)
7   ci[1] <= mu && mu <= ci[2]
8 })
9 round(mean(covered), 4)

```

```
[[1] 0.9503
```

Four things it does not mean

“There is a 95% probability that μ is between 29.3 and 33.3.” No. Both numbers are fixed and μ is fixed; the probability is 0 or 1, you just do not know which. The 95% attached to the procedure before you saw the data, and it does not transfer to the particular answer afterwards.

“95% of the data lie in this interval.” No, and the size gives it away. With $n = 200$ the interval above is about four minutes wide while commutes scatter over fifty. That is the SD-versus-SE confusion in interval form, and it is the most common one in student reports.

“95% of future sample means will fall in it.” Also no—that is roughly 83%, not 95%, because a future mean has its own sampling error to add. If you want an interval for a future observation you need a [prediction interval](#).

“The result is significant because the interval is narrow.” Narrow means precise, not correct. As [Where the Data Came From](#) argued, a large biased sample produces a narrow interval around the wrong number—and the narrowness makes it more persuasive, not less wrong.

THE SENTENCE TO WRITE

“We estimate the mean commute at **31.3 minutes (95% CI: 29.3 to 33.3)**.”

Then, if the reader needs it: “*an interval constructed this way contains the true mean about 95% of the time.*” That is defensible, and it is short.

What controls the width

The half-width is $1.96 \cdot s / \sqrt{n}$, so exactly three things move it.

R Code

```
1 set.seed(6101)
2 half_width <- function(n, conf = 0.95, sd_pop = 14) {
3   qt(1 - (1 - conf) / 2, df = n - 1) * sd_pop / sqrt(n)
4 }
5
6 round(rbind(
7   `n = 50` = c(`90%` = half_width(50, 0.90), `95%` = half_width(50), `99%` = half_width(50, 0.99)),
8   `n = 200` = c(half_width(200, 0.90), half_width(200), half_width(200, 0.99)),
9   `n = 800` = c(half_width(800, 0.90), half_width(800), half_width(800, 0.99))
10 ), 3)
```

	90%	95%	99%
n = 50	3.319	3.979	5.306
n = 200	1.636	1.952	2.575
n = 800	0.815	0.972	1.278

Sample size, through $\sqrt{n}$ —so quadrupling the data halves the width, the same k^2 rule as before. **Confidence level**, which buys certainty with width: going from 95% to 99% costs about a third more interval. And the **variability of the data**, which is a property of the world rather than a choice.

There is no fourth lever. In particular there is no way to have a narrow interval and a high confidence level on a small noisy sample, and an analysis that appears to manage it has made an error somewhere.

EXERCISE: MEASURE THE COVERAGE YOURSELF

Run 10,000 studies. Each draws $n = 40$ from a **skewed** population—`rexp(40, rate = 1/31)`, whose true mean is 31—and builds the usual t -interval. Return the **proportion of intervals containing 31**.

Exercise

```
1 # Each study: x <- rexp(40, rate = 1/31)
2 # Interval:   mean(x) + c(-1, 1) * qt(0.975, 39) * sd(x) / sqrt(40)
3 # Return the proportion of the 10,000 intervals that contain 31.
4 -----
```

Please replace ----- with valid code.

EXERCISE: WHICH STATEMENT IS LEGITIMATE

A study reports a mean satisfaction score of 7.2 with a 95% confidence interval of 6.9 to 7.5. Assign the letter, for example `answer <- "A"`.

Exercise

```
1 # A. There is a 95% probability that the true mean satisfaction is between
2 #   6.9 and 7.5.
3 # B. 95% of respondents rated satisfaction between 6.9 and 7.5.
4 # C. If this study were repeated many times, about 95% of the intervals
5 #   constructed this way would contain the true mean satisfaction.
6 # D. We can be 95% sure the next respondent will rate between 6.9 and 7.5.
7 answer <- -----
```

Please replace ----- with valid code.

Summary

- $\bar{x} \pm 1.96 \cdot s / \sqrt{n}$, and the derivation is one line from the central limit theorem plus the standard error.
- **The interval is random; the parameter is not.** Confidence is a property of the procedure across repetitions, not of the interval you are holding.
- Simulation makes this concrete: 100 studies gave 100 different intervals, a few missed, and the misses looked exactly like the hits from the inside.
- Coverage is **checkable**. For a Normal population the recipe delivered 95%; for a skewed population at $n = 40$ it delivered 93.2%.
- Width is controlled by n , the confidence level, and the variability of the data. There is no fourth lever.
- A narrow interval is precise, not correct. Bias is invisible to it.

Next: what to do about the fact that we substituted s for σ and carried on as though nothing had happened.

None of these are graded and none are auto-checked. They exist so that you arrive on Monday with something to argue about. Several have no single right answer; those are the useful ones.

Predict the output

Say what each prints **before** you run it. Getting it wrong and understanding why is the point.

R Code	
1	# 1.
2	<code>qt(0.975, df = 1e6) - qnorm(0.975)</code>
3	
4	# 2.
5	<code>set.seed(1)</code>
6	<code>x <- rnorm(9, mean = 50, sd = 12)</code>
7	<code>c(diff(t.test(x)\$conf.int),</code>
8	<code>diff(t.test(x, conf.level = 0.99)\$conf.int))</code>
9	
10	# 3.
11	<code>n <- 100</code>
12	<code>c(sqrt(0.5 * 0.5 / n), sqrt(0.05 * 0.95 / n))</code>

```
[1] 2.372274e-06
[1] 14.97457 21.78898
[1] 0.05000000 0.02179449
```

DISCUSSION



This runs and is still wrong

Each of these produces a number. Each number is wrong, or right for a question nobody asked. Find the error.

R Code	
1	# 4. Reporting the spread of the data.
2	<code>set.seed(3)</code>
3	<code>scores <- rnorm(400, mean = 72, sd = 11)</code>
4	<code>se <- sd(scores) / sqrt(length(scores))</code>
5	<code>cat("Most students scored between",</code>
6	<code>round(mean(scores) - 1.96 * se, 1), "and",</code>
7	<code>round(mean(scores) + 1.96 * se, 1), "\n")</code>

```
Most students scored between 71.2 and 73.4
```

R Code	
1	# 5. A conversion rate from a small pilot.
2	<code>x <- 1; n <- 40</code>
3	<code>ph <- x / n</code>
4	<code>round(ph + c(-1, 1) * 1.96 * sqrt(ph * (1 - ph) / n), 4)</code>

```
[1] -0.0234 0.0734
```

R Code

```
1 # 6. "The sample is big, so the estimate is good."
2 set.seed(5)
3 # 5,000 responses, but only from people who opted in to the mailing list,
4 # who are more satisfied than the customer base as a whole.
5 opted_in <- rnorm(5000, mean = 8.1, sd = 1.2) # true overall mean is 6.4
6 ci <- t.test(opted_in)$conf.int
7 round(c(ci, width = diff(ci)), 3)
```

```
      width
8.094 8.161 0.067
```

DISCUSSION



What does this license you to claim?

For each statement, decide whether it is defensible from a 95% confidence interval of [2.1, 5.7] for a mean difference. Write a sentence for each.

7. "The difference is between 2.1 and 5.7."
8. "There is a 95% chance the difference is between 2.1 and 5.7."
9. "The difference is not zero."
10. "The difference is probably around 3.9."
11. "A difference of 5.6 is consistent with our data."
12. "If we ran the study again we would get an interval between 2.1 and 5.7."

DISCUSSION



Bring to class

13. **The coverage you can measure.** Pick a population shape you think your project data resembles—skewed, bimodal, heavy-tailed, bounded. Simulate it, build 10,000 t -intervals at the sample size you expect to have, and report the actual coverage. Bring the number. If it is not close to 0.95, bring an opinion about what to do.
14. **An interval in the wild.** Find a reported confidence interval in a news article, a company blog post, or a paper. Bring it, and answer: is it an interval for a mean, a proportion, or a difference? Is the sample size given? Does the surrounding text interpret it correctly?
15. **Argue about this.** A colleague says: "We have 40,000 users in the experiment. The sample is so large that confidence intervals are pointless—every difference will be significant." They are partly right and mostly wrong. Work out which part is which, and be ready to defend it.

A conversion rate, a click-through rate, a defect rate, a poll result. These are the quantities you will be asked about most often, and [The Standard Error of a Mean](#) already gave you their standard error, because a proportion is the mean of a column of zeros and ones:

$$SE(\hat{p}) = \sqrt{\frac{\hat{p}(1 - \hat{p})}{n}}$$

So the obvious interval is the obvious one:

$$\hat{p} \pm 1.96 \sqrt{\frac{\hat{p}(1 - \hat{p})}{n}}$$

This is the **Wald interval**. It is what every introductory course teaches, what most people reach for, and it is unreliable in precisely the situations that matter commercially.

Checking it the way we checked the others

The method from [Confidence Intervals](#) applies unchanged: fix a true p , simulate many studies, count how often the interval covers.

R Code

```
1 set.seed(2)
2 wald_coverage <- function(n, p, reps = 20000) {
3   mean(replicate(reps, {
4     x <- rbinom(1, n, p)
5     ph <- x / n
6     se <- sqrt(ph * (1 - ph) / n)
7     abs(ph - p) < 1.96 * se
8   }))
9 }
10
11 round(c(p_0.50 = wald_coverage(100, 0.50),
12        p_0.10 = wald_coverage(100, 0.10),
13        p_0.03 = wald_coverage(100, 0.03)), 4)
```

```
p_0.50 p_0.10 p_0.03
0.9428 0.9329 0.8088
```

At $p = 0.5$ it is close enough. At $p = 0.03$ —a perfectly ordinary conversion rate—an interval sold as 95% covers **about 80% of the time**. One in five such intervals is wrong, and there is no warning anywhere in the output.

Why it breaks

Two failures compound, and both get worse as p approaches 0 or 1.

The standard error is estimated from $\hat{p}$. When the true rate is 3% and $n = 100$, a study that happens to observe zero conversions gets $\hat{p} = 0$, hence $SE = 0$, hence an interval of zero width that is certainly wrong. Less extreme versions of the same thing happen constantly: an unluckily low $\hat{p}$ produces an unluckily small SE, so the interval is too narrow exactly when it is most badly centered.

The Normal approximation is poor near the boundary. The sampling distribution of $\hat{p}$ is discrete and skewed when np is small, and a symmetric interval cannot describe it. The interval also runs off the end—

R Code

```

1 n <- 100; x <- 2
2 ph <- x / n
3 round(ph + c(-1, 1) * 1.96 * sqrt(ph * (1 - ph) / n), 4)

[1] -0.0074 0.0474

```

—a lower limit below zero, for a quantity that cannot be negative.

The fix, which is one function call

Rather than estimating the standard error at $\hat{p}$ and inverting, the **Wilson interval** asks which values of p are consistent with the data and solves for them directly. It is not more complicated to use:

R Code

```

1 # 2 conversions out of 100.
2 prop.test(2, 100, correct = FALSE)$conf.int

[1] 0.005501968 0.070011791
attr(,"conf.level")
[1] 0.95

```

Nonnegative, asymmetric, and it does not collapse. Its coverage:

R Code

```

1 set.seed(2)
2 wilson_coverage <- function(n, p, reps = 20000) {
3   mean(replicate(reps, {
4     x <- rbinom(1, n, p)
5     ci <- prop.test(x, n, correct = FALSE)$conf.int
6     p >= ci[1] && p <= ci[2]
7   }))
8 }
9
10 round(c(p_0.50 = wilson_coverage(100, 0.50),
11        p_0.10 = wilson_coverage(100, 0.10),
12        p_0.03 = wilson_coverage(100, 0.03)), 4)

p_0.50 p_0.10 p_0.03
0.9428 0.9369 0.9688

```

Around 95% across the board, including the case where the Wald interval was catastrophic.

WHAT TO DO

Use `prop.test()` or `binom.test()` for a proportion. They are one call, they are in base R, and they do not fail quietly.

If you must compute by hand, the **Agresti–Coull** rule is a good approximation and easy to remember: **add two successes and two failures**, then apply the Wald formula to the adjusted counts. Two conversions in 100 becomes four in 104.

The rule of thumb, and why it is not enough

The traditional check is $n\hat{p} \geq 10$ and $n(1 - \hat{p}) \geq 10$. Here $n\hat{p} = 3$, so the rule does flag this case—which is a point in its favor.

The difficulty is that the rule uses $\hat{p}$, the very quantity whose unreliability is the problem. A study that observes 3 conversions out of 100 when the truth is 12% passes nothing and looks fine. And the rule offers no guidance at all on what to do instead.

Using `prop.test()` **unconditionally is simpler than remembering the rule**, and it is right in the cases where the rule would have said the Wald interval was fine.

Why this matters more than it looks

Rare events are the commercially interesting ones. Conversion, churn, fraud, click-through, adverse reactions—all sit at a few percent, which is exactly where the Wald interval is worst.

Combine that with the relative-precision arithmetic from the standard error chapter:

R Code				
1	<code>n <- c(1000, 10000, 100000)</code>			
2	<code>p <- 0.03</code>			
3	<code>se <- sqrt(p * (1 - p) / n)</code>			
4	<code>round(data.frame(n, se, half_width = 1.96 * se, relative = 1.96 * se / p), 4)</code>			
	<code>n</code>	<code>se</code>	<code>half_width</code>	<code>relative</code>
1	1e+03	0.0054	0.0106	0.3524
2	1e+04	0.0017	0.0033	0.1115
3	1e+05	0.0005	0.0011	0.0352

To pin a 3% rate down to within a tenth of itself you need tens of thousands of observations. That is the honest answer to “can we call the experiment yet”, and it is why the interval being *correct* matters: at these sample sizes the decision often hinges on whether the interval includes the current rate.

EXERCISE: THE TWO INTERVALS, SIDE BY SIDE

A trial converts **7 of 250** visitors. Return the **width** of the 95% Wilson interval from `prop.test(7, 250, correct = FALSE)` —a single number.

Exercise	
1	<code># The conf.int component holds the two endpoints.</code>
2	<code>_____</code>

Please replace `_____` with valid code.

Summary

- A proportion is a mean of zeros and ones, so $SE(\hat{p}) = \sqrt{\hat{p}(1 - \hat{p})/n}$ and the **Wald interval** is the obvious construction.
- It is **unreliable when p is near 0 or 1**: at $p = 0.03$ with $n = 100$ a nominal 95% interval covered 80%.
- It fails two ways at once—the standard error is estimated at $\hat{p}$, so an unlucky sample gets a too-small SE, and a symmetric interval cannot describe a skewed sampling distribution. It also produces impossible negative limits.
- `prop.test(x, n, correct = FALSE)` gives the Wilson interval, which held near 95% everywhere tested. Use it unconditionally.
- Agresti–Coull—**add two successes and two failures**—is the hand version.
- This is not an edge case. Conversion, churn and fraud all live at a few percent, which is where the standard recipe is worst.

Next: practice problems, and then a term of putting these intervals to work.

The derivation used σ , the population standard deviation. You have never had one and you never will. Every interval you have built substituted s , the sample standard deviation, and carried on as if nothing had happened.

Something did happen. s is itself an estimate, computed from the same handful of observations, and it wobbles. Replacing a known constant with a noisy estimate and keeping the same 1.96 quietly claims a precision you have not earned.

How much it costs

Measure it. Build intervals with the Normal critical value 1.96, using s in place of σ , and see how often they cover.

R Code

```
1 set.seed(6101)
2 coverage <- function(n, crit) {
3   mean(replicate(20000, {
4     x <- rnorm(n, mean = 100, sd = 15)
5     ci <- mean(x) + c(-1, 1) * crit * sd(x) / sqrt(n)
6     ci[1] <= 100 && 100 <= ci[2]
7   })))
8 }
9
10 round(c(n_5 = coverage(5, 1.96),
11         n_10 = coverage(10, 1.96),
12         n_30 = coverage(30, 1.96),
13         n_200 = coverage(200, 1.96)), 4)
```

n_5	n_10	n_30	n_200
0.8814	0.9182	0.9408	0.9483

At $n = 200$ the damage is negligible. At $n = 5$ an interval advertised as 95% covers **88% of the time**—it misses more than twice as often as it claims. The population here is exactly Normal, so nothing is wrong with the central limit theorem. The error is entirely from pretending s is σ .

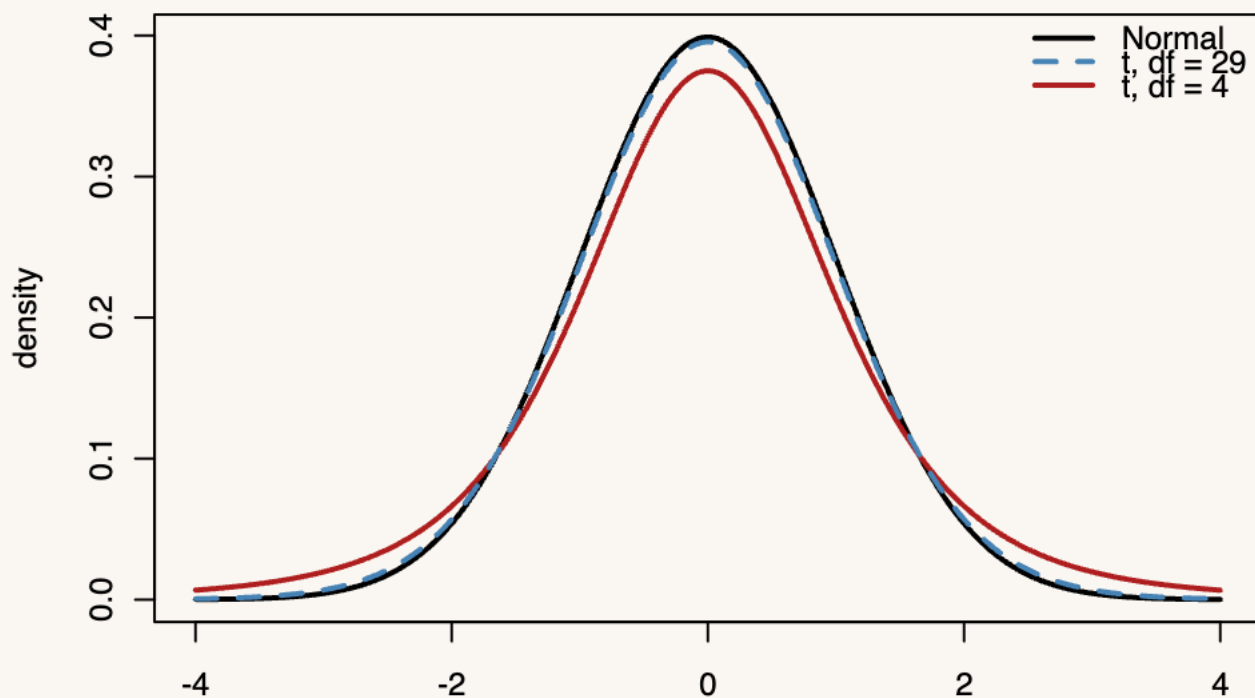
The fix

$$t = \frac{\bar{X} - \mu}{s/\sqrt{n}}$$

does not follow a Normal distribution. It follows **Student's t** with $n - 1$ degrees of freedom—the same $n - 1$ that appeared in the variance divisor, for the same reason: one degree of freedom was spent estimating $\bar{x}$ before s could be computed.

The t looks like a Normal with heavier tails. Heavier tails mean a larger critical value, which means a wider interval—which is precisely the extra allowance you owe for not knowing σ .

R Code	
1	<code>x <- seq(-4, 4, length.out = 400)</code>
2	<code>plot(x, dnorm(x), type = "l", lwd = 2, ylab = "density", xlab = "")</code>
3	<code>lines(x, dt(x, df = 4), lwd = 2, col = "firebrick")</code>
4	<code>lines(x, dt(x, df = 29), lwd = 2, col = "steelblue", lty = 2)</code>
5	<code>legend("topright", bty = "n", lwd = 2,</code>
6	<code> col = c("black", "steelblue", "firebrick"), lty = c(1, 2, 1),</code>
7	<code> legend = c("Normal", "t, df = 29", "t, df = 4"))</code>



R Code

```
1 round(c(df_4 = qt(0.975, 4), df_9 = qt(0.975, 9), df_29 = qt(0.975, 29),
2       df_199 = qt(0.975, 199), normal = qnorm(0.975)), 3)
```

df_4	df_9	df_29	df_199	normal
2.776	2.262	2.045	1.972	1.960

With four degrees of freedom you need **2.776** standard errors rather than 1.96—an interval 42% wider. By $df = 29$ it is 2.045, and by 199 it is 1.972, which is 1.96 for any practical purpose.

Does it work?

R Code

```
1 set.seed(6101)
2 coverage_t <- function(n) {
3   mean(replicate(20000, {
4     x <- rnorm(n, mean = 100, sd = 15)
5     ci <- mean(x) + c(-1, 1) * qt(0.975, df = n - 1) * sd(x) / sqrt(n)
6     ci[1] <= 100 && 100 <= ci[2]
7   })))
8 }
9 round(c(n_5 = coverage_t(5), n_10 = coverage_t(10), n_30 = coverage_t(30)), 4)
```

n_5	n_10	n_30
0.9500	0.9488	0.9500

95% at every sample size, including $n = 5$. The t distribution is not an approximation that improves with n —for a Normal population it is **exact**, and the convergence to 1.96 is the correction becoming unnecessary rather than becoming accurate.

IN PRACTICE

Use `t.test()`, which does all of this and prints the interval:

```
## {r}  
t.test(x)$conf.int
```

Use t always. At large n it agrees with the Normal to three decimals, so there is no situation in which choosing the Normal is worth the risk of being wrong at small n .

The assumption you have now taken on

The t interval is exact **for a Normal population**. That is a stronger requirement than the central limit theorem needed, and it bites in exactly the case where you reached for t in the first place.

R Code

```
1 set.seed(6101)  
2 # Same n = 8, two populations: Normal, and skewed with the same mean.  
3 norm_cov <- mean(replicate(20000, {  
4   x <- rnorm(8, mean = 10, sd = 10)  
5   ci <- mean(x) + c(-1, 1) * qt(0.975, 7) * sd(x) / sqrt(8)  
6   ci[1] <= 10 && 10 <= ci[2]  
7 }))  
8 skew_cov <- mean(replicate(20000, {  
9   x <- rexp(8, rate = 1/10)  
10  ci <- mean(x) + c(-1, 1) * qt(0.975, 7) * sd(x) / sqrt(8)  
11  ci[1] <= 10 && 10 <= ci[2]  
12 }))  
13 round(c(normal_population = norm_cov, skewed_population = skew_cov), 4)
```

```
normal_population skewed_population  
0.9484            0.8954
```

The t correction fixes the *unknown- σ* problem and does nothing about skew. On a skewed population at $n = 8$ the interval still under-covers badly, and there is no critical value that rescues it—the sampling distribution is not symmetric, so no symmetric interval can be right.

Small n and skewed data is the case where the formulas have nothing to offer. It is one of the situations [Why Resample](#) was written for.

EXERCISE: THE CRITICAL VALUE

A study has $n = 12$. Return the critical value for a **90%** confidence interval for the mean—a single number.

Exercise

```
1 # Careful with two things: the degrees of freedom, and which tail probability  
2 # qt() wants for a 90% two-sided interval.  
3 -----
```

Please replace _____ with valid code.

Summary

- Substituting s for σ adds uncertainty that the Normal critical value ignores. At $n = 5$ a nominal 95% interval covered **88%**.
- $(\bar{X} - \mu)/(s/\sqrt{n})$ follows a t **distribution with $n - 1$ degrees of freedom**, whose heavier tails give a larger critical value and a wider interval.
- `qt(0.975, 4)` = 2.776 against 1.96; by $df = 199$ it is 1.972. **Use t always**—at large n the choice does not matter, and at small n it does.
- The t interval is **exact for a Normal population**, not an approximation that improves with n .
- It corrects for unknown σ and **nothing else**. Small n with skewed data under-covers no matter which critical value you use, because a non-symmetric sampling distribution cannot be captured by a symmetric interval.

Next: the same construction for a proportion, where the standard recipe fails in a way that matters for every conversion rate you will ever report.

Monday, October 5

Post your questions on any of these pages by **Monday at 12:00 noon**, the morning of class.

Last week produced an interval and a careful sentence about what it claims. This week narrows the question: *is the data compatible with this particular claim?*

The machinery is small—one statistic, one tail area—and you have already built most of it. What the week is really about is the gap between what a test computes and what people report it as having computed. That gap is where a large share of published findings that later fail to replicate came from, and it is worth understanding as a mechanism rather than as a scandal.

Two of these chapters cover material that is usually left out of an introductory course. **Power** determines what a non-significant result means and is the reason underpowered studies overstate every effect they publish. **Multiple comparisons** explains why an honest analyst, making defensible choices one at a time, arrives at a finding that is not there. Neither is advanced. Both are asked about in interviews, and both will decide whether your project's conclusions hold up.

What is here

THE LOGIC OF A TEST

Assume the claim, measure surprise. The test statistic as a standardized distance, the p-value as a tail area you can simulate rather than look up, and the exact correspondence between a two-sided test and a confidence interval.

WHAT A P-VALUE LICENSES

Under a true null, p-values are uniform—which explains both why 0.049 and 0.051 are the same evidence and why small p-values are guaranteed if you look enough times. Then the five misreadings, and why significance is not importance.

POWER

The half of the picture nobody reports. How many observations you need, what “we found nothing” is worth, and the winner's curse: at 25% power, the studies that reached significance overstated a true effect of 0.3 as 0.57.

MULTIPLE COMPARISONS

Twenty tests, a 64% chance of a false positive. Where it really happens— subgroups, outcomes, peeking, specification hunting—and what Bonferroni and Benjamini-Hochberg each cost you.

PRACTICE PROBLEMS

Including three analyses that run, return a number, and mislead: an underpowered null result, a peeked A/B test, and the subgroup that worked.

A habit for this week

Report the interval, then the p-value if you must. The interval carries the effect size, the precision, and the p-value's information; the p-value alone carries the least of the three and is the one most likely to be misquoted back at you.

DUE THIS WEEK

HW 4 is due Sunday at 11:59 p.m., the night before class. And as always, post your questions on these pages by Monday noon—that afternoon's session is built out of them.

A test at $\alpha = 0.05$ produces a false positive one time in twenty. That is not a defect; it is the specification.

It becomes a defect the moment you run more than one test, because the guarantee was per test and the claim people make is about the collection.

The arithmetic

If m independent tests are run and every null is true, the chance that **at least one** comes out significant is $1 - 0.95^m$.

R Code

```
1 m <- c(1, 2, 5, 10, 20, 50, 100)
2 round(data.frame(tests = m, P_at_least_one_false_positive = 1 - 0.95^m), 3)
```

	tests	P_at_least_one_false_positive
1	1	0.050
2	2	0.098
3	5	0.226
4	10	0.401
5	20	0.642
6	50	0.923
7	100	0.994

Twenty tests and it is more likely than not. A hundred and it is essentially certain. Twenty is not a large number of tests—it is one outcome across twenty subgroups, or five outcomes at four time points, or a dashboard.

R Code

```
1 set.seed(6101)
2 # 20 comparisons, all null: two groups drawn from the same population.
3 pvals <- replicate(20, {
4   a <- rnorm(50); b <- rnorm(50)
5   t.test(a, b)$p.value
6 })
7 round(sort(pvals), 4)
8 cat("significant at 0.05:", sum(pvals < 0.05), "\n")
```

```
[1] 0.0062 0.0360 0.0693 0.1649 0.2972 0.3017 0.3344 0.3589 0.4706 0.6027
[11] 0.6425 0.6635 0.6907 0.8599 0.9253 0.9357 0.9454 0.9585 0.9623 0.9833
significant at 0.05: 2
```

Nothing is different between any of these groups. Report only the smallest p-value and you have a finding, a plausible story to explain it, and no way for a reader to tell.

Where it actually happens

Rarely as an explicit set of twenty tests. Usually as one of these:

Subgroup hunting. The treatment did not work overall, so you check men, women, under-40s, over-40s, each region, each product line. Twelve subgroups is twelve tests.

Outcome hunting. Revenue did not move, so you look at sessions, retention, clicks, time on page, support tickets.

Peeking. Checking an A/B test daily for three weeks and stopping when it crosses 0.05 is not one test. It is twenty-one opportunities, and the false positive rate is far above 5%.

Specification hunting. Trying the analysis with and without outliers, with and without a covariate, logged and unlogged, and reporting the version that worked.

The last is the hardest to see from the inside, because each individual choice is defensible. **The problem is not any one decision; it is that the decisions were made after seeing which way they pushed the answer.** No p-value in the final write-up records the paths not taken.

Two corrections

Bonferroni controls the probability of *any* false positive: compare each p-value to α/m instead of α . It is exact, it is one line, and it is brutally conservative—with 100 tests each must clear 0.0005.

Benjamini-Hochberg controls the **false discovery rate**: the expected proportion of your rejections that are false. It is the right tool when you are screening many candidates and can tolerate some duds in the shortlist.

R Code

```
1 set.seed(11)
2 # 100 tests. The first 10 have a real effect; the other 90 are null.
3 p <- c(replicate(10, t.test(rnorm(50, mean = 0.5))$p.value),
4       replicate(90, t.test(rnorm(50, mean = 0))$p.value))
5 truth <- c(rep(TRUE, 10), rep(FALSE, 90))
6
7 report <- function(keep, label) {
8   c(method = label, discoveries = sum(keep),
9     true_ones = sum(keep & truth), false_ones = sum(keep & !truth))
10 }
11
12 rbind(
13   report(p < 0.05, "raw p < 0.05"),
14   report(p.adjust(p, "bonferroni") < 0.05, "Bonferroni"),
15   report(p.adjust(p, "BH") < 0.05, "Benjamini-Hochberg")
16 )
```

	method	discoveries	true_ones	false_ones
[1,]	"raw p < 0.05"	"12"	"9"	"3"
[2,]	"Bonferroni"	"5"	"5"	"0"
[3,]	"Benjamini-Hochberg"	"8"	"8"	"0"

Averaged over 2,000 runs of that setup, the pattern is stable:

METHOD	DISCOVERIES	TRUE	FALSE
raw $p < 0.05$	13.7	9.3	4.4
Bonferroni	4.4	4.3	0.05
Benjamini-Hochberg	7.2	6.8	0.4

Read the trade honestly. **Uncorrected, a third of your findings are false.** Bonferroni removes essentially all of them and finds fewer than half the real effects. BH finds two-thirds of the real effects at the cost of about one false one in twenty reported.

WHICH TO USE

Bonferroni when a single false positive is expensive—a clinical claim, a regulatory filing, a public statement.

Benjamini-Hochberg when you are generating a shortlist for follow-up and the cost of a dud is another experiment.

Neither, if you can instead **pre-register the one comparison you care about**. That is not a correction; it is not needing one.

Correction is not the real fix

Adjusting p-values assumes you know m . The dangerous case is the one where you do not—where a dozen analytic choices were made fluidly while looking at the data, and no one, including you, could reconstruct how many comparisons were implicitly made.

Three habits work better than any adjustment.

Decide the primary analysis before seeing the outcome. One outcome, one comparison, written down. Everything else is explicitly labeled exploratory.

Report everything you ran. A table of twenty p-values, uncorrected, is more honest than one corrected p-value from an unstated twenty.

Treat exploratory findings as hypotheses. They are the input to the next study, not the conclusion of this one. Held to that standard, subgroup hunting is a good use of data.

⌕ EXERCISE: HOW BAD DOES IT GET?

You run 12 independent tests, all of them null, at $\alpha = 0.05$. Return the probability that **at least one** is significant.

Exercise	
1	# The complement rule from probability: 1 minus the chance that none of them fire.
2	_____

Please replace _____ with valid code.

Summary

- The 5% guarantee is **per test**. Twenty tests give a 64% chance of at least one false positive; a hundred give 99%.
- It arrives as **subgroup hunting, outcome hunting, peeking, and specification hunting**—rarely as an explicit list of tests.
- **Bonferroni** (α/m) controls any false positive and is conservative. **Benjamini-Hochberg** controls the false discovery rate and finds more.
- In the 100-test simulation: uncorrected found 13.7 things of which 4.4 were false; BH found 7.2 of which 0.4 were false.
- Correction requires knowing m , and the worst cases are the ones where you do not. **Pre-specify the primary comparison, report everything you ran, and treat exploratory results as hypotheses.**

Next: the practice problems, and then a term of deciding which test the question actually calls for.

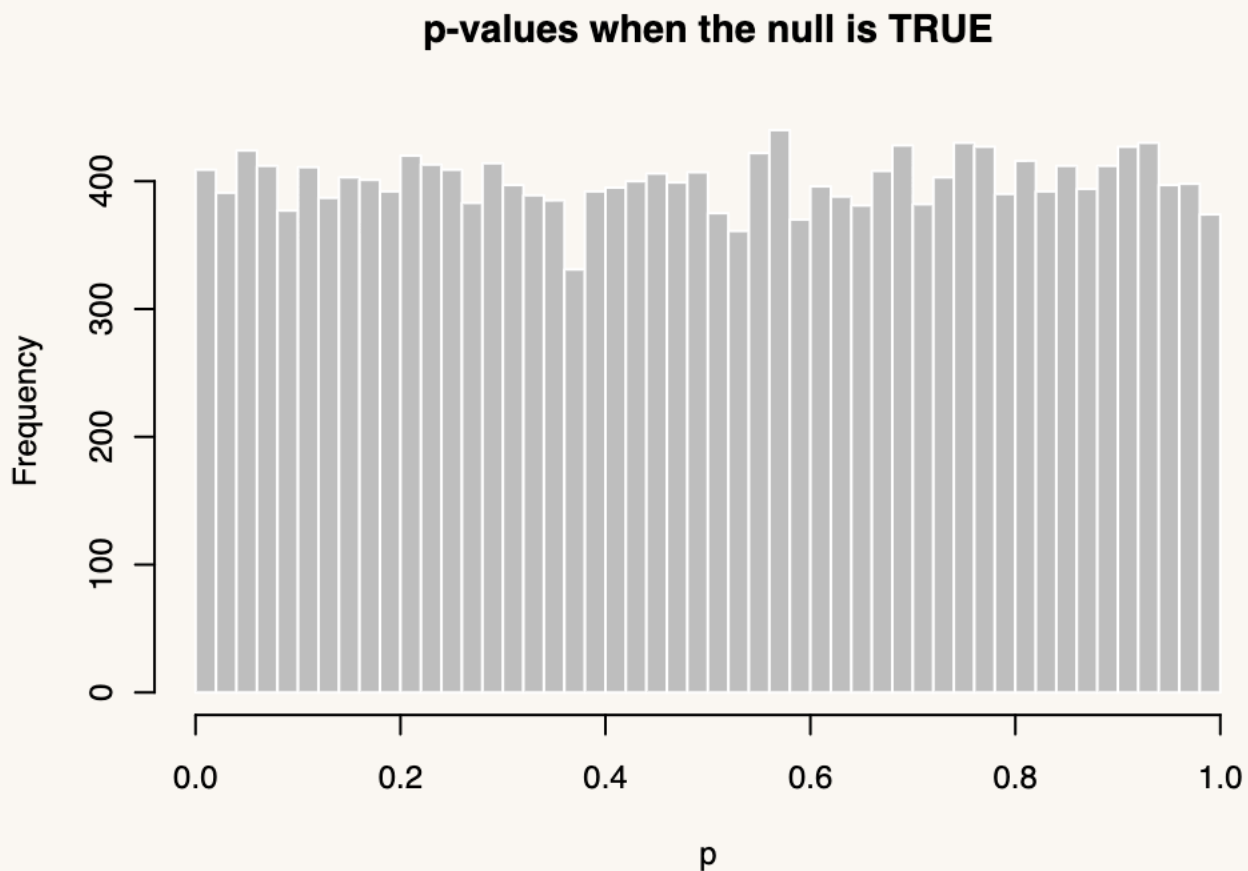
The p-value is the most reported and least understood number in applied statistics. This chapter is short because its content is mostly subtraction: here is the one thing it says, and here are the five things people take it to say.

Under the null, a p-value is uniform

Start with a fact that makes everything else easier to reason about.

R Code

```
1 set.seed(6101)
2 # 20,000 studies in a world where the null is exactly true.
3 p <- replicate(20000, t.test(rnorm(30), mu = 0)$p.value)
4
5 hist(p, breaks = 40, col = "gray75", border = "white",
6      main = "p-values when the null is TRUE", xlab = "p")
7
8 round(c(P_below_0.05 = mean(p < 0.05),
9        P_below_0.50 = mean(p < 0.50),
10       P_below_0.90 = mean(p < 0.90)), 4)
```



P_below_0.05	P_below_0.50	P_below_0.90
0.0506	0.4974	0.8987

Flat. When the null is true, every p-value is as likely as every other: 5% below 0.05, half below 0.5, and so on. That is not an accident; it is the definition working. A threshold of 0.05 produces false positives 5% of the time precisely because the distribution is uniform.

Two consequences follow immediately.

A p-value of 0.049 and one of 0.051 are the same evidence. Nothing happens at the boundary. The distribution has no feature there, and the only thing that changes is which side of a convention you land on.

Under the null, small p-values are guaranteed to occur. Run twenty honest tests of true nulls and you should *expect* one below 0.05. That is the whole of [Multiple Comparisons](#), visible already in this histogram.

The five misreadings

“ $p = 0.03$, so there is a 3% chance the null is true.” The p-value is $P(\text{data} \mid H_0)$, not $P(H_0 \mid \text{data})$. Getting from one to the other requires a prior, which is what [Conditional Probability and Bayes](#) was about. Without one, the second quantity is simply not available.

“ $p = 0.03$, so there is a 97% chance the effect is real.” Same inversion, wearing a hat.

“ $p > 0.05$, so there is no effect.” A non-significant result is compatible with no effect *and* with a real effect the study was too small to detect. Which of those you are in is a question about **power**, and the p-value cannot answer it. Absence of evidence is not evidence of absence.

“ $p = 0.001$, so the effect is large.” The p-value confounds effect size with sample size. A trivial effect measured on a million users produces a tiny p-value; an enormous effect measured on nine people may not.

R Code			
1	set.seed(6101)		
2	# A negligible difference, measured on a lot of people.		
3	big <- t.test(rnorm(100000, mean = 0.01), mu = 0)		
4	# A large difference, measured on very few.		
5	small <- t.test(rnorm(9, mean = 1.2), mu = 0)		
6			
7	round(c(tiny_effect_huge_n_p = big\$p.value, estimate_1 = big\$estimate,		
8	big_effect_tiny_n_p = small\$p.value, estimate_2 = small\$estimate), 5)		
	tiny_effect_huge_n_p	estimate_1.mean of x	big_effect_tiny_n_p
	0.00010	0.01233	0.00044
	estimate_2.mean of x		
	1.55238		

The first result is “highly significant” and describes an effect of 0.01. The second is a hundred times larger and less significant. **Significance is not importance**, and the only way to tell them apart is to look at the estimate and its interval.

“The result replicated, because both studies had $p < 0.05$.” Two studies agreeing on which side of a threshold they fall is a weak form of agreement. Compare the intervals instead.

WHAT A SMALL P-VALUE DOES LICENSE

“If the null were true, data like this would be uncommon.”

That is it. It is a real thing to know and it is worth reporting—but it is a statement about the data under an assumption, not about the assumption.

Effect size, and why it is not optional

Every test in this course has an effect size sitting next to it, and it is usually the number the audience actually wanted.

R Code

```

1 set.seed(6101)
2 control <- rnorm(60, mean = 72, sd = 11)
3 treatment <- rnorm(60, mean = 77, sd = 11)
4
5 tt <- t.test(treatment, control)
6 d <- (mean(treatment) - mean(control)) /
7     sqrt((var(treatment) + var(control)) / 2) # Cohen's d
8
9 round(c(difference = diff(range(c(mean(control), mean(treatment))))),
10     ci_low = tt$conf.int[1], ci_high = tt$conf.int[2],
11     p = tt$p.value, cohens_d = d), 4)

```

difference	ci_low	ci_high	p	cohens_d
-9.9373	5.9554	13.9192	0.0000	0.9023

Report the **difference and its interval** first. Cohen's d —the difference in standard deviations—is useful when the units are not meaningful to the reader, with the usual rough anchors of 0.2 small, 0.5 medium, 0.8 large. Treat those as vocabulary rather than law; a d of 0.1 on a mortality outcome is enormous.

✧ EXERCISE: WHICH CONCLUSION IS AVAILABLE

A study of a new onboarding flow reports: *mean time to first action fell by 0.4 seconds, 95% CI $[-1.9, 1.1]$, $p = 0.60$, $n = 5,000$.*

Assign the letter, for example `answer <- "A"`.

Exercise

```

1 # A. The new flow has no effect on time to first action.
2 # B. There is a 60% probability that the null hypothesis is true.
3 # C. The data are consistent with no effect, and also with effects up to about
4 #   2 seconds in either direction; the study cannot distinguish them.
5 # D. The effect is real but too small to matter.
6 answer <- _____

```

Please replace _____ with valid code.

Summary

- Under a true null, p-values are **uniform**. Five percent fall below 0.05 by construction, and 0.049 versus 0.051 is not a distinction.
- A p-value is $P(\text{data} \mid H_0)$. It is not the probability the null is true, nor the probability the effect is real.
- **Non-significant is not null**. That distinction is a question about power.
- The p-value confounds **effect size with sample size**: 0.01 on 100,000 people beat 1.2 on nine.
- Report the **estimate and its interval**. Add the p-value if you like; it is the least informative of the three.

Next: the missing half of every non-significant result.

α is chosen. It is the false positive rate, it is set at 0.05 by convention, and everyone reports it.

β is inherited. It is the false *negative* rate, it follows from the sample size and the size of the effect, almost nobody computes it, and it is the reason most published findings that fail to replicate ever got published.

Power is $1 - \beta$: the probability that a study finds an effect that is really there.

$$\text{power} = P(\text{reject } H_0 \mid H_0 \text{ is false})$$

Two questions it answers

Both are asked constantly, and neither has any other route to an answer.

“How many observations do we need?” — the sample-size calculation, before the study.

“We found nothing. What does that mean?” — the interpretation, after it. A non-significant result from a well-powered study is informative. The same result from an underpowered study is not evidence of anything.

Measuring it by simulation

Power is a probability, so simulate it: build worlds where the effect is real, run the test, count how often it fires.

R Code	
1	<code>set.seed(6101)</code>
2	<code># True effect of 0.5 standard deviations. How often does n = 30 detect it?</code>
3	<code>detected <- replicate(10000, {</code>
4	<code> x <- rnorm(30, mean = 0.5, sd = 1)</code>
5	<code> t.test(x, mu = 0)\$p.value < 0.05</code>
6	<code>})</code>
7	<code>round(mean(detected), 3)</code>
[1] 0.757	

About 75%. So a quarter of studies run this way will miss a real, medium-sized effect and report nothing—and nothing in their output will indicate that this is what happened.

R also computes it directly.

R Code	
1	<code>power.t.test(n = 30, delta = 0.5, sd = 1, type = "one.sample")</code>
One-sample t test power calculation n = 30 delta = 0.5 sd = 1 sig.level = 0.05 power = 0.7539627 alternative = two.sided	

The three levers

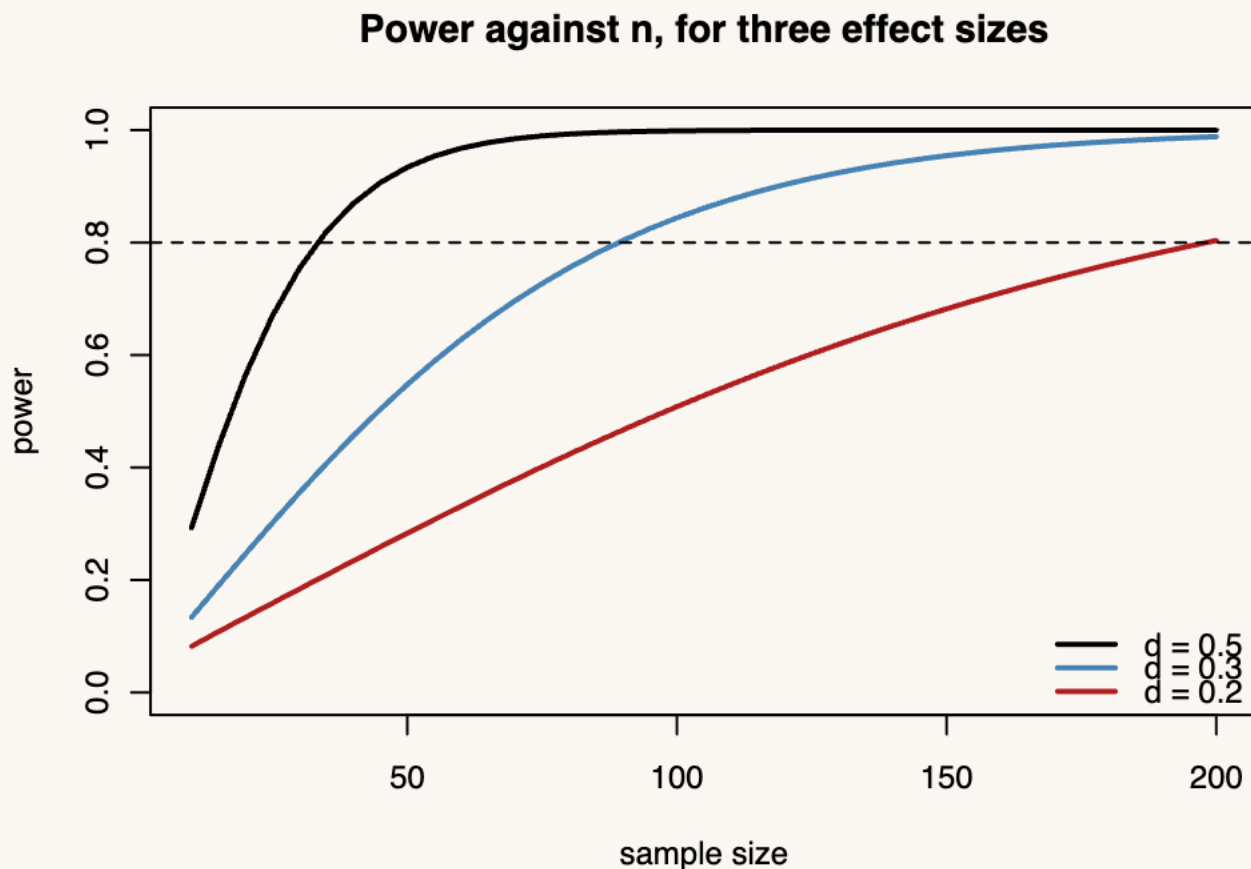
Power rises with the **effect size**, the **sample size**, and α . Only two of those are yours to choose.

R Code

```

1 n <- seq(10, 200, by = 5)
2 pw <- function(d) sapply(n, function(k)
3   power.t.test(n = k, delta = d, sd = 1, type = "one.sample")$power)
4
5 plot(n, pw(0.5), type = "l", lwd = 2, ylim = c(0, 1),
6   xlab = "sample size", ylab = "power",
7   main = "Power against n, for three effect sizes")
8 lines(n, pw(0.3), lwd = 2, col = "steelblue")
9 lines(n, pw(0.2), lwd = 2, col = "firebrick")
10 abline(h = 0.8, lty = 2)
11 legend("bottomright", bty = "n", lwd = 2,
12   col = c("black", "steelblue", "firebrick"),
13   legend = c("d = 0.5", "d = 0.3", "d = 0.2"))

```



The dashed line is 80%, the conventional target. Read off where each curve crosses it:

R Code

```

1 for (d in c(0.8, 0.5, 0.3, 0.2, 0.1)) {
2   n80 <- ceiling(power.t.test(delta = d, sd = 1, power = 0.8,
3     type = "one.sample")$n)
4   cat("d =", format(d, nsmall = 1), " n for 80% power:", n80, "\n")
5 }

```

```
d = 0.8  n for 80% power: 15
d = 0.5  n for 80% power: 34
d = 0.3  n for 80% power: 90
d = 0.2  n for 80% power: 199
d = 0.1  n for 80% power: 787
```

Halving the effect quadruples the sample. That is the same k^2 relationship as everywhere else this term, and it is why detecting small effects is expensive and why claims of small effects from small samples deserve suspicion.

The winner's curse

This is the part with consequences beyond your own study, and it is best shown rather than described.

Suppose a real effect of 0.3 exists, and studies of 20 people go looking for it.

R Code

```
1 set.seed(3)
2 res <- replicate(20000, {
3   x <- rnorm(20, mean = 0.3, sd = 1)
4   tt <- t.test(x, mu = 0)
5   c(p = tt$p.value, estimate = mean(x))
6 })
7
8 sig <- res["p", ] < 0.05
9
10 round(c(true_effect      = 0.3,
11         power            = mean(sig),
12         mean_estimate_all = mean(res["estimate", ]),
13         mean_estimate_if_sig = mean(res["estimate", sig])), 3)
```

```
      true_effect      power  mean_estimate_all
mean_estimate_if_sig
      0.300      0.247      0.300
      0.568
```

Three numbers worth sitting with.

Power is 25%. Three quarters of these studies find nothing.

Averaged over all studies, the estimate is unbiased—0.30, exactly right.

Averaged over the studies that reached significance, the estimate is 0.57— nearly **double the truth**.

That is not a coding error and it is not a bias in the estimator. It is selection. To clear $p < 0.05$ on twenty observations, a study needs a large observed effect; studies that happened to see the true 0.3 do not qualify. So the significance filter systematically passes overestimates.

WHY REPLICATION FAILS

The published literature is the significant subset. In an underpowered field, that subset overstates every effect it contains, so the next team powers their replication using the inflated number, is underpowered again, and finds nothing.

The original was not fraudulent. It was underpowered, and being underpowered guarantees that whatever you *do* publish is exaggerated.

Post-hoc power is not the fix

Having found nothing, it is tempting to compute power using the effect size you observed. Do not. That quantity is a deterministic function of the p-value—it tells you nothing the p-value did not, and it is always low when the result was non-significant, which makes it look like an explanation while being a restatement.

The useful version is the confidence interval. If the interval is $[-1.9, 1.1]$, then effects up to 1.9 remain consistent with your data and you say so. If it is $[-0.05, 0.04]$, you have genuinely ruled out anything large, and *that* is evidence of absence.

🔗 EXERCISE: HOW MANY DO WE NEED?

You are planning an A/B test on a continuous outcome. The smallest difference worth acting on is **0.25 standard deviations**. You want **80% power** at $\alpha = 0.05$, two-sided, with equal group sizes.

Return the required **number per group**, rounded up.

Exercise

```
1 # power.t.test() solves for whichever argument you leave out.
2 # type = "two.sample" for two independent groups.
3 -----
```

Please replace ----- with valid code.

Summary

- **Power** $= 1 - \beta$, the chance of detecting an effect that is really there. It follows from the design; it is not chosen.
- Simulate it in four lines, or use `power.t.test()`.
- **Halving the effect quadruples the sample.** Detecting $d = 0.5$ takes 34 observations; $d = 0.2$ takes 199.
- The **winner's curse**: at 25% power, significant studies overstated a true effect of 0.3 as 0.57. The estimator is unbiased; the *filter* is not.
- Underpowered research does not merely fail to find things. It guarantees that what it does publish is exaggerated, which is the mechanism behind failed replication.
- **Post-hoc power is not diagnostic.** Report the confidence interval, which says which effects remain on the table.

Next: what happens when you run not one test but forty.

Not graded, not auto-checked. Bring your answers and your disagreements.

Predict the output

R Code

```
1 # 1. Under a true null, how are p-values distributed?
2 set.seed(1)
3 p <- replicate(5000, t.test(rnorm(25), mu = 0)$p.value)
4 round(quantile(p, c(0.1, 0.25, 0.5, 0.75, 0.9)), 3)
5
6 # 2.
7 set.seed(2)
8 x <- rnorm(2000, mean = 0.05)
9 c(p = t.test(x, mu = 0)$p.value, estimate = mean(x))
10
11 # 3.
12 round(c(one_test = 0.05, twenty_tests = 1 - 0.95^20,
13         bonferroni_threshold = 0.05 / 20), 4)
```

```
10% 25% 50% 75% 90%
0.109 0.258 0.511 0.746 0.893
      p      estimate
4.800965e-05 9.150843e-02
      one_test      twenty_tests bonferroni_threshold
      0.0500      0.6415      0.0025
```

DISCUSSION



This runs and is still wrong

R Code

```
1 # 4. "We found no difference between the groups."
2 set.seed(7)
3 a <- rnorm(12, mean = 100, sd = 20)
4 b <- rnorm(12, mean = 112, sd = 20) # a real 12-point difference
5 tt <- t.test(a, b)
6 round(c(p = tt$p.value, ci = tt$conf.int), 3)
```

```
      p      ci1      ci2
0.138 -36.391  5.388
```

R Code

```
1 # 5. Peeking at an A/B test.
2 set.seed(4)
3 x <- rnorm(2000); y <- rnorm(2000) # no true difference at all
4 ps <- sapply(seq(100, 2000, by = 100), function(k) t.test(x[1:k], y[1:k])$p.value)
5 round(ps, 3)
6 cat("ever crossed 0.05:", any(ps < 0.05), "\n")
```

```
[1] 0.125 0.735 0.902 0.563 0.419 0.308 0.163 0.291 0.404 0.249 0.259 0.303
[13] 0.227 0.295 0.240 0.178 0.335 0.234 0.389 0.689
ever crossed 0.05: FALSE
```

R Code

```
1 # 6. The subgroup that worked.
2 set.seed(9)
3 region <- rep(c("N", "S", "E", "W"), each = 60)
4 tier <- rep(c("free", "paid", "ent"), times = 80)
5 y <- rnorm(240) # no effect anywhere
6 g <- sample(c("A", "B"), 240, replace = TRUE)
7 ps <- sapply(split(seq_along(y), list(region, tier)),
8             function(i) t.test(y[i] ~ g[i])$p.value)
9 round(sort(ps)[1:4], 4)
```

```
E.paid N.paid W.ent N.free
0.2388 0.2837 0.2871 0.5560
```

DISCUSSION



What does this license you to claim?

A trial reports: *treatment reduced average recovery time by 1.2 days, 95% CI [0.1, 2.3], $p = 0.03$, $n = 180$.*

7. “The treatment works.”
8. “The treatment reduces recovery time by 1.2 days.”
9. “There is a 3% chance the treatment has no effect.”
10. “A reduction of 2 days is consistent with these data.”
11. “The effect is clinically important.”
12. Now suppose the same trial reported $p = 0.03$ for one of nine outcomes it measured, and the other eight are not shown. What changes?

DISCUSSION



Bring to class

13. **Power for your project.** Name the effect you would care about, guess the standard deviation, and compute the sample size you would need for 80% power. If your project data are already collected, compute the power you have instead. Bring the number, and be ready to say whether the study can answer its question.
14. **Find a p-value in the wild.** A news article, a press release, a paper. Bring it and answer: how many comparisons could plausibly have been made? Is an effect size reported alongside? Would the interval have told a different story?
15. **Argue about this.** “We should lower α to 0.005 so that fewer false findings get published.” Work out what that costs, who pays it, and whether it addresses the mechanism in [Power](#) or merely moves it.

A confidence interval answers *how big is it, and how precisely do we know?* A hypothesis test answers a narrower question: *is the data compatible with this particular claim?*

The second question is more often asked and more often misunderstood, and almost all of the misunderstanding comes from one structural feature that this chapter exists to make unmissable. **A test assumes the claim is true and then measures surprise.** It never evaluates whether the claim is true; it evaluates the data, on the supposition that it is.

The four steps

1. **State a null hypothesis H_0** —a specific claim precise enough to compute with. “The mean is 100.” “The two groups have the same rate.”
2. **Choose a test statistic**—one number summarizing how far the data sits from that claim, in units of its own standard error.
3. **Work out how that statistic behaves if H_0 is true.** This is the part the whole term has been building towards, and it is where the standard error and the central limit theorem get used.
4. **Ask how often that behavior would produce something at least as extreme as what you saw.** That proportion is the **p-value**.

The statistic is a standardized distance

Step 2 is always the same shape:

$$t = \frac{\text{estimate} - \text{claim}}{\text{SE}(\text{estimate})}$$

How many standard errors away is what I saw from what you claimed? Every test in this course is that expression with different things substituted in.

R Code

```
1 set.seed(8)
2 # A supplier claims mean fill weight is 500 g. We weigh 40 units.
3 fill <- rnorm(40, mean = 496, sd = 9)
4
5 est  <- mean(fill)
6 se   <- sd(fill) / sqrt(length(fill))
7 tstat <- (est - 500) / se
8
9 round(c(estimate = est, se = se, t = tstat), 4)
```

estimate	se	t
495.8582	1.5335	-2.7009

The sample mean is about 2.7 standard errors below the claim. Whether that is surprising depends on how often a t of that size arises when the claim is true—which is step 3, and which you already know: it follows a t distribution with $n - 1$ degrees of freedom.

The p-value is a tail area

R Code

```
1 set.seed(8)
2 fill <- rnorm(40, mean = 496, sd = 9)
3 tstat <- (mean(fill) - 500) / (sd(fill) / sqrt(40))
4
```

```

5 # Two-sided: as extreme in EITHER direction.
6 p <- 2 * pt(-abs(tstat), df = 39)
7 round(c(t = tstat, p_value = p), 5)
8
9 t.test(fill ~ mu, mu = 500)

```

```

      t      p_value
-2.70086  0.01018

      One Sample t-test

data:  fill
t = -2.7009, df = 39, p-value = 0.01018
alternative hypothesis: true mean is not equal to 500
95 percent confidence interval:
 492.7564 498.9600
sample estimates:
mean of x
 495.8582

```

`t.test()` does all of it and reports the confidence interval alongside, which is worth noticing: the interval excludes 500, and the p-value is below 0.05. **Those are the same fact stated twice.** A two-sided test at level α rejects exactly when the $1 - \alpha$ confidence interval excludes the claimed value.

PREFER THE INTERVAL

Given the choice, report the interval. It contains the p-value's information and adds what the p-value omits—the size of the effect and the precision of the estimate. “Mean fill 495.9 g, 95% CI 492.8 to 499.0, against a claimed 500” says everything “ $p = 0.010$ ” says, plus the two things a reader actually needs.

Seeing the null distribution

Nothing about step 3 requires trusting a formula. Simulate the world in which H_0 holds and look at what it produces.

R Code

```

1 set.seed(6101)
2 # If the supplier were telling the truth, what t-statistics would we see?
3 null_ts <- replicate(20000, {
4   x <- rnorm(40, mean = 500, sd = 9)
5   (mean(x) - 500) / (sd(x) / sqrt(40))
6 })
7
8 hist(null_ts, breaks = 60, col = "gray75", border = "white",
9       main = "t-statistics when the null is true", xlab = "t")
10 abline(v = c(-2.72, 2.72), col = "firebrick", lwd = 2)
11
12 # How often is |t| at least as extreme as ours?
13 round(mean(abs(null_ts) >= 2.72), 4)

```

t-statistics when the null is true



[1] 0.01

The simulated proportion matches `t.test()`'s p-value. A **p-value is a frequency in a world you constructed**, and that is the whole of its meaning.

What the p-value is a probability of

Write it precisely, because the order of the conditioning is where everything goes wrong:

$$p = P(\text{data at least this extreme} \mid H_0 \text{ true})$$

It is **not** $P(H_0 \text{ true} \mid \text{data})$. Those are different quantities, and swapping them is the same inversion as [Conditional Probability and Bayes](#) warned about with medical tests—where the two conditionals differed by a factor of thirty.

A small p-value says: *if the claim were true, data like this would be rare*. Rare things happen. And a claim being incompatible with the data does not identify which of your assumptions is at fault—the null is one assumption among several, alongside independence, the sampling frame, and the measurement.

Rejecting, and what the two errors are

Choosing a threshold α before looking converts the p-value into a decision, and creates two ways to be wrong.

	H_0 TRUE	H_0 FALSE
Reject	Type I error, rate α	Correct
Do not reject	Correct	Type II error, rate β

α is chosen. β is not—it follows from the sample size and the size of the effect, and it is the subject of [Power](#).

The asymmetry matters. “**Do not reject**” is not “**accept**”. A test that fails to find evidence has not produced evidence of absence, and the difference between those is entirely a question of how much power the study had.

🔗 EXERCISE: COMPUTE A P-VALUE FROM SCRATCH



x holds 25 measurements. Test $H_0 : \mu = 12$ against a two-sided alternative **without using** `t.test()`. Return the p-value.

Exercise	
1	# t = (mean - 12) / (sd / sqrt(n)), then two times the lower tail of t .
2	_____

Please replace _____ with valid code.

Summary

- A test **assumes H_0 and measures surprise**. It never evaluates whether H_0 is true.
- The statistic is always (estimate – claim)/SE—how many standard errors away.
- The p-value is $P(\text{data this extreme} \mid H_0)$, and it can be obtained by simulation as easily as by formula. It is **not** $P(H_0 \mid \text{data})$.
- A two-sided test at level α and the $1 - \alpha$ confidence interval agree exactly. **Report the interval**—it carries the effect size and the precision too.
- α is chosen; β is inherited from the design. “**Do not reject**” is not “**accept**.”

Next: what a p-value licenses you to say, and the four claims it does not support.

Every test in this course is $(\text{estimate} - \text{claim}) / \text{SE}$ with different things substituted in, so choosing among them is not deep. It is a question about what kind of variables you have, and it gets asked in interviews because a surprising number of people cannot answer it.

The decision

OUTCOME	COMPARISON	TEST	IN R
Numeric	One group against a claimed value	one-sample t	<code>t.test(x, mu =)</code>
Numeric	Two independent groups	two-sample t	<code>t.test(x ~ g)</code>
Numeric	Same units measured twice	paired t	<code>t.test(x, y, paired = TRUE)</code>
Numeric	Skewed, or a statistic with no formula	permutation	Permutation Tests
Binary	Two proportions	two-proportion	<code>prop.test(x, n)</code>
Categorical	Association between two categorical variables	chi-squared	<code>chisq.test(table(a, b))</code>
Categorical	Small expected counts	Fisher's exact	<code>fisher.test(table(a, b))</code>

Two questions get you to the row: **what kind of thing is the outcome**, and **how many groups are being compared**. Everything else is detail.

Paired is a different test, not a nicer one

The most consequential row is the third, because getting it wrong throws away most of your power.

R Code	
1	<code>set.seed(6101)</code>
2	<code># 30 subjects, measured before and after. The effect is small; the</code>
3	<code># between-subject variation is large.</code>
4	<code>subject <- rnorm(30, mean = 100, sd = 20)</code>
5	<code>before <- subject + rnorm(30, 0, 2)</code>
6	<code>after <- subject + 3 + rnorm(30, 0, 2)</code>
7	
8	<code>round(c(unpaired_p = t.test(after, before)\$p.value,</code>
9	<code>paired_p = t.test(after, before, paired = TRUE)\$p.value), 6)</code>
	<code>unpaired_p paired_p</code>
	<code>0.490281 0.000000</code>

The same data, the same 3-unit effect. Unpaired, the subject-to-subject spread of 20 swamps it and the test finds nothing. Paired, each subject serves as their own control, that spread cancels, and the effect is unmissable.

If the same unit appears in both groups, the observations are not independent, and the paired test is not an optional refinement—it is the one that respects the design.

Chi-squared, for categorical association

The chi-squared test compares a table of counts with the counts you would expect if the two variables were unrelated.

R Code

```
1 # Did the plan a customer chose relate to which region they came from?
2 tbl <- matrix(c(120, 90, 60,
3               150, 95, 55), nrow = 2, byrow = TRUE,
4               dimnames = list(region = c("north", "south"),
5                                   plan  = c("free", "pro", "ent")))
6 tbl
7
8 chisq.test(tbl)
```

```
      plan
region free pro ent
north  120  90  60
south  150  95  55

Pearson's Chi-squared test

data:  tbl
X-squared = 2.1128, df = 2, p-value = 0.3477
```

The statistic is the same idea as always—observed minus expected, standardized and squared—and the expected counts assume independence.

R Code

```
1 tbl <- matrix(c(120, 90, 60, 150, 95, 55), nrow = 2, byrow = TRUE,
2               dimnames = list(region = c("north", "south"),
3                                   plan  = c("free", "pro", "ent")))
4 round(chisq.test(tbl)$expected, 1)
```

```
      plan
region free pro ent
north 127.9 87.6 54.5
south 142.1 97.4 60.5
```

The rule of thumb is that every expected count should be at least 5. Below that, the approximation degrades and `fisher.test()` is the honest choice; R warns you when it happens, and the warning is worth reading rather than suppressing.

A two-proportion test is the two-by-two special case. `prop.test` and `chisq.test` on the same 2×2 table give the identical p-value, which is worth seeing once:

R Code

```
1 round(c(prop  = prop.test(c(1210, 1120), c(24000, 24000), correct = FALSE)$p.value,
2               chisq = chisq.test(matrix(c(1210, 24000 - 1210,
3               1120, 24000 - 1120), nrow = 2),
4               correct = FALSE)$p.value), 6)
```

```
      prop      chisq
0.055943 0.055943
```

Prefer `prop.test`: it reports the **difference and its confidence interval**, which chi-squared does not, and by now you know why that matters.

What about ANOVA?

You will meet it, so here is the honest position. ANOVA tests whether several group means are all equal. It is the natural extension of the two-sample *t* test to three or more groups.

It is not taught in this course because **a linear model with a categorical predictor does the same job and more**: it gives you the individual group differences and their intervals rather than a single “something differs” verdict, it extends to covariates, and it is where November goes anyway. If you need it, `aov()` exists and `summary(lm(y ~ g))` tells you more.

EXERCISE: WHICH TEST

Forty participants each rate a website before and after a redesign, on a 1–100 scale. You want to know whether the redesign changed ratings.

Assign the letter, for example `answer <- "A"`.

Exercise

```
1 # A. Two-sample t-test comparing the before ratings with the after ratings.
2 # B. Paired t-test, because each participant appears in both sets.
3 # C. Chi-squared test, because ratings are categories from 1 to 100.
4 # D. Two-proportion test on the share who rated above 50.
5 answer <- _____
```

Please replace _____ with valid code.

Summary

- Two questions choose the test: **what kind of outcome**, and **how many groups**.
- **Paired when the same unit appears twice**. In the simulation the unpaired test found nothing and the paired test found the effect easily, on identical data.
- **Chi-squared** for categorical association, with expected counts of at least 5; **Fisher's exact** below that.
- A two-proportion test is chi-squared on a 2×2 table. Use `prop.test`, because it reports the difference and its interval.
- **ANOVA is a linear model with a categorical predictor**, and the model tells you more.

Next: the practice problems, and then a month of modeling.

An A/B test is a randomized experiment with a marketing name. Everything in [What Randomization Buys](#) applies, and this chapter is the checklist that turns it into something you can run on Monday.

The order matters. **Every decision below is made before any data arrives**, and writing them down is what separates an experiment from an exercise in finding something.

1. One primary metric

Name the single number that decides the question. Not a dashboard—one number, chosen in advance.

The reason is [Multiple Comparisons](#). Five metrics at $\alpha = 0.05$ give a 23% chance of a false positive under a complete absence of effect, and the write-up will report whichever one moved.

Secondary metrics are fine and useful. They are labeled secondary, they do not decide the ship, and they are reported whether or not they moved.

2. Guardrails

Guardrail metrics are the ones you are not trying to improve and are not willing to break: latency, error rate, unsubscribes, support tickets, revenue.

They exist because optimizing one number almost always costs another. A change that lifts click-through by making the button misleading will lift click-through. The guardrail is how you find out what it cost, and the decision rule is asymmetric—**you need evidence of improvement on the primary metric and absence of harm on the guardrails**.

3. The smallest effect worth acting on

Before computing any sample size you have to answer a business question: *how big would the effect have to be for us to do anything differently?*

This is the hardest step and the one most often skipped, because it is not a statistical question and there is no formula for it. It is also where most of the value of the exercise is—teams frequently discover at this point that no plausible effect would change the decision, which means the experiment need not be run at all.

4. Sample size, from power

Now it is arithmetic. A conversion rate is a proportion, so `power.prop.test()` does the work.

R Code	
1	# Baseline conversion 5%. How many per group to detect various improvements?
2	for (p2 in c(0.055, 0.06, 0.07)) {
3	n <- ceiling(power.prop.test(p1 = 0.05, p2 = p2, power = 0.8, sig.level = 0.05)\$n)
4	cat("5.0% ->", format(100 * p2, nsmall = 1), "% :", format(n, big.mark = ","),
5	"per group\n")
6	}
5.0% -> 5.5 % : 31,234 per group 5.0% -> 6.0 % : 8,158 per group 5.0% -> 7.0 % : 2,213 per group	

Detecting a lift from 5% to 6% takes 8,158 per group. Halving the target effect, to 5.5%, takes **31,234**—almost four times as many, which is the same k^2 relationship as everywhere else this term.

This is where the previous step earns its keep. If your traffic is 5,000 users a week and you need 62,000, the experiment takes three months, and that is worth knowing before you start rather than in week six.

GUESS THE BASELINE HONESTLY

`power.prop.test` needs the baseline rate, which you usually know from historical data. Take it from the same population, the same days of week, and the same measurement definition you will use in the test. A baseline borrowed from a different segment is the most common reason a carefully computed sample size turns out to be wrong.

5. A stopping rule, fixed in advance

Write down the sample size, and stop there.

The Oct 5 problems showed twenty looks at data with no effect whatsoever crossing 0.05 along the way. Checking daily and stopping when it looks good is not one test; it is as many tests as you took looks, and the false positive rate is far above the 5% you think you are paying.

There are legitimate sequential designs—group sequential methods, always-valid inference—that let you look repeatedly with the arithmetic adjusted. Use one, or fix n in advance. What is not available is looking repeatedly with the arithmetic of a single test.

6. Analysis, written down before the data

The analysis is two lines and there is no reason not to write them first.

R Code

```
1 # 24,000 users per arm. Control converts 4.67%, treatment 5.04%.
2 res <- prop.test(x = c(1210, 1120), n = c(24000, 24000), correct = FALSE)
3
4 round(c(treatment = 1210 / 24000, control = 1120 / 24000,
5         difference = (1210 - 1120) / 24000,
6         p_value = res$p.value,
7         ci_low = res$conf.int[1], ci_high = res$conf.int[2]), 5)
```

treatment	control	difference	p_value	ci_low	ci_high
0.05042	0.04667	0.00375	0.05594	-0.00009	0.00759

A lift of 0.375 percentage points, $p = 0.056$, with an interval running from -0.009 to $+0.76$ percentage points.

Report it that way. “Not significant” is a poor summary—the interval says the data are consistent with no effect and also with a lift of three quarters of a percentage point, and whether that ambiguity matters depends on the decision threshold you set in step 3.

EXERCISE: HOW LONG WILL THIS TAKE?

Your checkout page converts at 12%. The product team will act on an improvement of one percentage point or more. You want 80% power at $\alpha = 0.05$.

Return the required number per group, rounded up.

Exercise

```
1 # power.prop.test(p1 = , p2 = , power = ) solves for n per group.
2 -----
```

Please replace _____ with valid code.

The checklist

BEFORE YOU START

Primary metric	One, named in advance
Guardrails	What you refuse to break
Minimum effect	The smallest difference worth acting on
Sample size	From power, using an honest baseline
Stopping rule	Fixed n , or a sequential method
Analysis	Written down before the data exists

If you cannot fill in every row, the experiment is not ready to run—and the row that is usually blank is the third one.

Summary

- **One primary metric**, chosen in advance. Everything else is secondary and reported regardless.
- **Guardrails** catch what the improvement cost, with an asymmetric decision rule.
- Name **the smallest effect worth acting on** before computing anything. It is a business question, and answering it sometimes cancels the experiment.
- Sample size follows from power: 5%→6% needs 8,158 per group, 5%→5.5% needs 31,234.
- **Fix the stopping rule.** Peeking with single-test arithmetic is not a slightly optimistic analysis, it is a different false positive rate.
- Report the **difference and its interval**, not “significant” or not.

Next: what goes wrong once it is running.

Monday, October 19

Post your questions on any of these pages by **Monday at 12:00 noon**, the morning of class.

Everything so far has described data you were handed. This week is about data you **cause to exist**, and about the reason that distinction is worth a session of its own.

Every question anyone actually wants answered is causal—*does this work, should we ship it, did the change help*—and no amount of care applied to observational data answers one. Randomization does, and it is the only thing that does. This week is what it buys, how to run one properly, and the specific ways a well-designed experiment still ends up producing the wrong answer.

The machinery is small, because you already have it: a proportion and its standard error from [Intervals for a Proportion](#), and power from [Power](#). What is new is the design, and the discipline of writing it down before the data exists.

What is here

WHAT RANDOMIZATION BUYS

Self-selected groups differed by 1.62 on an unrecorded variable; randomized groups by 0.046. Why that matters more than any adjustment you could make, what randomization does not buy, and why you randomize the user rather than the session.

DESIGNING AN A/B TEST

Six decisions, all made before any data arrives: one primary metric, guardrails, the smallest effect worth acting on, the sample size that follows from it, a stopping rule, and the analysis written down in advance.

WHAT GOES WRONG

Sample ratio mismatch, checked before any outcome and invalidating the experiment when it fires. Then novelty and primacy, interference in marketplaces and social products, and a 0.05 percentage point lift with a p-value of 3×10^{-7} .

CHOOSING A TEST

The decision table, the paired test that found an effect the unpaired test missed on identical data, chi-squared for categorical association, and an honest note on why ANOVA is not in this course.

PRACTICE PROBLEMS

Including an experiment that looks clean and is broken, and a design that randomized sessions when it should have randomized users.

The sentence this week is aiming at

“We ran an experiment, and here is what it licenses us to claim.”

Half of that sentence is design and half is honesty about scope. A randomized experiment gives you a fair comparison among the people you ran it on—and that is a genuinely strong thing to have, provided you do not quietly extend it to people who were never in the study.

November takes up the harder case: what you can say when randomizing was never an option.

DUE THIS WEEK

Your project proposal. One page, mostly a question—see [S.M.A.R.T. Questions](#). And as always, post your questions on these pages by Monday noon.

Not graded, not auto-checked. Bring your answers and your disagreements.

Predict the output

R Code

```
1 # 1. What does randomization do to an unmeasured variable?
2 set.seed(5)
3 conf <- rnorm(5000)
4 g <- sample(c("A", "B"), 5000, replace = TRUE)
5 round(c(randomized = mean(conf[g == "A"]) - mean(conf[g == "B"]),
6         self_chosen = mean(conf[conf > 0]) - mean(conf[conf <= 0])), 3)
7
8 # 2. The cost of halving the effect you want to detect.
9 round(sapply(c(0.13, 0.125, 0.1225),
10             function(p2) ceiling(power.prop.test(p1 = 0.12, p2 = p2, power = 0.8)$n)))
11
12 # 3. Two ways to the same p-value.
13 c(prop = prop.test(c(300, 260), c(5000, 5000), correct = FALSE)$p.value,
14    chisq = chisq.test(matrix(c(300, 4700, 260, 4740), nrow = 2),
15                             correct = FALSE)$p.value)
```

```
randomized self_chosen
-0.005      1.606
[1] 17169 67496 267611
prop      chisq
0.08190776 0.08190776
```

DISCUSSION



This runs and is still wrong

R Code

```
1 # 4. A clean-looking experiment.
2 #   Intended 50/50. Observed:
3 chisq.test(c(20450, 19550), p = c(0.5, 0.5))
```

```
Chi-squared test for given probabilities

data:  c(20450, 19550)
X-squared = 20.25, df = 1, p-value = 6.795e-06
```

R Code

```
1 # 5. Randomizing the wrong unit.
2 set.seed(8)
3 user <- rep(1:200, each = 10)           # 200 users, 10 sessions each
4 u_level <- rnorm(200, sd = 4)           # users differ a lot from one another
5 y <- u_level[user] + rnorm(2000, sd = 1)
6 g <- sample(c("A", "B"), 2000, replace = TRUE) # assigned per SESSION
7
8 round(c(se_treating_sessions_as_independent = t.test(y ~ g)$stderr,
```

```

9  ✓      se_from_user_means = {
10      um <- tapply(y, user, mean); ug <- tapply(g == "A", user, mean) > 0.5
11      t.test(um ~ ug)$stderr
12      }) 4)

```

```

se_treating_sessions_as_independent
0.1950

```

```

se_from_user_means
0.6374

```

R Code

```

1  # 6. Reporting a result.
2  res <- prop.test(c(2040, 2000), c(40000, 40000), correct = FALSE)
3  round(c(lift_pp = 100 * (2040 - 2000) / 40000,
4          p = res$p.value, ci_pp = 100 * res$conf.int), 4)

```

```

lift_pp      p  ci_pp1  ci_pp2
0.1000  0.5184 -0.2035  0.4035

```

DISCUSSION



What does this license you to claim?

A team reports: *users in the treatment arm completed onboarding 8% more often, $p = 0.01$, over one week, with a 50.4/49.6 split across 90,000 users.*

7. “The new onboarding causes an 8% lift.”
8. “We should ship it.”
9. What would you check before believing any of it?
10. The effect was +14% on day one and +2% by day seven. What is that, and what does it do to the headline number?
11. The team also reports that support tickets rose 6%, “but not significantly.” How should that enter the decision?

DISCUSSION



Bring to class

12. **Design one.** Take your project question, or invent a product change. Write the six rows of the design checklist: primary metric, guardrails, minimum effect worth acting on, sample size, stopping rule, analysis. Bring the sample size and how long it would take at a realistic traffic level.
13. **Find an experiment in the wild.** A company engineering blog, a paper, a press release. What was randomized? What was the primary metric? Can you tell how many metrics were examined?
14. **Argue about this.** “*We cannot randomize—it would be unfair to give some users a worse experience.*” Work out when that objection is right, when it is an argument for randomizing sooner rather than later, and what [Where the Data Came From](#) says about the alternative.

Every question anyone actually cares about is causal. *Does the new onboarding flow increase retention? Does the drug work? Should we ship this?* None of those are questions about association, and all of them are routinely answered with an association and a hopeful tone.

This chapter is about the one procedure that closes the gap, why it works, and what it costs.

The problem, stated honestly

You want to compare outcomes under treatment with outcomes without it. For any individual you observe exactly one of those. The other is not missing data in the sense of [Where the Data Came From](#)—it is unobservable in principle, because that person either got the treatment or did not.

So every causal claim is a comparison between **different people**, and it is only as good as the claim that those groups were otherwise alike.

Why “otherwise alike” almost never holds

Suppose there is some characteristic that affects the outcome—motivation, severity, prior spend, whatever. Call it a confounder. Now compare groups formed by letting people choose.

R Code	
1	<code>set.seed(6101)</code>
2	<code>n <- 2000</code>
3	<code>conf <- rnorm(n)</code> # something that affects the outcome. We happen to see it here.
4	
5	<code># Self-selection: keener people opt in.</code>
6	<code>picked <- ifelse(conf > 0.2, "A", "B")</code>
7	<code>round(mean(conf[picked == "A"]) - mean(conf[picked == "B"]), 3)</code>
[[1]] 1.621	

The groups differ by 1.62 units on a variable nobody recorded. Any difference in outcome will be a mixture of the treatment and that gap, and the data cannot separate them.

Now assign at random instead.

R Code	
1	<code>set.seed(6101)</code>
2	<code>n <- 2000</code>
3	<code>conf <- rnorm(n)</code>
4	
5	<code>imbalance <- replicate(2000, {</code>
6	<code> g <- sample(c("A", "B"), n, replace = TRUE)</code>
7	<code> mean(conf[g == "A"]) - mean(conf[g == "B"])</code>
8	<code>})</code>
9	
10	<code>round(c(mean_imbalance = mean(imbalance),</code>
11	<code> sd_imbalance = sd(imbalance),</code>
12	<code> within_95pct = quantile(abs(imbalance), 0.95)), 4)</code>
mean_imbalance sd_imbalance within_95pct.95% -0.0010 0.0457 0.0876	

Centered on zero, with a spread of 0.046—**thirty-five times smaller than the self-selected gap**, and shrinking like $1/\sqrt{n}$ in the familiar way.

THE CLAIM RANDOMIZATION ACTUALLY SUPPORTS

Randomization does not make the groups identical. It makes them **identical in expectation on every variable at once**—including the ones you did not measure, did not think of, and could not have measured.

That last clause is the whole value. Statistical adjustment can only control for variables you have. Randomization handles the ones you do not, which are the ones that ruin observational studies.

What it does not buy

It does not fix a bad outcome measure. Randomizing perfectly and then measuring the wrong thing yields a clean estimate of something irrelevant.

It does not license generalization. A randomized experiment on your existing users tells you about your existing users. Randomization gives you *internal* validity—the comparison is fair. *External* validity is a separate question, and it is the [Where the Data Came From](#) question again.

It does not survive interference. The arithmetic assumes one person's assignment does not affect another's outcome. In a social product, a marketplace, or anywhere users interact, that fails—and it fails in the direction of making the treatment look better or worse than it is depending on which side of the market you touched.

It does not make the result large enough to matter. That is [Practical significance](#).

Randomize the unit that is independent

The commonest design error is randomizing the wrong thing.

If one user makes twenty visits and you randomize *visits*, the same person appears in both arms, the arms are correlated, and your standard error is too small for the reason [The Standard Error of a Mean](#) gave: n is the number of independent units, and fifty measurements on ten patients is not $n = 50$.

Randomize the user, not the session. If users interact in teams, randomize the team. The unit of randomization should be the unit you are willing to call independent, and it is usually larger than the unit you would like it to be.

✧ EXERCISE: WHAT DOES THE COMPARISON SUPPORT?

A company emails all customers offering a new premium tier. Three months later, customers who took it have 40% higher retention than those who did not.

Assign the letter, for example `answer <- "A"`.

Exercise

```
1 # A. The premium tier causes a 40% increase in retention.
2 # B. Nothing causal: customers chose the tier, so the groups differ in whatever
3 #   made them choose, and that same thing plausibly affects retention.
4 # C. The claim is fine as long as the sample is large enough.
5 # D. The claim is fine if you control for age, region and tenure.
6 answer <- _____
```

Please replace _____ with valid code.

Summary

- Every causal question compares outcomes you cannot both observe, so it becomes a comparison between **different people**.
- Self-selected groups differed by 1.62 on an unrecorded variable; randomized groups differed by 0.046. Randomization balances **every** variable in expectation, including the unmeasured ones.
- It buys **internal validity only**. Generalization, measurement, and interference are separate problems it does not touch.
- **Randomize the unit you are willing to call independent**—usually the user, sometimes the team, rarely the session.

Next: turning that into an experiment somebody can actually run.

The design is correct, the randomization is correct, and the experiment still produces the wrong answer. This chapter is the short list of ways that happens, in roughly the order you should check for them.

Sample ratio mismatch

Check this first, before looking at any outcome.

You assigned users 50/50. Count how many actually landed in each arm. If the split is off by more than chance, something in the pipeline is broken, and the outcome comparison is worthless no matter how good it looks.

R Code	
1	# Three experiments, all intended to be 50/50.
2	for (obs in list(c(49850, 50150), c(49000, 51000), c(48500, 51500))) {
3	p <- chisq.test(obs, p = c(0.5, 0.5))\$p.value
4	cat(sprintf("%6d / %6d (%.2f%%) p = %s\n",
5	obs[1], obs[2], 100 * obs[1] / sum(obs), format.pval(p, digits = 3)))
6	}
	49850 / 50150 (49.85%) p = 0.343
	49000 / 51000 (49.00%) p = 2.54e-10
	48500 / 51500 (48.50%) p = <2e-16

The first is ordinary sampling noise. The second is a **2% imbalance** and its p-value is 2.5×10^{-10} —at this scale, a discrepancy far too small to notice by eye is overwhelming evidence that assignment is broken.

That sensitivity is the point. SRM is the cheapest diagnostic you have and it catches the expensive failures: a redirect that drops users before logging, a bot filter applied to one arm, a caching layer serving control to returning visitors, an assignment hash that is not uniform.

AN SRM INVALIDATES THE EXPERIMENT

It does not mean “be careful interpreting this”. It means the mechanism that was supposed to make the groups comparable did not run as specified, so you no longer know what the groups are. Find the cause, fix it, and rerun.

The tempting move—dropping the excess users to restore balance—makes it worse, because whatever selected them is now doing the selecting.

Novelty and primacy

Users react to *change*, not only to the change.

Novelty: a new interface gets clicked because it is new. The effect decays. Measured in week one and extrapolated to a year, it is a fantasy.

Primacy: the opposite. Existing users are slowed down by an unfamiliar layout and the treatment looks worse than it is, until they adapt.

Both are diagnosed the same way: **plot the effect over time**, and look at new users separately from established ones. If the treatment effect is large on day one and gone by day fourteen, you have measured a reaction to novelty. If it is negative on day one and positive by day fourteen, you have measured primacy and should run longer.

This is also why “run the experiment for a full week” is standard advice—weekday and weekend users differ, and an experiment ending on a Wednesday has an unbalanced mix.

Interference

The arithmetic assumes one user's assignment does not affect another user's outcome. Common situations where it plainly does:

Marketplaces. Show treatment users better search results and they buy the scarce inventory, so control users see less of it. The treatment looks good partly by making control worse, and the measured difference overstates what happens when everyone gets it.

Social features. A user in treatment posts more; their friends in control see more content. The effect leaks into the control arm and the difference understates the truth.

Shared resources. A treatment that increases server load slows everyone down, including control.

The fix is to randomize a larger unit—by market, by region, by time period—so that interference happens *within* an arm rather than between them. It costs enormous power, because the number of independent units collapses from millions of users to a handful of markets, and that trade is the whole difficulty of experimentation in marketplaces.

Practical against statistical significance

The last failure is one of reporting rather than mechanism.

R Code				
1	# An enormous experiment on a trivial effect.			
2	big <- prop.test(x = c(505000, 500000), n = c(1e7, 1e7), correct = FALSE)			
3				
4	c(rate_treatment = 505000 / 1e7, rate_control = 500000 / 1e7,			
5	lift_pp = 100 * (505000 - 500000) / 1e7,			
6	p_value = signif(big\$p.value, 3))			
	rate_treatment	rate_control	lift_pp	p_value
	5.05e-02	5.00e-02	5.00e-02	3.09e-07

A lift of 0.05 percentage points, with a p-value of 3×10^{-7} . It is real, it is precisely measured, and it is almost certainly not worth the engineering cost of shipping it.

Statistical significance answers “is it there?”. **Practical significance answers “does it matter?”.** The second question is settled by the threshold you wrote down in step 3 of the design, before the data existed—which is the reason that step is not optional.

🔗 EXERCISE: CHECK FOR SRM

An experiment intended a 50/50 split and observed **12,412** users in control and **12,088** in treatment. Return the **p-value** of a chi-squared test against a 50/50 expectation.

Exercise	
1	# chisq.test() takes the observed counts and the expected proportions.
2	-----

Please replace ----- with valid code.

The order to check things

1. **SRM.** Before any outcome.
2. **The metric definition,** computed identically in both arms.
3. **Effect over time,** for novelty and primacy.

4. **Guardrails.**
5. **The primary metric**, with its interval.
6. **Practical significance**, against the threshold you set in advance.

Everything above the primary metric is a check on whether the experiment ran. Only once those pass does the estimate mean anything.

Summary

- **Sample ratio mismatch invalidates an experiment.** A 2% imbalance on 100,000 users gives $p = 2.5 \times 10^{-10}$; check it first, and never fix it by dropping users.
- **Novelty and primacy** are diagnosed by plotting the effect over time and splitting new from established users.
- **Interference** breaks the independence assumption in marketplaces, social products and shared infrastructure. The fix—randomizing markets or regions—costs most of your power.
- **Significant is not important.** A 0.05 percentage point lift on twenty million users had $p = 3 \times 10^{-7}$ and is probably not worth shipping.

Next: the machinery, and a decision table for choosing among tests.

Getting messy data into shape, and asking who is missing from it.

HOW THIS SESSION WORKS

This lab is ungraded—the only one all semester. Nothing here counts. Its purpose is to show you what every other Monday looks like, while the stakes are zero.

Tonight is Part B only, at the keyboard, in fours, with a checkpoint in the middle. We take it slowly, because the point is the format rather than the finishing.

Part A is the spoken half—a pair mock interview—and it is how every Monday from Aug 31 opens. Tonight you see one question of it demonstrated from the front instead of running it yourselves. The other three are below, and they are worth working through with a partner before Aug 31, when this half starts carrying a point.

You may not use an AI assistant in any lab, including this one. The honest reason is in Part A, and it is worth reading even though we are not running it tonight: interviews are moving toward asking whether you can catch what an assistant got wrong, which is not a thing you can do for work you could not have done yourself.

Part A—Discussion

NOT RUN TONIGHT

A1 is demonstrated from the front; the rest are yours to practice. From Aug 31 this half runs in pairs at the start of every lab and is one of the three points for the session. Read the format below so that none of it is new then, and work A2 to A4 with someone before that.

⌘ FORMAT: PAIR MOCK INTERVIEW

Work in pairs. One plays the **interviewer** and holds the rubric card; the other answers **out loud**, without typing. **Swap after each question**, so that across the four you each ask twice and answer twice. Do not read the rubric card when you are the one answering.

Holding the rubric is not the easy seat. Knowing what a good answer contains is harder than producing one, and it is the skill that makes you useful in a code review.

Questions are labeled by **role and stage**—the kind of screen they belong to—never by company. Attributions to named firms are usually wrong, stale, or under NDA, and a wrong attribution misleads whoever is preparing.

The five-minute clock is part of the simulation. Real interview answers are time-boxed, and an answer that arrives at minute nine is a different answer.

That is also the honest reply to *why can't I use AI in labs*, though the reply has changed. It used to be that no interview let you use one. As of this year some do: Google is piloting a coding round with an approved assistant, Meta and others are moving the same way, and about a quarter of employers in New York now permit it. What those interviews assess is not whether the assistant produces code. It is whether you catch what it got wrong—prompt, then validate, then debug. You cannot do that for work you could not have done yourself, which is what an unaided lab is for. The lab is not simulating a workplace. It is simulating the gate you have to pass to reach one.

Nothing in Part A is submitted.

A1. THE FIRST FIVE MINUTES

Phone screen, data analyst.

“A CSV lands in your inbox. 40,000 rows, 12 columns, no documentation, and your manager wants a number by Thursday. What do you do in the first five minutes, and why those things?”

 RUBRIC CARD—INTERVIEWER ONLY

A2. WHO IS NOT IN THIS FILE

Case-study round, data scientist.

“A team surveys its users and reports 85% satisfaction. Before I believe it, what do you want to know?”

 RUBRIC CARD—INTERVIEWER ONLY

A3. THE JOIN THAT LOST ROWS

Technical screen, analytics engineer.

“You join customers to orders and end up with fewer rows than you started with. Walk me through what happened and what you do about it.”

 RUBRIC CARD—INTERVIEWER ONLY

A4. DEFENDING A DELETION

Hiring manager round, any data role.

“In your last project, did you delete any rows? Convince me it was the right call.”

 RUBRIC CARD—INTERVIEWER ONLY

Part B—Build

ROLES IN THIS LAB

Groups of two or three, rotating from last week. Eight of us is four pairs, or three groups if you would rather be a trio. **Everyone is at a keyboard.** There is no single driver and no waiting for a turn: if you want to try something, try it.

One job still rotates, and it is the one that disappears when everybody is typing. The **Skeptic** answers, for every result the group is about to accept: *what would tell us this is wrong?* Hand it on at the checkpoint.

Typing in parallel is not the same as working in parallel. **Agree what each of you is trying before you try it, and compare answers instead of racing to one.** Three people who quietly write the same thing three times have got less done than one person who was being watched.

At the checkpoint I will pose a question, give you a minute to confer, and call on someone at random to answer for the group. Confer until everyone at your table could answer it.

HOW PART B WORKS

Every problem has a **self-check** cell below it. Run it after your answer. It asserts the properties your answer must have; it does not contain the answer, and passing it is necessary but not sufficient—a self-check that passes on nonsense is one you outwitted, not a correct answer.

You are not expected to know R yet. Nothing has been assigned, and the [R Primer](#) is what you start after tonight. Each cell names the functions you need in a comment, the self-check tells you which misconception you hit rather than only that you were wrong, and the worked solution is one click away. Use all three. The part of this lab that is actually hard is not the typing.

Cells share one R session, top to bottom. Run the setup cell first, then work in order.

Two halves with a **checkpoint** between them. If you are not through B3 when the checkpoint is called, say so—that is information, not a confession.

We may not reach B4 to B6 tonight, and that is fine. First days run long: someone's R will not install, someone will arrive late, the projector will do something. Whatever we do not get to is yours to finish this week, with the worked solutions on this page and the self-checks to tell you whether you have it. Nothing here counts toward anything.

Run this first. It builds the file you have been handed.

```
set.seed(6101)

n <- 240
clinic <- data.frame(
  patient = sprintf("P%04d", c(1:235, 231:235)), # note the tail
  site = sample(c("north", "North", "NORTH", "south", "South"), n,
    replace = TRUE, prob = c(.3, .1, .05, .4, .15)),
  age = c(sample(19:88, n - 4, replace = TRUE), -999, -999),
  wait_min = round(rlnorm(n, meanlog = 3.1, sdlog = 0.7), 1),
  score = sample(c(1:10, NA), n, replace = TRUE,
    prob = c(rep(.09, 10), .10))
)
clinic$wait_min[sample(n, 18)] <- NA
str(clinic)
```

'data.frame': 240 obs. of 5 variables:
\$ patient : chr "P0001" "P0002" "P0003" "P0004" ...
\$ site : chr "south" "South" "south" "north" ...
\$ age : num 61 63 32 58 55 41 86 47 54 54 ...
\$ wait_min: num 52.1 92.5 NA 18.6 25.3 56.8 14.4 42.7 8.4 59 ...
\$ score : int 2 1 8 9 NA 4 8 8 4 9 ...

```
head(clinic, 4)
```

	patient	site	age	wait_min	score
1	P0001	south	61	52.1	2
2	P0002	South	63	92.5	1
3	P0003	south	32	NA	8
4	P0004	north	58	18.6	9

First half

B1. TAKE INVENTORY BEFORE YOU TOUCH ANYTHING

Produce one table with a row per column of `clinic` giving its **type**, its **number of missing values**, and its **number of distinct values**.

Do not clean anything yet. The point is to know what you have.

Exercise

```
1 # sapply(clinic, f) runs f down each column and returns one value per column.
2 # class(x)[1] is the type; sum(is.na(x)) counts missing; length(unique(x)) counts
3 # distinct. data.frame() glues three such vectors into a table.
4 inventory <- _____
5
6 inventory
```

Please replace _____ with valid code.

B2. THE IDENTIFIER THAT IS NOT ONE

Return the number of `patient` values that appear **more than once**.

Then, before you touch it: decide as a group whether a repeated patient ID is a duplicate row to delete, or a patient with two visits. Your answer determines what you do next, and the data cannot tell you which it is.

Exercise

```
1 # table(x) counts how often each value occurs. You want how many of those
2 # counts are greater than one -- sum() of a logical vector counts the TRUEs.
3 n_repeated <- _____
4
5 n_repeated
```

Please replace _____ with valid code.

B3. FIVE SITES THAT ARE TWO

Create `clinic$site_clean`, a factor with exactly two levels, `north` and `south`.

Exercise

```
1 # trimws(x) drops leading and trailing spaces; tolower(x) lowers the case;
2 # factor(x) turns the result into a factor. All three, in an order you choose.
3 clinic$site_clean <- _____
4
5 table(clinic$site_clean, useNA = "ifany")
```

Please replace _____ with valid code.

⚠ CHECKPOINT

Stop typing. I will ask one question and call on someone at random.

Confer first: **you found four defects in this file. Which one would have been hardest to notice if you had gone straight to computing the average wait time—and which would have done the most damage?**

Second half

🔗 B4. SENTINELS ARE NOT DATA



`age` contains impossible values. Create `clinic$age_clean` with those converted to `NA`, and return nothing else changed.

Decide as a group what counts as impossible **before** you look at how many rows it removes.

Exercise

```
1 # ifelse(test, yes, no) picks element by element, so it can return the age
2 # where the test holds and NA where it does not.
3 clinic$age_clean <- _____
4
5 summary(clinic$age_clean)
```

Please replace `_____` with valid code.

🔗 B5. WHAT DID THE CLEANING COST



Return the **proportion of rows** that are complete on `age_clean`, `wait_min` and `score`—a number between 0 and 1.

Exercise

```
1 # complete.cases(df) gives one TRUE per row with nothing missing. The
2 # proportion of TRUEs in a logical vector is its mean().
3 kept <- _____
4
5 kept
```

Please replace `_____` with valid code.

🔗 B6. WHO DID YOU LOSE



The last question is the one that matters and it has no single right answer.

Compare the rows you **kept** with the rows you **dropped**. Are they alike? Use whatever comparison you think is fair—by site, by age, by wait time.

Then write one sentence, as a comment, that you would be willing to put in a report.

Exercise

```
1 complete <- complete.cases(clinic[, c("age_clean", "wait_min", "score")])
2
3 # Compare the two groups on something that matters.
4 -----
5
6 # One sentence you would put in a report:
7 #
```

Please replace ----- with valid code.

BEFORE YOU LEAVE

You did no statistics today. You did the part that decides whether the statistics will mean anything—and every decision you made in Part B was a judgment the data could not make for you.

That is the shape of the term.

Before you go, post a comment on any page of these notes. One line is enough—it does not have to be a good question tonight. It proves your GitHub account works with the comment system, which is how every week's reading credit is earned. A student who finds out on the morning of the first deadline that this does not work has already lost the week.

Then, for next Monday: read [Exploratory Data Analysis](#)—all three chapters, about an hour and forty minutes—and post your questions by **Monday at 12:00 noon**.

[Questions, and Data That Can Answer Them](#) is the part behind tonight, and it is worth reading in full this week. It carries no deadline and no reading questions. [Where the Data Came From](#) is the one to read first: it is what B6 was really asking, and we will come back to it next Monday from the front.

Start the [R Primer](#) this week and keep it at alongside the course; no session is ever spent on R. The [Reading Schedule](#) has every week laid out.

Exploratory analysis on a real dataset, and asking who is missing from it.

HOW THIS SESSION WORKS

This is the first session that counts. Last week we ran the build only and demonstrated the interview from the front; tonight both halves run for real. Nothing here is submitted—the score comes from what I see you doing.

Three points, one for each of:

Interview	You took both roles in Part A, and your partner signed your rubric card.
Discussion	You said something substantive at the checkpoint.
Build	You were working on Part B when I came round, and could say where you were stuck.

None of the three asks you to be right. **Hand your signed card in before you leave**—that card is the Interview point, and there is no other record of it. The two lowest sessions of the term are dropped.

The session opens with **question triage**, then **Part A**, spoken and away from the keyboard, in pairs. Eight of us is four pairs, so everyone gets both seats without anyone having to sit out a turn. **Part B** is at the keyboard in groups of two or three—all of you typing—with a checkpoint in the middle.

No AI assistants. Documentation and these notes are fine.

Question triage

Seven of you posted, all before the deadline. The seven questions turn out to be two questions:

- “The number and the picture disagree—which do I trust?” – five of you
- “Why is the formula shaped like that?” – the other two

1. A rule that fires when it shouldn’t
2. What correlation cannot see, and what can
3. Why $n - 1$, and why variance at all
4. What the box is actually showing; when an axis may be cut

1. A rule that fires when it shouldn’t

“The text said that a general rule is that if the mean is less than the median then that indicates a left skew, however, in the commute example the histogram doesn’t appear to be left skewed.”

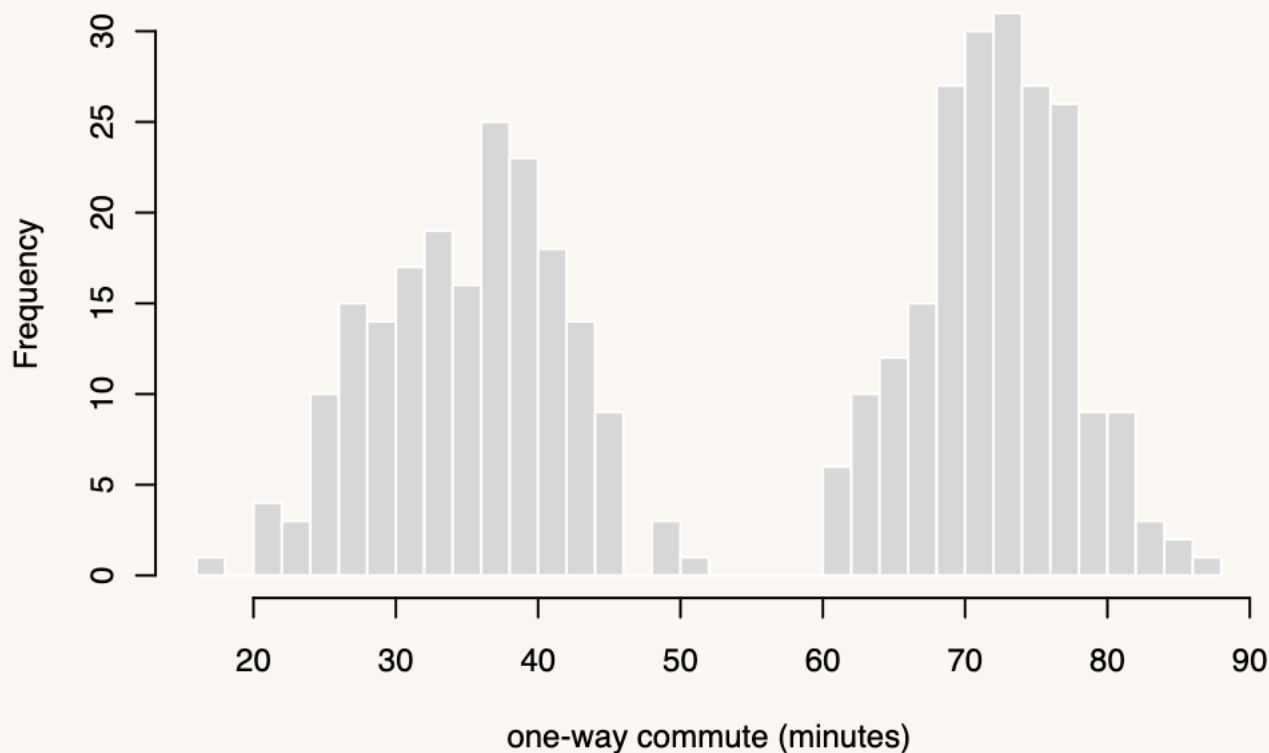
The chapter states the rule as *the single most useful diagnostic in this chapter* and then, four hundred lines later, builds a variable that breaks it. Both at once:

R Code

```
1 set.seed(6101)
2 grp <- sample(c(0, 1), 400, replace = TRUE)
3 commute <- rnorm(400, mean = ifelse(grp == 1, 72, 34), sd = 6)
4
5 c(mean = mean(commute), median = median(commute)) # mean < median: "left tail"
```

```
6 hist(commute, breaks = 30, col = "gray85", border = "white",
```

```
7     mean     median
  54.31841  62.33429
```



Where is the left tail? **The gap tells you the mean and the median disagree. It does not tell you why.** Skew is one reason; two populations is another, and it produces the same gap with no tail anywhere. The chapter has been corrected.

2. What correlation cannot see, and what can

“Can two variables have a covariance close to zero but still have a strong relationship if that relationship is nonlinear?”

“Is there a mathematical way of calculating a non-linear relationship similar to correlation or covariance, or is the best method visual?”

The first has a one-line answer: $r = \text{cov}(x, y) / (s_x s_y)$, and the two standard deviations are positive, so r and the covariance are **zero together**. They are one question.

The second is the one the chapter had no answer to. Run this:

R Code

```
1 x <- seq(-3, 3, length.out = 400)
2
3 rbind(monotone_curve = c(pearson = cor(x, exp(x)),
4                           spearman = cor(x, exp(x), method = "spearman")),
5       parabola      = c(pearson = cor(x, x^2),
6                           spearman = cor(x, x^2, method = "spearman")))
```

	pearson	spearman
monotone_curve	8.189781e-01	1.000000000
parabola	-2.108214e-16	-0.002328116

Spearman correlates the **ranks**, so it reports 1 for the perfect curved relationship that Pearson undersells at 0.82. Then look at row two. **Spearman buys you monotone curves and nothing more.** Distance correlation and mutual information go further; none of them tells you the shape.

3. Why $n - 1$, and why variance at all

“I get that we use $n-1$ because to do so otherwise would systematically understate the variance in the true population, but since when are we calculating the variance in the true population? I thought we were just doing calculations on the data from the sample.”

The chapter defends $n - 1$ given that you are estimating a population. It never argues the “given”, which is exactly what is being asked. If you only want to describe the numbers in front of you, divide by n and you are done—that is the population variance, and it is correct. You are almost never asking that.

“If variance and SD basically say the same thing about spread, why do we use variance at all instead of just always using SD?”

Because **variances add and standard deviations do not**. Every formula in the rest of this course—standard error, R^2 , ANOVA—is additive in the variance and has no clean form in the SD. Compute in variance, report in SD.

4. Two quick ones

“When interpreting IQR/SD, is it assessing for outliers or rather where the bulk of the data is?”

Three different jobs, and only the third is about outliers: the **IQR** is where the middle half sits; **IQR vs SD** is tail weight; **$1.5 \times \text{IQR}$ past a quartile** flags individual points.

“When is it acceptable to truncate an axis without exaggerating the difference?”

Not a rule to memorize—a test. **Is the reader meant to read a quantity off how long a mark is?** Bars yes, so zero. Lines and dot plots no, so cut freely and label the range.

You did not need to have posted to get something here—most of this is a confusion more than one person had. And post early rather than at 11:58: a question posted on Saturday can change what we do in the room; one posted at noon can only be answered in writing.

Part A—Discussion

🔗 FORMAT: PAIR MOCK INTERVIEW

One of you is the **interviewer** and holds the rubric card; the other answers **out loud**, without typing. Then swap. The rubric cards are collapsed—**do not open the one for the question you are answering.**

Questions are labeled by role and stage, never by company.

A1. MEAN OR MEDIAN

Phone screen, data analyst.

“When would you report a median instead of a mean? And is there a case where you would report neither?”

 RUBRIC CARD—INTERVIEWER ONLY

A2. A CORRELATION OF ZERO

Technical screen, data scientist.

“Two variables have a correlation of 0.02. Are they unrelated?”

 RUBRIC CARD—INTERVIEWER ONLY

A3. THE FLOOR IN THE DATA

Case-study round, data scientist.

“You are given a dataset of recorded earthquakes. The smallest magnitude in it is exactly 4.0, and 46 events sit at that value. What does that tell you?”

 RUBRIC CARD—INTERVIEWER ONLY

A4. EXPLAINING A DISTRIBUTION UPWARD

Hiring manager round, any data role.

“Describe a distribution to me in one sentence, as if I have no statistics background and have to make a decision from it.”

 RUBRIC CARD—INTERVIEWER ONLY

Part B—Build

ROLES IN THIS LAB

Groups of two or three, rotating from last week. Eight of us is four pairs, or three groups if you would rather be a trio. **Everyone is at a keyboard.** There is no single driver and no waiting for a turn: if you want to try something, try it.

One job still rotates, and it is the one that disappears when everybody is typing. The **Skeptic** answers, for every result the group is about to accept: *what would tell us this is wrong?* Hand it on at the checkpoint.

Typing in parallel is not the same as working in parallel. **Agree what each of you is trying before you try it, and compare answers instead of racing to one.** Three people who quietly write the same thing three times have got less done than one person who was being watched.

The Skeptic has the hardest job today. This dataset looks clean.

📄 HOW PART B WORKS

Self-check cells assert properties, not answers. Passing is necessary, not sufficient.

Cells share one R session, top to bottom. Two halves with a **checkpoint**.

`quakes` is built into R: 1,000 seismic events recorded near Fiji, with latitude, longitude, depth in km, magnitude, and the number of stations that reported each one.

```
data      (quakes)
str       (quakes)

'data.frame':  1000 obs. of  5 variables:
 $ lat      : num  -20.4 -20.6 -26 -18 -20.4 ...
 $ long     : num   182 181 184 182 182 ...
 $ depth    : int   562 650 42 626 649 195 82 194 211 622 ...
 $ mag      : num    4.8 4.2 5.4 4.1 4 4 4.8 4.4 4.7 4.3 ...
 $ stations: int    41 15 43 19 11 12 43 15 35 19 ...

summary   (quakes)

      lat      long      depth      mag
Min.   :-38.59  Min.   :165.7   Min.    : 40.0   Min.    :4.00
1st Qu.: -23.47 1st Qu.:179.6   1st Qu.: 99.0   1st Qu.:4.30
Median : -20.30 Median :181.4   Median :247.0   Median :4.60
Mean   : -20.64 Mean   :179.5   Mean   :311.4   Mean   :4.62
3rd Qu.: -17.64 3rd Qu.:183.2   3rd Qu.:543.0   3rd Qu.:4.90
Max.   : -10.72 Max.   :188.1   Max.    :680.0   Max.    :6.40
 stations
Min.    : 10.00
1st Qu.: 18.00
Median  : 27.00
Mean    : 33.42
3rd Qu.: 42.00
Max.    :132.00
```

Nothing is missing. No sentinel values, no duplicate keys, no misspelled categories. Last week's checklist finds nothing wrong with this file.

🔗 B1. DESCRIBE MAG HONESTLY



Return a named vector with the **mean**, **median**, **standard deviation**, **minimum** and **maximum** of `mag`.

Then, as a group, say one sentence about what kind of distribution those five numbers describe. Write it down. You will be asked to revise it.

Exercise

```
1 # c(mean = ..., median = ...) builds a named vector; mean(), median(),
2 # sd(), min() and max() supply the five numbers.
3 five <- _____
4
5 round(five, 3)
```

Please replace _____ with valid code.

🔗 B2. A COLUMN THAT LIES TO SUMMARY()



`depth` has mean 311 and median 247—a mild right skew, on those numbers.

Draw a histogram of `depth` with enough bins to see what is going on, and then return the **number of events within 50 km of the mean depth**.

Exercise

```
1 # A logical vector counts with sum(): how many depths sit within 50 km of the
2 # mean? abs(x - mean(x)) <= 50 is the test.
3 hist(quakes$depth, breaks = 30, col = "gray75", border = "white",
4       main = "Depth", xlab = "km")
5
6 near_mean <- _____
7
8 near_mean
```

Please replace _____ with valid code.

🔗 B3. FIND THE FLOORS



Return a named vector giving, for `mag` and for `stations`, the minimum value and how many events sit exactly on it.

Exercise

```
1 # min() gives the floor; sum(x == min(x)) counts how many sit exactly on it.
2 # c(mag_min = ..., mag_n = ..., stations_min = ..., stations_n = ...).
3 floors <- _____
4
5 floors
```

Please replace _____ with valid code.

🛑 CHECKPOINT

Stop typing. Confer, then I call on someone at random.

Revise the sentence you wrote in B1. You said something about the average earthquake. What is the correct subject of that sentence now, and roughly how many events do you think are missing from this file?

🔗 B4. THE RELATIONSHIP THAT EXPLAINS THE FILTER



Return the correlation between `mag` and `stations`, and plot one against the other.

Exercise

```
1 # cor(x, y) returns the correlation of two numeric vectors.
2 plot(quakes$mag, quakes$stations, pch = 19, col = rgb(0, 0, 0, 0.25),
3       xlab = "magnitude", ylab = "stations reporting")
4
5 r <- _____
6
7 round(r, 3)
```

Please replace _____ with valid code.

🔗 B5. DOES DEPTH RELATE TO MAGNITUDE

Compare mean magnitude across three depth bands: shallow (under 200 km), intermediate (200–400), and deep (over 400).

Return the three means as a named vector.

Exercise

```
1 # tapply(values, group, f) applies f to values within each level of group.
2 band <- cut(quakes$depth, breaks = c(0, 200, 400, 700),
3           labels = c("shallow", "intermediate", "deep"))
4
5 by_band <- _____
6
7 round(by_band, 3)
```

Please replace _____ with valid code.

🔗 B6. WHAT THIS FILE CANNOT TELL YOU

No single right answer. Write three bullet points, as comments, that you would put in the limitations section of a report using this data.

At least one must name something you can **sign**—a bias whose direction you can state even though you cannot measure it.

Exercise

```
1 # Three limitations, each a full sentence a reader could act on.
2 limitations <- c(
3   "",
4   "",
5   ""
6 )
7
8 limitations
```

```
[1] "" "" ""
```

Expected three limitations as a character vector, each a full sentence a reader could act on – not a phrase. `limitations <- c("...", "...", "...")`

BEFORE YOU LEAVE

The file had no missing values, no bad types, and no duplicates. Every defect today was in what the file **is**, not in how it was written—and none of it was visible in `summary()`.

Next week the question changes from *what is in front of me* to *what could this have been if I had collected it again*. Read [Probability and Random Variables](#) before Sep 14.

Simulating probabilities before computing them.

HOW THIS SESSION WORKS

Part A is spoken, in pairs. **Part B** is at the keyboard in threes, with a checkpoint in the middle.

One rule specific to today: **for every problem in Part B, write your predicted answer as a comment before you run anything.** Being wrong cheaply is the entire mechanism by which a mental model gets built, and today is designed around it.

No AI assistants.

Part A—Discussion

🔗 FORMAT: PAIR MOCK INTERVIEW

One is the **interviewer** and holds the rubric card; the other answers **out loud**. Then swap.

🔊 A1. TWO WORDS PEOPLE SWAP

Phone screen, quantitative role.

“Mutually exclusive and independent—what is the difference? And can two events be both?”

📋 RUBRIC CARD—INTERVIEWER ONLY

🔊 A2. THE POSITIVE TEST, IN SYMBOLS

Technical screen, data scientist. Asked more often than any other probability question, and most often lost in the first ten seconds.

“A disease affects 1 in 1,000 people. A test is 99% accurate in both directions. You test positive. What is the chance you have it?”

Don’t give me a number yet. First, on paper, write the question as probability statements: name your events, tell me what each number I said is a probability of, and write down exactly what I asked for.”

📋 RUBRIC CARD—INTERVIEWER ONLY

🔊 A3. WHEN YOU CANNOT DO THE ALGEBRA

Case-study round, data scientist.

“You need a probability and the derivation is beyond you, or beyond the time you have. What do you do?”

📋 RUBRIC CARD—INTERVIEWER ONLY

A4. WHICH RULE LICENSES EACH STEP?

Technical screen, quantitative role. The interviewer writes a derivation on the board and asks you to defend it.

“Two fair dice. Let A be the first die shows a 6 and B be the second die shows a 6. Below, $P(\text{at least one six})$ is computed two ways, one equality at a time. For every numbered step, tell me which axiom or rule allows it, and whether it also leans on an assumption about the dice.”

First way.

$$P(\text{at least one six}) = P(A \cup B) \quad (1)$$

$$= P(A) + P(B) - P(A \cap B) \quad (2)$$

$$= P(A) + P(B) - P(A)P(B) \quad (3)$$

$$= \frac{1}{6} + \frac{1}{6} - \frac{1}{6} \cdot \frac{1}{6} \quad (4)$$

$$= \frac{11}{36} \quad (5)$$

Second way.

$$P(\text{at least one six}) = 1 - P(\text{no six}) \quad (6)$$

$$= 1 - P(A^c \cap B^c) \quad (7)$$

$$= 1 - P(A^c)P(B^c) \quad (8)$$

$$= 1 - (1 - P(A))(1 - P(B)) \quad (9)$$

$$= 1 - \frac{5}{6} \cdot \frac{5}{6} = \frac{11}{36} \quad (10)$$

 RUBRIC CARD—INTERVIEWER ONLY

Part B—Build

ROLES IN THIS LAB

Groups of two or three, rotating. **Everyone is at a keyboard**—no single driver, no waiting for a turn. One job rotates: the **Skeptic** owns predictions, and before any cell is run states the group’s expected answer out loud.

Today the Skeptic is graded on being wrong usefully, not on being right.

HOW PART B WORKS

Every problem: **predict, then run**. Self-checks assert properties, not answers.

One R session, top to bottom. Two halves with a **checkpoint**.

```
set.seed(6101)
cat("R", R.version.string, "\n")
```

R R version 4.5.1 (2025-06-13)

First half

B1. AT LEAST ONE SIX



Roll four dice. Estimate the probability of **at least one six** by simulating 50,000 times.

Predict first. Most people say 4/6.

Exercise



```
1 # Prediction: _____
2
3 p_sim <- _____
4
5 round(p_sim, 4)
```

Please replace _____ with valid code.

B2. THE BIRTHDAY PROBLEM



In a room of 23 people, estimate the probability that **at least two share a birthday**. Ignore leap years.

Predict first, and write down the number. This class has fewer than 23 people in it, so you can check the answer against the room.

Exercise

```
1 # Prediction: _____
2
3 p23 <- _____
4
5 round(p23, 4)
```

Please replace _____ with valid code.

Three doors: a car behind one, goats behind the other two. The game always runs the same way.

1. You pick a door.
2. The host, who knows where the car is, opens one of the *other two* doors, and always one with a goat behind it. If both of those hide goats, because you picked the car, he chooses between them at random.
3. He then offers you a choice. **Staying** means keeping the door you picked. **Switching** means giving it up for the one door still closed: the door that is neither your pick nor the one he opened.

You win if the door you end with hides the car.

`play()` below plays one game: `play(FALSE)` stays, `play(TRUE)` switches, and each returns `TRUE` for a win. Estimate the win probability for **staying** and for **switching**, over 30,000 games each, and return the two as a vector, staying first.

Then, on paper, work out both probabilities exactly: condition on whether your first pick hides the car, say what staying and switching give in each case, and check that the simulation agrees.

Predict first, as a group, and commit to it out loud before running anything.

Exercise

```

1 # Prediction - stay: _____ switch: _____
2
3 play <- function(switch_door) {
4   car <- sample(3, 1)
5   pick <- sample(3, 1)
6   # The host opens a door that is neither your pick nor the car.
7   options <- setdiff(1:3, c(car, pick))
8   host <- if (length(options) == 1) options else sample(options, 1)
9   final <- if (switch_door) setdiff(1:3, c(pick, host)) else pick
10  final == car
11 }
12
13 results <- _____
14
15 round(results, 4)

```

Please replace _____ with valid code.

⚡ CHECKPOINT: WHY SWITCHING WINS, STEP BY STEP

Stop typing. Your simulation says staying wins about a third of the time and switching about two thirds. Below is the calculation that says why. **As a group, work through every step until anyone at your table could explain any one of them.** Then I call on a group and name a step, and whoever I point to explains it to the class.

Let W be *your first pick hides the car*.

1. $P(W) = \frac{1}{3}$, whatever rule you use to pick a door.
2. $P(W^c) = \frac{2}{3}$.
3. Given W , staying wins and switching loses, whichever goat the host shows.
4. Given W^c , switching wins and staying loses.
5. $P(\text{win by staying}) = 1 \cdot P(W) + 0 \cdot P(W^c) = \frac{1}{3}$, and $P(\text{win by switching}) = 0 \cdot P(W) + 1 \cdot P(W^c) = \frac{2}{3}$.
6. Now change one thing: the host does **not** know where the car is. He opens one of the other two doors with equal chance, and happens to show a goat; call that G . Then $P(G | W) = 1$, $P(G | W^c) = \frac{1}{2}$, $P(G) = \frac{2}{3}$, and $P(W | G) = \frac{1}{2}$: staying and switching are even.

For each step, be ready to say **which rule or fact it uses** and **why it holds here**. For step 6, also say **which of steps 3 and 4 stops being true**, and why.

Solution. Facilitator notes. What a good explanation of each step contains.

1. **Independence, then total probability.** You pick without knowing where the car is, so your pick is independent of the car's position, and $P(W) = \sum_i P(\text{pick door } i) \cdot \frac{1}{3} = \frac{1}{3}$ however the picks are weighted. Listen for "because there are three doors" with nothing about independence: a player who could see the car would not have $P(W) = \frac{1}{3}$.
2. **The complement rule.**
3. **The host's choice cannot matter.** Both doors he might open hide goats, so after he opens one, the other closed door hides a goat too.
4. **The host is forced.** Of the two doors that are not your pick, one hides the car and he may not open it, so he opens the other goat door and the one closed door left hides the car. This is the step that carries the whole result.
5. **The law of total probability**, with W and W^c as the two cases.
6. **Step 4 breaks.** A random host is not forced: when your pick hides a goat he reveals the car half the time, and the condition "he showed a goat" throws those games away. That is where $P(G | W^c) = \frac{1}{2}$ comes from, and Bayes' rule gives $P(W | G) = \frac{1 \cdot \frac{1}{3}}{\frac{2}{3}} = \frac{1}{2}$. Strong groups say it in words: a goat is certain when your door hides the car but a coin flip when it does not, and that pulls the odds back to even.

The one sentence to leave on the board: **switching wins exactly when your first pick was wrong, because the host's rule forces him to reveal the other goat.**

Second half

🔗 B4. BUILD THE POPULATION, THEN COUNT



Simulate 200,000 people. One in a thousand has the disease. The test is right 99% of the time in both directions.

Return the proportion of **positive testers** who actually have it.

Do not use Bayes' theorem. Build the population and count.

Exercise

```
1 # Prediction: _____
2
3 N <- 200000
4 disease <- runif(N) < 0.001
5 positive <- ifelse(disease, runif(N) < 0.99, runif(N) < 0.01)
6
7 ppv <- _____
8
9 round(ppv, 4)
```

Please replace _____ with valid code.

🔗 B5. CHECK INDEPENDENCE BY COUNTING



Roll two dice, 200,000 times. Let A be the sum exceeds 7 and B be the first die shows 6.

Return $P(A \cap B)$ and $P(A)P(B)$ as a named vector of two numbers, and say whether the events are independent.

Exercise

```
1 # Prediction – independent? _____
2
3 d1 <- sample(6, 200000, replace = TRUE)
4 d2 <- sample(6, 200000, replace = TRUE)
5
6 check <- _____
7
8 round(check, 4)
```

Please replace _____ with valid code.

BEFORE YOU LEAVE

Every answer today came from playing a game many times and counting. No formula was required, and in two of the five cases the formula would have been harder than the simulation.

Next week the game becomes *draw a sample and take its mean*, played thousands of times—and the thing you count is how much the mean moves. Read [Expectation, Variance, and the Standard Error](#) before Sep 21.

Watching $\sigma/\sqrt{n}$ emerge by simulation.

HOW THIS SESSION WORKS
Part A on paper, in your group. Part B at the keyboard, in threes, with a checkpoint.

Part B is built backwards on purpose. You will **measure** the standard error before anybody writes the formula down, and the formula will arrive as a description of something you already watched happen.

No AI assistants.

Part A—Discussion

🔗 **FORMAT: WORK IT OUT TOGETHER**
No interviewer and no cards tonight. Work these **in your group**, on paper, and argue until everyone can explain the answer. I will call on someone at random to give the group’s answer.

Each problem carries its **final answers**, folded away. Work it first, then open it and check—if you disagree with it, that is the interesting case and I want to hear it.

A1 to A6 are last week’s material—what kind of variable it is, then expectation and variance by hand. They are here because tonight is one long statement about the variance of a mean, and you cannot watch that happen without being able to compute one.

🔊 **A1. DISCRETE OR CONTINUOUS?**

Before any arithmetic, agree on what kind of random variable each of these is, and say what decided it:

- a. The number of support tickets one customer opens in a month.
- b. The time a visitor spends on a page.
- c. A commute time, recorded to the nearest minute.
- d. The proportion of 400 surveyed residents who answer yes.

📝 **ANSWERS**

📄 **A2. A SCRATCH CARD, EXACTLY**

The card from the reading costs \$2 and pays a prize X :

PRIZE	\$0	\$1	\$5	\$20
Probability	0.70	0.20	0.08	0.02

- a. $E[X]$.
- b. $E[X^2]$, then $\text{Var}(X)$ and $\text{SD}(X)$.
- c. Your **profit** on one card is $X - 2$. Its expectation and variance?
- d. The shop sells 10,000 cards tonight. What is the standard deviation of the shop’s **average** profit per card?

📝 **ANSWERS**

A3. WHICH OF THESE ARE DENSITIES?

For each, decide whether it is a valid probability density. If it is not, say **which requirement fails**, and whether multiplying by a constant would fix it.

- $f(x) = 3x^2$ on $[0, 1]$
- $f(x) = x - \frac{1}{2}$ on $[0, 2]$
- $f(x) = \sin x$ on $[0, \pi]$
- $f(x) = 1/x^2$ on $[1, \infty)$

 ANSWERS

A4. A CONTINUOUS ONE, BY INTEGRATION

A machine is busy for a fraction X of each shift, with density $f(x) = 2x$ on $[0, 1]$ —busier shifts are more likely than idle ones.

- Check that f is a density.
- $E[X]$ and $E[X^2]$.
- $\text{Var}(X)$ and $\text{SD}(X)$.
- Over 50 shifts, what is the standard deviation of the **average** busy fraction?

 ANSWERS

A5. ONE TRIP, TWO TRIPS, TWICE THE TRIP

A delivery leg has distance D with $E[D] = 12$ km and $\text{SD}(D) = 3$. Cost in dollars is $C = 4 + 2.5D$.

- $E[C]$, $\text{Var}(C)$ and $\text{SD}(C)$.
- Two **independent** legs, $D_1 + D_2$: expectation and variance.
- Their **average**, $(D_1 + D_2)/2$: expectation and variance.
- Compare $2D$ with $D_1 + D_2$. Same expectation—are they the same variable? Which has the larger spread, and why?

 ANSWERS

A6. WHEN THE TWO ARE NOT INDEPENDENT

$E[X] = 3$, $E[Y] = 5$, $\text{Var}(X) = 4$, $\text{Var}(Y) = 9$, and $\text{Cov}(X, Y) = -2$.

- $E[X + Y]$. Did you need the covariance?
- $E[2X - 3Y + 1]$.
- $\text{Var}(2X - 3Y + 1)$ and its SD.
- What would the variance have been if X and Y were independent?

 ANSWERS

A7. IS IT I.I.D.?

Tonight's formula assumes the draws are **independent and identically distributed**. For each of these, decide whether that holds—and if it does not, say **which half fails**:

- Twenty rolls of a fair die.
- One thousand sessions, from forty users, twenty-five sessions each.
- The temperature at noon on thirty consecutive days.
- Ten students drawn, one at a time and without replacement, from a class of fifty.
- Dial a random number; if someone picks up, survey everyone in that family.
- Dial random numbers, but hang up without surveying anyone under 30.

ANSWERS

Part B—Build

ROLES IN THIS LAB

Groups of two or three, rotating. **Everyone is at a keyboard**—no single driver, no waiting for a turn. One job rotates: the **Skeptic** answers, for every result the group accepts, *what would tell us this is wrong?*

Today the Skeptic has one extra job: after B2, the Skeptic states the pattern the group thinks it has found, **before** anyone checks it against a formula.

HOW PART B WORKS

Self-checks assert properties, not answers. One R session, top to bottom. Two halves with a **checkpoint**.

Nothing here uses a formula until B3. Resist looking one up.

```
set.seed(6101)
# A population you can see all of, which is the one luxury simulation gives you.
population <- rnorm(1e6, mean = 100, sd = 12)
round(c(true_mean = mean(population), true_sd = sd(population)), 3)
```

true_mean	true_sd
99.995	12.011

First half

B1. HOW MUCH DOES A MEAN MOVE



Draw 5,000 samples of size **16** from `population`, take the mean of each, and return the **standard deviation of those 5,000 means**.

Predict it first. The population SD is 12.

Exercise



```
1 # Prediction: _____
2
3 se16 <- _____
4
5 round(se16, 4)
```

Please replace _____ with valid code.

B2. FIND THE PATTERN



Repeat B1 at $n = 4, 16, 64, 256$. Return a named vector of the four spreads.

Then—**Skeptic, out loud, before checking anything**—state the rule connecting n to the spread.

Exercise

```
1 # Prediction for the pattern: _____
2
3 spread <- _____
4
5 round(spread, 3)
```

Please replace _____ with valid code.

B3. DOES IT CARE WHAT THE POPULATION LOOKS LIKE



The population above was Normal. Repeat B2 with a strongly right-skewed population—`rexp(n, rate = 1)`, which has mean 1 and SD 1.

Return the four spreads.

Exercise

```
1 # Prediction: does the pattern survive? _____
2
3 spread_skew <- _____
4
5 round(spread_skew, 3)
```

Please replace _____ with valid code.

Follow-up. B4 takes this apart: the rule needs a **finite** variance, and you are about to meet two populations that do not have one.

⊞ B4. WHEN THERE IS NO σ TO DIVIDE

$\sigma/\sqrt{n}$ needs a **finite** σ . The Cauchy distribution does not have one—it has no variance, and no mean either.

The setup gives you `spread(rng)`, which is exactly B3's measurement: draw 2,000 samples at each of $n = 4, 16, 64, 256$, take their means, and report the SD of those means. Build `spreads` with one row per population—`rexp(n, rate = 1)` then `rcauchy(n)`—using `rbind`.

Exercise

```
1 # What does the Cauchy row do that the exponential row does not? _____
2
3 spreads <- _____
4
5 round(spreads, 3)
```

Please replace _____ with valid code.

Run the cell a second time. Does the Cauchy row repeat?

⚠ PRACTICAL NOTE: WHEN DOES THIS ACTUALLY HAPPEN?

Not often with heights or commute times. It happens wherever a single observation can outweigh everything you have already collected:

- **Money with a tail.** Insurance claims, trading losses, damages awarded, revenue per customer. One claim can exceed the sum of the other ten thousand.
- **Sizes that grow by multiplying.** City populations, firm sizes, file sizes, traffic per flow. These are the power laws, and when the tail index is 2 or below, the variance is infinite.
- **Ratios.** Anything of the form X/Y where Y can land near zero is Cauchy-like—cost per click when clicks are few, a slope divided by another slope, an estimate divided by a noisy baseline.

How you notice it in your own data: the sample SD keeps climbing as rows arrive rather than settling; the mean jumps when one record lands; and dropping the single largest observation moves the answer more than you are comfortable with. Plot the running mean and the running SD—if neither flattens, stop quoting $\sigma/\sqrt{n}$.

What to do instead: report a median, with an interval built from percentiles. The mean and the SD are **moments**—averages of X and of $(X - \mu)^2$ —and a heavy enough tail leaves them infinite or undefined, which is what you just watched. A median is read straight off the sorted data and survives any tail at all.

⌘ CHECKPOINT

Stop typing. Confer, then I call on someone at random.

Why $\sqrt{n}$ and not n ? A complete answer says what quantity actually adds when you sum independent things—and where, in the four lines of that argument, independence is used.

Second half

🔗 B5. ONE SAMPLE, TWO NUMBERS

You will not usually have a population to sample from repeatedly. From **one** sample of 150, return both the SD of the data and the SE of the mean.

Then answer, as a group: a city asks “*how much do commutes vary across residents?*” Which number answers it?

Exercise

```
1 one_sample <- sample(population, 150)
2
3 two <- _____
4
5 round(two, 4)
```

Please replace _____ with valid code.

🔗 B6. SIZE YOUR OWN STUDY

No single right answer.

Take a quantity from your project—or invent one. Decide, as a group, **how precisely you need to know its mean**, then compute the sample size that delivers it.

You will need a guess at σ . Say where your guess came from; that assumption is doing as much work as the arithmetic.

Exercise

```
1 # The quantity:
2 # The precision we need (in its units):
3 # Our guess at sigma, and where it came from:
4
5 n_needed <- _____
6
7 n_needed
```

Please replace _____ with valid code.

BEFORE YOU LEAVE

You measured $\sigma/\sqrt{n}$ before you were given it, watched it hold for a population shaped nothing like a bell curve, saw it fall apart when the variance is infinite, and then used it to size a study of your own.

Next week you find out what **shape** the sampling distribution has—which is the last missing piece before those numbers become a statement. Read [The Central Limit Theorem and Confidence Intervals](#) before Sep 28.

Where the data came from, and what the summaries hide.

Assigned	Monday, Aug 31
Due	Sunday, Sep 13, 11:59 p.m. , on Blackboard
Weight	One of eight assignments; homework is 20% of the grade and no score is dropped
AI	Encouraged , with the disclosure described at the end
Extensions	48 hours, no reason needed, twice in the term. Say in the submission that you are using one.

You have two weeks because Labor Day falls in the middle. You do not need two weeks. Start it the weekend after class, while the lab is still fresh.

What you submit

Two files, both uploaded to Blackboard:

1. `hw1-lastname.Rmd` — your R Markdown source.
2. `hw1-lastname.html` — the knitted output.

Download the starter file: `hw1-template.Rmd` — the headings, the YAML, and the data-loading chunk, so that none of your time goes on scaffolding. It is deliberately thin: deciding what code belongs under each heading is part of what is being graded. Rename it with your last name.

Your `.Rmd` must knit **in a fresh R session** on a machine that is not yours. That is the two-computer test from the [R Primer](#), and it is the most common way this assignment loses points: a chunk that works because something is still sitting in your environment from an hour ago. Restart R, knit, and read the output before you upload it.

EVERY NUMBER IN YOUR PROSE MUST BE COMPUTED, NOT TYPED

If you write “about 2.8% of flights have no arrival time,” that percentage must be produced by inline R code in the sentence itself – backtick, `r`, your expression, backtick – not read off the console and typed. This is the [self-updating-sentence](#) exercise, applied for real. A typed number that disagrees with your own code is the single easiest error for me to find, and I do look.

The data

```
install.packages("nycflights13") # once
library(nycflights13)
data(flights)
```

`flights` is every flight that **departed one of the three New York City airports in 2013**: 336,776 rows, 19 columns. Read `?flights` before you touch it. Nothing here needs `dplyr`; base R is fine, and so is the tidyverse if you prefer it. Plots may be base R or `ggplot2`.

You may not use a different dataset. Everyone works on the same file so that the write-ups are comparable, and so that a claim you make can be checked.

Part 1 · The question (8 points)

Write **one S.M.A.R.T. question** that this file could actually answer, and then write **one that it could not**, in the same subject area.

For each, one short paragraph. For the second, the paragraph must say *what specifically about this file* blocks it — not “the data is limited,” but the column, the population, or the year that is in the way.

No code in this part.

Part 2 · One variable, honestly (13 points)

Take `arr_delay`, arrival delay in minutes.

- Report its mean, median, standard deviation, minimum, maximum, and IQR.
- The mean and the median disagree. Say which is larger, what that implies about the shape, and then **check the implication against a histogram** rather than asserting it. Show the histogram.
- Compute the proportion of flights that arrive *earlier* than the mean arrival delay. Write one sentence explaining to a non-statistician why that number is not a mistake.
- Compare the IQR to the standard deviation. What does that ratio tell you that neither number tells you alone? One sentence.

Part 2 is deliberately the same shape as B1 in Lab 2. The difference is that this variable has a genuine long tail, and `quakes` did not.

Part 3 · The flights that are not there (23 points)

`arr_delay` has missing values. This part is the centre of the assignment.

- How many rows have a missing `arr_delay`? What percentage is that?
- Missingness is a mechanism, not a nuisance.** Some of those rows are missing `dep_delay` as well, and some are not. Count both groups. Then, using the other columns, work out what physically happened to each group — what kind of event produces a missing arrival time *and* a missing departure time, and what kind produces only the former. Name both.
- `mean(flights$arr_delay, na.rm = TRUE)` is the line almost everyone writes. Say precisely what population that number describes. Your sentence should start “The average arrival delay of...” and the part after “of” is the whole point of this question.
- Now the consequence. Compute, for each carrier, both the **mean arrival delay** among the flights that have one and the **proportion of that carrier’s flights with no arrival time at all**. Put them side by side, with the number of flights per carrier. Then answer:

Does `na.rm = TRUE` treat all carriers equally?

Support the answer with a number, not an impression.

- One paragraph: if you were ranking carriers for a customer-facing “on-time” score, what would you do about the rows dropped in (c)? There is more than one defensible answer. An answer that says what it costs is worth more than one that claims to be free.
-

Part 4 · A ranking that is not what it looks like (18 points)

- Rank the carriers by mean arrival delay, worst first. Show the table.
- Look at the top five, and at how many flights each of those carriers actually operated out of New York in 2013. **At least one of them should stop you.** Say which, and why.

- c. This is the same failure as one of the four panels in [What correlation cannot see](#). Say which panel, and what the two cases have in common. Two or three sentences.
- d. Propose a rule — a minimum number of flights, an interval, a shrunken estimate, anything you can defend — that would stop the table in (a) from misleading a reader. Apply it and show the revised ranking. You are not expected to know the standard solution to this; you are expected to notice that the problem exists and do something honest about it.
-

Part 5 · A plot that misleads, and its honest twin (13 points)

Produce **two plots of the same comparison**, side by side or one after the other:

- One built to **overstate** the difference between carriers. Legal moves only: choose the axis range, choose the mark, choose which carriers appear, choose the aspect ratio. You may not alter a number.
- One that represents the same comparison **honestly**.
- Then, in one paragraph, name every choice you changed between them and say which single change did the most work. Apply the test from [Visualization That Does Not Mislead](#): is the reader meant to read a quantity off how long a mark is?

Both plots need axis labels and a caption a stranger could read.

Part 6 · Who is missing (15 points)

A colleague reads your write-up and says:

“Great — so now we know the average flight delay in the United States.”

Write **one paragraph, at most 150 words**, correcting them.

Name the three most important ways this file is not a sample of US flights, and for each one say which direction it pushes the answer — bigger, smaller, or unknown — rather than only that it is a problem. “Unknown” is a legitimate and sometimes the most honest answer, but say why it is unknown.

This paragraph is the one I read most carefully. It is [Where the Data Came From](#) applied to a file you have now spent several hours inside.

AI disclosure (required)

Homework is the component where AI use is **encouraged**. It is not the component where it goes undeclared.

Include, at the end of your knitted document, a section titled `## AI disclosure` containing:

- The prompt log.** Every prompt you sent, and enough of what came back to show what you were working from. A link to a shared conversation is fine.
- A critique**, 150-300 words. What the critique has to show is that **you interrogated the assistant rather than accepting it**. There are two ways to show that, and they are worth the same:
 - You caught something.** Name the part it happened in, what the assistant said, and what the consequence would have been had you accepted it.
 - You pushed and it held up.** Show the prompts you used to test it – the ones where you tried to make it fail – and say what you were probing for. “I asked it three times what `na.rm = TRUE` was excluding and it was right every time” is a full-marks answer, and it is a more interesting result than a caught error.

“I did not use an AI assistant on this assignment” is always an acceptable answer, and needs no justification. Write exactly that and you have full marks on the disclosure.

WHAT SCORES ZERO, AND WHAT DOES NOT

Zero is for not having interrogated the assistant – not for the assistant having done well. A critique that says only “it was helpful and accurate” scores zero because it reports a verdict with no test behind it. The same student who adds the three prompts they used to try to break it scores full marks.

This distinction matters and it is deliberate: **you are never worse off for telling me the truth**. If you used an assistant and it performed well, say so and show the pushing. Do not omit it, and do not invent an error that did not happen.

Where they reliably do go wrong on this dataset, if you want somewhere to push: Parts 3(b), 4(b) and 6. They will hand you `na.rm = TRUE` without a word about what it drops, rank sixteen carriers without noticing that one flew thirty-two times, and describe this file as US flight data.

A fabricated log is falsification of data under the Academic Integrity Code and is reported. The bar for full marks here is honesty, not sophistication.

How the critique is scored

	POINTS	WHAT IT LOOKS LIKE
Absent	0	No disclosure section, or a log with no critique. Also: no declaration either way.
Generic	3	True of any assistant on any assignment. “It sometimes invents function names.” No part of <i>this</i> assignment named.
Specific	7	Names a real error or a real test, in a named part, on this dataset. Says what happened.
Consequential	10	The above, plus what it would have cost – which number would have been wrong, which claim unsupported – or a probe that targeted something the assignment is actually about.

A declaration of no AI use scores **10**.

How it is graded

PART	POINTS	WHAT EARNS THEM
1 · The question	8	The second question names a specific obstruction in <i>this</i> file
2 · One variable	13	The shape claim is checked against a picture, not asserted
3 · Missingness	23	Both mechanisms named; (c) says who the average is <i>of</i> ; (d) is answered with a number
4 · The ranking	18	The small-n carrier is caught, and (d) does something about it
5 · Two plots	13	The misleading plot is genuinely misleading and every change is named
6 · Who is missing	15	Three distinct mechanisms, each with a <i>direction</i>
— · AI disclosure	10	Scored on the four-level scale above; a no-AI declaration earns full marks
Total	100	

Correct code that answers a question I did not ask earns nothing, and a correct number with no sentence around it earns half. Every part asks you to say what the number *means*; that sentence is what is being graded.

THE ONE-LINE VERSION OF THIS ASSIGNMENT

Nothing in `flights` is dirty. There are no misspelled categories, no duplicate keys, no sentinel values. Every defect you will find is in **what the file is** and **who is not in it** — and none of it shows up in `summary()`.

Probability on a file that does not cooperate.

Assigned	Monday, Sep 14
Due	Sunday, Sep 20, 11:59 p.m. , on Blackboard
Weight	One of eight assignments; homework is 20% of the grade and no score is dropped
AI	Encouraged , with the disclosure described at the end
Extensions	48 hours, no reason needed, twice in the term. Say in the submission that you are using one.

The practice problems this week built their dice, their weather, and their fraud model by simulation, so every assumption held because someone put it there. This assignment runs the same ideas on the flights file from HW 1, where nobody arranged for independence and nothing is Poisson because a chapter said so. Most of the marks are for noticing where a textbook rule breaks and saying why.

What you submit

Two files, both uploaded to Blackboard:

1. `hw2-lastname.Rmd` , your R Markdown source.
2. `hw2-lastname.html` , the knitted output.

Download the starter file: `hw2-template.Rmd` . It has the headings, the YAML, and the data-loading chunk. Deciding what code belongs under each heading is part of what is being graded.

Your `.Rmd` must knit **in a fresh R session**, the two-computer test from the [R Primer](#). Restart R, knit, and read the output before you upload it.

EVERY NUMBER IN YOUR PROSE MUST BE COMPUTED, NOT TYPED

As in HW 1: a probability you state in a sentence comes from inline R code in that sentence, not from the console. A typed number that disagrees with your own code is the easiest error for me to find, and I do look.

The data, and the definitions

```
library(nycflights13)
data(flights)
```

Use these definitions throughout, **exactly**, so that your numbers can be checked. Where a part needs something different, it says so.

- **The flights you use.** Rows where both `dep_delay` and `arr_delay` are present: 327,346 of the 336,776. Call this data frame `d` .
- **A:** the flight arrived more than 15 minutes late, `arr_delay > 15` .
- **D:** the flight departed more than 15 minutes late, `dep_delay > 15` .
- **A day** is a calendar date built from `year` , `month` and `day` .
- **Morning** and **evening** use the *actual* departure hour, `dep_time %/% 100` : morning is below 12, evening is 17 or later.

Nothing here needs `dplyr` . A logical vector, `mean()` , `table()` , `tapply()` , `var()` and `pnorm()` are the whole toolkit, as in the chapters.

Part 1 · Events are logical vectors (10 points)

- Build **A** and **D** as logical vectors. Report $P(A)$, $P(D)$ and $P(A \cap D)$.
 - Compute $P(A \cup D)$ two ways: directly, with `|`, and by the [addition rule](#). Show both. They must agree to every digit, and if they do not, one of your vectors is wrong.
 - Compute the probability that a flight is neither, and say which rule gets you there in one line.
-

Part 2 · Independence, checked rather than assumed (18 points)

- If A and D were independent, $P(A \cap D)$ would equal $P(A)P(D)$. Compute both and their ratio. Then compute $P(A | D)$ and say, in one sentence, what it tells a passenger whose flight has just pushed back 20 minutes late.
- Compute $P(A | \text{origin})$ for EWR, JFK and LGA. Is arriving late independent of the airport a flight leaves from? Answer with the numbers, then say whether the difference is large enough to matter to a traveler choosing an airport. Both halves count.
- Two flights, same day.** For each day, compute the proportion of morning flights that arrive late and the proportion of evening flights that do. Then:
 - report the correlation between those two daily proportions across days;
 - report the average morning proportion and the average evening proportion, and their product;
 - estimate the probability that a morning flight and an evening flight on the **same** day are both late, as the average over days of (morning proportion $\times$ evening proportion), and compare it with the product;
 - name what the two flights share.

Practice problem B3 built this dependence with a `weather` variable. Here nobody built it.

Part 3 · A rule that works, and the same rule that does not (22 points)

A gate agent flags every flight that departs more than 30 minutes late (`dep_delay > 30`) as one that “will arrive late”.

- Take “arrive late” to mean `arr_delay > 15`. Build the 2×2 table of flag against outcome, and report the rule’s **sensitivity**, **specificity** and **precision**.
 - Keep the flag exactly as it is. Change only the outcome, to `arr_delay > 180`: three hours late. Report the same three numbers, and the counts of true positives and false positives.
 - Recompute the precision in (b) from its sensitivity, its specificity, and the base rate alone, using Bayes’ rule as in [the medical test](#). Show that it matches the precision you counted.
 - One paragraph. The rule did not change, it catches essentially every three-hour delay, and its precision fell from about 95% to 8%. Explain why to a manager who is about to use it as an alarm for three-hour delays.
-

Part 4 · Which distribution, and why not (22 points)

- One fingerprint, two counts.** A Poisson count has its mean equal to its variance ([the Poisson’s fingerprint](#)). Use **all 336,776 rows** for this part, not `d`, because a canceled flight has no delays to keep it in `d`.
 - Cancellations.** For each day, count the flights with a missing `dep_time`.
 - Scheduled departures.** For each day, count the flights from JFK with scheduled `hour == 17`.

Report the mean and variance of each count. Neither is Poisson, and they fail in opposite directions. For each one, say in one or two sentences what about *how the count is produced* breaks the Poisson.

- For JFK→LAX flights in `d`, count for each day how many arrive late (A) and how many flights there were. Let p be the pooled late rate: all late JFK→LAX flights over all JFK→LAX flights. If each day’s count were binomial, its variance would be about $np(1 - p)$. Report p , the observed variance of the daily late count, and the average of $np(1 - p)$ over days. Name which of the binomial’s conditions fails, and connect it to Part 2(c).

c. Check the 68–95–99.7 rule on `arr_delay` (all of `d`) and on `air_time` for JFK→LAX. Report the three proportions for each. Then:

- for the variable that passes, compare `pnorm`'s estimate of $P(\text{air_time} > 360)$ with the actual proportion;
- for the one that fails, compare `pnorm`'s estimate of $P(\text{arr_delay} < -30)$ with the actual proportion, say which way it is wrong, and say why the shape you found in HW 1 predicts that.

Part 5 · < or ≤ (8 points)

- a. Using the daily late counts from 4(b), compute $P(X < 5)$ and $P(X \leq 5)$. Explain the difference, and say which probability it equals.
- b. `air_time` is a measurement, and [Random Variables](#) says that for a continuous variable $<$ and $\leq$ give the same probability. Compute $P(\text{air_time} < 330)$ and $P(\text{air_time} \leq 330)$ for JFK→LAX. They differ. Explain why, using how the column is recorded.

Part 6 · The staffing sentence (10 points)

A colleague plans next month's rebooking staff this way:

"About 24% of flights arrive late, and flights are independent, so on a day with 30 JFK–LAX flights we should expect about 7 late, give or take a couple."

Write **one paragraph, at most 150 words**, correcting the plan. Use at least two numbers from this assignment, name the assumption that fails, and say which way it pushes the staffing: too many staff, too few, or both, on different days.

AI disclosure (required)

Homework is the component where AI use is **encouraged**. It is not the component where it goes undeclared. The rules are the same as HW 1.

Include, at the end of your knitted document, a section titled `## AI disclosure` containing:

1. **The prompt log.** Every prompt you sent, and enough of what came back to show what you were working from. A link to a shared conversation is fine.
2. **A critique**, 150–300 words, showing that **you interrogated the assistant rather than accepting it**: either something it got wrong and what that would have cost, or the prompts you used to try to break it when it held up. Both earn full marks.

"I did not use an AI assistant on this assignment" is always an acceptable answer, needs no justification, and earns full marks on the disclosure.

WHERE ASSISTANTS RELIABLY GO WRONG ON THIS ASSIGNMENT

If you want somewhere to push: they multiply marginal probabilities without checking independence (Parts 2 and 6), report accuracy when asked whether an alarm is useful (Part 3), call a count Poisson because it is a count (Part 4), and treat `air_time` as continuous because it is a time (Part 5).

A fabricated log is falsification of data under the Academic Integrity Code and is reported. The bar for full marks here is honesty, not sophistication.

	POINTS	WHAT IT LOOKS LIKE
Absent	0	No disclosure section, or a log with no critique. Also: no declaration either way.
Generic	3	True of any assistant on any assignment. No part of <i>this</i> assignment named.
Specific	7	Names a real error or a real test, in a named part, on this dataset.
Consequential	10	The above, plus what it would have cost, or a probe aimed at something the assignment is actually about.

A declaration of no AI use scores **10**.

How it is graded

PART	POINTS	WHAT EARNS THEM
1 · Events	10	The addition rule reproduces the direct count exactly
2 · Independence	18	Every comparison carried by a number; (b) judged for size; (c) names the shared cause
3 · Base rates	22	The precision collapse in (b) explained by the base rate, and matched by Bayes' rule
4 · Distributions	22	Each failure explained by how the data were produced, not only detected
5 · $<$ or $\leq$	8	(b) names the whole-minute recording
6 · Staffing	10	The failed assumption named, its direction stated, numbers used
AI disclosure	10	The four-level scale above; a no-AI declaration earns full marks
Total	100	

A correct number with no sentence around it earns half. Every part asks what the number *means*, and that sentence is what is being graded.

THE ONE-LINE VERSION OF THIS ASSIGNMENT

Every rule this week is true under its assumptions. On real data the work is checking them, and the checking is one `mean()` away.

How far off is the average?

Assigned	Monday, Sep 21
Due	Monday, Sep 28, 11:59 p.m. , on Blackboard
Weight	One of eight assignments; homework is 20% of the grade and no score is dropped
AI	Encouraged , with the disclosure described at the end
Extensions	48 hours, no reason needed, twice in the term. Say in the submission that you are using one.

The chapters this week derived $SE(\bar{X}) = \sigma/\sqrt{n}$ in four lines, and the lab watched it appear from `rnorm` and `rexp`. Both had something real data never gives you: a population whose μ and σ were known.

This assignment gives you one. The 327,346 flights in `d` are every completed flight out of New York in 2013, so for once you can compute the truth, draw samples from it, and check how far the samples land. Most of the marks are for Part 4, where the formula, used the way software uses it, is wrong by far more than you would guess, and the reason is one word in its derivation.

What you submit

Two files, both uploaded to Blackboard:

1. `hw3-lastname.Rmd`, your R Markdown source.
2. `hw3-lastname.html`, the knitted output.

Download the starter file: `hw3-template.Rmd`. It has the headings, the YAML, and the data-loading chunk. Deciding what code belongs under each heading is part of what is being graded.

Your `.Rmd` must knit **in a fresh R session**, the two-computer test from the [R Primer](#). Restart R, knit, and read the output before you upload it.

EVERY NUMBER IN YOUR PROSE MUST BE COMPUTED, NOT TYPED

As in HW 1 and HW 2: a number you state in a sentence comes from inline R code in that sentence, not from the console. A typed number that disagrees with your own code is the easiest error for me to find, and I do look.

The data, and the definitions

```
library(nycflights13)
data(flights)
```

Use these definitions throughout, **exactly**, so that your numbers can be checked. Where a part needs something different, it says so.

- **The flights you use.** Rows where both `dep_delay` and `arr_delay` are present: 327,346 of the 336,776, as in HW 2. Call this data frame `d`. Part 1 is the one exception and uses all 336,776 rows.
- **Late** means `arr_delay > 15`.
- **A day** is a calendar date built from `year`, `month` and `day`.
- **The population.** In Parts 3 to 6, treat `d$arr_delay` as the entire population. Its mean is μ and its standard deviation is σ . `sd()` divides by $n - 1$ rather than by the population size; with 327,346 rows the difference is in the fifth significant digit, so either is fine.
- **Simulations.** Put `set.seed(6101)` on the line immediately before each simulation, as the chapters do. If you draw in a different order your simulated numbers will differ a little from mine. That is expected and is not marked down; a difference

in the first significant digit is a bug.

Nothing here needs `dplyr`. `mean()`, `var()`, `sd()`, `cov()`, `tapply()`, `sample()`, `replicate()` and `split()` are the whole toolkit, as in the chapters.

Part 1 · A payout table (12 points)

An airline's compensation policy pays, for every scheduled flight:

OUTCOME	RULE	PAYOUT
Canceled	<code>dep_delay</code> missing	\$400
Diverted	<code>dep_delay</code> present, <code>arr_delay</code> missing	\$200
Over three hours late	<code>arr_delay</code> > 180	\$200
One to three hours late	<code>60</code> < <code>arr_delay</code> <= <code>180</code>	\$50
Anything else		\$0

Use **all 336,776 rows** for this part. A canceled flight is exactly the kind `d` leaves out.

- Build the payout for every flight. Report the probability of each payout value, then compute $E[\text{payout}]$ two ways: from the table, as $\sum x P(X = x)$ in [Expectation](#), and as `mean()` of the payout column. They must agree to every digit.
- What share of the expected payout comes from cancellations, and what share of flights are canceled? One sentence on what that says about where an expectation's weight can sit.
- An insurer offers to cover the policy for a flat fee per scheduled flight. What fee is fair? Then give the expected cost of a day with 900 scheduled flights, and say in one sentence why that calculation needs no assumption at all about whether flights on the same day are canceled together.

Part 2 · Sums are free, variances are not (16 points)

For each day and each origin (EWR, JFK, LGA), count the late flights. That gives three daily counts, and their sum is the day's total.

- Report the mean and variance of each airport's daily count, and of the daily total. Check that the mean of the total is the sum of the three means. Then compare the variance of the total with the sum of the three variances, and give the ratio.
- Compute the three covariances between airports, and show that [the general rule](#) with the covariance terms reproduces the variance of the total exactly. Report the three correlations too.
- A planner adds the three airports' variances to size a region-wide rebooking team. In two or three sentences: what do the airports share that makes the covariances positive, and does the planner's figure overstate or understate the day-to-day swings?

Part 3 · A population you can see (18 points)

- Report μ and σ for the population.
- For $n = 25, 100, 400$ and 1600 , draw 5,000 samples of n flights from the population, take the mean of each, and report the standard deviation of those 5,000 means next to $\sigma/\sqrt{n}$. `arr_delay` has the long right tail you found in HW 1. Does the formula care? Say what in [the derivation](#) tells you whether it should.
- Draw **one** sample of 400 flights. Report its mean, its standard deviation, and the standard error of its mean. Then write two sentences, one using the SD and one using the SE, each saying what its number describes. A reader must be able to tell from

the wording alone which is which.

- d. How many randomly sampled flights would pin the mean arrival delay down to a standard error of 1 minute? Of half a minute? Say what the second answer costs compared with the first.

Part 4 · When n is not n (24 points)

An analyst who cannot afford the whole file samples **days** instead of flights: pick 16 days at random, and average every flight on those days.

- a. Do it 2,000 times. Report the average number of flights per replicate and the standard deviation of the 2,000 means. That standard deviation is the real standard error of this design.

Speed: `by_day <- split(d$arr_delay, d$date)` once, outside the loop, and then `unlist(by_day[sample(names(by_day), 16)])` inside it. Filtering all of `d` 2,000 times also works, but it is slow.

- b. Now compute what $\sigma/\sqrt{n}$ says the standard error should be, using the average number of flights from (a) as n . Then check it: draw 2,000 random samples of that many flights and report the standard deviation of their means. Give the ratio of the real standard error in (a) to the formula's.
- c. How many **randomly sampled** flights would give the same standard error as the 16 days did? That is the design's *effective* sample size. Report it, and what fraction of the flights the analyst actually averaged it amounts to.
- d. Compute the mean arrival delay of each of the 365 days, take the standard deviation of those daily means, and divide by $\sqrt{16}$. Compare the result with (a), and say in one sentence what that says the independent units in this design are.
- e. One paragraph. The derivation has four steps and exactly one assumes something about how the observations relate to each other. Name the step, say what fails here and why the covariances are positive, and say which way it pushes a standard error that software computes from 14,000 rows.

Part 5 · A rare rate (12 points)

A proportion is a mean of zeros and ones, so its standard error is $\sqrt{p(1-p)/n}$.

- a. Using the population rates, compute the standard error of $\hat{p}$ in a random sample of 1,000 flights for two outcomes: late (`arr_delay > 15`), and over three hours late (`arr_delay > 180`). Report each standard error, and each as a fraction of its own p .
- b. For each outcome, how many flights would bring the standard error down to 10% of p ? One sentence on why the rarer outcome needs so many more.
- c. Both answers assume the 1,000 flights are a random sample. In one sentence, using Part 4: what would happen to them if the 1,000 flights came from three days?

Part 6 · The dashboard sentence (8 points)

The operations dashboard reports:

“March mean arrival delay: 5.8 minutes, standard error 0.27 ($n = 27,902$ flights). A month whose mean differs by more than half a minute has really changed.”

Write **one paragraph, at most 150 words**, correcting it. Use at least two numbers you computed, name the assumption behind the 0.27, and say what following the dashboard's rule would do: how often would it declare a real change?

AI disclosure (required)

Homework is the component where AI use is **encouraged**. It is not the component where it goes undeclared. The rules are the same as HW 1 and HW 2.

Include, at the end of your knitted document, a section titled `## AI disclosure` containing:

1. **The prompt log.** Every prompt you sent, and enough of what came back to show what you were working from. A link to a shared conversation is fine.
2. **A critique**, 150–300 words, showing that **you interrogated the assistant rather than accepting it**: either something it got wrong and what that would have cost, or the prompts you used to try to break it when it held up. Both earn full marks.

“I did not use an AI assistant on this assignment” is always an acceptable answer, needs no justification, and earns full marks on the disclosure.

WHERE ASSISTANTS RELIABLY GO WRONG ON THIS ASSIGNMENT

If you want somewhere to push: they compute $\sigma/\sqrt{n}$ with n equal to the number of rows, whatever the rows are (Parts 4 and 6), add standard deviations instead of variances (Part 2), call a small standard error precise without comparing it with p (Part 5), and describe an expected value as what will happen on a given day (Part 1).

A fabricated log is falsification of data under the Academic Integrity Code and is reported. The bar for full marks here is honesty, not sophistication.

	POINTS	WHAT IT LOOKS LIKE
Absent	0	No disclosure section, or a log with no critique. Also: no declaration either way.
Generic	3	True of any assistant on any assignment. No part of <i>this</i> assignment named.
Specific	7	Names a real error or a real test, in a named part, on this dataset.
Consequential	10	The above, plus what it would have cost, or a probe aimed at something the assignment is actually about.

A declaration of no AI use scores **10**.

How it is graded

PART	POINTS	WHAT EARNS THEM
1 · Payout table	12	Both routes to the expectation agree; (c) says why linearity needs no independence
2 · Sums and variances	16	The covariance terms recover the total exactly; the direction of the planner's error is stated
3 · A population you can see	18	The formula checked against simulation, and the SD and SE sentences kept apart
4 · When n is not n	24	The real standard error measured, the effective n computed, and the failed step named
5 · A rare rate	12	Standard errors judged against p , not in absolute terms
6 · Dashboard	8	The assumption named, its consequence for the alarm stated, numbers used
AI disclosure	10	The four-level scale above; a no-AI declaration earns full marks
Total	100	

A correct number with no sentence around it earns half. Every part asks what the number *means*, and that sentence is what is being graded.

THE ONE-LINE VERSION OF THIS ASSIGNMENT

$\sigma/\sqrt{n}$ is exact when the n observations are independent. The work is knowing what your n independent things are, and a file with 14,000 rows can turn out to hold 16 of them.